

ФСП Талант
Сопроводительная документация

команда Дмитрия Конькова

Цифровая платформа-агрегатор ИТ-вакансий с верифицированным профилем достижений участника Федерации спортивного программирования. Самостоятельный проект, лицензия MIT.

Репозиторий: https://github.com/KonkovDV/fsp-talent
Сборка документа: 20.10.2026
Исходные тексты — Markdown в каталоге docs/, этот файл собран скриптом docs/tools/build_pdf.py.

Содержание

1. Сопроводительная документация
2. Соответствие техническому заданию
3. Сборка, развёртывание и запуск
4. Руководство пользователя стенда
5. Проверка теста на грейд
6. Проверка подбора
7. Нагрузочная проверка
8. Интеграция с ФСП
9. Защита от фиктивных вакансий и ATS
10. Коды ошибок API
11. Дизайн-система
12. Сторонние компоненты и лицензии

ФСП Талант: сопроводительная документация

Этот документ отвечает на раздел 3.4 технического задания: каждому пункту раздела мы отвели свою главу и сохранили порядок. Все цифры получены запуском кода, команды приведены рядом.

Пункт ТЗ 3.4	Глава
Функциональная и компонентная архитектура	1, 2
Механика тестирования и устойчивость к распространению заданий	3
Механика подбора и логика категоризации	4
Процедура валидации и её результаты	5
Схема интеграции с реестром ФСП	6
Описание API	7
Библиотеки и компоненты с версиями	8
Инструкция по сборке, развёртыванию и запуску	9

0. Решение в двух абзацах

Человек называет грейд. Тест его зажигает. Работодатель не ждёт отклика: он приходит сам, с вилкой, и видит имя только после «да».

Кандидат регистрируется по email или входит через ФСП ID, проходит опрос (направление и специализация, цель: работа или стажировка, предметная область по желанию), выбирает грейд и подтверждает его адаптивным тестом. Из этого складывается категория «специализация × грейд». Результаты соревнований ФСП не меняют грейд: это отдельная подтверждённая ось профиля. По ФСП ID без входа видны только дисциплина, год, корзина места и статус соревнования. Протокол соревнования — не цифровой след и не грейд. Если реестр молчит, кандидат может загрузить файл: строка появляется у него на странице и в балл не входит. Порядок внутри категории на странице «Режим жюри» — места по подтверждающему тесту, без имени и без номера аккаунта. Открытые вакансии рядом называет компанию и должность. Имени и контактов в этих ответах нет. Тот же текст может забрать мессенджер. Кандидат без истории ФСП ничего не теряет.

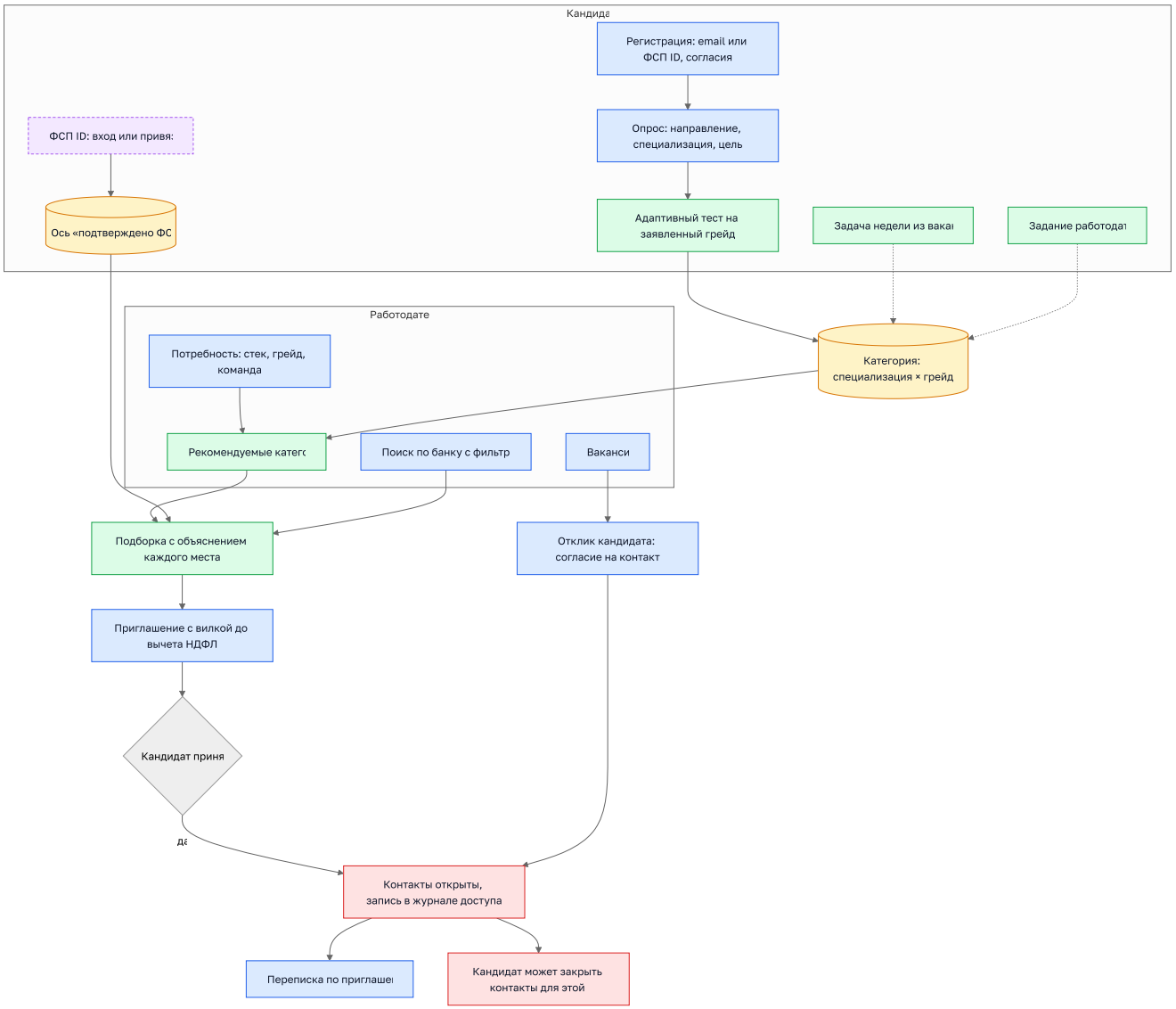
У компании есть публичная страница: название, открытые вакансии и уровень доверия. Почты компании и имён кандидатов на ней нет. Ссылка стоит в приглашении.

В шапке кабинета кнопка «Куда нажать» отвечает ссылкой на раздел этой роли. На странице «Режим жюри» тот же вопрос без входа ведёт только на публичные страницы. Вопрос на сервере не хранится и наружу не отправляется: ни в модель, ни в профиль.

Работодатель не ждёт откликов. Он описывает потребность: стек, специализацию, грейд, чем занимается команда. Платформа показывает, в каких категориях искать, и даёт подборку, где у каждого места есть объяснение. Работодатель сам отправляет приглашение с вилкой до вычета НДФЛ (рядом показывается сумма на руки по шкале ст. 224 НК) и может дать кандидату своё задание. Имя и контакты кандидата открываются только после того, как кандидат принял приглашение. Логин публичного кода хранится как контакт: на карточке до согласия только пометка, в балл он не входит. Свидетельство о навыках добавляет цитату к навыку и грейд не ставит. Навык, уже подтверждённый тестом, файл не понижает. Дальше стороны переписываются на платформе, а кандидат может закрыть контакты для одной компании. Классический путь тоже есть: вакансии, отклик, статусы. В тексте вакансии подписываются просьба денег у кандидата, зарплата мимо оформления и отбор по внешности. Такая вакансия остаётся открытой, грейд не меняется.

1. Функциональная архитектура

Роли и сценарии

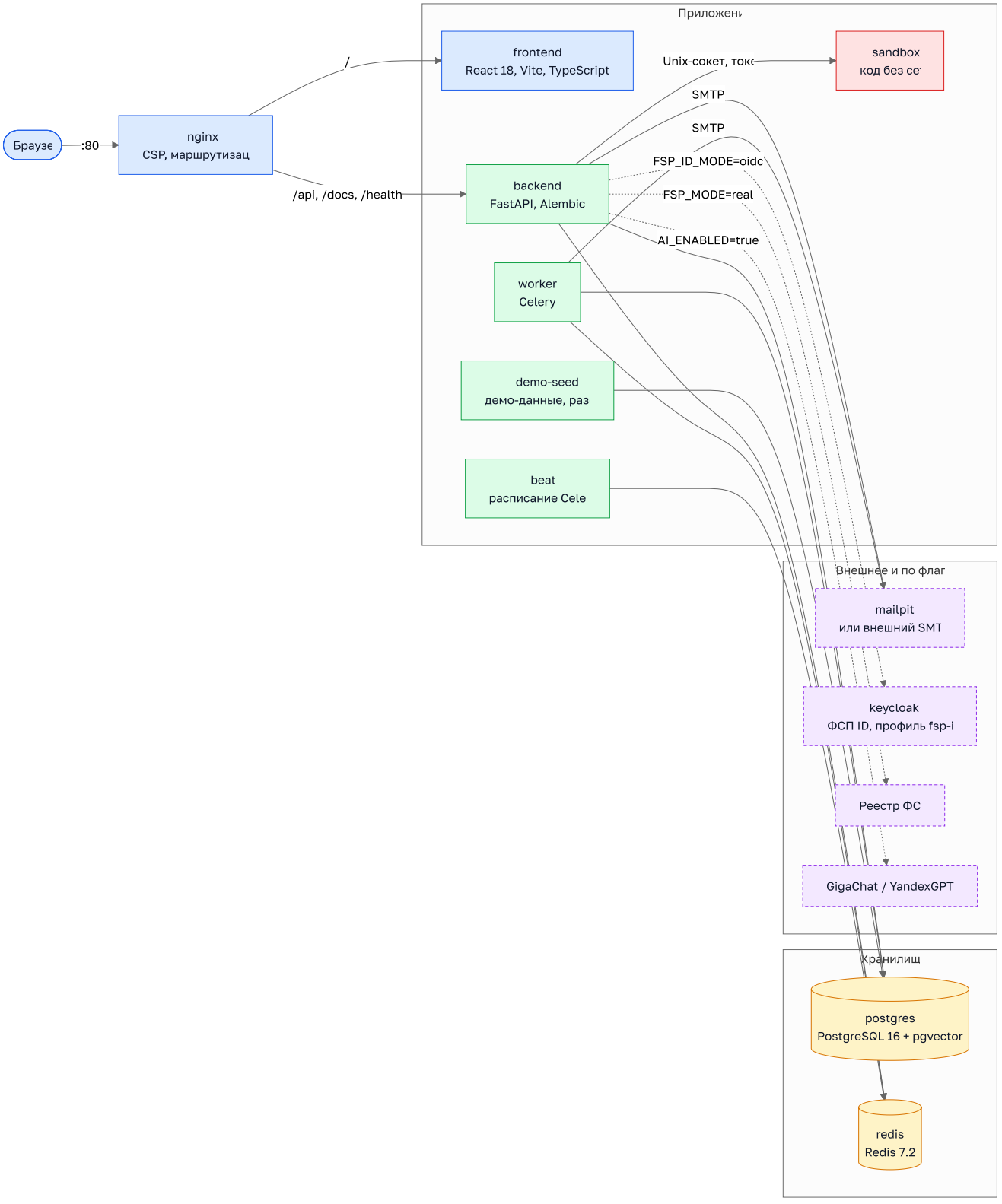


Что умеет каждый кабинет

Кандидат (ТЗ 2.1)	Работодатель (ТЗ 2.1)
Профиль вручную или из PDF-резюме: ФИО, контакты, стек, грейд, стаж, софт-скиллы. Каждое поле предлагается с цитатой и переносится по кнопке	Профиль компании: сфера, сайт, описание, ИНН с проверкой контрольной суммы. Предпросмотр приглашения глазами кандидата
Опрос, выбор грейда, адаптивный тест, итог по темам, история попыток. Путь к следующему грейду: навыки, которые есть у людей этой ступени и которых нет в профиле; навык виден, если его назвали не меньше пяти. Рядом — открытые потребности той же специализации, до которых не хватает одного обязательного навыка. Грейд и балл подбора эти списки не меняют	Потребность с версиями: изменения не теряют прежнюю подборку и отобранных
Категория и грейд, подтверждённые тестом. Пересдача на грейд ниже или выше с ограничением частоты. На стенде есть пробный проход из 15 заданий и 5 минут	Рекомендуемые категории и подборка, у каждого кандидата — разбор факторов и «лучше N % категории»
Задания от работодателей (общие для специализации и адресные), задача недели из вакансии и короткая практика по слабой теме	Поиск по всему банку: специализация, грейд, стек, дисциплина и разряд ФСП, только подтверждённые тестом
Входящие приглашения с вакансией, вилкой и профилем компании: принять или отклонить, переписка после принятия, закрыть контакты для одной компании. Шкала ожиданий своей категории: четверти только если названных сумм не меньше пяти	Сравнение до четырёх кандидатов рядом, без имён. Свои задания кандидатам: с эталоном (проверяет система) или открытые (проверяет работодатель)
Вакансии под профиль, отклик, статусы откликов	Приглашение без привязки к вакансии, статусы: отправлено, просмотрено, принято, отклонено, отозвано, истекло

Кандидат (ТЗ 2.1)	Работодатель (ТЗ 2.1)
Приватность: видимость карточки, согласия и их отзыв, скачать свои данные, сменить пароль, удаление аккаунта, вход через ФСП ID и привязка	Вакансии с редактированием, отклики со сменой статуса. В разделе «Компания» — скачать свои данные и сменить пароль
Уведомления на сайте и письма	Контакты только после согласия кандидата, переписка по принятому приглашению, кнопка «Какой тест получится» у вакансии и вкладка «Как проверим» у потребности

2. Компонентная архитектура



Монолит с явными слоями (ТЗ 3.5): роутеры принимают запрос и проверяют права, бизнес-логика живёт в сервисах, которые не знают про HTTP, модели отвечают только за хранение. Внешние системы (реестр ФСП, ФСП ID, GigaChat или YandexGPT, почта) подключены через адаптеры с флагом режима: в демо всё работает без внешних сервисов. На публичном стенде вместо nginx стоит Caddy 2.10: он принимает 80 и 443 и сам получает сертификат. Локальная схема на рисунке от этого не меняется.

Модули backend

Слой	Каталог	Что внутри
API	`app/api/v1/`	24 модуля роутеров, собранных в `router.py`: auth, candidates, resumes, favorites, search, needs, vacancies, offers, interviews, messages, tasks, assessment, employer, moderation, methodist, integrations, passport, legal, notifications, public, problems, guide, ai, fsp-id-demo. Единый формат ошибок `app/api/errors.py`
Тест	`app/items/`, `app/services/cat.py`, `assessment.py`, `item_monitor.py`, `practice.py`, `vacancy_exam.py`	Параметрические семейства заданий, IRT 3PL, адаптивный выбор, мониторинг утечки, задача недели, тест по вакансии, пробный проход
Подбор	`app/services/matching.py`, `search/pipeline.py`, `categorization.py`, `skills.py`	Оценка соответствия, SQL-фильтры, полнотекстовый ранг и RRF, категории, нормализация навыков
ФСП	`app/services/fsp/`, `app/services/scoring/`	Реестр (mock и HTTP), ФСП ID (OIDC), ось ФСП: класс результата, давность, байесовское сжатие
Доверие	`app/services/vc/`, `identity.py`, `privacy.py`, `notify.py`, `inn.py`	Удостоверения VC 2.0, коды подтверждения, согласия и удаление данных, уведомления, проверка ИНН
Задания и переписка	`app/services/tasks.py`, `app/api/v1/tasks.py`, `messages.py`	Задания от работодателя, проверка ответа, строка в карточке, переписка по приглашению
Фоновые задачи	`app/workers/`	Пересчёт профилей, истечение приглашений, задача недели, очистка по 152-ФЗ
Проверка	`app/validation/`	Симуляция теста, проверка подбора, калибровка заданий
Данные	`app/models/`, `alembic/versions/`	`0001_initial` — вся схема одним шагом, включая историю опроса, результат теста по навыку, условие «да, если» и гостевую подборку; у ревизии есть откат

Данные

Главные таблицы: `participants`, `skills`, `results` (строки протоколов ФСП), `discipline_scores` (ось ФСП), `category_assignments`, `test_sessions` и `test_session_items`, `grade_confirmations`, `item_stats` (статистика ключей заданий), `practice_attempts`, `employer_tasks` и `task_submissions` (задания работодателя и ответы), `employers`, `needs` и `need_versions`, `need_shortlist`, `offers`, `offer_messages`, `vacancies`, `applications`, `consents`, `consent_logs`, `access_logs`, `notifications`, `verifiable_credentials`.

У `participants` в начальной миграции есть `last_login_at`. По нему `GET /api/v1/public/jury-free` (только при `DEMO_MODE`) возвращает тот из адресов `jury1` … `jury10`, в который дольше всех не входили. В ответе нет ничего, кроме адреса. Ночной сброс это поле не очищает.

3. Механика тестирования

Единый набор заданий расходится по рынку. Задание, сочиненное моделью под одного человека, нельзя сравнить с соседним по сложности.

Параметрические семейства. Семейство — шаблон задания с параметрами и кодом, который вычисляет правильный ответ. Каждый показ — новый изоморф: другие числа, строки и фрагменты кода. В оценке изоморфы одного уровня делят родителя. Различимость `a` — априор семейства, по ответам в банк она не записывается. Трудность `b` задаёт уровень (от −2,2 до 1,8). Страница «Как устроен тест» показывает `a` каждого семейства. Сложность не берётся из случайных чисел конкретного показа. Двадцать семейств с вычисляемым ответом (Python, SQL, JavaScript, алгоритмы, графы, сети, безопасность, ML-метрики, встраиваемые системы, границы и расхождения тестов, метрики и доли аналитика, непокрытые требования и шаги цепочки, пакеты и опоздания в конвейере данных, Docker, Kubernetes, Linux shell) и два концептуальных пула — всего 22 в `backend/app/items/registry.py`. Пулы вопросов на понимание: у специализаций с вопросами на понимание не меньше шести идей, у каждой идеи несколько формулировок. У тестирования план из четырёх семейств с вычисляемым ответом. У аналитика их шесть, у инженерии данных четыре плюс вопросы на понимание конвейера. Юнит-тест `test_item_families.py` перезапускает каждый сгенерированный фрагмент в Python, Node.js, Go, Java, Kotlin, C# или SQLite и сверяет ключ. Языковая модель во время теста не используется.

Почему утечка не помогает:

- выученный ответ не подходит к новым параметрам;
- если ответ — целое число, его вводят с клавиатуры, вариантов на экране нет. В модели остаётся малый шанс c = 0,02; у четырёх вариантов c = 0,25. На границе грейда информация вводимого числа 0,240 против 0,150, то есть в 1,60 раза больше (`cut_information`);

- правильный ответ не уходит в браузер до конца теста, ответ принимается только на текущее задание по HMAC-токену, время засекает сервер. Ключ в строке задания подписан отдельно: если номер правильного варианта в базе не сходится с подписью, ответ не входит в θ. Повтор той же отправки несёт Idempotency-Key и не открывает вторую попытку;
- если доля верных ответов по ключу растёт сверх ожидаемой, ключ уходит в карантин после 30 ответов: $z \geq 3{,}5$ или сумма $S = \max(0, S + (u - P - 0{,}12)) \geq 7$. θ для этой вероятности считается без заданий того же семейства. Статистика открыта: `GET /api/v1/public/assessment/items`;
- вторая попытка не повторяет вопросы на понимание: общих не больше 40 %.

Модель угроз: скрипт или ассистент, который отвечает за кандидата. Цель атакующего — получить подтверждённый грейд выше настоящего. Что стоит на пути сегодня:

Защита	Как работает	Где в коде
Новый экземпляр на каждый показ	Числа и фрагменты кода генерируются заново, ответ чужого экземпляра совпадает в коде в 9,3 %, в вопросах на понимание в 36,9 %; с чужим ключом сдают 69,2 % против 58,5 % честно	`backend/app/items/`, `python -m app.validation.construction_check`
Контроль экспозиции и карантин	Выбор задания штрафует частые ключи (Симпсон–Хеттер); ключ с избытком верных ответов уходит в карантин при $n \geq 30$ и $z \geq 3{,}5$ либо при сумме $S \geq 7$ (приращение $u - P - 0{,}12$). θ считается без заданий этого семейства	`services/cat.py`, `services/item_monitor.py`
Быстрый верный ответ	Верный ответ на задание с кодом быстрее 4 секунд считается по семейству; три таких в одном семействе помечают попытку для разбора. Отдельно идёт накопленная сумма (CUSUM) доли верных и скорости по семейству	`services/assessment.py:59-60, 84-97, 100-127`
Уход со вкладки и копирование условия	Каждое событие пишется в отметки сессии	`services/session_marks.py`
Несогласованность ответов	$lz^* < -1{,}645$ даёт отметку и предложение пересдать	`services/person_fit.py`, `services/assessment.py`
Разрыв двух форм	Остаток на заданиях с фиксированной формулировкой минус остаток на свежих числах, при опорной оценке θ только по заданиям с кодом. Порог 0,40. На `detector_curves` (зёрна 201–203, заложенное совпадение чужого ключа в коде 9,2 %) ложных 2,1 %, поймано 24,9 %. Отметка для просмотра. Грейд и место не меняются	`services/form_gap.py`, `validation/detector_curves.py`
Отметка теста	Сигналы браузера кандидат видит у себя. В выдаче работодателя они не меняют оценку и не пишутся. Грейд не меняется	`services/session_marks.py`, `services/search/pipeline.py`
Вопросы на понимание	До 30 % плана теста — задания на понимание с лимитом 75 секунд; вторая попытка повторяет их не больше чем на 40 %	`items/registry.py:46`, `services/assessment.py:57, 453-490`

Что атакующий получает в лучшем случае. Один тест подтверждает только заявленный грейд: на следующую ступень нужен отдельный тест, автоматического прыжка нет (`services/cat.py`). Повтор того же грейда — не раньше чем через 7 дней (`RETAKE_INTERVAL_DAYS`, `services/assessment.py`). Отмеченная попытка место в выдаче не опускает: множитель выключен, работодатель отметку в карточке не видит. Контакты открываются только после приглашения и согласия, а дальше идут задания работодателя и собеседование, где решение принимает человек.

Следующие шаги. Прогон локальной модели на 200 изоморфах каждой специализации с долей решённых по семействам и уровням; семейства, которые модель решает чаще 80 %, получают меньший вес в плане теста. Задания «объясни подход» с проверкой работодателем. Прокторинг для тестов, которые идут в найм напрямую. Задания теста, где кандидат пишет и запускает код, — в той же песочнице без сети, где уже работают практические задачи (`services/sandbox/`). Аномальное время ответа как признак подсказанного задания — van der Linden & Guo, 2008.

Задачи прошлых соревнований ФСП (концепция расширения). Постановщик допускает их как дополнительные, если защититься от тех, кто их уже решал. Задачи из открытых архивов ФСП — отдельный необязательный блок, в грейд он не входит, θ не меняет и показывается отдельной строкой. Кандидат, у которого в реестре есть запись об участии в соревновании X, задач X не получает. Задачи с опубликованным разбором старше заданного срока исключаются: разбор делает их выученными. Приёмочный тест: участник соревнования X не получает ни одной задачи X.

Адаптивность. Уровень считается как среднее апостериорного распределения. Следующее задание берётся там, где оно лучше всего разделяет людей у порога заявленного грейда. Из пяти лучших одно выбирается с ограничением показов. Пока по специализации нет двадцати законченных сессий, выбор среди пяти равномерный. Дальше ключ, который видели чаще 40 % этих сессий, берётся реже: с вероятностью 0,40, делённой на долю. Последний в пятёрке берётся всегда. Опубликованная таблица считает долю по всем сессиям сразу: самый частый ключ занимает 41 % всех показов и до 72 % сессий mobile. Тот же отбор, что на стенде, на этой когорте даёт внутри специализации 49,7–65,0 %. План тем специализации учитывается.

Тест длится от 8 до 20 заданий и заканчивается, когда вероятность уровня выходит за 10 % или 90 %. Если вопрос о следующем уровне уже закрыт, раньше может остановить обобщённое отношение правдоподобия у порога (зона ±0,25, порог $\ln 9$). На экране кандидат видит уверенность и оценку θ с погрешностью.

После основного теста, если ответы плохо сходятся с оценкой (поправка Снейдерса ниже −1,645), платформа предлагает пересдать. Это не вывод о списывании. Верный ответ на задание с кодом быстрее четырёх секунд копится по семейству: три таких ответа помечают попытку для разбора. Грейд и карантин ключа от пометки не меняются. Кривые информации, погрешность и показы по семействам открыты на странице «Как устроен тест» (`/method`).

Решатель одной семьи. В сессии не больше пяти заданий одной семьи. Перебор потолков 3, 4 и 5 и правило складного ножа на 2000 кандидатах и зерне 7 записан в VALIDATION_ASSESSMENT.md. В живом тесте потолок 5.

Решение о грейде. Грейд даёт только основной тест специализации: не тест по вакансии, не результаты ФСП и не буква в номере сертификата. Буква сохраняется и на карточке кандидата написано, что в поиск она не входит. Кандидат сам выбирает уровень, тест его подтверждает или нет; грейд подтверждён при вероятности уровня от 75 %. Тест на уровень подтверждает только этот уровень. Если результат уверенно выше (от 90 % за следующий порог), платформа предлагает отдельный тест на ступень выше, но сама грейд не повышает. Подтверждённый грейд не понижается (ТЗ 2). Если кандидат не сдал, тест на уровень ниже доступен сразу, и попытка не сгорает. Если те же ответы подтверждают ступень ниже по правилу теста на неё (априор в её пороге, вероятность от 75 %, тот же нож по семействам), кандидат может принять её кнопкой без отдельного теста. Предложение снимается, как только начат новый основной тест этой специализации. Пересдача того же уровня — через 7 дней, прерванная попытка засчитывается. После смены грейда следующее повышение — через 90 дней. Для стажировки есть грейд «стажёр» на ступень ниже junior. Грейд в опросе выбирается обязательно: без него опрос не сохраняется. Стаж только подсказывает выбор («по стажу 6 лет обычно выбирают сеньора») и сам грейд не назначает. **Правило первого подтверждения:** первый подтверждённый грейд — точка отсчёта, а не смена, поэтому паузу не запускает. Кандидат, который осторожно выбрал junior и подтвердил его, может сразу пройти тест на ступень выше; уже это повышение запускает паузу 90 дней. Так осторожный выбор не наказывается, а лестница без теста не проходится.

Пробный проход на стенде. Когда включён демо-режим, кандидат может пройти 15 заданий за 5 минут (по 20 секунд на задание). Итог подписан как пробный: грейд не подтверждается, категория не создаётся, поиск эту попытку не видит.

Длина теста. Сессия заканчивается на 20-м задании. Одно продолжение до 6 заданий на 2000 кандидатах и зерне 7 даёт 76,9 % сдавших среди тех, кто на уровне при пороге 77 % и ложные сдачи 9,8 % при пороге 8,5 %. Складной нож при потолке 5: 69,0 % сдавших среди тех, кто на уровне, при пороге 71 %, точность вдали от порога 88,9 % при пороге 91 %. Живой тест идёт до 20 заданий и с потолком 5.

Неотправленный ответ текущего задания пишется на сервер с номером версии. После перезагрузки поле заполняется снова. Вкладка со старым номером получает отказ и не затирает более новый черновик. В балл черновик не входит, пока кандидат не нажмёт «Ответить».

На обзоре кандидата одна строка говорит, какой подтверждённый навык закрыл бы больше всего открытых вакансий. Грейд от этой подсказки не меняется.

Тест зависит от вакансии. Навыки, названные в тексте вакансии или потребности, подтягивают свои семейства (`vacancy_exam.py`). Работодатель до публикации видит доли тем, цитаты, по которым определены навыки, и три примера заданий без ответа. Кандидат может пройти такой тест: он покажет результат по темам вакансии и подтвердит навыки, но грейд не изменит. В демо девять вакансий с описаниями: на них видно, как разные тексты дают разный тест.

Регулярные задания от работодателя (ТЗ 2.1). Федерация уточнила, что регулярное задание пишет работодатель. Работодатель создаёт задание в разделе «Задания» или кнопкой «Дать задание» в карточке кандидата. Тип ответа — число (эталон — целое), выбор одного из 2–8 вариантов, код из лаборатории, короткий шаблон или открытый текст до 4000 знаков. Число, выбор, код и шаблон проверяет система. Модель ответ не оценивает. Эталон кандидату не показывается, пока он сам не ответил. Открытый ответ получает статус «на проверке», работодатель ставит «зачтено / не зачтено» с комментарием. Задание можно выложить для всех кандидатов специализации или дать одному кандидату из поиска или подборки (по анонимному id карточки), можно привязать к вакансии и задать срок. Один ответ на задание, не больше 30 заданий в сутки от компании. Вердикт появляется в анонимной карточке строкой «Задание работодателя «…»: зачтено, вчера». Грейд задание не меняет. Уведомления (колокольчик и письмо) не содержат данных другой стороны.

Задача недели. Кроме заданий работодателя, раз в неделю планировщик выбирает каждому активному кандидату открытую вакансию его специализации и присылает «задачу недели», собранную платформой из текста вакансии на тех же семействах. Грейд она не меняет, работодатель видит в карточке тему, результат и давность. После теста платформа ещё предлагает практику по самой слабой теме.

Подробно, с формулами и таблицами: VALIDATION_ASSESSMENT.md.

4. Механика подбора и категоризация

Категория — специализация из опроса и грейд. Грейд берётся из теста по этой специализации. До теста грейд заявленный: в выдаче он помечен как заявленный и в оценке соответствия весит 0,85 от подтверждённого, поэтому кандидат с неподтверждённым грейдом стоит ниже такого же с подтверждённым. Справочник: одиннадцать специализаций (backend, frontend, fullstack, mobile, ML и системы ИИ, DevOps, информационная безопасность, встраиваемые системы и робототехника, тестирование, аналитика, инженерия данных) и пять грейдов (стажёр, junior, middle, senior, lead). У человека одна категория. Фильтр дисциплины в поиске ограничивает нижнюю границу оси ФСП выбранной дисциплиной.

Ось ФСП считается отдельно (методика `v4.2.0`: участие не входит в балл, LCB — лучшее подмножество до пяти результатов, ось монотонна). Она двигает место внутри категории и грейд не ставит. Разряд берётся из реестра и в балл не входит.

Заявленный грейд весит 0,85 от подтверждённого, `GRADE_MODEL` = `legacy`. Ожидаемая релевантность на трёх зёрнах: nDCG@10 0,658, P@10 0,632, завышенный грейд в первой десятке 22,1 % (`python -m app.validation.matching --check`).

Все соревнования весят одинаково, статус не учитывается. В подборке по потребности профильная дисциплина входит целиком, остальные — с весом 0,5. Для безопасности результат по ИБ важнее робототехники. Если у специализации нет своей дисциплины, берётся лучший результат по всем. Фильтр дисциплины оставляет только выбранную. Фактор «Запрос» смотрит на слова и, если вектор включён, на смысл запроса в доле 15% к 85% слов. Модель выключена или ничего не вернула — остаётся прежний порядок по словам. Ось ФСП и результат теста в этот фактор не входят. Таблица подбора от этого не сдвигается: симулятор хранит одну нижнюю границу на человека, без разбивки по дисциплинам. Пара «ИБ 0,5 против алгоритмики 0,6» проверяется в `tests/unit/test_fsp_relevance.py`. Команда `python -m app.validation.matching --check` проходит.

Место в протоколе переводится в класс: победитель 1,00, призёр 0,85, топ-10 % 0,65, верхняя половина 0,45, участие 0. Давность затухает с полупериодом 18 месяцев. Командный результат делит вес доказательства на `√m`. Поле из N человек умножает вес на 1 − e^(−N/30). При одинаковой давности и личном зачёте пять призов в поле из шести остаются ниже одной свежей личной победы среди двухсот той же дисциплины. Та победа уже ниже этих пяти, если ей больше 10 месяцев, если в команде от трёх человек или если дисциплина чужая. В одном подмножестве не больше двух результатов за календарный месяц, и поиск видит все результаты этого месяца, а не два высших класса (v4.2.0). Участие (класс 0) остаётся в профиле и в балл не входит. Нижняя граница — максимум по подмножествам результатов не длиннее пяти, пустое подмножество включено: новый результат никогда не понижает оценку, а история не бывает ниже пустой (с v4.0.0). Оценка сжимается к среднему и ранжируется по нижней границе LCB = μ − σ. Один случайный успех не обгоняет стабильную серию. Участия без призов среднее не поднимают. Разряд из реестра показывается для справки и в оценку не входит. На карточке рядом пишется та же оценка как среднее ± неопределённость и число событий. В балл и в грейд входит только нижняя граница, среднее минус неопределённость. Elo-MMR по полным протоколам сюда не подставляется: реестр отдаёт место и размер поля, а не рейтинг каждого соперника.

Статус поиска работы. «Ищу работу» попадает в поиск и в подборку. «Открыт к предложениям» тоже виден, но карточка это помечает. «Не ищу» уходит из выдачи, и приглашение ему не отправляется.

Порядок выдачи задаёт одна оценка соответствия, разложимая на части (`app/services/matching.py`). Ключ сортировки: минус оценка соответствия, минус слитый ранг, минус нижняя граница оси ФСП, идентификатор (`order_pool` в `app/services/search/pipeline.py`). Ярус и порядок «сначала категория» в поиске не используются; их таблицы — в VALIDATION_MATCHING.md.

- попадание в список решает соответствие потребности: навыки 0,45, грейд 0,30, специализация 0,25, текст 0,05, предметная область 0,05, город и формат 0,05, когда вакансия их называет. Рядом, отдельно от этого балла и от нижней границы ФСП, стоит взаимная пригодность: квалификация в степени 0,55, возможность (вилка и формат) в степени 0,30, намерение искать в степени 0,15. Нулевая возможность обнуляет взаимное число, навыки его не спасают. Человек из списка не исчезает: на карточке написано, что не сошлась вилка или формат;
- навык весит по силе доказательства: подтверждён тестом 1,0, найден цитатой в резюме 0,75, заявлен 0,55. Сосед по группе технологий даёт 0,3 этого веса. Более узкий инструмент закрывает общий навык сильнее: PostgreSQL закрывает SQL на 0,85 веса доказательства, SQL PostgreSQL сильнее 0,3 не закрывает. Строка «проверены тестом, не подтверждены» место не опускает ниже заявления: иначе человек, сдавший тест, стоял бы ниже того, кто навык только написал;
- регион в балл не входит. Команда `python -m app.validation.fairness` проверяет правило четырёх пятых на двух метках с одинаковыми баллами и не печатает цифру живого банка. Чужой город, который человек готов сменить, даёт 0,7 вместо 0 и остаётся ниже точного совпадения. Тот же коэффициент входит в отдельный шанс ответа на приглашение; список от этого шанса не двигается. Пока город скрыт, флаг переезда в балл не входит;
- у стажёра и младшего разработчика (junior) две и больше приметы роста (активный поиск, четыре навыка, навык с теста, стажёр который ищет работу) пишутся строкой на карточке. Балл и грейд не двигаются;
- подтверждение старше полугода оценку уровня не снижает: строка говорит, что погрешность шире. Даты нет — строка не появляется, опубликованный порядок без дат не сдвигается. Грейд от давности не меняется. Опубликованные цифры подбора сняты, когда давность ещё умножала вклад теста;
- внутри категории выше тот, у кого сильнее тест и протокол ФСП (ТЗ 2.1). Тест и ФСП входят с весом 0,30 к соответствию, причём ФСП только поднимает доказательства над тестом (`T + 0,5·max(0, F - T)`). Вес умножается на близость `g·r·(0,5 + 0,5·покрытие навыков)`: точный грейд 1, на ступень выше 0,6, на ступень ниже 0,35, дальше 0; та же специализация 1, соседняя 0,5, другая 0. Сильный профиль из соседней категории остаётся ниже подходящего, а внутри категории подтверждённый грейд и сильный результат стоят выше одного лишнего заявленного навыка. Вклад теста и ФСП виден в карточке («+9 к баллу»). Проверка: Спирмен порядка внутри категории со скрытой силой 0,410 ± 0,021 против 0,011 у сортировки по резюме, `python -m app.validation.matching` (docs/VALIDATION_MATCHING.md);
- неприменимый фактор не учитывается: нет истории ФСП — ось выпадает, кандидат не теряет баллы;
- если и потребность, и кандидат назвали вилку до вычета НДФЛ, карточка пишет, покрывает ли она ожидание, и хранит вероятность согласия отдельно от места. Место эта вероятность не двигает. Прежний отказ из-за вилки снижает только её (множитель 0,85, не больше трёх раз). Отказ из-за формата снижает отдельно показанный шанс ответа, когда потребность назвала формат, и место тоже не двигает. Если поднять верх вилки на 15 % закроет ожидание ещё хотя бы одному человеку и таких ожиданий не меньше пяти, подборка говорит, до какой суммы и скольких это коснётся;

- когда у подтверждения есть погрешность, фактор теста берёт вероятность на нижней границе $\theta - 1{,}64 \cdot SE$. Если у сессии есть число верных ответов, туда же входит нижняя граница Уилсона этой доли, и фактор оставляет меньшую из двух. На стенде это число читает агрегат `app_answer_counts`: работодатель видит счётчик, а не текст задания и не ключ. В синтетике погрешности нет, а доля берётся как 12 ответов с числом верных, округлённым из вероятности уровня. Доля верных по теме на экране результата считается тем же интервалом и место в списке не задаёт;
- кандидат без теста занимает одно место в каждом десятке, если его оценка не хуже последнего в десятке больше чем на 0,08. Уже стоящий в десятке остаётся на своём месте;
- больше двух открытых приглашений опускает карточку на 0,02 за каждое следующее, но не больше чем на 0,10;
- имя, пол, возраст и регион в оценку не входят. Скрытая группа с теми же профессиональными фактами проходит правило четырёх пятых. Это проверяет тест `test_fairness.py`. В выборе заданий теста региона тоже нет: на `python -m app.validation.assessment --n 2000 --seed 7` четыре метки назначены отдельным генератором, максимальная доля одного ключа в группах 40 %, 42 %, 39 % и 43 %, отношение меньшей к большей 0,91.

Объяснимость. В карточке — строки «плюс / минус / инфо» по каждому фактору, у каждой строки машинный код и, где источник однозначен, метка «тест», «ФСП», «заявлено», «вилка» или «очередь». Рядом веса, «лучше N % кандидатов категории», результат задачи недели и вердикт по заданию работодателя с давностью. Сравнение до четырёх кандидатов ставит факторы рядом. У каждого места, кроме последнего, одна строка называет первый по порядку фактор, чей вклад в балл выше, чем у следующего места. Более поздний и больший разрыв её не заменяет. Место, отданное правилом разнообразия, этой строки не получает. Если кого-то из выбранных уже нет в выдаче, сравнение отвечает 404. Если один обязательный навык — единственный пробел хотя бы у одного человека и в пуле не меньше пяти, подборка называет этот навык. Пробел считается той же направленной связью, что и балл: FastAPI закрывает требование Python, сосед по группе вроде MySQL вместо PostgreSQL пробелом остаётся. Подсказка порядок списка не меняет.

Уточнение без потери подборки (ТЗ 2.1). Текст вакансии можно вставить в форму: правила вытаскивают специализацию, грейд, обязательный и желательный стек, вилку, формат и город. Навыки, похожие по словам на словарь, пишутся отдельно: «проверьте перед сохранением». Сами в обязательный стек они не попадают. Внешняя модель для этого не нужна. Вставленный текст сам сайт не открывает. Отдельная кнопка читает одну открытую ссылку: переход по редиректу не выполняется, внутренний адрес отклоняется, соединение идёт на адрес, который уже прошёл эту проверку. Прокси из окружения процесса не используется. Обхода проверки сайта вакансий нет. Черновик не сохраняется, пока работодатель сам не нажмёт «Создать». Над подборкой написано, сколько человек в банке и сколько осталось по условиям этой потребности. Потребность хранится версиями. Отобранные кандидаты помнят, по какой версии их выбрали, и остаются в списке после правки фильтров. Над списком три числа воронки, а кнопки оставляют на экране один шаг: отобрано, приглашено или отказ. Счётчики при этом не сужаются. Скрытая карточка тоже остаётся скрытой: пометка относится к потребности, а не к версии, и чужую подборку не меняет. Прошлую версию можно вернуть: её параметры становятся новой версией, список отобранных не трогается. Правка с устаревшим номером версии отклоняется.

Жалоба. Кандидат жалуется на приглашение или открытую вакансию. Три разных человека и нижняя граница Уилсона не ниже 10 % жалоб на приглашения за 30 дней ставят компанию на паузу: новые приглашения отвечают «Приглашения приостановлены до проверки». Снять паузу может только команда `python -m app.services.trust release <инн>`. Подробности — в TRUST_AND_ATS.md.

Практика. Приглашение можно пометить как практику. Этап сдвигается только вперёд: отправлено, отбор, пробный срок, выход. Балл подборки и грейд от этапа не меняются.

Интерес. В избранном компания ставит карточке «приоритет». Специалист отмечает интерес к компании. Компания видит этот факт на анонимной карточке и в приглашении, без имени.

Собеседование. После принятия компания предлагает от одного до трёх времён (дата, время по Москве, формат, ссылка или адрес). Там же можно дать задание на код: из библиотеки или своим текстом. Черновик кандидата сохраняется, прошлые снимки листаются. Пометка «спор» остаётся заметкой и не меняет грейд. Модель это задание не оценивает. Кандидат выбирает одно. Обе стороны скачивают один файл календаря: начало записано как `DTSTART;TZID=Europe/Moscow`. Итог «прошёл», «не прошёл» или «не пришёл» меняет вероятность согласия на следующее приглашение этой же компании: прохождение умножает её на 1,10 и не поднимает выше 1, непрохождение — на 0,85, неявка — на 0,70. Чужие компании этот итог не видят. Внешний календарь не подключается.

Приглашение. Вилка ниже ожиданий кандидата показывает подсказку и не блокирует отправку. Повтор той же отправки с тем же ключом не создаёт второе приглашение. Одно ожидающее приглашение на пару работодатель–кандидат. Отказ без причины не сохраняется: зарплата, формат, стек, «не ищет» или другое. Доли причин и квартили ожиданий по категории показываются, когда в ячейке не меньше пяти человек. Условия отправленного приглашения не редактируются.

Поиск по банку. Сначала SQL отсекает закрытые профили, несовершеннолетних без согласия представителя, неподходящие специализацию, грейд, дисциплину и разряд. Анкета «Как вы нанимаете» не показывает компании уровень. Пока ответов нет, список прежний. Меньше трёх «да» убирает из списка подтверждённых тестом senior и lead и оставляет на карточке специализацию, грейд и стек: город, стаж и строки ФСП скрыты. Имя и контакты по-прежнему только после согласия. На экране только число скрытых карточек. Если заданы и специализация, и грейд, в списке по умолчанию остаётся точная пара, соседний грейд той же специализации и смежная специализация того же грейда. Два шага сразу не входят. Фильтр поиска может оставить только точную пару или любой грейд этой специализации: балл от этого не меняется, сужается только список. У потребности и у вакансии поле `grade_match`: `exact`, `plus_minus` или `any`, по умолчанию `plus_minus`. Плюс-минус — та же специализация и один шаг

грейда. Смежная специализация в этот режим не входит. На карточке это уже написано: «смежная специализация» или «грейд выше / ниже». Точная пара по-прежнему выше. Затем полнотекстовый ранг PostgreSQL и триграммы объединяются методом RRF (K = 60) с рангом по результату теста. Okapi BM25 считается отдельно (`app/services/search/bm25.py`). Выдача объединяет полнотекстовый ранг PostgreSQL, триграммы и ранг по результату теста. Смысловой поиск по векторным представлениям текста (эмбеддингам) включается флагом `EMBEDDINGS_MODE=model`. Отдельная модель пары «запрос–текст навыков» включается только вместе: `CROSS_ENCODER_MODE=model` и имя локальной модели в `CROSS_ENCODER_MODEL`, например `BAAI/bge-reranker-v2-m3`. Она переставляет верхние 50 этого фактора. Хвост списка остаётся в прежнем порядке слов. Библиотеки или файла весов нет — порядок остаётся прежним. Навыки, грейд и нижняя граница ФСП в этот фактор не входят. Платный адрес для этого не вызывается.

Искусственный интеллект: какие модели, как и зачем

Платформа работает и без моделей: навыки из резюме достаёт словарь с правилами, грейд даёт тест с проверкой ответа кодом, порядок выдачи задаёт формула из этого раздела. Модели только подсказывают и выключаются флагами.

Модель	Где работает	Что делает	Как включить
GigaChat (по умолчанию GigaChat-2-Pro)	облако Сбера, Россия	предлагает дополнительные навыки из резюме; черновики учебных заданий для автора банка	`AI_ENABLED=true` и ключ `GIGACHAT_CREDENTIALS`
YandexGPT, Cloud.ru Foundation Models, MWS GPT	облака Яндекса, Cloud.ru и МТС, Россия	то же, что GigaChat, вместо него	стол «Модерация» → «ИИ-провайдер»: только сотрудник с правами с консоли и под вторым фактором
YandexGPT Pro 5.1	облако Яндекса	основная модель, когда выбран YandexGPT	`YANDEX_MODEL=yandexgpt-5.1/latest`
YandexGPT Lite 5	облако Яндекса	вторая модель: сверяет черновик задания и видит только условие	`YANDEX_CHECK_MODEL=yandexgpt-5-lite/latest`
deepvk/USER-bge-m3	на своём сервере, библиотека sentence-transformers	векторы профилей для поиска по смыслу запроса	`EMBEDDINGS_MODE=model`
локальная модель пары, например BAAI/bge-reranker-v2-m3	свой сервер, sentence-transformers	переставляет верхние 50 фактора «Запрос». Навыки, грейд и нижняя граница ФСП не меняются	`CROSS_ENCODER_MODE=off`, пока не задано имя модели

> **URI моделей YandexGPT сверяются при включении.** Официальная страница моделей Yandex AI Studio из нашей сети отдаёт капчу. Вторичные источники 2026 года расходятся: `gpt://<folder>/yandexgpt-5.1` и `yandexgpt-5-lite` (toolarium, май 2026) против `yandexgpt/rc` для Pro 5.1 и `yandexgpt/latest` для Pro 5 (aisrc.ru). Ключа для проверки вызовом у нас нет. Перед включением YandexGPT сверьте `YANDEX_MODEL` со страницей моделей. По умолчанию модель не используется (`AI_ENABLED=false`).

- В модель уходит только обезличенный текст: функция `for_model` (`app/services/ai/assistant.py`) заменяет имя, email, телефон и Telegram метками, и слой провайдеров (`app/services/ai/providers.py`) применяет её к каждому запросу. Без ключа запросов во внешний сервис нет.
- Провайдера можно сменить без перезапуска, но только из закрытого списка: адреса зашиты в код, поля «URL» нет. Ключ хранится зашифрованным AES-GCM и в ответах API не возвращается. Перед вызовом имя хоста должно резолвиться только в публичные адреса, редиректы запрещены (OWASP SSRF Prevention). Яндексу уходит заголовок `x-data-logging-enabled: false`, чтобы запросы не сохранялись. Менять ключи можно только после `python -m app.services.totp enroll`; без включённого провайдера работает GigaChat из настроек сервера.
- **Мнение ИИ по карточке** (Потребности → карточка кандидата) — только совет рекрутеру, на балл и порядок не влияет. Модель получает пронумерованные факты обезличенной карточки, без имени, контактов, города и текста резюме. Каждый пункт должен ссылаться на факты. Пункт без ссылки, с числом не из фактов или о защищённом признаке (возраст, пол, семья, здоровье, национальность, место жительства) отбрасывается, и экран показывает, сколько отброшено. Ответ помечен как сгенерированный ИИ, решение остаётся за человеком (EU AI Act, ст. 14 и 50; NIST AI RMF). Ответ хранится сутки, ключ — хеш фактов, лимит 30 вызовов в час на компанию, `app/services/ai/second_opinion.py`.
- Навык от модели принимается, только если он дословно цитируется в резюме и сводится к словарю навыков. Кандидат подтверждает каждый навык сам.
- Черновик задания пишет одна модель, проверяет другая и видит только условие. Черновик в тест не попадает, его разбирает человек: на столе «Модерация» очередь `item_drafts` со статусами «ждёт», «одобрен», «отклонён» и заметкой без имён и контактов. Одобренный черновик — заготовка для автора семейства заданий.
- Модели не ставят грейд, не ранжируют кандидатов и не проверяют ответы теста. Решение о приглашении принимает работодатель (ст. 16 152-ФЗ).
- Письменный ответ (VS.3) разрешает облачные модели, в том числе зарубежные, и предпочитает открытые. Мы оставили GigaChat: запрос обезличен, оплату API берём на себя, на машине проверки видеокарты нет.

- Векторная модель открытая и работает локально: данные кандидатов за пределы сервера не уходят. Без неё поиск остаётся лексическим и по формуле.
- Передача обезличенного резюме в GigaChat описана в согласии кандидата (`backend/app/legal/docs/pd_candidate.md`, версия 1.2).

Против массовых рассылок: компания с подтверждённой почтой отправляет до 10 приглашений в сутки, после проверки ИНН, почты на домене сайта или статуса партнёра ФСП — до 50 (`INVITE_LIMIT` в `backend/app/services/trust_level.py`). Одному кандидату компания держит одно активное приглашение; после отказа повторно пригласить можно через 30 дней (`DECLINE_PAUSE` в `backend/app/services/offers.py`). Если хранилище лимитов недоступно, приглашение не уходит.

5. Процедура валидации и результаты

Размеченных пар найма нет, поэтому валидация построена на синтетике **со скрытой правдой**: модель видит только зашумлённые наблюдения, а качество считается по правде, которую модель не видела. Все проверки детерминированы и запускаются одной командой.

Что проверяем (критерий ТЗ 6.2)	Результат	Команда (из `backend/`)
Соответствие эталону: точность решения вдали от границы грейда, 2000 кандидатов	91,7 %	`python -m app.validation.assessment --n 2000 --seed 7`
Согласованность: каппа Коэна повторной сдачи с новыми заданиями	0,49	та же
Дискриминативность: медиана точечно-бисериальной корреляции, вводимый ответ	0,24	та же
Устойчивость к распространению: утечка у 30 % кандидатов, точность вдали от границы	88,9 % с карантином ключей	та же
Ложные сдачи при утечке без карантина и с ним, на тех же людях	19,2 % и 13,4 %	та же
Лестница: тест на middle сдают настоящие junior, middle, senior	6,4 %, 70,5 %, 99,2 %	та же
Угадывание проходит тест	0 %	та же
Точность классификации по Руднеру у порога заявленного грейда	83,5 %	та же
Перекрытие заданий между кандидатами одной симуляции	15,2 %	та же
Подбор, nDCG@10 на трёх зёрнах: полная модель и сортировка по резюме	0,658 ± 0,022 и 0,620 ± 0,005	`python -m app.validation.matching`
Порядок внутри категории, Спирмен со скрытой силой: полная модель, без теста, без ФСП, по резюме	0,410, 0,162, 0,384, 0,011	та же
Завышенный грейд в первой десятке: полная модель и по резюме	22,1 % и 28,1 %	та же
Настоящий грейд ниже грейда потребности, первая десятка: полная модель и по резюме	13,6 % и 30,9 %	та же
Чужие объекты по идентификатору (другой работодатель, другой кандидат)	0 ответов 2xx из 240 запросов	`python tests/smoke_api.py`
Справедливость: смена имени, пола, региона не меняет порядок	выполняется	та же процедура подбора
Нагрузка: 40 пользователей одновременно	0 ошибок, p95 поиска 69 мс	`python tests/load_api.py`

Короткая проверка порогов на 300 кандидатах и подборе запускается командой `make evaluate`. Список требований и одна команда без базы — `make accept` (requirements_trace.md). Подбор можно проверить на своей разметке: `python -m app.validation.matching --pairs ../docs/examples/jury_pairs.jsonl`.

Слепая разметка видит только заявленный профиль и в одном случае из пяти сдвигает оценку на ступень. Против меток, посчитанных по скрытой правде, каппа Коэна 0,46 при зерне разметчика 2027 (`python -m app.validation.matching --check`). Это другое зерно, чем зерно набора, и веса подбора по этой каппе не подгонялись. Набор профилей для проверки лежит в `app/validation/resources` и проверяется командой `python -m app.validation.jury_pack app/validation/resources/jury_profiles.json` (и тем же CSV). В базу он не пишется. Страница «Режим жюри» (`/jury`) ведёт к двум демо-входам и к публичным страницам: отдельной роли для проверяющих в системе нет. `make evaluate` гоняет эту проверку вместе с тестом на 300 кандидатах (зерно 7), подбором, набором профилей и полными прогонами теста на 2 000 и 5 000. В CI остаётся короткий прогон на 300: полные таблицы длиннее обычной проверки стенда.

На 5000 кандидатах, то же зерно 7, точность вдали от границы 92,4 %, каппа пересдачи 0,49, точность по Руднеру 83,6 %, перекрытие заданий 16,9 %. Команда: `python -m app.validation.assessment --n 5000 --seed 7`. Опубликованная таблица выше считается на 2000.

Подробно: VALIDATION_ASSESSMENT.md, VALIDATION_MATCHING.md, LOAD_TEST.md.

6. Интеграция с реестром ФСП

- **Вход через ФСП ID** — OpenID Connect, поток кода с PKCE. Платформа — конфиденциальный клиент Keycloak: пароль ФСП вводится только на странице провайдера, ФСП ID приходит в проверенном ID-токене. В демо роль Keycloak играет помеченная страница этого сайта с тем же протоколом.
- **Реестр.** Нам нужен один метод только на чтение, он отдаёт строки протокола участника. Пол, контакты и документы мы не запрашиваем. В демо работает синтетический реестр, а в режиме `FSP_MODE=real` включается HTTP-клиент по предложенному контракту.
- **Удостоверения** — каждая новая строка протокола выдаётся как W3C Verifiable Credential 2.0 типа OpenBadgeCredential с подписью `eddsa-jcs-2022`. У хранимого удостоверения есть статус отзыва (`GET /api/v1/public/credentials/{id}/status`): подпись при отзыве не переписывается, статус начинает отвечать, что удостоверение отозвано. Предъявление SD-JWT с тем же `jti` проверяется отдельно (`POST /api/v1/public/credentials/verify-sd-jwt`): подпись и строка отзыва. Функция одной подписи базу не читает. Краткая карточка подписывает только год, статус соревнования и корзину места, без названия события и без имени. Проверка: http://localhost/badge. Тот же ключ опубликован как did:web хоста: `/.well-known/did.json`. Имени в этом документе нет.

Схема обмена, контракт, настройка Keycloak и дельта-синхронизация: FSP_INTEGRATION.md.

7. API

Спецификация OpenAPI 3.1: http://localhost/openapi.json, Swagger UI: http://localhost/docs. Swagger отдаётся с того же адреса, без внешних CDN и встроенных скриптов, поэтому работает при строгой политике `script-src 'self'`. У каждого метода описаны запрос, ответ и коды ошибок.

Группа	Методов	Что делает
candidates	35	Профиль, опрос, видимость, резюме и PDF-профиль, результаты ФСП и удостоверения, приглашения и ответ на них, закрытие контактов для компании, привязка ФСП ID, согласие законного представителя, жалобы
public	27	Проверка удостоверений, предъявления SD-JWT и их статус, DID издателя, публичный профиль и компании, рейтинг категории, методика и веса, статистика заданий, проверка подбора на своей разметке, согласие представителя, флаг демо-стенда и свободный демо-аккаунт
moderation	18	Стол модерации: второй фактор TOTP, жалобы и выгрузка CSV, аккаунты, подозрительные вилки, черновики заданий, закрытие вакансий, провайдеры языковой модели
auth	17	Регистрация по email с кодом, вход кандидата и работодателя, гостевой вход, вход и регистрация через ФСП ID, сброс и смена пароля, обновление токена, выгрузка своих данных
assessment	17	План теста, старт, ответы и черновик, пауза между попытками, история и разбор, оспаривание, отказ от понижения, практика и задача недели
needs	17	Потребности с версиями и черновиком из текста или ссылки, подборка, сравнение, тест по потребности, отобранные и скрытые, заметка модели по карточке
vacancies	11	Вакансии, редактирование, отклики и их статусы, контакты по отклику
employer	10	Профиль компании, практика найма, репутация, избранное, песочница ATS
interviews	9	Слоты собеседования, выбор времени, файл календаря, итог; задание с кодом по приглашению и его оспаривание
tasks	8	Задания от работодателя: создание, лаборатория, ответы, проверка открытого ответа, закрытие; задания для кандидата
integrations	11	Вебхуки и API-ключи для ATS: адреса, смена ключа, журнал доставок, проверочная отправка, выгрузка
offers	7	Предпросмотр и отправка приглашения, рыночные квартили вилки, список, отзыв, контакты после согласия
legal	6	Тексты согласий, политика, выдача и отзыв согласий
methodist	4	Мониторинг банка заданий: карантин, возврат задания, журнал
notifications	3	Колокольчик: список и прочтение
passport	3	Паспорт навыков: выпуск и публичная страница
search	2	Поиск по банку кандидатов; витрина банка «специализация × грейд»
ai	2	Разбор резюме и текста запроса
problems	2	Каталог кодов ошибок RFC 9457
messages	2	Переписка по принятому приглашению
guide	1	«Где это сделать»: ссылка на страницу кабинета по вопросу, без профиля и без модели
fsp-id-demo	1	Демо-провайдер ФСП ID
health	1	Проверка здоровья

Всего 214 методов. Счёт по первому тегу метода в спецификации OpenAPI: `python -c "from app.main import app; print(sum(k in ('get', 'post', 'put', 'patch', 'delete') for m in app.openapi()['paths'].values() for k in m))"` в `backend/`.

Ошибки приходят в одном формате с машинным кодом: `{"error": {"code", "message", "details"}}`. Полный список кодов: API_ERRORS.md.

8. Библиотеки и компоненты

Версии, на которых проходят тесты. Backend ставит их из `backend/requirements.lock`, frontend — из `frontend/package-lock.json`. Полный перечень, включая тестовые пакеты, инструменты документации и материалы, с лицензиями — THIRD_PARTY.md.

Backend (Python 3.12)

Библиотека	Версия	Лицензия	Зачем
FastAPI	0.142.2	MIT	HTTP API, OpenAPI
Starlette	1.7.0	BSD-3	ASGI-основа FastAPI
Uvicorn	0.54.0	BSD-3	ASGI-сервер
Pydantic / pydantic-settings	2.13.5 / 2.15.0	MIT	Схемы, валидация, настройки
SQLAlchemy	2.1.3	MIT	ORM, асинхронный доступ
asyncpg	0.31.0	Apache-2.0	Драйвер PostgreSQL
Alembic	1.20.0	MIT	Миграции
pgvector (Python)	0.5.0	MIT	Тип vector для эмбеддингов
redis-py	6.4.0	MIT	Лимиты, состояние входа ФСП ID
Celery	5.6.3	BSD-3	Фоновые задачи и расписание
PyJWT	2.15.1	MIT	JWT и проверка подписи ID-токена ФСП ID по набору ключей поставщика
argon2-cffi	25.1.0	MIT	Хеши новых паролей, Argon2id
bcrypt	5.0.0	Apache-2.0	Проверка старых хешей паролей
cryptography	50.0.2	Apache-2.0 / BSD	Ed25519 для удостоверений
httpx	0.28.1	BSD-3	Клиент реестра ФСП и Keycloak
pypdf	6.19.0	BSD-3	Текст из PDF-резюме
fpdf2	2.8.9	LGPL-3.0	PDF-профиль кандидата
pytest	9.1.1	MIT	Тесты
gigachat (необязательно)	≥ 0.1.39	MIT	ИИ-советник для резюме
sentence-transformers, torch (необязательно)	≥ 3.0, ≥ 2.3	Apache-2.0, BSD-3	Смысловой поиск

Frontend (Node.js 20)

Библиотека	Версия	Лицензия	Зачем
React, React DOM	18.3.1	MIT	Интерфейс
React Router	6.30.6	MIT	Маршруты
TanStack Query	5.104.1	MIT	Запросы и кеш
Zustand	4.5.7	MIT	Состояние входа
React Hook Form, Zod, @hookform/resolvers	7.89.0, 3.25.76, 3.10.0	MIT	Формы и проверка
Axios	1.20.0	MIT	HTTP-клиент
lucide-react	0.447.0	ISC	Иконки
qrcode-generator	2.0.4	MIT	QR-код ссылки на удостоверение и паспорт навыков, рисуется в браузере
date-fns, clsx, tailwind-merge	3.6.0, 2.1.1, 2.6.1	MIT	Утилиты
@fontsource-variable/jetbrains-mono, golos-text	5.3.0	OFL-1.1	Шрифты брендбука, без внешних запросов
Vite, @vitejs/plugin-react	6.4.3, 4.7.0	MIT	Сборка
TypeScript	5.9.3	Apache-2.0	Типы
Tailwind CSS, PostCSS, Autoprefixer	3.4.19, 8.5.28, 10.6.1	MIT	Стили

Инфраструктура

Компонент	Версия	Лицензия
PostgreSQL + pgvector + pg_trgm	16 (образ `pgvector/pgvector:pg16`)	PostgreSQL License
Redis	7.2 (`redis:7.2-alpine`)	BSD-3
nginx	`nginx:1.28-alpine`	BSD-2
Mailpit	`axllent/mailpit:v1.31.4`	MIT
Caddy (публичный стенд)	2.10 (`caddy:2.10-alpine`)	Apache-2.0
Swagger UI	5.33.1 (в репозитории)	Apache-2.0

Всё — открытое ПО (ТЗ 3.5). Исходный код решения открыт, не скомпилирован и не обфусцирован, лицензия MIT (LICENSE).

9. Сборка, развёртывание и запуск

```
git clone https://github.com/KonkovDV/fsp-talent.git && cd fsp-talent
make demo          # Windows: см. docs/INSTALL.md, три команды PowerShell
```

Сайт http://localhost, API http://localhost/docs, письма http://localhost:8025. Пошаговая инструкция с требованиями, проверкой, тестами, сбросом и частыми проблемами: INSTALL.md. Публичный стенд с HTTPS: deploy/README.md.

10. Безопасность и 152-ФЗ

- Отдельные согласия на обработку (ч. 1 ст. 9 152-ФЗ) и на распространение для публичной страницы (ст. 10.1). В базе хранятся версия и SHA-256 принятого текста.
- Контакты кандидата видны только после принятия приглашения или собственного отклика и только при действующем согласии. Каждый просмотр — запись в журнале доступа.
- Отзыв согласия закрывает контакты сразу, через 30 дней задача стирает имя, контакты, ФСП ID и навыки (ч. 5 ст. 21). Перед отзывом диалог перечисляет, что именно произойдёт: карточка уйдёт из поиска, контакты закроются, страница выключится или запишется только сам отзыв. Удаление аккаунта стирает данные немедленно.
- При эксплуатации персональные данные граждан РФ ведутся в базах на территории РФ (ч. 5 ст. 18). Демонстрационный стенд запускается там, где его подняли, и эту локализацию не обещает.
- Неправомерная или случайная передача персональных данных: уведомление Роскомнадзора в течение 24 часов и результаты внутреннего расследования в течение 72 часов (ч. 3.1 ст. 21, редакция от 24.06.2025). Стенд такое уведомление не отправляет.
- Если модель включена, резюме уходит к ней без имени, почты, телефона и Telegram. Разбор запроса работодателя идёт правилами, без модели. Черновик задания запрашивается только когда задана вторая, другая модель: в запрос входят специализация, тема и уровень, без резюме. Черновик сохраняется как принятый, лишь если обе модели назвали один ключ и этого ключа нет в тексте условия. В банк заданий и в сессию теста он не попадает.
- Решение задачи с кодом выполняется в контейнере `sandbox` без сети (`network_mode: none`). В нём Python, Node и один SELECT к памяти SQLite. Запись, вторая команда и ответ задачи в контейнер не входят. Корень только для чтения, capabilities сброшены, не больше 64 процессов, 256 МБ и полпроцессора. Переменных из `.env` у контейнера нет. API обращается к нему только через Unix-сокет на общем томе, с токеном. На запуск — новая папка, 3 секунды, 2 секунды процессора. В песочницу уходят исходник и входы примеров. Ожидаемые ответы остаются в API, он и сравнивает. Если песочница не отвечает, сдача не сохраняется и не считается неверной (503). Без контейнера, только локально, Python запускается дочерним процессом с запретом сокетов.
- Песочница: решению на Python и SQL ядро не даёт ни `fork`, ни потоков (`RLIMIT_NPROC` = 0), поэтому оно не уйдёт через `setsid()` из группы процессов, которую убивает таймаут. Node ограничен `pids_limit` контейнера. Код кандидата выполняется под отдельным uid 10001 в отдельном контейнере, без `SECRET_KEY` и без доступа к базе. Этот uid общий с сервером песочницы: щит в процессе закрывает `os.unlink` сокета, перезагрузку модуля `os` и импорт `ctypes`, а сервер вешает сокет заново, если путь исчез. Загрузка файла стандартной библиотеки своим загрузчиком щит обходит. Сокет имеет права `0660`, к нему подключается только API: его контейнер входит в группу 10001 (`group_add`). `SANDBOX_REQUIRED` по умолчанию `true`: запасной запуск кода внутри API нужно включить явно (`SANDBOX_REQUIRED=false`, только для тестов и ноутбука).
- Несовершеннолетние скрыты из поиска и рейтинга без согласия законного представителя. Согласие не заменяет дату рождения.
- Возраст и согласие. Регистрация открыта с 15 лет. По позиции Роскомнадзора (письмо от 11.03.2022 № 08-15154; разъяснения 27.08.2025) с 14 лет человек может сам дать согласие на обработку своих данных, до 14 лет его даёт законный представитель. Мы строже: карточка кандидата 15–17 лет не показывается работодателям, пока представитель не подтвердит согласие по подписанной ссылке (`backend/app/services/guardian.py`). Причина — речь о трудоустройстве: с 15 лет допускается только лёгкий труд (ст. 63 ТК РФ), а при уточнении требований (Q&A) просили участия родителей. Согласие на обработку ПДн оформляется отдельным документом (ст. 9 152-ФЗ в редакции с 01.09.2025).

- Новые пароли — Argon2id (19 МиБ, две итерации, одна дорожка). Старый хеш bcrypt при входе проверяется и сразу переписывается. Ключ подписи токенов уникален для установки. Пароль ФСП платформа не видит (OIDC). Коды подтверждения хранятся только в виде HMAC.
- Срок access-токена — решение по дизайну. Access-токен — подписанный JWT на 60 минут (`ACCESS_TOKEN_EXPIRE_MINUTES`, `backend/app/core/config.py:68`), сервер не хранит список выданных токенов. Каждый запрос сверяет эпоху токена с аккаунтом (`backend/app/api/deps.py:37`), поэтому смена и сброс пароля (`api/v1/auth.py:501, 535`), повтор старого refresh (`auth.py:614–616`), закрытие входа модератором (`api/v1/moderation.py:198–199`), удаление и блокировка аккаунта гасят все access-токены сразу. Выход гасит refresh-поколение на сервере (`auth.py:623–649`) и стирает access-токен в браузере (`frontend/src/api/client.ts:180–181`); копия токена, снятая до выхода, живёт не дольше оставшихся до 60 минут. Чтобы закрыть все сессии немедленно, достаточно сменить пароль.
- Ошибка API — RFC 9457 (`application/problem+json`): класс E01–E20, путь в `type`, конкретный код в `error.code`. Каталог классов открыт без входа: `GET /api/v1/problems`. Поле `detail` для старых клиентов не менялось.
- Почта, телефон, Telegram, почта представителя и способ связи в приглашении хранятся как AES-256-GCM (NIST SP 800-38D: nonce 96 бит, метка 128 бит, дополнительные данные — таблица, поле и строка). Вход ищет HMAC почты, не шифртекст.
- Лимиты на вход, регистрацию, письма, приглашения и поиск. Если хранилище лимитов недоступно, вход и регистрация закрываются, а не пропускают запросы.
- Решение о найме принимает человек: платформа ранжирует и объясняет, но не отсеивает автоматически (ст. 16 152-ФЗ).
- Стол модерации открыт только сотруднику, которому права выданы с консоли сервера (`python -m app.services.staff`, проверка `is_staff` в `backend/app/api/v1/moderation.py:48`); ни демо-режим, ни адрес его не открывают, остальным раздел отвечает 404. Стол показывает числа по банку, выгружает жалобы в CSV без имени пожаловавшегося и может закрыть вход компании или кандидата. Печатный вход `hr@sibcode.demo` и собственный вход стола закрыть нельзя. Строка в базе остаётся, карточка уходит из поиска.
- Чужая потребность, чужое задание, чужая сессия теста и чужая переписка отвечают так же, как отсутствующие. Те же таблицы закрыты политиками строк. На публичном стенде API, worker и beat входят как `fsp_app` и не читают чужие строки от имени владельца базы. Локальный Compose входит как владелец и переключается на `fsp_app`. Работодатель не видит ключи заданий.

Сверка с OWASP ASVS уровня 1 — не сертификат. Строка говорит, что уже есть в коде и чего нет.

Тема	Есть	Нет
Вход	Пароли Argon2id, старый bcrypt переписывается при входе, подтверждение почты кодом, лимит попыток. Если хранилище лимитов недоступно, вход закрывается. Второй фактор TOTP (RFC 6238) для стола модерации, включается консольной командой `python -m app.services.totp enroll`	Второй фактор для кандидатов и работодателей
Сессия	Access-токен на 60 минут в заголовке. Refresh на 30 дней лежит в HttpOnly-cookie и при обмене получает новый номер поколения. Пять неверных паролей подряд с одного адреса закрывают вход в этот аккаунт с этого адреса на 15 минут; владелец с другого адреса входит. Весь аккаунт закрывается только после 100 неверных подряд (NIST SP 800-63B §5.2.2), опубликованные демо-аккаунты целиком не закрываются. Окна в одну минуту на «аккаунт × адрес» и на адрес в Redis остаются. Обновление и выход по этой cookie требуют второй cookie `fsp_csrf` и тот же токен в заголовке `X-CSRF-Token`. Клиент API, который присылает refresh в теле, эту проверку не проходит. Выход увеличивает номер поколения, смена пароля и удаление аккаунта увеличивают эпоху. Вход через ФСП ID — PKCE S256. Keycloak realm `fsp` входит в Compose	Access-токен лежит в `localStorage` и до истечения срока выход его не гасит. Демо-вход ФСП ID остаётся страницей провайдера, пока `FSP_ID_MODE=mock`
Доступ к чужим объектам	Чужая потребность, задание, сессия теста и переписка отвечают так же, как отсутствующие. RLS на потребностях, приглашениях, сессиях и подтверждениях грейда. На публичном стенде соединение — роль `fsp_app` без прав суперпользователя и без `BYPASSRLS`. Ключи заданий работодатель не читает	Миграции идут от владельца базы, он суперпользователь. Загрузка демо-данных, фоновые задачи (worker) и публичные страницы включают обход политик флагом приложения: у `fsp_app` такого права нет, сброс роли к владельцу не возвращает
Криптография и канал	Подпись удостоверений Ed25519, коды подтверждения — HMAC. Заголовки nginx и Caddy: `X-Frame-Options`, `nosniff`, `Referrer-Policy`, CSP с `script-src 'self'`, `script-src-attr 'none'`, `base-uri 'self'` и `form-action 'self'`. Встроенного скрипта на странице нет, поэтому nonce в разметку не подставляется, `Permissions-Policy` без камеры, микрофона и геолокации. Тот же набор ставит и сам API. Вход ограничен через Redis и закрывается, если Redis недоступен. Тело больше 10 МБ получает 413. `/.env` и `/.git` отвечают 404	Локальный стенд отвечает по HTTP. HTTPS и HSTS есть у публичного стенда за Caddy
Данные	Отдельные согласия с версией и SHA-256, отзыв, удаление аккаунта (слово «удалить» и пароль, если он есть), журнал просмотра контактов	Уведомление Роскомнадзора стенд не отправляет
Ошибки	RFC 9457, классы E01–E20, код машины в `error.code`. Каталог: `GET /api/v1/problems`	В интерфейсе человек видит текст из `error.message`
Зависимости	Перечень Python-зависимостей собран из `requirements.lock`	Подпись образов в реестре

11. Дальше

Защита от фиктивных вакансий и связь с ATS описаны в TRUST_AND_ATS.md. Боевой ФСП ID и реестр — в FSP_INTEGRATION.md и разделе 11 ниже.

Публичная страница кандидата получает короткий адрес вместе с постоянным именем стенда. Трудность заданий и веса подбора калибруются командой `python -m app.validation.calibrate` на настоящих ответах и откликах. Карантин ключа считает z и CUSUM; вернуть ключ можно после разбора. По сессии копится остаток ответа относительно модели: это пометка семейства, не карантин и не грейд. Удостоверение подписывается `eddsa-jcs-2022`, вход идёт по OIDC. OID4VCI и ЕСИА подключаются на контурах ФСП и государства.

Грейд ставит тест специализации. Сессия заканчивается на 20-м задании, в одной семье не больше пяти заданий. Заявленный грейд весит 0,85 от подтверждённого.

Основная категория одна. Вторая специализация, подтверждённая отдельным тестом, в выдаче этой роли стоит на карточке, а основная остаётся строкой рядом. В подборке профильная дисциплина входит целиком, результат другой дисциплины учитывается с весом 0,5. Фильтр дисциплины ограничивает нижнюю границу одной дисциплиной. Уход со вкладки во время теста записывается как пометка и грейд не меняет. Законченный тест можно оспорить: запись не видна работодателю, пересмотр ручной, грейд кнопкой не меняется.

12. Словарь терминов

Термин	Что значит
θ (тета)	Оценка способности кандидата на общей шкале заданий. Чем выше, тем сильнее
IRT	Теория ответа на задание: вероятность верного ответа зависит от θ и параметров задания
EAP	Оценка θ как среднего апостериорного распределения после каждого ответа
SE	Стандартная ошибка оценки θ: чем меньше, тем увереннее тест
Вероятность уровня	Доля апостериорного распределения θ выше порога грейда. Грейд подтверждается от 75 %
LCB	Нижняя граница доверительного интервала. Ось ФСП и фактор теста берут её, а не точечную оценку
CUSUM	Накопительная сумма отклонений. По ключу это второй сигнал утечки и карантин. По сессии это остаток ответа относительно модели: только пометка, грейд не меняется
Руднер	Точность и согласованность классификации по грейду, вычисленные из θ и SE
nDCG@10	Насколько первая десятка выдачи близка к идеальному порядку, от 0 до 1
P@10, MAP	Доля релевантных в первой десятке; средняя точность по всему списку
Спирмен	Ранговая корреляция; в проверке подбора — порядок внутри категории против скрытой силы
ДИ, бутстреп	Доверительный интервал; бутстреп — оценка интервала повторной выборкой потребностей
Каппа Коэна	Согласие двух оценок сверх случайного, от 0 до 1
RRF	Слияние нескольких ранжирований по обратному рангу в поиске
HMAC	Подпись сообщения секретным ключом: ключ ответа, ссылка представителя
JWT	Подписанный токен доступа
OIDC, PKCE	Протокол входа через ФСП ID и защита кода авторизации от перехвата
RLS	Политики строк в PostgreSQL: работодатель и кандидат видят только свои строки
IDOR	Доступ к чужому объекту по его идентификатору; проверка стенда обходит все такие маршруты
SBOM	Перечень зависимостей сборки, подписывается в CI

Соответствие техническому заданию

Каждое требование ТЗ — где его увидеть в интерфейсе и где оно реализовано в коде. Экраны указаны по пунктам меню: «Кандидат → Профиль» значит раздел «Профиль» в кабинете кандидата. Демо-входы — в README.

Колонка «Тест» — файл автотеста в `backend/tests/` (`unit/`, `integration/`) или `frontend/src` (`*.test.tsx`); юнит-тесты — `make test`.

Статусы: ✅ сделано · ◐ сделано частично, остаток указан · ◇ концепция (ТЗ просит концепцию, а не код).

Раздел 2. Описание задачи

Требование	Статус	Где увидеть	Где в коде	Тест
Обратная механика: работодатель сам находит категорию и выходит на кандидата с предложением и зарплатой	✅	Работодатель → Потребности → подборка → «Пригласить»	`app/api/v1/needs.py`, `offers.py`	`test_regressions_offers_vacancies.py`
Публичная страница компании: название, вакансии, доверие. Почта и имена кандидатов не публикуются	✅ сверх ТЗ	Приглашения → «Страница компании», `/companies/:id`	`GET /public/companies/{id}`	`test_public_pii.py`
Категория определяется опросом, выбором грейда и тестом, а не резюме	✅	Кандидат → Обзор → опрос; Тест на грейд	`app/services/survey.py`, `categorization.py`, `assessment.py`	`test_categorization.py`
Категория = специализация × грейд, справочник специализаций и грейдов	✅	Работодатель → Поиск, фильтры	`app/services/survey.py` (11 специализаций, 5 грейдов)	`test_categorization.py`
Внутри категории выше те, у кого сильнее подтверждённый профиль и есть опыт ФСП	✅	Подборка: полосы факторов, «Почему в выдаче»	`app/services/matching.py`	`test_matching.py`, `test_reason_labels.py`
Взаимная пригодность: навыки не вытягивают нулевую вилку или формат. Человек остаётся в списке, причина написана	✅ сверх ТЗ	Карточка → «Взаимно N»	`mutual_fit` в `app/services/matching.py`	`test_mutual_fit.py`, `test_matching.py`
Смысл запроса, если модель включена, весит 15%. Без модели остаётся порядок по словам	✅ сверх ТЗ	Поиск: строка под числом кандидатов	`fuse_text` в `app/services/search/pipeline.py`	`test_text_blend.py`
Слово специализации в запросе («бэкенд») заполняет пустой фильтр роли	✅ сверх ТЗ	Поиск	`implied_role` в `app/api/v1/search.py`	`test_role_alias.py`
Подсказка «Куда нажать»: раздел только этой роли, вопрос на сервере не хранится и никуда не отправляется. Без входа подсказка ведёт только на публичные страницы	✅ сверх ТЗ	Шапка кабинета; Режим жюри	`POST /guide/ask`, `POST /public/guide`	`test_guide.py`

Требование	Статус	Где увидеть	Где в коде	Тест
Тест устойчив к распространению заданий и даёт сопоставимую сложность	✅	Тест на грейд: новые параметры у каждого	`app/items/`, `app/services/cat.py`, VALIDATION_ASSESSMENT.md	`test_error_echo_clip.py`
Неотправленный ответ теста сохраняется и возвращается после перезагрузки; старая вкладка его не затирает	✅ сверх ТЗ	Тест на грейд	`PUT /assessment/{id}/draft`	`test_answer_draft.py`
Несколько описаний вакансий, на их основе система формирует задания	✅	Кандидат → Вакансии (9 вакансий); Работодатель → Потребность → «Как проверим», Вакансии → «Какой тест получится». Навык без семейства заданий назван отдельно	`app/services/vacancy_exam.py`, `seed/generate_seed.py`	`test_vacancy_exam.py`, `test_need_exam.py`
Грейд не понижается, можно пройти тест ниже или выше, смена грейда ограничена по времени	✅	Тест на грейд: выбор уровня, срок до пересдачи, рекомендация теста на ступень выше	`app/services/assessment.py` (`decide_grade`, `retake_available_at`: 90 и 7 дней, тест ниже после провала — сразу)	`test_repo_integrity.py`, `test_assessment_policy.py`
Регулярные короткие задания от работодателя	✅	Работодатель → Задания или «Дать задание» в карточке; Кандидат → Задания. Плюс «Задача недели», собранная платформой из вакансии	`app/services/tasks.py`, `app/api/v1/tasks.py`; задача недели — `app/services/practice.py`, `offer_weekly_tasks`	`test_practice.py`
Песочница без сети запускает Python, JavaScript и один SELECT. Ответ в контейнер не передаётся	✅ сверх ТЗ	Контейнер `sandbox`	`sandbox/harness_sql.py`, `sandbox/runner.py`	`test_sql_sandbox.py`, `test_sandbox_hardening.py`
Сценарий контакта, отличный от отклика (ориентир LHH)	✅	Работодатель → Приглашения; Кандидат → Приглашения	`app/api/v1/offers.py`, `candidates.py`	`test_regressions_offers_vacancies.py`
Связь профиля с ID участника ФСП, совместимость с Keycloak	✅	Вход → «Войти через ФСП ID» (единая точка входа); Кандидат → Профиль → привязка	`app/services/fsp/identity.py`, `app/api/v1/auth.py` (`/fsp/*`), FSP_INTEGRATION.md	`test_identity.py`, `test_fsp_signin.py`
Команда сама решает, какие данные ФСП получать и как показывать	✅	Кандидат → Результаты ФСП; карточка в выдаче	`app/services/fsp/registry.py`, `scoring/engine.py`	`test_scoring.py`
Корректная обработка кандидата без истории ФСП	✅	Вход `nofsp@demo.talent`	`matching.py`: ось выпадает без штрафа	`test_matching.py`
Чтение протоколов по ФСП ID (только публичный профиль совершеннолетнего с согласием на распространение, ст. 10.1 152-ФЗ; лимиты 10/мин на IP и 300/мин на сайт, fail-closed) и открытых вакансий без кабинета: без имени и без смены грейда	✅ сверх ТЗ	Режим жюри → «Протоколы» / «Вакансии». Там же порядок внутри категории по тесту, без имени. На демо-стенде страница «Как устроен тест» показывает веса балла. Свой файл протокола добавляет строки на страницу результатов и в балл не входит	`GET /public/telegram/achievements`, `POST /candidates/me/protocols`	`test_consent_pd_distribution_text.py`

Требование	Статус	Где увидеть	Где в коде	Тест
Собственная процедура оценки решения и её результаты	✅	–	SOLUTION.md, гл. 5	`test_validation.py`
Защита от фиктивных вакансий и недобросовестных работодателей (плюс)	◐	Компания: ИНН. Кандидат → Приглашения → «Пожаловаться». Модерация: сводка, CSV жалоб, закрыть вход	Работает: ИНН, вилка, 50 приглашений в день, жалоба и пауза компании, стол модерации. Проверка по реестру и отзыв паузы вручную остаются в концепции: TRUST_AND_ATS.md	`test_trust.py`
Интеграция с ATS (плюс)	◐	Компания → «Квитанция ATS»	Концепция в TRUST_AND_ATS.md. На стенде пример события подписывается HMAC, адрес наружу не вызывается	`test_webhooks.py`, `test_api_keys.py`

Раздел 2.1. Кабинет кандидата

Требование	Статус	Где увидеть	Где в коде	Тест
Профиль вручную или загрузка PDF с распознаванием (ФИО, контакты, стек, грейд, роли, стаж, софт-скиллы)	✅	Профиль → «Загрузить PDF»: поля с цитатой и кнопка «Применить». Три шага: текст, цитаты, подставить	`app/services/pdf_profile.py`, `ai/assistant.py`	`test_pdf_profile.py`, `test_api_layer.py`
Автогенерация стандартизированного PDF-профиля	✅	Профиль → «Скачать PDF профиля»	`GET /candidates/me/profile.pdf`	`test_pdf_profile.py`
Опрос, выбор грейда, тест, автоматическая категория	✅	Обзор → опрос; Тест на грейд	`PUT /candidates/me/survey`, `/assessment/*`	`test_guide.py`
Регулярные задания влияют на актуальность профиля	✅	Задания, Обзор → «Задача недели»; у работодателя строки «Задание работодателя «...»: зачтено, вчера» и «Короткая практика ... N дн. назад»	`app/services/tasks.py` (`card_signal`), `app/services/search/pipeline.py`	`test_employer_tasks.py`
Подсказка, какой один подтверждённый навык закроет больше всего открытых вакансий	✅ сверх ТЗ	Кандидат → Обзор	`GET /candidates/me/career`	нет автотеста, проверка на стенде
Категория и грейд, история опроса и теста, тест ниже или выше с ограничением частоты	✅	Обзор; Профиль → история опроса; Тест на грейд → история	`GET /candidates/me/survey/history`, `GET /assessment/history`, `/assessment/cooldown`	`integration/test_survey_history.py`

Требование	Статус	Где увидеть	Где в коде	Тест
Приглашения с вакансией и зарплатой, принять или отклонить	✅	Приглашения; после принятия — «Переписка», «Собеседование», задание на код и «Закрыть контакты для этой компании». На обзоре работодателя воронка: шорт-лист → отправлено → просмотрено → принято → контакты. В приглашении можно отметить условия, в том числе компенсацию турниров ФСП. Практика идёт отдельно: отправлено → отбор → пробный срок → выход. Спор по заданию не меняет грейд	`POST /candidates/me/offers/{id}/respond`, `/revoke-contacts`, `/offers/{id}/messages`, `/offers/{id}/slots`, `/offers/{id}/code`, `POST /offers/{id}/practice`	`test_practice.py`
Вакансии, самостоятельный отклик, свои отклики и статусы	✅	Вакансии → «Мои отклики». Просьба денег, зарплата мимо оформления, отбор по внешности и давление вместо условий (переработки, «мы одна семья», вилка только после собеседования) подписаны на карточке. Вакансия остаётся открытой	`app/services/vacancy_flags.py`	`test_vacancy_flags.py`
Настройки: приватность, согласия, связь с ФСП ID	✅	Профиль → видимость, согласия, ФСП ID, удаление аккаунта. Слово «удалить» и пароль, если вход по паролю	`PATCH /candidates/me/visibility`, `/me/consents`, `DELETE /candidates/me`	`test_openapi_problem_json.py`
Перед отзывом согласия видно, что именно изменится	✅ сверх ТЗ	Профиль → Согласия → «Отозвать»	`if_withdrawn` в `GET /me/consents`	`ConsentsPanel.test.tsx`
Кандидат видит только свои приглашения и отклики	✅	—	проверка владельца в каждом методе	`test_access.py`

Раздел 2.1. Кабинет работодателя

Требование	Статус	Где увидеть	Где в коде	Тест
Профиль компании: описание, направление, контакты	✅	Компания	`GET/PATCH /employer/me`	`integration/test_cipher_refresh.py`
Описание потребности: специалисты, чем занимается команда, стек, специализация, грейд	✅	Потребности → создать. Рядом с грейдом: точный, плюс-минус один шаг той же специализации или любой. По умолчанию плюс-минус. Кнопки «Высокая нагрузка», «Искусственный интеллект», «Робототехника» только заполняют форму	`app/api/v1/needs.py` (`grade_match`)	`test_api_layer.py`

Требование	Статус	Где увидеть	Где в коде	Тест
Вставленный текст вакансии заполняет черновик потребности: специализация, грейд, стек, вилка, формат, город. Одна открытая ссылка читается так же: без перехода дальше, без внутреннего адреса и с соединением только на уже проверенный публичный адрес. Похожие по словам навыки помечаются отдельно и сами в стек не попадают. Просьба денег, зарплата мимо оформления и отбор по внешности подписываются и здесь	✅ сверх ТЗ	Потребности → «Разобрать текст» / «Разобрать страницу»	`POST /needs/draft`, `POST /needs/draft-url`	`test_need_draft.py`
Подборка показывает, сколько человек в банке и сколько осталось по условиям потребности	✅ сверх ТЗ	Потребности → «N из M в банке»	`GET /needs/{id}/matches` (`base_total`, `total`)	`Needs.test.tsx`
Шорт-лист считает воронку и умеет показать один шаг: отобрано, приглашено или отказ	✅ сверх ТЗ	Потребности → Шорт-лист	`GET /needs/{id}/shortlist?status=`	`test_shortlist_page.py`
Рекомендованные категории и кандидаты, обоснование попадания	✅	Потребность → «Где искать», подборка, «Почему в выдаче». Оси факторов — диаграмма без имени	`recommended_categories`, `matching.score`	`test_bank_grid.py`, `test_matching.py`
Поиск по всей базе: специализация, грейд, категория, стек, достижения ФСП	✅	Поиск. Рядом с точной парой остаются один шаг грейда и смежная специализация того же грейда; два шага в список не входят. Анкета «Как вы нанимаете» без цифры уровня: пока ответов нет, банк прежний; меньше трёх «да» скрывает подтверждённых senior и lead, а на оставшихся карточках прячет город, стаж и строки ФСП. Имя до согласия не открывается	`GET /search/candidates`, `POST /employer/me/hiring-practice`	`test_practice.py`

Требование	Статус	Где увидеть	Где в коде	Тест
До трёх резюме у кандидата: основное из опроса и ещё две специализации с заявленным грейдом	✅ сверх ТЗ	Профиль → «Резюме»; поиск по специализации находит человека, на карточке пометка «грейд заявлен»	`app/api/v1/resumes.py`, `app/services/resumes.py`	`test_resumes.py`
Избранное компании вне потребностей: анонимные карточки, только кто ещё в поиске	✅ сверх ТЗ	«В избранное» и «приоритет» на карточке без имени. Специалист отмечает интерес к компании, компания видит только факт	`PUT /employer/favorites/{id}`, `POST /candidates/me/interests/{id}`	`test_api_layer.py`
Второй фактор (TOTP, RFC 6238) для стола модерации: включается только консольной командой, код одноразовый	✅ сверх ТЗ	`python -m app.services.totp enroll <почта>`; Модерация спрашивает код и открывается на 15 минут	`app/services/totp.py`, `app/api/v1/moderation.py`	`test_totp.py`
Смена провайдера языковой модели без перезапуска: только GigaChat, YandexGPT, Cloud.ru и MWS GPT, ключ зашифрован, защита от SSRF	✅ сверх ТЗ	Модерация → «ИИ-провайдер» (под вторым фактором)	`app/services/ai/providers.py`, `app/api/v1/moderation.py`	`test_ai_providers.py`
Мнение ИИ по карточке кандидата: только совет, балл и порядок не меняет, каждый пункт ссылается на факты карточки	✅ сверх ТЗ	Потребности → «Мнение ИИ по карточке»	`app/services/ai/second_opinion.py`, `app/api/v1/needs.py`	`test_second_opinion.py`
Паспорт навыков по ссылке и QR: подпись Ed25519, без имени и контактов, новый отзывает старый	✅ сверх ТЗ	Профиль → «Паспорт навыков»; `/passport/<id>`	`app/api/v1/passport.py`, `app/services/passport.py`	`test_passport.py`
Приглашение с описанием и зарплатой без обязательной вакансии, статусы	✅	«Пригласить» в подборке и поиске; Приглашения	`POST /offers/`	`test_api_layer.py`
Вилка «на руки» рядом с суммой до НДФЛ: шкала ст. 224 НК с 2025 года, в базе хранится сумма до налога. В приглашении можно ввести и сумму на руки: перед сохранением она переводится	✅ сверх ТЗ	Приглашение → «до вычета НДФЛ / на руки»	`app/services/salary_net.py`	`test_salary_net.py`
Строка «выше / ниже среднего по категории» рядом с процентом	✅ сверх ТЗ	«Почему в выдаче»	`standing_line`	`test_standing.py`

Требование	Статус	Где увидеть	Где в коде	Тест
Кандидат не показывается компании, которую скрыл, и в предметной области, от которой отказался	✅ сверх ТЗ	Приглашение → «Не показывать этой компании»; Профиль → «Куда не предлагать»	`app/services/exclusions.py`	`test_exclusions.py`
Повторный тест специализации начинает оценку с ответов прошлого теста, а не с нуля	✅ сверх ТЗ	Итог теста: «В старт вошли ответы прошлого теста»	`seed_carried`	`test_cat_carry.py`
Публичная страница показывает цифры проверки и состав теста по вставленному тексту вакансии, без ответов и без входа	✅ сверх ТЗ	«Как устроен тест»	`POST /public/vacancy-preview`, `/method`	`test_guide.py`
Навык на карточке подписан: подтверждён тестом, по цитате, заявлен; отсутствие — «нет …»	✅ сверх ТЗ	Карточка кандидата	`skillChip`, веса в `matching.TRUST`	`test_matching.py`
Сквозные сценарии сами берут код из Mailpit: регистрация доходит до входа без ручной почты	✅	`npx playwright test e2e/journeys.spec.ts` при `make demo`	`frontend/e2e/journeys.spec.ts`	`test_fixture_contacts.py`
Скрыть кандидата в потребности: пометка живёт при смене версии, подборка других компаний не меняется	✅ сверх ТЗ	Потребности → «Скрыть», «Показать скрытых»	`need_hidden`, `GET /needs/{id}/matches`	`test_need_hidden.py`, `test_migration_order.py`
Вилка против медианы категории: ниже половины или выше трёх раз, без блокировки отправки; подозрительная остаётся на столе модерации	✅ сверх ТЗ	Подсказка в приглашении и в вакансии; у кандидата «вилка ниже рынка»; Модерация → «Вилки вне рынка»	`app/services/market.py`, `offer_flags`	`test_market.py`
Уровень доверия компании из фактов: лимит приглашений и открытых вакансий, метка у кандидата, партнёр только командой	✅ сверх ТЗ	Компания → уровень; приглашение показывает метку	`app/services/trust_level.py`	`test_trust_level.py`, `test_api_layer.py`

Требование	Статус	Где увидеть	Где в коде	Тест
Репутация компании из фактов за 180 дней: ответ, собеседование, жалобы, срок; за 90 дней: доля откликов, открытых в течение 7 дней, и медиана дней до этого, плюс доля принятых приглашений. Доли с нижней границей Уилсона, меньше пяти случаев не показываются, общей оценки нет	✅ сверх ТЗ	Приглашение кандидата; Компания	`app/services/reputation.py`, `GET /employer/reputation`, `GET /public/companies/{id}`, `GET /vacancies/for-me`	`test_reputation.py`, `test_vacancies.py`
Живые вебхуки в ATS: Standard Webhooks, повтор доставки, только https на публичный хост, приёмник стенда, экспорт согласившихся	✅ сверх ТЗ	Интеграции	`app/services/webhooks.py`, `app/api/v1/integrations.py`	`test_webhooks.py`
Роль методиста: черновики заданий, возврат ключа из карантина с заметкой в журнале, пороги только для чтения. Роль ставится командой и не пишется из кабинета	✅ сверх ТЗ	Методист. В вакансии по-прежнему доли тем и примеры без ответов	`app/services/staff.py`, `app/api/v1/methodist.py`	`test_methodist.py`
Кандидат сам выбирает, что печатать на анонимной карточке: город, стек, опыт, результаты ФСП. Скрытый город не входит в балл, чтобы его нельзя было вычислить фильтром. Готовность к переезду смягчает чужой показанный город до 0,7 и в балле, и в отдельном шансе ответа на приглашение. Пока город скрыт, флаг не читается	✅ сверх ТЗ	Профиль → «Что видно на анонимной карточке»	`show_city`, `relocation`, `shown_city_match`	`test_matching.py`
Первые сутки после опроса грейд можно пересдать сразу, без паузы 7 и 90 дней	✅ сверх ТЗ	Тест на грейд, строка в правилах	`calibration_open`	`test_calibration.py`

Требование	Статус	Где увидеть	Где в коде	Тест
Быстрые верные ответы или уход со вкладки на тесте грейда кандидат видит у себя. В выдаче работодателя строка не пишется. Место, оценка соответствия и грейд не меняются: множитель 0,85 выключен	✅ сверх ТЗ	Результат теста у кандидата	`session_marks.penalised`, `APPLY_INTEGRITY_FACTOR`	`test_integrity_rank.py`
Свежесть подтверждения: оценка уровня прежняя, после полугода строка говорит, что погрешность шире. Без даты строка не появляется, грейд не меняется	✅ сверх ТЗ	«Почему в выдаче»	`freshness_of`	`test_test_freshness.py`
У стажёра и младшего разработчика (junior) две и больше приметы роста пишутся строкой. Балл и грейд не меняются	✅ сверх ТЗ	«Почему в выдаче»	`ambition_note`	`test_ambition.py`
Кандидат может указать логин публичного кода. На карточке до согласия логина нет. В балл и в грейд он не входит	✅ сверх ТЗ	Профиль → «Публичный код»; контакты после согласия	`code_profile_line`	`test_code_profile.py`
Свидетельство о навыках принимается как цитата. Грейд от файла не меняется, навык с теста не понижается	✅ сверх ТЗ	Профиль → «Свидетельство о навыках»	`POST /candidates/me/certificate`	`test_certificate_evidence.py`
Доля попадания в верхний срез по внешней метке региона: меньше четырёх пятых от самой высокой доли — правило нарушено. Регион в балл не входит, живую цифру банка команда не печатает	✅ сверх ТЗ	`python -m app.validation.fairness`	`impact_report`	`test_validation.py`, `test_fairness.py`
Рядом с нижней границей ФСП показывается разброс той же оценки. В балл и в грейд он не входит	✅ сверх ТЗ	«Почему в выдаче», если результаты ФСП видны	`elo_stretch`	`test_elo_stretch.py`

Требование	Статус	Где увидеть	Где в коде	Тест
Модель пары «запрос–текст навыков» по желанию переставляет верхние 50 только в факторе запроса. Выключена, пока не заданы режим и локальное имя модели. Платный адрес не вызывается. Навыки, грейд и нижняя граница ФСП не меняются	✅ сверх ТЗ	Поиск, флаг сервера	`CROSS_ENCODER_MODE`	`test_cross_rank.py`
Страница не выполняет скрипт из атрибута, не подменяет свой адрес и не отправляет форму на чужой сайт. Стили на элементе остаются	✅ сверх ТЗ	Заголовок CSP у nginx, Caddy и API	`app/core/csp.py`	`test_csp.py`
Ключ подписи удостоверения опубликован как did:web. Имя в документ не входит. На странице проверки есть QR этого документа	✅ сверх ТЗ	Проверка удостоверения, `/.well-known/did.json`	`GET /api/v1/public/did.json`	`test_did_web.py`
Мягкие навыки — поле профиля из фиксированного списка, на карточке, в балл не входят	✅ сверх ТЗ	Профиль → «Мягкие навыки»	`app/services/soft_skills.py`	`test_soft_skills.py`, `test_api_layer.py`
Три атаки на банк заданий одной таблицей: угадывание, утечка семейства, чужой ключ	✅	Проверка теста	`python -m app.validation.assessment --n 2000 --seed 7`	`test_validation.py`
Публикация вакансий, редактирование, отклики и работа с ними	✅	Вакансии. Отклик проходит при грейде не дальше одного шага; можно оставить только точный грейд или не фильтровать его	`PUT /vacancies/{id}`, `PATCH /vacancies/applications/{id}/status`, `grade_match`	`test_vacancies.py`, `test_regressions_offers_vacancies.py`
Контакты закрыты до принятия приглашения или отклика	✅	Приглашения: контакты появляются после «Принять»; если кандидат закрыл контакты для компании — 403 `contacts_revoked`	`GET /offers/{id}/contacts`, журнал `access_logs`	`integration/test_contacts.py`
Регулярные задания кандидатам и проверка ответов	✅	Задания → «Новое задание», ответы, «Зачтено / не зачтено»; «Дать задание» в поиске и подборке	`app/api/v1/tasks.py`	`test_employer_tasks.py`
Переписка с кандидатом после принятия приглашения	✅	Приглашения → «Переписка»	`app/api/v1/messages.py`	`test_access.py`

Раздел 2.1. Поиск и подбор

Требование	Статус	Где в коде	Тест
Выдача по категориям из опроса и теста, а не по резюме	✅	`category_assignments`, `matching.py`	`test_matching.py`
Ранжирование по силе подтверждённого профиля и соответствию потребности, краткая объяснимость	✅	`matching.py`: факторы с весами и строки «плюс / минус / инфо»	`test_matching.py`
Фильтрация и уточнение без потери подборки	✅	версии потребности `need_versions`, отобранные `need_shortlist` помнят версию	`test_api_layer.py`

Раздел 2.2. Данные и правила видимости

Правило	Статус	Где в коде
1. Зарплата «от–до» в рублях обязательна в вакансии и приглашении	✅	`OfferCreate`, `VacancyIn`: обе границы обязательны, «до» ≥ «от»
2. Приглашение содержит описание, зарплату, компанию, способ связи; контакты после согласия	✅	`offers.py`, `candidates.py` (`/me/offers`)
3. Статусы приглашений и откликов видны обеим сторонам	✅	`offers.status`, `applications.status`, уведомления
4. Регистрация по email с подтверждением адреса	✅	`app/services/identity.py`: код из 6 цифр или ссылка
Сверх ТЗ, только демо-стенд: гостевой вход без почты	✅	`POST /auth/guest`, `app/services/guest.py`: те же согласия, пустой аккаунт на 48 часов, гостя нет в поиске; вне DEMO_MODE ответ 404
Сверх ТЗ: пример кабинета, если сервер не ответил	✅	Вход → «Пример кабинета». Плашка «это пример». Запись с экрана не уходит; `frontend/src/demoFallback.ts`

Уточнения требований (Q&A) и как они учтены

Те же решения с доказательством тестами (имя теста или файл на каждое, проверяется CI): REQUIREMENTS_QA.md.

Уточнение	Как сделано	Где в коде
Грейд определяется только тестом категории	Грейд даёт только основной тест специализации. Тест по вакансии, задачи недели, задания работодателя и результаты ФСП грейд не меняют	`app/services/assessment.py`, `is_vacancy_session`
Неподтверждённый грейд показывается ниже	До теста грейд помечен как заявленный и в оценке соответствия весит 0,85 от подтверждённого; в карточке строка «указан кандидатом, тестом не подтверждён»	`app/services/matching.py`
Повышение грейда — отдельным тестом	Тест на уровень подтверждает только этот уровень. Уверенный результат выше (от 90 % за следующий порог) даёт рекомендацию пройти тест на ступень выше, но грейд сам не растёт	`app/services/cat.py` (`RECOMMEND_NEXT_P`), `assessment.py`
Ограничение по времени, но тест ниже без потери попытки	Не сдали — тест на уровень ниже доступен сразу. Пересдача того же уровня — через 7 дней, прерванная попытка засчитывается. После смены грейда следующее повышение — через 90 дней	`retake_available_at`, `GRADE_COOLDOWN_DAYS`, `RETAKE_INTERVAL_DAYS`
Тест устойчив к утечке банка	Параметрические семейства вместо банка, вводимый ответ, карантин утёкших ключей, вопросы на понимание не повторяются во второй попытке больше чем на 40 %	`app/items/`, `item_monitor.py`, VALIDATION_ASSESSMENT.md
Языковая модель допустима, открытые предпочтительнее, видеокарта не нужна	Письменный ответ разрешает облачные модели, в том числе зарубежные. Мы берём GigaChat или YandexGPT и только обезличенный текст: навык принимается, если цитата из резюме дословная. Без ключа работает разбор правилами. Балл теста, грейд и порядок подбора модель не считает; мнение по карточке — только совет со ссылками на факты	`app/services/ai/assistant.py`
Вакансии не импортируем	Вакансии создают только работодатели на платформе, демо-вакансии вымышлены	`app/api/v1/vacancies.py`, `seed/`
Регулярные задания пишет работодатель, проверяет система или работодатель	Число, выбор, код и короткий шаблон проверяет система. Открытый ответ проверяет работодатель. Модель ответ не оценивает. Задание для всех кандидатов специализации или одному кандидату	`app/services/tasks.py`, `graded_by`
ФСП ID — единая точка входа, вход можно имитировать	«Войти через ФСП ID» на входе и регистрации: OIDC с PKCE, демо-провайдер вместо Keycloak. Новый человек проходит экран согласий, аккаунт создаётся из реестра. Регистрация по email осталась	`app/api/v1/auth.py`, FSP_INTEGRATION.md
Кандидат может закрыть контакты	Глобальный отзыв согласия и закрытие контактов для одной компании: компания получает 403 `contacts_revoked`, событие пишется в журналы	`candidates.py` (`/revoke-contacts`), TRUST_AND_ATS.md

Уточнение	Как сделано	Где в коде
Нужна связь после принятия приглашения	Переписка по каждому принятому приглашению, после закрытия контактов — только чтение	`app/api/v1/messages.py`
Достижения ФСП: соревнования не ранжировать по статусу, учитывать результат и количество	Методика `v4.2.0`: класс результата по месту (победитель, призёр, топ-10 %, верхняя половина, участие с весом 0), статус не учитывается. Количество результатов работает через байесовское сжатие, нижняя граница — лучшее подмножество до пяти результатов, не больше двух за один месяц, поиск видит все результаты месяца: новое событие никогда не опускает нижнюю границу, участие её не поднимает. Разряд — факт реестра и в балл не входит	`app/services/scoring/engine.py`, README
Демо-аккаунты	Пока включён демо-режим, смена пароля и удаление закрыты только у опубликованных демо-адресов: `fsp001@demo.talent`, `fsp002@demo.talent`, `nofsp@demo.talent`, `hr@sibcode.demo`, `hr@quantumlab.demo`, `hr@perimeter.demo`, `hr@cloudwhale.demo`, `hr@robotech.demo` и `jury1@demo.talent` … `jury10@demo.talent`. Зарегистрировать такой адрес нельзя. Адрес вроде `ivan@x.demo` — обычный аккаунт. Кнопка на входе берёт адрес, в который дольше всех не входили (`GET /api/v1/public/jury-free`), и не отдаёт других данных. Ночной возврат в 04:10 МСК задаёт `DEMO_RESET_CRON`; пустая строка его выключает. Время входа сброс не стирает	`app/core/demo_guard.py`, `seed/reset_demo.py`, `app/api/v1/public.py`
Длина теста	Тест заканчивается на 20-м задании. Одно продолжение до 6 заданий на 2000 кандидатах, зерно 7: 76,9 % сдавших среди тех, кто на уровне при пороге 77 %, ложные сдачи 9,8 % при пороге 8,5 %	`MAX_ITEMS` в `app/services/cat.py`
Несовершеннолетние участники	Скрыты из поиска и рейтинга без согласия законного представителя. Согласие не заменяет дату рождения: без даты карточка в поиск не попадает	`app/services/profile.py` (`adult_cutoff`), `app/services/search/pipeline.py` (`_in_search`)
Зарплата — до вычета НДФЛ	Поля вилки названы «Зарплата до вычета НДФЛ, ₽ в месяц», в API — `salary_from_gross`, `salary_to_gross`	`InviteDialog.tsx`, `app/models/offer.py`
Ресурсы стенда	Обычная машина: девять сервисов и разовый `demo-seed`, около 1,5 ГБ памяти, без видеокарты	INSTALL.md
«Отрасль» в ТЗ — направление ИТ	Шаг опроса называется «Направление и специализация», поле предметной области необязательно	`frontend/src/components/SurveyCard.tsx`, `app/services/survey.py`

Уточнения из таблицы вопросов, как это сделано сейчас:

Код	Уточнение	Где в коде
E08	Достижения ФСП поднимают место внутри грейда и не заменяют тест. Дисциплины равны; для потребности по ИБ результат по ИБ может весить больше, чем по робототехнике	Грейд от ФСП не зависит (`app/services/assessment.py`). В подборке по потребности профильная дисциплина входит целиком, остальные — с весом 0,5 (`app/services/fsp/relevance.py`, `OTHER_DISCIPLINE`); для специализации без профильной дисциплины берётся лучший результат по всем. Ручной фильтр „дисциплина“ оставляет только её (`app/services/search/pipeline.py`). Статус соревнования не весит
E09	Регулярное задание адресное. Задание на всю категорию не запрещено. Способ оценки свой	Адресное задание и задание на специализацию: `app/api/v1/tasks.py`. Число, выбор и функция `solve` проверяет система, открытый ответ — работодатель. Код не меняет грейд. Задача недели собирается из вакансии специализации (`app/services/practice.py`)
PR.1	Процедуру валидации предлагает команда: повторный тест и разделяющая сила, симуляция допустима	`python -m app.validation.assessment --n 2000 --seed 7`: повтор, каппа, медиана r. Подбор — `python -m app.validation.matching`
VS.1	Грейд — уровень навыков по тесту, не стаж в годах	Стаж только подсказывает уровень в опросе и в оценку грейда не входит (`app/services/categorization.py`)
VS.2	Несколько категорий не требуются. Для MVP достаточно одной, несколько будут плюсом	Основная категория — из опроса. Вторую специализацию подтверждает отдельный тест (`GradeConfirmation`). В поиске этой роли человек один. Карточка этой выдачи показывает подтверждённую пару, основная остаётся строкой
VS.3	Облачные языковые модели допустимы. Если в модель уходят персональные данные, риски 152-ФЗ на команде	GigaChat или YandexGPT только по флагу и только на обезличенном тексте. Балл теста и подбора считается без модели (`app/services/ai/`)
VS.4	ФИО и места работы показывать можно. Закрыты только контакты до принятия приглашения	Карточка анонимна до принятия. Это строже ответа: имя не показывается раньше согласия

Код	Уточнение	Где в коде
VS.5	Первичный тест единый под категорию при регистрации, не один и тот же набор для всех. Задания под вакансию — отдельный блок	Параметрические семейства основного теста. Тест по вакансии грейд не меняет (`app/services/vacancy_exam.py`)
FTT.1	Жёсткого лимита на образ и память нет. Расход должен быть оправдан	Около 1,5 ГБ, без GPU (INSTALL.md)
FTT.2	Важнее качество подбора, чем скорость. Основные операции — за приемлемое время	Качество подбора проверяет отдельная процедура (VALIDATION_MATCHING.md). Подборка на 10 000 профилях, p95 360 мс (LOAD_TEST.md)
FTT.3	Категории достаточно как связки специализации и грейда. Стек и формат работы — плюс	Категория — специализация и грейд. Город и формат — один фактор балла, когда вакансия их называет. Несовпадение снижает балл и человека из выдачи не убирает. Вилка входит в согласие сторон
FTT.4	Официальный грейд только по тесту. Проверка на списывание не требуется и будет плюсом	Грейд ставит только тест. Темп, lz*, CUSUM, чужой вариант, похожий код, лимит лаборатории, уход со вкладки и копирование текста оставляют пометку. Грейд от пометки не меняется. Законченный тест можно оспорить. Работодатель эту запись не видит
FTT.5	Интеграция с ATS в MVP не требуется. Нужна концепция	TRUST_AND_ATS.md
FTT.6	Свои данные: синтетика или открытые источники. Парсинг чужих вакансий — не продуктовая механика	Демо-данные и симуляция свои. Импорта вакансий нет
WA.1	Достаточно основных направлений, если путь от теста до приглашения целый	Одиннадцать специализаций и пять грейдов. Путь от теста до приглашения целый
WA.2	Справочник может включать дизайнеров. Приоритет — роли, которые проверяются тестом	В справочнике роли, которые проверяет тест

Целевая машина — около 32–64 ГБ памяти и порядка 8 ядер, без видеокарты. Стенд проекта занимает около 1,5 ГБ и видеокарты не требует. В обычном запуске вход ФСП ID идёт через демо-провайдер по тому же OIDC. Домен почты на решение не влияет.

Раздел 3. Программные и архитектурные требования

Требование	Статус	Где	Тест
3.1 Обычная машина или недорогой сервер	✅	Девять сервисов и разовый `demo-seed`, около 1,5 ГБ памяти (INSTALL.md)	н/п (документ)
3.2 ML и NLP для резюме, категоризации и подбора — преимущество	✅	Разбор резюме с цитатами, IRT-модель теста, ранжирование, GigaChat и смысловой поиск (эмбеддинги) по флагу	`test_resumes.py`, `test_skills.py`
3.3 Ссылка на стенд или видео и инструкция локального запуска	✅	видео, INSTALL.md, deploy/README.md	н/п (документ)
3.3 Сквозной сценарий и объяснение механик	✅	Руководство пользователя: полный сценарий, README: 5 минут	н/п (документ)
Одна команда приёмки по таблице требований	✅ сверх ТЗ	`make accept`, requirements_trace.md; `python -m app.validation.accept`	`test_accept.py`, `test_validation.py`
Проверка дампа базы и восстановление только с явным подтверждением	✅ сверх ТЗ	deploy/README.md; `deploy/verify_backup.py`, `deploy/restore_db.sh`	`test_backup_check.py`
3.4 Сопроводительная документация по восьми пунктам	✅	SOLUTION.md	н/п (документ)
3.5 Открытый код и открытые библиотеки	✅	Публичный репозиторий, LICENSE (MIT), таблица лицензий в SOLUTION.md, гл. 8	н/п (документ)
3.5 Frontend, Backend и слой хранения разделены	✅	`frontend/`, `backend/`, PostgreSQL	`test_api_layer.py`
3.5 Docker, Dockerfile, docker-compose.yml	✅	`docker-compose.yml`, `backend/Dockerfile`, `frontend/Dockerfile`, проверка в CI	`test_compose_gpu_optional.py`
3.5 OpenAPI: запросы, ответы, коды ошибок	✅	`/docs`, `/openapi.json`, API_ERRORS.md	`test_openapi_paths.py`
Ошибка API в форме RFC 9457: класс E01–E20 и прежний код	✅ сверх ТЗ	`GET /api/v1/problems`, API_ERRORS.md; `app/api/problems.py`	`test_problems.py`

Требование	Статус	Где	Тест
3.5 152-ФЗ, согласия, роли, защита паролей и токенов	✅	Пять неверных паролей с одного адреса закрывают вход с этого адреса на 15 минут; аккаунт целиком — после 100 (NIST 800-63B), опубликованные демо-аккаунты не закрываются. Минутные окна Redis остаются. SOLUTION.md, гл. 10	`test_security.py`, `test_login_lock.py`
Пароли Argon2id, refresh в HttpOnly-cookie, двойная cookie против подделки запроса	✅ сверх ТЗ	Вход и обновление сессии; `app/core/security.py`, `app/core/csrf.py`	`test_security.py`, `test_doc_facts.py`
Заголовки браузера закрывают камеру, микрофон и геолокацию. Вход ограничен через Redis	✅ сверх ТЗ	Ответ `/health` и nginx; `BrowserLock` в `app/main.py`	`test_browser_lock.py`
3.5 Совместимость с Keycloak	✅	OIDC-клиент, FSP_INTEGRATION.md	`test_fsp_signin.py`
3.5 Кроссплатформенность, адаптивная вёрстка	✅	Все экраны от 390 px, светлая и тёмная темы (DESIGN.md)	`Responsiveness.test.tsx`
3.5 Десятки пользователей одновременно	✅	40 пользователей, 0 ошибок (LOAD_TEST.md)	`load_api.py`
Гонка черновика ответа и гонка ответа на приглашение: остаётся один победитель	✅ сверх ТЗ	LOAD_TEST.md, сценарий `races`; `tests/load_api.py`	`test_load_race.py`
3.5 Комментарии к сложным решениям, код соответствует документации	✅	Докстринги модулей теста, подбора и ФСП; цифры документации сверены с выводом команд	`test_doc_facts.py`

Материалы проекта

Материал	Где
Репозиторий с кодом	публичный репозиторий https://github.com/KonkovDV/fsp-talent
Сопроводительная документация в doc или pdf	docs/FSP_Talent_documentation.pdf — сборка всех документов, `make docs-pdf`
Презентация	PDF, PPTX — 31 слайд
Инструкция локального запуска	INSTALL.md

Сборка, развёртывание и локальный запуск

Инструкция для эксперта, который разворачивает решение на своей машине, и для оператора сервера (ТЗ 3.3, 3.4). Разделы 1–10 — локальный запуск, раздел 11 — продуктивная установка с HTTPS, резервными копиями, обновлением и откатом. Тот же путь, что в разделах 3–4, описан в `.github/workflows/ci.yml` (задание `compose-e2e`: `make demo` и `make check` на чистой машине).

Первый запуск занимает 5–10 минут, столько уходит на скачивание и сборку образов. Повторный проходит за секунды.

1. Что понадобится

Что	Версия	Как проверить
Docker Engine или Docker Desktop	24 и новее	`docker --version`
Docker Compose v2	2.24 и новее (необязательный `.env` и флаг `--wait`)	`docker compose version`
Git	любая	`git --version`
Свободная память	4 ГБ	всё вместе занимает около 1,5 ГБ
Место на диске	5 ГБ	образы и зависимости
Свободные порты	80 и 8025	порт 80 можно поменять, см. раздел 9

Всё из таблицы проверяет `make doctor` (скрипт `scripts/doctor.sh`, ничего не меняет): строка ✓ или ✗ и что исправить, при любом ✗ команда завершается с ошибкой. Порты, занятые уже запущенным стендом, не считаются ошибкой; ключ `SECRET_KEY` из `.env.example` тоже: стенд заменит его своим при первом запуске. Для запуска без Docker (раздел 10) — `make doctor LOCAL=1`: Python 3.12, Node.js 20+, PostgreSQL 16 и Redis 7 отвечают, `SECRET_KEY` задан.

`make` нужен только для коротких команд. В Windows его обычно нет, поэтому для PowerShell рядом даны те же команды без `make`.

Подходит Linux, macOS (Intel и Apple Silicon) и Windows 10/11 с Docker Desktop на WSL 2. Python и Node.js на машине не нужны: всё собирается внутри контейнеров.

Рабочая станция на 32–64 ГБ памяти и около 8 ядер без видеокарты подходит с запасом: видеокарта решению не нужна.

2. Получить код

```
git clone https://github.com/KonkovDV/fsp-talent.git
cd fsp-talent
```

Если код пришёл архивом, распакуйте его и откройте терминал в папке `fsp-talent`.

3. Запустить одной командой

Linux, macOS, Windows (PowerShell) — одинаково

```
docker compose up -d --build --wait
```

`.env` создавать не нужно: настройки берутся из `.env.example`, свой `.env` (если есть) их перекрывает. Команда завершается, когда все контейнеры `healthy` и демо-данные загружены. С `make` то же самое: `make demo`. На Windows можно и так: `powershell -ExecutionPolicy Bypass -File scripts\start-local.ps1`.

Письмо с кодом подтверждения на этом запуске не приходит в Gmail и на другую почту. Откройте http://localhost:8025: это локальный ящик Mailpit. Своя почта нужна только серверной установке и задаётся в `deploy/.env` (`SMTP_HOST` и соседние поля).

Что происходит внутри:

1. Настройки читаются из `.env.example` и необязательного `.env`. Ключ подписи токенов контейнер сам создаёт при первом старте и сохраняет в томе `vc_keys`, так что опубликованных секретов в работе нет.
2. Образ backend собирается один раз и используется для API, worker, beat и загрузчика демо-данных. Контейнеры стартуют по проверкам здоровья: база и Redis, затем API (миграции применяются при его старте), затем worker и beat.
3. Сервис `demo-seed` после готовности API загружает демо-данные и становится `healthy`; `--wait` ждёт именно его. Повторный запуск безопасен: уже загруженное не дублируется. При `DEMO_MODE=false` загрузка пропускается.

Проверка демо-данных: `docker compose run --rm backend python -m seed.demo_check` (или `make demo-check`), ожидается «18/18 OK».

В конце команда печатает адреса и демо-входы.

4. Проверить, что всё работает

Адрес	Что там
http://localhost	Сайт: кабинеты кандидата и работодателя
http://localhost/docs	Swagger UI: все методы API с запросами, ответами и кодами ошибок
http://localhost/openapi.json	Спецификация OpenAPI 3.1
http://localhost/health	`{"status":"ok",...,"build":"<sha коммита>"}`: `build` — коммит сборки (`make demo` передаёт `git rev-parse HEAD`; без git — `unknown`)
http://localhost:8025	Mailpit: письма с кодами подтверждения и уведомлениями

Команда `docker compose ps` должна показать в состоянии `running` девять сервисов — nginx, frontend, backend, worker, beat, postgres, redis, mailpit и sandbox — и загрузчик `demo-seed`, который после загрузки данных остаётся ждать. Sandbox запускает решения задач с кодом, сети у него нет. Проверка здоровья есть у базы, backend, worker, beat, frontend и sandbox. У redis, Mailpit и nginx её нет. Keycloak стартует отдельно, с профилем `fsp-id`.

Сквозная проверка API на запущенном стенде:

```
make check
```

Команда регистрирует нового кандидата (код подтверждения она берёт из Mailpit), привязывает ФСП ID через демо-провайдера, проходит адаптивный тест, ищет кандидата, отправляет приглашение и проверяет раскрытие контактов. Она же проверяет задания от работодателя (с проверкой системой и работодателем), вход через ФСП ID привязанного и нового кандидата, переписку по принятому приглашению и закрытие контактов для одной компании. Она же проверяет демо-аккаунты `jury1` … `jury10`, согласие законного представителя и обходит все маршруты с идентификатором объекта: другой работодатель и другой кандидат не получают чужих объектов. Успешный прогон заканчивается строкой `SMOKE OK`.

> `make check` меняет демо-данные: например, подтверждает тестом грейд кандидата `fsp001@demo.talent`. Чтобы вернуть стенд к исходному виду, выполните `make reset` и снова `make demo`.

5. Демо-входы

Пароль у всех демо-аккаунтов: `demo12345`. Компании и люди вымышленные.

Роль	Email	Что показывает
Кандидат	`fsp001@demo.talent`	Привязан ФСП ID, результаты из протоколов, ожидающие приглашения с зарплатой, задача недели
Кандидат	`fsp002@demo.talent`	Специализация «информационная безопасность», командные результаты ФСП
Кандидат	`nofsp@demo.talent`	Без истории ФСП: категория строится только по опросу и тесту
Кандидат с нуля	`jury1@demo.talent` … `jury10@demo.talent`	Пустой опрос: путь кандидата с первого шага. Дата рождения уже указана, на показе её не вводят. При `DEMO_MODE=true` полный тест проходится без пауз и лимитов попыток, результат сразу попадает в категорию и поиск
Работодатель	`hr@sibcode.demo`	Три потребности с подборками, приглашения в статусах «ожидает», «принято» и «отклонено», вакансии и отклики, три задания для кандидатов (одно открытое ждёт проверки)

Чтобы показать регистрацию с нуля, заведите новый аккаунт на http://localhost/register. Код подтверждения придёт в Mailpit. Для привязки ФСП ID в демо-реестре свободны номера FSP2000–FSP2099, пароль `test`. Войти без регистрации можно кнопкой «Войти через ФСП ID» на странице входа: `FSP001` / `test` открывает аккаунт `fsp001@demo.talent`, свободный номер из FSP2000–FSP2099 / `test` создаёт новый аккаунт после экрана согласий.

Пошаговый сценарий из ТЗ 3.3 и описание всех экранов — в руководстве пользователя. Ниже два разных прохода: короткий показ и самостоятельная проверка.

Показ за три минуты

За десять минут до показа стенд уже поднят (`make demo`). В демо-режиме стенд каждую ночь в 04:10 МСК возвращается в исходное состояние, вручную — `make demo-reset`. После сброса у `jury1@demo.talent` … `jury10@demo.talent` пустой опрос и нет ни одного теста: ни пробного, ни полного. Подтверждённый грейд появится, только если перед показом пройти опрос и полный тест под `jury1@demo.talent` (backend, middle). Во второй вкладке вошёл `hr@sibcode.demo`, в третьей — `fsp001@demo.talent`. Пароль у всех `demo12345`. В одной вкладке включена тёмная тема. Если показ сокращают до двух минут, строку 0:20–0:50 пропускают.

Время	Кто	Что сделать	Что сказать
0:00–0:20	—	Вступление	Работодатель находит человека, а не наоборот. Грейд даёт только тест, ФСП — отдельная подтверждённая ось, контакты — после согласия.
0:20–0:50	Кандидат	Вход → «Войти через ФСП ID» → `FSP001` / `test` → «Результаты»	ФСП ID по протоколу OpenID Connect с PKCE, как будет в Keycloak Федерации. Без ФСП человек ничего не теряет.
0:50–1:20	Кандидат	`jury1@demo.talent` / `demo12345`: опрос (специализация и грейд) → «Тест на грейд» → «Пробный тест», 3–4 задания, видна шкала уверенности → «Прервать». В окне подтверждения сказано, что грейд не меняется и паузы нет. В истории — только этот пробный проход; полный тест (8–20 заданий) запускается с той же страницы	Задание каждый раз новое: в коде совпадает ответ у 9,3 % пар свежих экземпляров, в вопросах на понимание у 36,9 % (`construction_check`, зерно 7).
1:20–1:50	Работодатель	«Поиск». На широком экране витрина уже раскрыта: нажать число в ячейке Backend / мидл, не заголовок витрины. В карточке видны грейд, тест и ФСП	Внутри категории выше тот, у кого сильнее тест и ФСП: вклад виден в баллах.
1:50–2:20	Работодатель	«Потребности» → подсказка «Если поднять верх вилки до …, ожидание закроется ещё у N из M» → «Пригласить» с вилкой до вычета НДФЛ	Место задаёт соответствие; вилка и вероятность согласия видны на карточке и порядок не двигают.
2:20–2:45	Кандидат	`fsp001`: «Приглашения» → «Принять»	До принятия имени и контактов нет ни в карточке, ни в ответе API.
2:45–3:00	Работодатель	Контакты открылись, кнопка «Переписка»	Дальше — переписка на платформе. Проверить самим: `jury1@demo.talent` … `jury10@demo.talent`, пароль `demo12345`.

Если спросят про точность подбора, говорить так: тест и ФСП упорядочивают людей внутри категории. Спирмен со скрытой силой 0,410 против 0,011 у сортировки по резюме, завышенный грейд в первой десятке 22,1 % против 28,1 %. nDCG@10 полной модели 0,658, у сортировки по резюме 0,620, у строки «без теста и без ФСП» 0,669. Выдача сортируется одной оценкой соответствия. Цифры — `python -m app.validation.matching --check`.

Ответы, если спросят

Короткие ответы на частые вопросы по цифрам. Команды — из каталога `backend/`. Прогон на 300 кандидатах только проверяет пороги и в эти ответы не подставляется.

- Каппа 0,49. Вдали от порога то же решение в 87 % пар, корреляция двух оценок θ — 0,86, взвешенная каппа — 0,56, согласованность по Руднеру — 79,1 %. Команда: `python -m app.validation.assessment --n 2000 --seed 7`.
- Параметры заданий задаёт уровень семейства. В банк пишется априор: средняя ошибка a 0,146 и 0,283. Калибровка по живым ответам — `python -m app.validation.calibrate`.
- Симуляция отвечает по той же модели, которой тест оценивает, поэтому отдельно прогнана ошибка параметров заданий. При σ = 0,30 точность вдали от порога 93,1 %, ложные сдачи 9,1 %. Атака «20 чужих сессий» от модели не зависит: на 400 людях на каждом из шести зёрен (всего 2400, `python -m app.validation.assessment --stolen`) сдача после атаки 5,7 % (4,8–6,7 %) при честной 5,3 % (4,5–6,3 %), в пределах 95 % ДИ честной; до расширения параметров узких семейств было 6,2 % (5,3–7,2 %). Чужой ключ от другого экземпляра с ответом своими силами там, где ключ не подходит: 69,2 % сдач против 58,5 % честно у тех же 400 человек (+10,8 п. п., `python -m app.validation.construction_check`); прежние 0 % получались по построению прогона (подробно в VALIDATION_ASSESSMENT.md, раздел «Что держится по построению»).
- Семейств с вычисляемым ответом двадцать, плюс два пула вопросов на понимание. На прогоне в 2000 человек другой кандидат видел в среднем 15,2 % заданий. Ответ чужого экземпляра совпадает в коде в 9,3 % случаев; с чужим ключом сдают 69,2 % против 58,5 % честно.
- Решение с подсказкой модели на MVP не требуется. Есть лимит времени, сигнал несоответствия ответа модели и накопленная сумма по времени и доле верных. Сигнал — повод пересдать, грейд он сам не снимает.
- В тесте на грейд код не запускается: ответ — число, его считает эталон на каждом языке, эталоны гоняет сборка. Задачи с кодом от работодателя выполняются в контейнере `sandbox` без сети и грейд не меняют.
- Цифры воспроизводятся так: `python -m app.validation.assessment --n 2000 --seed 7` и `python -m app.validation.matching --check`. SHA-256 набора подбора на Linux — `1ebff9bbea1c3a3efa08684a86e3094bc6ec88a1867014ad655e9bd12ffc7e00`, на Windows он другой из-за сборки математики. Цифры этого раздела — с прогона на Linux.
- Сборка: `python -m pytest tests/unit --collect-only -q` собирает 1752 теста. Последний прогон с покрытием (Python 3.12, все компиляторы, как в CI) напечатал 74,7 % по всему приложению, сервисы 77 %, API 61 %. Порог сборки — 65 %. Интеграционные тесты не проходят, если общее покрытие ниже 72 %, а `pipeline` или `assessment` ниже 60 %. Тестов интерфейса 210, проверок Playwright 67: семь сквозных сценариев и 60 проверок доступности.
- Сорок пользователей — это масштаб из задания. На банке 10 000 профилей и двух процессах API 95-й перцентиль подборки 360 мс, ошибок не было. Команда: `python tests/load_api.py --scenario shortlist --seconds 60 --reuse`.
- Строка «без теста и без ФСП» даёт nDCG@10 0,669, полная модель — 0,658. Польза теста — порядок внутри категории и меньше завышенных грейдов, см. абзац выше. Ярус в этот порядок не вошёл.

- «Завышенный грейд» у нас — показанный грейд выше настоящего: 22,1 % против 28,1 %. Второе определение, настоящий грейд ниже грейда потребности: 13,6 % против 30,9 %. Чужая цифра вроде 4 % посчитана на другом наборе. Оба наших числа печатает `python -m app.validation.matching --check`.
- Метки подбора считаются по скрытой правде, формула видит только заявленное. Слепая каппа оракула на зерне 2027 — 0,46. Процедура считает межэкспертную каппу двух разметчиков, как только к той же команде добавлены файлы пар.
- Проверка подбора вызывает тот же `order_pool`, что и выдача.
- У сервисов есть проверка живости и перезапуск `unless-stopped`; выкладку на сервер можно заморозить переменной `DEPLOY_FROZEN=true`.
- Командное событие весит `1/vn`: так несколько командных явок не перевешивают одну личную победу. Вес одинаков для дисциплин и статусов.
- Клиент OpenID Connect с PKCE готов. Keycloak с демо-реалмом поднимается отдельно и для локального показа не обязателен.
- Строки в PostgreSQL закрыты ролью приложения. В серверной установке API, worker и beat входят как `fsp_app`. Локальный Compose входит как владелец и переключается на `fsp_app`. Обход чужих идентификаторов: 0 ответов 2xx из 240 запросов. Эпоха токена закрывает старые сессии после смены пароля.
- Работодатель видит витрину и подсказку по вилке и приглашает подходящих, а не разбирает всю ленту откликов.
- Статус «не ищу» убирает человека из поиска. «Открыт к предложениям» снижает вероятность согласия.
- Причина отказа видна в долях рынка, когда в ячейке не меньше пяти человек. После отказа та же компания ждёт 30 дней.
- Доли рынка, четверти зарплат и публичный рейтинг категории считаются только по тем, кого работодатель видит в поиске: одно правило `visible_in_search` (дата рождения, правило для несовершеннолетних, «не ищу», удалённые и приостановленные). Рейтинг категории меньше чем из пяти человек не публикуется.
- Вилка в приглашении подписана «до вычета НДФЛ».
- Регистрация с 15 лет. Для 15–17 лет карточки нет в поиске, пока представитель не подтвердит согласие по ссылке.
- Согласия хранятся с версией текста. Отзыв сразу убирает карточку. В языковую модель уходит только обезличенный текст и только если флаг включён.
- У компании есть метка ИНН, лимит 50 приглашений в день и жалоба. Три разных человека и нижняя граница доли жалоб от 10 % ставят приглашения на паузу до ручной проверки.
- Каждое число в документации повторяется командой. Чужой ключ даёт прирост сдачи 10,8 п. п. (69,2 % против 58,5 %, `python -m app.validation.construction_check`). Место в подборке задаёт соответствие; вероятность, что человек примет вилку, на карточке есть и порядок не двигает. Тест можно пройти самому: `jury1@demo.talent` … `jury10@demo.talent`, пароль `demo12345`.

Проверить самим за десять минут

Самопроверка стенда: `make demo-check` (`python -m seed.demo_check`) печатает таблицу «аккаунт → ожидается → факт» по таблице выше и завершается с ошибкой, если хоть одна строка не сошлась. В `ci.yml` она идёт сразу после загрузки демо-данных.

Пароль демо-аккаунтов `demo12345`. На странице входа при включённом демо-режиме есть строка про демо-аккаунты и кнопка «Взять свободный аккаунт жюри». Кнопка берёт ответ `GET /api/v1/public/jury-free`: адрес из `jury1@demo.talent` … `jury10@demo.talent`, в который дольше всех не входили. В ответе только адрес. Если запрос не удался, номер выбирается случайно. Два нажатия до входа могут получить один адрес. Ночной сброс в 04:10 это время входа не очищает. Рядом с «Войти через ФСП ID» написано: демо `FSP001` / `test`.

Кто	Что сделать	Что увидеть
`jury1@demo.talent` … `jury10@demo.talent`	Опрос: специализация и грейд обязательны, стаж только подсказывает. «Пробный тест»: до 15 заданий и 5 минут, можно прервать. Затем полный тест, 8–20 заданий	Категория «специализация, грейд». После полного теста грейд подтверждён. У каждого из этих аккаунтов свои попытки, паузы между попытками нет
`hr@sibcode.demo`	«Поиск»: витрина, клик по ячейке. «Потребности»: подсказка по вилке, «Пригласить» с вилкой	Карточки без имён. Подсказка говорит, сколько людей добавится при другой вилке, и не блокирует отправку
Кандидат, которого пригласили	«Приглашения» → «Принять»	У работодателя появляются имя и контакты. До принятия их нет
Кандидат	«Вакансии» → «Откликнуться»	Работодатель в «Вакансиях» видит отклик и открывает контакты кнопкой «Показать контакты»
Компания и кандидат	После принятия: «Собеседование», одно–три времени по Москве, кандидат выбирает, обе стороны скачивают календарь	В файле начало записано как московское время. Итог «прошёл», «не прошёл» или «не пришёл» записывает компания

Регистрацию с нуля можно показать на http://localhost/register. На локальном стенде код подтверждения приходит в Mailpit (http://localhost:8025). На демо-стенде без почтовой учётки код показывается на экране подтверждения, если включены `DEMO_MODE` и `DEMO_SHOW_CODE`. Кандидат 15–17 лет указывает в профиле email представителя. Ссылка уходит письмом. На том же демо-стенде она дополнительно показывается на экране профиля самому участнику и только для адреса, который не кончается на `@demo.talent` или `.demo`. Экран «Забыли пароль?» при `DEMO_MODE` и `MAIL_MODE=console` пишет: «На демо-стенде письма не отправляются, используйте демо-аккаунты». Код сброса на экран не выводится.

6. Тесты и проверки

Все команды запускаются на поднятом стенде, внутри контейнера backend.

Что	Linux, macOS	Windows (PowerShell)
Юнит-тесты	`make test`	`docker compose exec -T backend python -m pytest tests/unit -q`
Стиль кода	`make lint`	`docker compose exec -T backend sh -c "pip install -q ruff==0.16.10 && ruff check app seed tests"`
Проверка теста и подбора на синтетике	`make validate`	`docker compose exec -T backend python -m app.validation.assessment --n 300 --seed 7 --check`, затем `docker compose exec -T backend python -m app.validation.matching`
Сквозная проверка API	`make check`	`docker compose exec -T -e SMOKE_BASE=http://localhost:8000/api/v1 -e MAILPIT_URL=http://mailpit:8025 backend python tests/smoke_api.py`
Нагрузка: 40 пользователей в течение минуты	`make load`	`docker compose exec -T -e SMOKE_BASE=http://localhost:8000/api/v1 -e MAILPIT_URL=http://mailpit:8025 backend python tests/load_api.py`

Внутри контейнера нет Node.js, Go, Java, Kotlin и .NET, поэтому тесты, которые перезапускают задания на этих языках, пропускаются с пометкой `skipped`. Полный прогон со всеми компиляторами — задание `test-backend` в `ci.yml` или локальный гейт с установленными Node.js, Go, JDK и .NET.

Полные цифры из документации воспроизводятся так:

```
docker compose exec -T backend python -m app.validation.assessment --n 2000 --seed 7
docker compose exec -T backend python -m app.validation.matching
docker compose exec -T backend python -m app.validation.matching --seed 7
```

7. Остановить, сбросить, обновить

Действие	Команда
Остановить, данные сохраняются	`make down` или `docker compose down`
Запустить снова	`make up` или `docker compose up -d --wait`
Удалить все данные (база, ключи)	`make reset` или `docker compose down -v`
Обновить код и пересобрать	`git pull`, затем `make demo`
Журнал API	`make logs` или `docker compose logs -f backend`
Консоль базы	`docker compose exec postgres psql -U fsp fsp_talent`

8. Что входит в стенд

Контейнер	Образ	Назначение
`nginx`	nginx:1.28-alpine	Единая точка входа: сайт, `/api`, `/docs`, заголовки безопасности (CSP)
`frontend`	собирается из `frontend/` (node:20 → nginx)	React-приложение для обеих ролей
`backend`	собирается из `backend/` (python:3.12-slim)	FastAPI, миграции Alembic при старте, 2 процесса uvicorn
`worker`	тот же образ	Celery: пересчёт профилей, истечение приглашений, задача недели, удаление данных по 152-ФЗ
`beat`	тот же образ	Расписание задач Celery
`sandbox`	собирается из `sandbox/` (python:3.12-slim и node)	Запуск кода из задач без сети, корень только для чтения, связь с API через Unix-сокет
`postgres`	pgvector/pgvector:pg16	PostgreSQL 16 с pgvector и pg_trgm
`redis`	redis:7.2-alpine	Лимиты запросов, состояние входа через ФСП ID, брокер Celery
`mailpit`	axllent/mailpit:v1.31.4	Почтовый ящик для писем на локальном стенде

Наружу открыт только порт nginx. API (8010), отдельный frontend (3010) и Mailpit (8025) слушают только 127.0.0.1. У базы и Redis портов на хосте нет.

9. Настройки `.env`

Для демо `.env` можно не трогать. Основные параметры:

Переменная	По умолчанию	Зачем
`SECRET_KEY`	заглушка	Ключ подписи токенов. Заглушку контейнер заменит случайным ключом. На публичном стенде задайте свой: `openssl rand -hex 32`
`HTTP_PORT`	80	Порт сайта на хосте. Если 80 занят: `HTTP_PORT=8081` и `PUBLIC_BASE_URL=http://localhost:8081`
`PUBLIC_BASE_URL`	http://localhost	Адрес, из которого строятся ссылки в письмах
`FSP_MODE`	mock	`mock` — тестовый реестр ФСП, `real` — HTTP-клиент по контракту из FSP_INTEGRATION.md
`FSP_ID_MODE`	mock в демо-сборке	В Compose и `.env` демо стоит `mock`: страница провайдера на этом сайте. Без переменной код берёт `oidc`. Мок при выключенном `DEMO_MODE` процесс не запускает. `oidc` — Keycloak Федерации (`FSP_ID_ISSUER`, `FSP_ID_CLIENT_ID`, `FSP_ID_CLIENT_SECRET`)
`AI_ENABLED`, `GIGACHAT_CREDENTIALS`	true, пусто	Без ключа модели резюме разбирается правилами. С ключом модель добавляет навыки с дословной цитатой, пишет черновики заданий для модератора и мнение по карточке; балл и порядок она не меняет
`YANDEX_API_KEY`, `YANDEX_FOLDER_ID`, `YANDEX_MODEL`, `YANDEX_CHECK_MODEL`	пусто, пусто, yandexgpt-5.1/latest (URI не сверен с официальной страницей, см. SOLUTION.md), в коде пусто, в `.env.example` — `yandexgpt-5-lite/latest`	YandexGPT вместо GigaChat, если `GIGACHAT_CREDENTIALS` пуст. Основная модель — YandexGPT Pro 5.1. Псевдоним `yandexgpt/latest` — это Pro 5, его не используют. Ключ сервисного аккаунта с ролью `ai.languageModels.user`; запросы уходят с `x-data-logging-enabled: false`. Для черновиков заданий нужна вторая модель, `yandexgpt-5-lite/latest`
`EMBEDDINGS_MODE`	off	`model` включает смысловой поиск по векторным представлениям текста (эмбеддингам). В сборке по умолчанию библиотека в образ не ставится: так образ остаётся лёгким и работает без видеокарты. Ставится аргументом сборки `INSTALL_RERANKER=1`: `make demo GPU=1` или `INSTALL_RERANKER=1 docker compose up -d --build`. По умолчанию выключено, образ backend без torch
`CROSS_ENCODER_MODE`, `CROSS_ENCODER_MODEL`, `CROSS_ENCODER_DEVICE`	off, пусто, auto	Локальная модель пары `BAAI/bge-reranker-v2-m3` переставляет верхние 50 фактора «Запрос». Веса лежат в `models/bge-reranker-v2-m3` и в репозиторий не входят. Устройство: `auto`, `cpu` или `gpu`. Пока режима нет или папки нет, порядок прежний. BM25 — формула без файла весов и в выдачу не входит. Навыки, грейд и нижняя граница ФСП не меняются
`MAIL_MODE`, `SMTP_*`	smtp → Mailpit	Настоящий SMTP на стенде
`MAIL_FILE_DIR`, `SMTP_STARTTLS`, `EMAIL_CODE_TTL_MINUTES`	/tmp/fsp-talent-mail, false, 30	Папка писем при `MAIL_MODE=file` (её читают CI и smoke, у параллельных прогонов своя папка); STARTTLS для SMTP; срок кода подтверждения в минутах
`ACCESS_TOKEN_EXPIRE_MINUTES`, `REFRESH_TOKEN_EXPIRE_DAYS`, `ALGORITHM`	60, 30, HS256	Срок access-токена и refresh-cookie, алгоритм подписи
`SECRET_KEY_FILE`	/secrets/secret_key	Файл случайного ключа подписи, если `SECRET_KEY` оставлен заглушкой: первый процесс создаёт ключ, остальные читают тот же файл
`MIGRATE_DATABASE_URL`, `APP_DB_PASSWORD`	пусто, пусто	Адрес суперпользователя только для миграций и пароль роли `fsp_app` для API, worker и beat. Пусто — локальный запуск под одной ролью
`SANDBOX_SOCKET`, `SANDBOX_REQUIRED`	пусто, true	Сокет контейнера песочницы. Без сокета код кандидата запускается только при явном `SANDBOX_REQUIRED=false` (тесты, ноутбук)
`DEMO_PROTECTED_SUFFIXES`	@demo.talent,.demo	Окончания адресов, которые на демо-стенде не закрываются целиком после серии неверных паролей: этими входами пользуются все посетители демо (`services/login_lock.py:51`). Смену пароля и удаление у опубликованных входов закрывает их список в `core/demo_guard.py`
`EMBEDDING_MODEL`, `EMBEDDING_DIM`	deepvk/USER-bge-m3, 1024	Модель векторов при `EMBEDDINGS_MODE=model` и размерность столбца pgvector
`GRADE_MODEL`	legacy	`legacy`: заявленный грейд весит 0,85 от подтверждённого; `expected`: ожидаемая релевантность по апостериорному распределению грейда

10. Запуск без Docker (для разработки)

Нужны Python 3.12, Node.js 20+, PostgreSQL 16 с расширением pgvector и Redis 7.

```
# backend
cd backend
python -m venv .venv && source .venv/bin/activate        # Windows: .venv\Scripts\activate
pip install -c requirements.lock -e ".[test]"
export DATABASE_URL=postgresql+asyncpg://fsp:fsp@localhost:5432/fsp_talent
export REDIS_URL=redis://localhost:6379/0
export SECRET_KEY=$(python -c "import secrets; print(secrets.token_hex(32))")
export MAIL_MODE=console VC_PRIVATE_KEY_PATH=./vc_ed25519.pem
alembic upgrade head
python -m app.core.schema_sync   # старая база получает новые таблицы и колонки, удалять ничего не нужно
python -m seed.generate_seed
uvicorn app.main:app --reload --port 8000

# frontend, в другом терминале
cd frontend
npm ci
npm run dev        # http://localhost:5173, запросы /api уходят на :8000
```

Коды подтверждения при `MAIL_MODE=console` печатаются в журнал uvicorn.

11. Развёртывание на сервере

Продуктивная сборка — `deploy/docker-compose.prod.yml`. Отличия от локальной: вход — Caddy 2.10 с автоматическим сертификатом Let's Encrypt вместо nginx, API, worker и beat входят в базу ролью `fsp_app`, Mailpit поднимается только с профилем `mailpit`, добавлен контейнер `backup`. Наружу открыты только 80 и 443 у Caddy. У API, базы и Redis портов на хосте нет.

> Docker-команды раздела сверены с `deploy/docker-compose.prod.yml`, `deploy/caddy/Caddyfile`, `deploy/backup.sh`, `deploy/restore_db.sh` и шагом `deploy` в `.github/workflows/ci.yml`. Именно этими командами обновляется серверная установка.

11.1. Сервер

Что	Минимум
ОС	Linux с Docker Engine 24+ и Compose v2
Процессор и память	1 vCPU и 4 ГБ. Публичный демо-стенд проекта работал на 8 ГБ, фоновый обработчик задач (worker) — один процесс
Диск	10 ГБ: образы, база, 14 ночных копий
Сеть	Открытые 80 и 443. Без своего домена подходит адрес вида `203-0-113-10.sslip.io`: sslip.io отвечает этим IP на такое имя
Почта	SMTP-ящик для кодов подтверждения или профиль `mailpit` для закрытого показа

Видеокарта не нужна. Образ backend по умолчанию не ставит sentence-transformers и torch (`INSTALL_RERANKER` выключен), `EMBEDDINGS_MODE` и `CROSS_ENCODER_MODE` — `off`.

11.2. Первый запуск

```
sudo mkdir -p /opt/fsp-talent && sudo chown "$USER" /opt/fsp-talent
git clone https://github.com/KonkovDV/fsp-talent.git /opt/fsp-talent
cd /opt/fsp-talent
cp deploy/.env.example deploy/.env
chmod 600 deploy/.env
openssl rand -hex 32   # SECRET_KEY, APP_DB_PASSWORD
openssl rand -hex 16   # POSTGRES_PASSWORD, SANDBOX_TOKEN
```

Заполните `deploy/.env` (таблица 11.3) и запустите:

```
GIT_SHA=$(git rev-parse HEAD) docker compose -f deploy/docker-compose.prod.yml --env-file deploy/.env up -d --build
docker compose -f deploy/docker-compose.prod.yml --env-file deploy/.env ps
```

При старте backend применяет миграции Alembic от владельца базы, досоздаёт новые таблицы и колонки (`app.core.schema_sync`), выставляет пароль роли `fsp_app` и только потом запускает uvicorn.

Демо-данные загружаются от пользователя `appuser`. Загрузка от root создаёт ключ подписи удостоверений, который приложение потом не прочитает, и проверка удостоверения отвечает 500.

```
docker compose -f deploy/docker-compose.prod.yml --env-file deploy/.env exec -T -u appuser backend python -m
seed.generate_seed
```

Проверка:

```
curl -s https://$SITE_ADDRESS/health          # "status":"ok" и "build" — коммит из GIT_SHA
SMOKE_BASE=https://$SITE_ADDRESS/api/v1 SMOKE_MAIL=demo python backend/tests/smoke_api.py
```

Второй командой быстрая проверка (smoke) смотрит стенд снаружи без доступа к почте, если включены `DEMO_MODE` и `DEMO_SHOW_CODE`. Он берёт код из ответа регистрации, проверяет, что `GET /public/issuer` отвечает 200, и пропускает шаги, которым нужны другие письма (сброс пароля, согласие законного представителя). Нужен только Python 3.12, сторонних пакетов скрипт не использует.

11.3. Файл `deploy/.env`

Обязательные ключи. Без первых трёх compose не стартует: в файле сборки стоит `${…:?}`.

Ключ	Что записать
`SITE_ADDRESS`	Домен сайта, например `203-0-113-10.sslip.io`. На это имя Caddy получает сертификат
`POSTGRES_PASSWORD`	Пароль владельца базы `fsp`: `openssl rand -hex 16`
`APP_DB_PASSWORD`	Пароль роли `fsp_app`, под которой работают API, worker и beat: `openssl rand -hex 32`. У роли нет прав суперпользователя, строки закрыты политиками строк
`SECRET_KEY`	Ключ подписи токенов: `openssl rand -hex 32`. Если оставить пустым, backend создаст случайный ключ в томе `vc_keys`; на сервере лучше задать явно
`SANDBOX_TOKEN`	Общий ключ API и песочницы кода: `openssl rand -hex 16`. Без него sandbox не запускается
`MAIL_FROM`, `SMTP_HOST`, `SMTP_PORT`, `SMTP_SSL`, `SMTP_USER`, `SMTP_PASSWORD`	Любой SMTP: Яндекс 360, Mail.ru для бизнеса, Unisender Go и другие. `MAIL_MODE=smtp`

Необязательные ключи:

Ключ	По умолчанию в примере	Зачем
`ACME_EMAIL`	пусто	Почта для напоминаний Let's Encrypt об окончании сертификата
`FIELD_ENCRYPTION_KEY`	не задан	Отдельный ключ шифрования контактов. Без него шифр берёт `SECRET_KEY`
`DEMO_MODE`	true	`true` — аккаунты `jury1@demo.talent` … `jury10@demo.talent`, пробный тест, демо-провайдер ФСП ID. Для настоящих пользователей `false`
`DEMO_SHOW_CODE`	false	Только при `DEMO_MODE=true`: код подтверждения показывается на экране тому, кто регистрируется. Для демо-стенда без почтовой учётки
`DEMO_RESET_CRON`	`10 4 * * *`	Ночной возврат демо-стенда к исходному виду, время московское. Пустая строка отключает сброс на время показа
`FSP_MODE`	mock	`mock` — демо-реестр протоколов ФСП, `real` — HTTP-клиент по контракту FSP_INTEGRATION.md
`FSP_ID_MODE`	mock	`mock` работает только при `DEMO_MODE=true`. Для настоящих пользователей — `oidc` с `FSP_ID_ISSUER`, `FSP_ID_CLIENT_ID`, `FSP_ID_CLIENT_SECRET` от Keycloak Федерации
`AI_ENABLED` и ключи модели	false	Включает языковую модель. Решение принимает оператор: меняется обработка ПДн. Ключи остаются только в `deploy/.env`

> В `deploy/.env.example` `DEMO_MODE=true` задан один раз, рядом с `DEMO_SHOW_CODE`: пример описывает демо-стенд с демо-провайдером ФСП ID (`FSP_ID_MODE=mock`), а мок без демо-режима backend не запускает. Для настоящих пользователей ставят `DEMO_MODE=false` и `FSP_ID_MODE=oidc`. Если ключ в файле повторяется, compose берёт последнее значение.

`deploy/.env` правит только оператор. Шаг выкладки в CI его не меняет.

11.4. HTTPS и Caddy

Конфиг — каталог `deploy/caddy` (`Caddyfile`). Caddy сам получает и продлевает сертификат для `SITE_ADDRESS`, нужен только открытый порт 80 для проверки Let's Encrypt. Он отдаёт статику frontend, проксирует `/api/*`, `/health`, `/docs`, `/openapi.json` и `/.well-known/did.json` на backend и выставляет заголовки: HSTS, CSP, `X-Frame-Options`, `Permissions-Policy` (камера, микрофон и геолокация закрыты). Тело больше 10 МБ получает ответ 413. Пути `/.env` и `/.git` отвечают 404.

Сертификаты лежат в томах `caddy_data` и `caddy_config`. Эти тома при обновлении не удаляют, иначе Caddy заново запросит сертификат и может упереться в лимит Let's Encrypt.

11.5. Почта

Есть SMTP-ящик — заполните `SMTP_*` и `MAIL_FROM`. Нет ящика — два варианта:

- закрытый показ: `--profile mailpit` в команде `up`, письма видны в Mailpit на `127.0.0.1:8025` сервера (через SSH-туннель);
- демо-стенд: `DEMO_MODE=true` и `DEMO_SHOW_CODE=true`, код регистрации показывается на экране подтверждения. Код сброса пароля на экран не выводится: «Забыли пароль?» сообщает, что письма со стенда не отправляются.

11.6. Резервные копии и восстановление

Контейнер `backup` каждую ночь в 03:30 по часам контейнера (часовой пояс в нём не задан, это UTC — 06:30 МСК) делает `pg_dump -Fc` в `deploy/backups/` и хранит 14 последних файлов с правами `600`.

Действие	Команда
Копия вручную	`docker compose -f deploy/docker-compose.prod.yml --env-file deploy/.env exec backup sh /backup.sh`
Проверить файл без записи в базу	`python3 deploy/verify_backup.py deploy/backups/fsp_talent-YYYYMMDD-HHMM.dump`
Восстановить	см. ниже

Восстановление заменяет данные в базе. Скрипт сначала проверяет файл тем же `verify_backup.py` и не запускается без `CONFIRM=restore`. Тома Docker он не удаляет.

```
CONFIRM=restore ENV_FILE=deploy/.env COMPOSE_FILE=deploy/docker-compose.prod.yml \
  sh deploy/restore_db.sh deploy/backups/fsp_talent-YYYYMMDD-HHMM.dump
```

В дампе есть персональные данные кандидатов: профили, резюме, согласия. Ключ приложения шифрует только часть полей, например контакты. Поэтому копию, которая уходит с сервера, шифруют, а ключ хранят отдельно от неё и от `deploy/.env`. Пример с age (открытый ключ получателя, закрытый на сервере не лежит):

```
age -r age1… -o fsp_talent-YYYYMMDD-HHMM.dump.age deploy/backups/fsp_talent-YYYYMMDD-HHMM.dump
# расшифровка на машине с закрытым ключом перед проверкой и восстановлением:
age -d -i key.txt -o fsp_talent-YYYYMMDD-HHMM.dump fsp_talent-YYYYMMDD-HHMM.dump.age
```

То же через GnuPG: `gpg --encrypt --recipient <ключ> файл.dump` (или `gpg --symmetric --cipher-algo AES256 файл.dump` с паролем). `verify_backup.py` и `restore_db.sh` работают с расшифрованным `.dump`. Сам контейнер `backup` пишет файлы без шифрования, с правами `600`, в `deploy/backups/` на сервере. `deploy/.env` с `SECRET_KEY` и `FIELD_ENCRYPTION_KEY` хранят отдельно от копий: без этих ключей контакты из дампа не прочитать.

11.7. Обновление

```
cd /opt/fsp-talent
docker compose -f deploy/docker-compose.prod.yml --env-file deploy/.env exec backup sh /backup.sh
git pull origin main
GIT_SHA=$(git rev-parse HEAD) docker compose -f deploy/docker-compose.prod.yml --env-file deploy/.env up -d --build
curl -s https://$SITE_ADDRESS/health
```

Этими же командами обновляет стенд шаг `deploy` в `.github/workflows/ci.yml`, без ручной копии перед `git pull`. Он запускается после проверок стиля, тестов backend и frontend, сквозного сценария и сборки образов. Если у репозитория стоит переменная `DEPLOY_FROZEN=true` или не задан `DEPLOY_HOST`, шаг пропускается. `DEPLOY_FROZEN=true` фиксирует код на сервере.

При обновлении не меняются:

- `SECRET_KEY` и `FIELD_ENCRYPTION_KEY` — от них зависит шифр уже записанных контактов;
- том `vc_keys` — в нём ключ подписи удостоверений, без него выданные подписи не проверить;
- тома `postgres_data`, `caddy_data`, `caddy_config`.

Поэтому `down -v` на сервере не выполняют.

11.8. Откат

```
cd /opt/fsp-talent
git log --oneline -n 10                # найти предыдущий коммит или тег
git checkout <коммит или тег>
GIT_SHA=$(git rev-parse HEAD) docker compose -f deploy/docker-compose.prod.yml --env-file deploy/.env up -d --build
curl -s https://$SITE_ADDRESS/health    # "build" — коммит отката
```

Схема базы при откате кода не возвращается: миграции и `schema_sync` только добавляют таблицы и колонки, предыдущая версия кода с лишними колонками работает. Если выпуск менял сами данные, после отката восстанавливают копию, снятую перед обновлением (раздел 11.6). Вернуться на основную ветку: `git checkout main` и та же команда `up`.

11.9. Демо-стенд

- Вернуть показ к исходному виду: `docker compose -f deploy/docker-compose.prod.yml --env-file deploy/.env exec -T -u appuser backend python -m seed.reset_demo`. После сброса у `jury1@demo.talent` … `jury10@demo.talent` снова пустой опрос, пароль `demo12345`.
- Ночной сброс делает beat по `DEMO_RESET_CRON`.
- Кнопка «Взять свободный аккаунт жюри» на странице входа берёт адрес из `GET /api/v1/public/jury-free`.
- Языковая модель на стенде: оператор вручную задаёт в `deploy/.env` `AI_ENABLED=true`, `YANDEX_MODEL=yandexgpt-5.1/latest`, `YANDEX_CHECK_MODEL=yandexgpt-5-lite/latest`, ключ и каталог, затем выполняет `up -d`.

11.10. Вход через ФСП ID

Для настоящих пользователей `FSP_ID_MODE=oidc` и данные клиента Keycloak Федерации. Проверить вход через настоящий OIDC-провайдер можно локально: `docker compose --profile fsp-id up -d` поднимает Keycloak 26.0.7 с демо-реалмом из `deploy/keycloak`, в `.env` ставится `FSP_ID_MODE=oidc`. Без профиля Keycloak не запускается, вход идёт через встроенного демо-провайдера.

Презентация лежит в `submission/03-presentation/` (`FSP_Talent_presentation.pdf` и `.pptx`). `docs/pitch.pdf`, `docs/pitch.pptx` и `make docs-pitch` — старый черновик, собранный скриптом; его цифры частично устарели.

12. Если что-то пошло не так

Симптом	Причина и решение
`port is already allocated`	Порт занят. Задайте в `.env` (или в окружении) `HTTP_PORT=8081` и `PUBLIC_BASE_URL=http://localhost:8081`; для почты, API и фронтенда — `MAIL_PORT`, `API_PORT`, `FRONTEND_PORT`. Затем повторите запуск
`unknown flag: --wait` или ошибка разбора `env_file`	Compose старше 2.24. Обновите Docker Desktop (4.27+) или плагин compose
Сборка падает на `apt-get` с `500 Internal Server Error`	Временный сбой зеркала Debian. Сборка повторяет попытку сама; если не помогло, запустите команду ещё раз
`demo-seed` завершился с ошибкой	`docker compose logs demo-seed`; повторный `docker compose up -d --wait` безопасен
`backend` не становится `healthy`	Посмотрите `docker compose logs backend`. Если там `SECRET_KEY is a published placeholder`, том `vc_keys` недоступен на запись: задайте `SECRET_KEY` в `.env`
Вход не работает, `503`	Redis не запущен: `docker compose ps redis`, затем `docker compose up -d redis`
Письмо с кодом не пришло	На локальном стенде откройте Mailpit http://localhost:8025. Код можно запросить повторно через минуту. На демо-стенде с `MAIL_MODE=console` письма не уходят: код регистрации показывается на экране, код сброса пароля — нет
Проверка удостоверения отвечает 500	Ключ в томе создан от root. Демо-данные загружают от `appuser` (раздел 11.2). Перезапуск backend отдаёт уже лежащий ключ пользователю `appuser`
Caddy не получает сертификат	Порт 80 закрыт или `SITE_ADDRESS` не указывает на этот сервер. Проверьте `docker compose -f deploy/docker-compose.prod.yml --env-file deploy/.env logs caddy`
`required variable … is missing a value`	В `deploy/.env` не задан `SITE_ADDRESS`, `POSTGRES_PASSWORD` или `APP_DB_PASSWORD`
sandbox перезапускается с `SANDBOX_TOKEN is required`	Задайте `SANDBOX_TOKEN` в `deploy/.env`
После `git pull` сайт показывает старую версию	Пересоберите образы: `make demo` (или `docker compose up -d --build --wait`)
Нужен чистый стенд	`make reset`, затем `make demo`
«Вход закрыт на 15 минут после нескольких неверных паролей» у демо-аккаунта	Кто-то пять раз ввёл неверный пароль к этому адресу. Войдите под другим: `jury2@demo.talent` … `jury10@demo.talent` или кнопка «Взять свободный аккаунт жюри»

Симптом	Причина и решение
Сборка падает на `npm ci` или `pip install`	Нет доступа к npm или PyPI. Проверьте прокси Docker Desktop (Settings → Resources → Proxies)

Руководство пользователя стенда

Здесь мы проходим по платформе за каждую из четырёх ролей: кандидат, работодатель, проверяющий и законный представитель несовершеннолетнего кандидата. Маршруты взяты из `frontend/src/App.tsx`, названия разделов — из меню кабинетов (`frontend/src/components/Layout.tsx`).

Локальная установка — http://localhost (запуск: INSTALL.md). Ниже пути даны от корня сайта: `/login` на локальном стенде — http://localhost/login. Пароль всех демо-аккаунтов — `demo12345`, список аккаунтов — README.

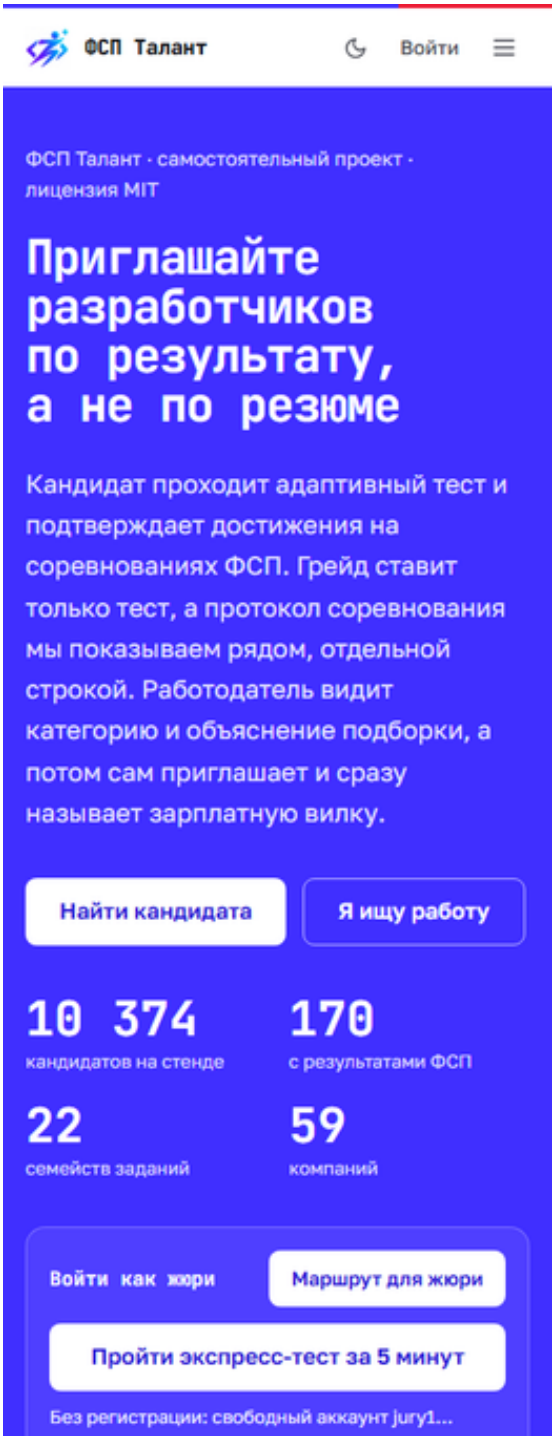
Карта страниц

Путь	Кто	Что там
`/`	все	Главная: что делает платформа, вход для обеих ролей
`/login`, `/register`, `/verify`, `/forgot`	все	Вход, регистрация, ввод кода из письма, сброс пароля
`/fsp-id/authorize`	кандидат	Страница провайдера ФСП ID на демо-стенде: номер участника и пароль
`/jury`	без входа	«Режим жюри»: быстрый вход в демо-аккаунты, методика, проверка удостоверения, подбор на своей разметке
`/method`	без входа	Как устроен тест, цифры проверки с командами
`/evaluate`	без входа	Расчёт nDCG@10 подбора на своих парах «потребность — кандидат»
`/badge`	без входа	Проверка подписи удостоверения ФСП (W3C VC 2.0)
`/leaderboard`	без входа	Порядок внутри категории по подтверждающему тесту, без имён: сначала вероятность уровня P(pass), при равенстве — оценка θ (поле `theta` в ответе API)
`/passport/:id`	без входа	Паспорт навыков кандидата по ссылке или QR-коду
`/p/:id`	без входа	Публичная страница кандидата, если он дал согласие на распространение
`/companies/:id`	без входа	Публичная страница компании: название, открытые вакансии, уровень доверия
`/guardian`	законный представитель	Подтверждение согласия на обработку данных кандидата 15–17 лет
`/legal`	все	Тексты согласий и правил
`/candidate/…`	кандидат	Обзор, Профиль, Тест на грейд, Задания, Приглашения, Вакансии, Результаты ФСП
`/employer/…`	работодатель	Обзор, Потребности, Поиск, Избранное, Задания, Приглашения, Вакансии, Компания, Интеграции, Модерация, Методист

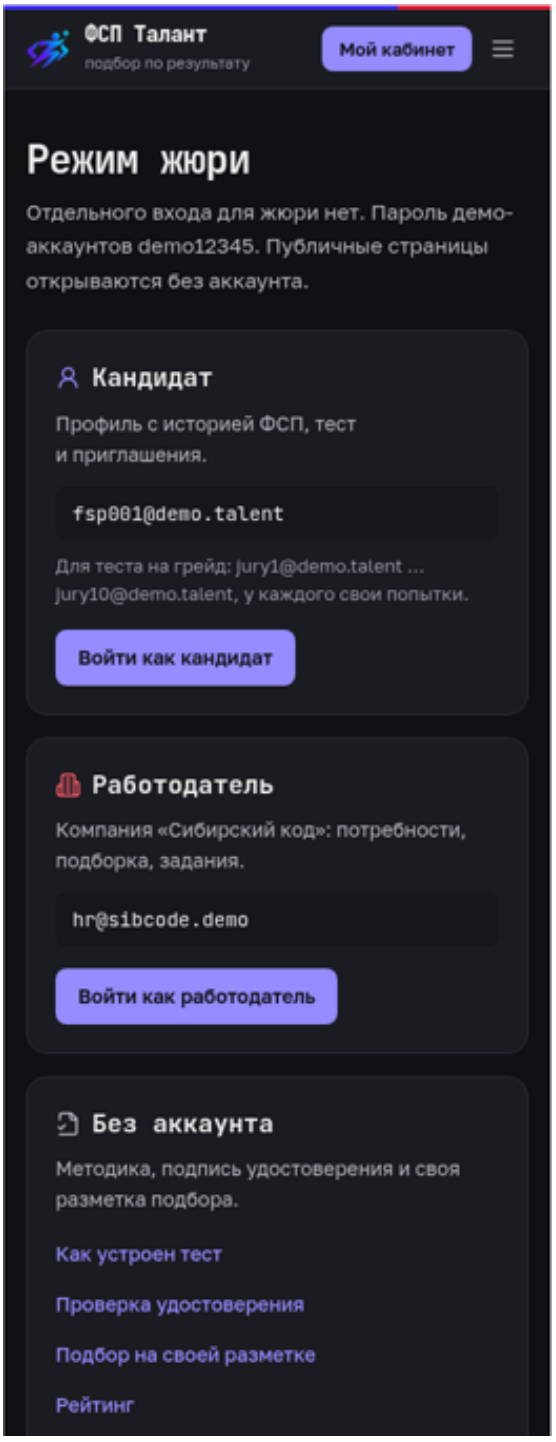
Путь за 12 экранов

Весь обратный найм — от первого входа до принятого приглашения — на двенадцати экранах стенда.

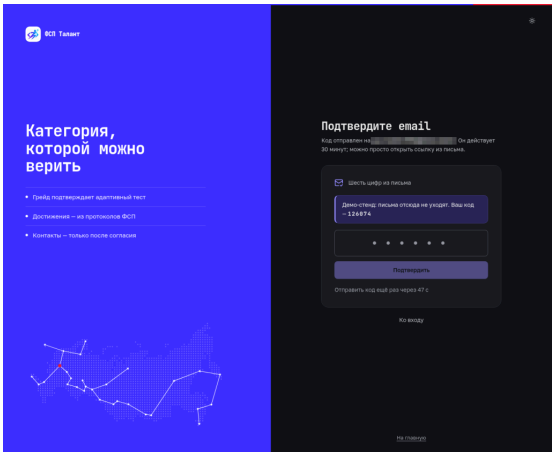
1. **Главная: обратный найм и демо-вход**



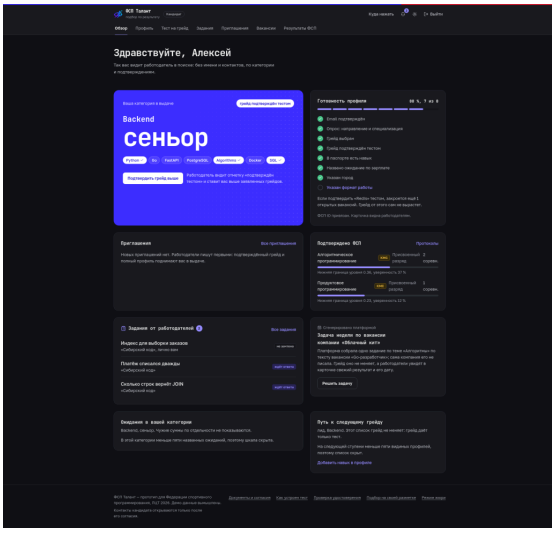
2. **«Режим жюри»: методика, проверка удостоверения, расчёт подбора**



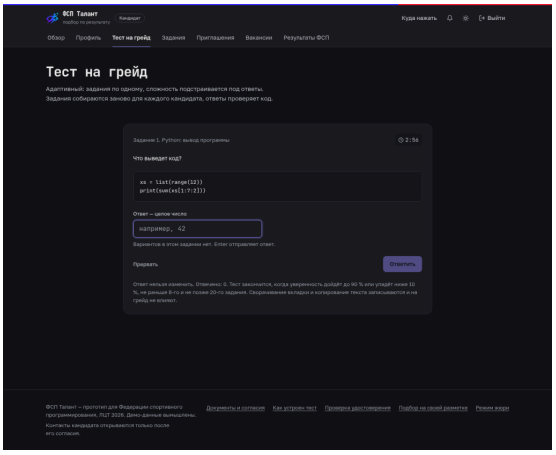
3. **Регистрация по почте с кодом подтверждения**



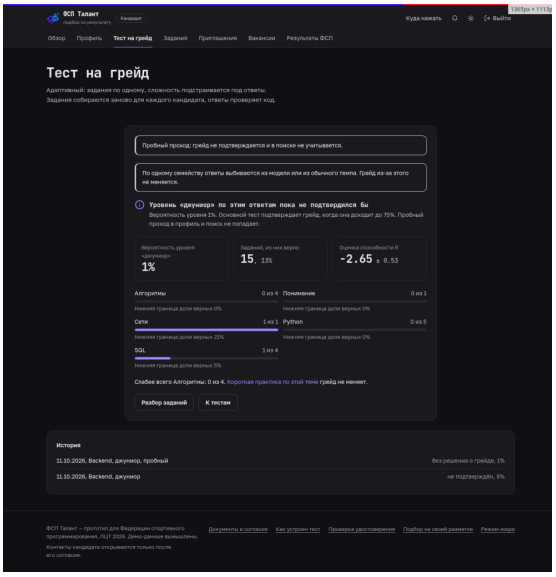
4. **Кабинет кандидата: категория, подтверждения и готовность профиля** (демо-аккаунт `fsp001@demo.talent`)



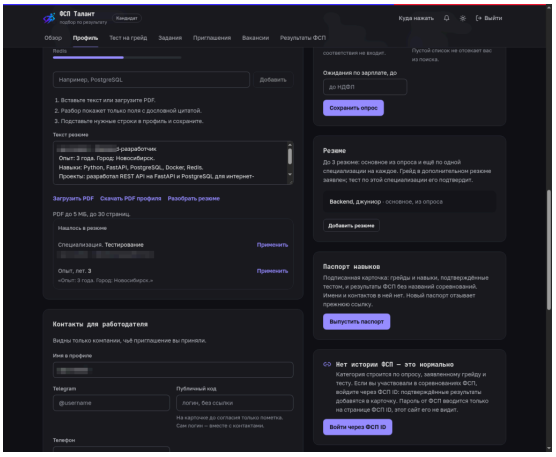
5. **Задание адаптивного теста: свои числа и код у каждого**



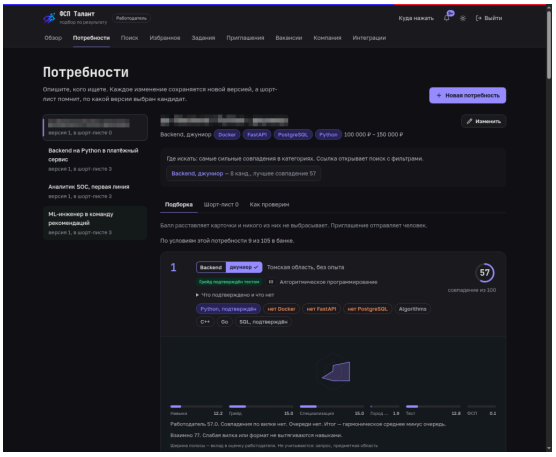
6. **Итог пробного теста: темы и вероятность грейда**



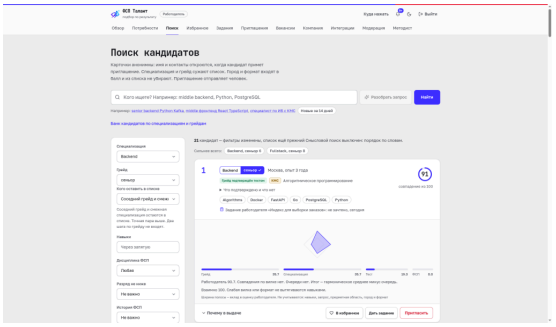
7. **Резюме PDF: навыки предлагаются с цитатой**



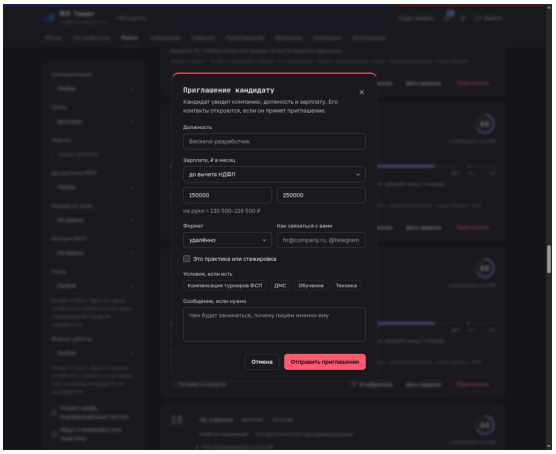
8. **Работодатель: потребность создана**



9. **Подборка: место, категория, подтверждения и объяснение**

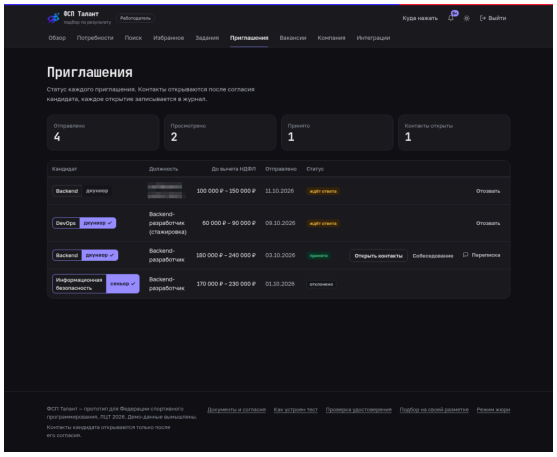


10. **Приглашение с вилкой «от–до»**



11. **Кандидат принимает приглашение — контакты открываются**

12. **Работодатель видит статусы приглашений**



Кандидат

Вход

- **По почте.** `/register` → роль «кандидат», email, пароль, отметка «Мне исполнилось 18 лет» или «Мне 15, 16 или 17 лет», согласия. На `/verify` вводится код из письма. Локально письмо лежит в Mailpit (http://localhost:8025). На стенде без почтовой учётки код показывается на экране подтверждения.
- **Через ФСП ID.** `/login` → «Войти через ФСП ID». Пароль вводится на странице провайдера, платформа получает проверенный ID-токен (OpenID Connect с PKCE). На демо-стенде провайдер — страница `/fsp-id/authorize`: `FSP001` / `test` входит в привязанный аккаунт `fsp001@demo.talent`, свободный номер из `FSP2000` … `FSP2099` / `test` после экрана согласий создаёт новый аккаунт из демо-реестра.
- **Готовые аккаунты.** `fsp001@demo.talent` (есть история ФСП и приглашения), `fsp002@demo.talent` (информационная безопасность), `nofsp@demo.talent` (без истории ФСП). Для прохождения с нуля — `jury1@demo.talent` … `jury10@demo.talent`.

Разделы кабинета

Раздел	Путь	Что делать
Обзор	`/candidate`	Опрос: направление и специализация, грейд, цель (работа или стажировка), предметная область по желанию. Здесь же категория, готовность профиля, задача недели, ожидающие приглашения и подсказка, какой навык откроет больше вакансий. Подсказка появляется после того, как карточку открыли три разные компании за 90 дней. Более узкий подтверждённый инструмент, который уже закрывает общий навык, в этот пробел не попадает. «Спрос на профиль» считает компании, которые открывали карточку за 90 дней, без имён, и середину вилок приглашений в категории, если таких приглашений не меньше пяти. Цифры место в подборке не двигают
Профиль	`/candidate/profile`	Профиль вручную или из PDF-резюме: каждое поле предлагается с цитатой и переносится по кнопке. Контакты для работодателя, мягкие навыки, «Куда не предлагать», статус поиска работы. Блок «История опроса»: каждое сохранение опроса, новые сверху. «Скачать PDF профиля». Согласия и их отзыв, видимость, привязка ФСП ID, «Паспорт навыков» → «Выпустить паспорт», выгрузка своих данных, смена пароля, удаление аккаунта
Тест на грейд	`/candidate/assessment`	Выбор уровня и языка кода (Python, JavaScript, Go, Java, Kotlin, C#) → «Начать тест». 8–20 заданий, ответ вводится. В конце — итог по темам и подтверждённый грейд. Не сдали — тест на уровень ниже доступен сразу; пересдача того же уровня — через 7 дней. На демо-стенде есть «Пробный тест» на 15 заданий и 5 минут, грейд он не подтверждает. Ниже — практическая проверка QA: до пяти пар «вход — ожидание» для функции с границами 0 и 100, сверка с исправной программой и с копиями, в которые внесены дефекты. Грейд от неё не меняется. Там же макет: пометка по контрасту и положению фигур. Копирование условия кладёт в буфер текст с одним другим числом; ответ этим числом остаётся пометкой. Грейд от макета и от этой пометки не меняется. Кейс по отрасли и уточняющий вопрос считаются одной рубрикой и грейд не меняют
Задания	`/candidate/tasks`	Задания работодателей: общие для специализации и адресные. С эталоном ответ проверяет система, открытое задание — работодатель. Грейд задания не меняют
Приглашения	`/candidate/offers`	Компания, вакансия, вилка до вычета НДФЛ и страница компании видны до решения. «Принять» открывает компании имя и контакты, затем доступны «Переписка» и выбор времени собеседования из одного–трёх, предложенных компанией (время московское). «Да, если…» оставляет приглашение в ожидании: контакты закрыты, место в подборке не меняется. «Закрыть контакты для этой компании» отзывает доступ одной компании, остальные не затрагиваются. Можно пожаловаться на приглашение
Вакансии	`/candidate/vacancies`	Вакансии под профиль, «Откликнуться», свои отклики и их статусы. Пока грейд не подтверждён тестом, видны все открытые вакансии, новые сверху; на вакансию с грейдом кнопка ведёт на тест («Сначала тест на грейд»). Отклик сам по себе даёт согласие на контакт с этой компанией

Раздел	Путь	Что делать
Результаты ФСП	`/candidate/results`	Строки протоколов из реестра ФСП и присвоенные разряды. Это отдельная ось профиля: она поднимает кандидата внутри категории, грейд не меняет. Нет истории — ничего не теряется. Файл протокола сначала показывается списком и крепится только к привязанному ФСП ID. Краткое удостоверение подписывается отдельно для каждой выдачи и не содержит идентификатор человека

Работодатель

Вход

- **Регистрация.** `/register` → роль «работодатель»: рабочий email, пароль, название компании, ИНН по желанию (проверяется контрольная сумма). Код подтверждения — как у кандидата.
- **Готовый аккаунт.** `hr@sibcode.demo` — компания «Сибирский код» с тремя потребностями, приглашениями во всех статусах, вакансиями с откликами и заданиями.

Разделы кабинета

Раздел	Путь	Что делать
Обзор	`/employer`	Сводка: потребности, приглашения, отклики, что ждёт решения
Потребности	`/employer/needs`	Создать потребность: специализация, грейд, обязательные и желательные навыки, предметная область, чем занимается команда, вилка, формат, город. Вставленный текст вакансии заполняет черновик. Дальше: «Где искать» (рекомендованные категории), подборка с полосами факторов и «Почему в выдаче», «Сравнить» до четырёх кандидатов без имён, вкладка «Как проверим» (каким получится тест), шорт-лист, версии потребности, «Пригласить». «Ссылка руководителю» отдаёт первые десять карточек без имён и контактов на семь дней; отметки «да / нет / позже» видны компании и место в подборке не двигают
Поиск	`/employer/search`	Поиск по всему банку: специализация, грейд, стек, «Только грейд, подтверждённый тестом», дисциплина и «Разряд не ниже». Витрина «специализация × грейд» с числом кандидатов в ячейке
Избранное	`/employer/favorites`	Отложенные карточки кандидатов
Задания	`/employer/tasks`	Своё задание для всех кандидатов специализации или «Дать задание» одному из карточки. С эталоном проверяет система, открытое проверяет работодатель: «Зачтено» или «Не зачтено» с комментарием. Вердикт появляется строкой в анонимной карточке
Приглашения	`/employer/offers`	Статусы: отправлено, просмотрено, принято, отклонено, отозвано, истекло. У принятых — имя, контакты, «Переписка», «Собеседование». Если кандидат закрыл контакты, компания видит `contacts_revoked`, переписка только для чтения. «Да, если…» само контакты не открывает. «Принять условие» открывает их, когда компания согласна с написанным условием. «Ваши отказы» считают причины ответов на приглашения этой компании. Рыночные доли по-прежнему скрыты, пока ответов меньше пяти
Вакансии	`/employer/vacancies`	Вакансии с вилкой «от–до», отклики и смена их статуса, «Показать контакты» по отклику
Компания	`/employer/company`	Профиль компании, ИНН, предпросмотр приглашения глазами кандидата, выгрузка данных, смена пароля
Интеграции	`/employer/integrations`	События для ATS (подписанные вебхуки) и журнал доставки; концепция — TRUST_AND_ATS.md
Модерация, Методист	`/employer/moderation`, `/employer/methodist`	Только для сотрудников с правами методиста. Права выдаются с консоли сервера (`python -m app.services.staff`), опубликованным демо-логинам они не даются: жалобы и пауза компании, статистика банка, карантин ключей заданий, черновики заданий. Вход на пульт модерации закрывается вторым фактором (TOTP)

Приглашение не отправить без вилки «от–до» и способа связи. Лимит — до 50 приглашений в сутки после проверки ИНН, до проверки — 10.

Если подборка пустая

Так бывает, когда в базе нет никого, кто подходит под обязательные навыки, грейд и вилку. Пустой список не прячется за общей ошибкой: работодатель видит, что совпадений нет, и получает подсказки, что поменять.

- В подборке по потребности появляется «Совпадений нет» с советом убрать часть обязательных навыков или грейд (`frontend/src/pages/employer/Needs.tsx`).
- Если кандидатов держит вилка, система считает, сколько людей добавится, если поднять её верх, и называет сумму. Если держит навык, называет, сколько добавится без него (`pay_hint` и `skill_hint`, `backend/app/services/search/pipeline.py`). Более узкий инструмент, который уже закрывает общий навык в балле, в этот пробел не попадает. Подсказка появляется, только когда кто-то действительно добавится, и порядок списка не меняет.
- В поиске по банку пустой результат выглядит как «Никого не нашли» с советом ослабить фильтры. Отдельно сказано, что в базе есть и кандидаты без истории ФСП (`frontend/src/pages/employer/Search.tsx`).
- Фильтры и прошлая подборка при этом не сбрасываются: можно поменять одно условие и сразу увидеть результат.

Когда список не пустой, силу совпадения показывают баллы и причины в карточках; отдельная плашка «сильных совпадений нет» — в плане развития.

Проверяющий (демо-режим)

1. Откройте `/jury`: там кнопки входа в демо-аккаунты и ссылки на страницы без входа.
2. Кандидатский путь с нуля: `/login` → «Взять свободный аккаунт жюри». Кнопка подставляет тот из `jury1@demo.talent` … `jury10@demo.talent`, в который дольше всех не входили. У этих аккаунтов пустой опрос, дата рождения уже указана, полный тест проходится без пауз и лимитов попыток, результат сразу попадает в категорию и поиск.
3. Работодательский путь: `hr@sibcode.demo` → «Потребности» → «Backend на Python в платёжный сервис».
4. Проверки без входа: методика теста `/method`, подпись удостоверения `/badge`, подбор на своей разметке `/evaluate` (до 500 пар, файл JSONL или вставка текста).
5. В демо-режиме стенд каждую ночь в 04:10 МСК возвращается в исходное состояние: опросы и тесты `jury1@demo.talent` … `jury10@demo.talent` очищаются, пароли демо-аккаунтов прежние.

Короткий маршрут на 5 минут — в README. Полный сценарий — ниже.

Полный сценарий из ТЗ 3.3

Шаги идут в том порядке, в каком их требует показать ТЗ. Примерно 10 минут.

#	Кто	Что сделать	Что увидеть
1	Новый кандидат	`/register`: email, пароль, «Мне исполнилось 18 лет», согласия. Код из письма (локально — Mailpit, на стенде без почты — на экране подтверждения)	Регистрация по email с подтверждением (ТЗ 2.2, п. 4). Истории ФСП не нужно
1а	Кандидат через ФСП ID	`/login` → «Войти через ФСП ID», свободный номер из FSP2000–FSP2099, пароль `test`	Единая точка входа: пароль вводится на странице провайдера, затем экран согласий, аккаунт создаётся из реестра. `FSP001` / `test` сразу входит в привязанный аккаунт
2	Он же	Обзор → опрос: направление и специализация, цель, предметная область (необязательно)	Категория появилась, грейд помечен как заявленный и в подборке стоит ниже подтверждённого
3	Он же	«Тест на грейд»: уровень и язык кода (Python, JavaScript, Go, Java, Kotlin, C#)	8–20 заданий, у каждого кандидата свои параметры. Числовой ответ вводится, вариантов нет. В конце — итог по темам и подтверждённый грейд; при уверенном результате выше — предложение отдельного теста на ступень выше
4	Он же	Профиль → «Войти через ФСП ID», FSP2000–FSP2099, пароль `test`	Вход как в Keycloak (OIDC): пароль вводится на странице провайдера. Результаты из протоколов появляются отдельной осью
5	Работодатель `hr@sibcode.demo`	Потребности → «Backend на Python в платёжный сервис»	«Где искать»: рекомендованные категории. Подборка по одной оценке соответствия с полосами факторов и «Почему в выдаче»
6	Он же	Отметить трёх кандидатов «Сравнить»; вкладка «Как проверим»	Факторы рядом, без имён. Доли тем из текста потребности, цитаты и три примера заданий
7	Он же	Поиск: специализация, грейд, стек, «Только грейд, подтверждённый тестом», «Разряд не ниже»	Поиск по всему банку с фильтрами (ТЗ 2.1)
8	Он же	Задания → «Платёж списался дважды» → ответ кандидата → «Зачтено» с комментарием; или «Дать задание» в карточке кандидата	Задание пишет работодатель. С эталоном проверяет система, открытое — работодатель. Вердикт появляется строкой в анонимной карточке
9	Он же	«Пригласить»: роль, вилка до вычета НДФЛ, способ связи	Без вилки приглашение не отправить. Кандидат получает уведомление и письмо
10	Кандидат `fsp001@demo.talent`	Задания → «Индекс для выборки заказов» → ответить	Адресное задание от работодателя проверено системой сразу, эталон виден только после ответа. Грейд не меняется
11	Он же	Приглашения → «Принять» → «Переписка»	Компания, вакансия, вилка и профиль компании видны до решения. У работодателя открываются контакты, в журнале доступа — запись. Стороны пишут друг другу в переписке по приглашению
12	Он же	Вакансии → «Откликнуться»; Обзор → «Задача недели»	Классический отклик со статусами. Короткое задание, собранное платформой из вакансии: грейд не меняет, работодатель видит свежий результат
13	Он же	Приглашения → «Закрыть контакты для этой компании»; или Профиль → согласия → отозвать согласие	Контакты закрываются сразу: для одной компании (она видит `contacts_revoked`, переписка только для чтения) или для всех

#	Кто	Что сделать	Что увидеть
14	Он же	Профиль → «Паспорт навыков» → «Выпустить паспорт»	Ссылка `/passport/<id>` и QR-код: подтверждённые грейды, навыки из теста и сводка ФСП без имени. Подпись Ed25519 проверяется на странице, при перевыпуске старый паспорт отзывается
15	Работодатель `hr@sibcode.demo`	Потребность → карточка → «Мнение ИИ по карточке» (если подключена модель)	Сильные стороны, что проверить и вопросы для интервью, у каждого пункта факт карточки. Пометка «не влияет на балл и порядок»

Законный представитель

Регистрация открыта с 15 лет. Кандидат 15–17 лет не виден работодателям, пока законный представитель не подтвердит согласие на обработку его данных (ч. 6 ст. 9 152-ФЗ).

1. Кандидат регистрируется с отметкой «Мне 15, 16 или 17 лет» и указывает дату рождения.
2. На «Обзоре» в карточке согласия представителя он вводит email родителя или другого законного представителя. Свой адрес входа указать нельзя. Писем представителю — не больше трёх в сутки.
3. Представитель получает письмо со ссылкой вида `/guardian#token=…`. Подпись в ссылке привязана к кандидату и к этому адресу: если адрес в профиле сменить, старая ссылка перестаёт работать. Токен стоит после `#` и не попадает в журналы сервера.
4. На странице `/guardian` представитель читает, что увидят работодатели, и нажимает «Даю согласие». Анонимная карточка кандидата появляется в поиске. Имя и контакты по-прежнему открываются только после того, как кандидат сам примет приглашение.

На демо-стенде без почты (`DEMO_MODE` и `DEMO_SHOW_CODE`) ссылка дополнительно показывается в той же карточке на «Обзоре» самого кандидата, кроме общих демо-адресов `@demo.talent` и `.demo`.

Если что-то не получается

Что случилось	Что сделать
Код подтверждения не пришёл	Локально — Mailpit http://localhost:8025. На стенде без почты код показан на экране подтверждения. Повторный код — через минуту
«Вход закрыт на 15 минут»	К этому адресу пять раз подряд ввели неверный пароль, в том числе кто-то посторонний: адреса демо-аккаунтов опубликованы. Подождите 15 минут или войдите под другим аккаунтом из `jury1@demo.talent` … `jury10@demo.talent` (кнопка «Взять свободный аккаунт жюри»)
Тест не даёт пересдать уровень	Пересдача того же уровня — через 7 дней, тест на уровень ниже — сразу. У `jury1@demo.talent` … `jury10@demo.talent` на демо-стенде пауз нет
В карточке нет имени и контактов	Так и задумано: они открываются только после принятия приглашения или отклика кандидата
Работодатель видит `contacts_revoked`	Кандидат закрыл контакты для этой компании; переписка остаётся только для чтения

Проверка теста на грейд

ТЗ (раздел 2) требует тест, устойчивый к утечке: без единого набора вопросов и без вызова языковой модели на каждого кандидата. Там же названы устойчивость повторной сдачи и различающая сила заданий. Все три свойства проверены симуляцией через те же функции, что работают в API (`app/services/cat.py`, `app/services/item_monitor.py`), так что цифры ниже описывают именно рабочий код.

Обозначения простыми словами:

Обозначение	Что значит
θ	Настоящий уровень знаний кандидата на общей шкале теста (в симуляции он известен заранее)
θ̂	Оценка этого уровня по ответам кандидата
SE	Погрешность оценки θ̂: чем меньше, тем увереннее тест
IRT 3PL	Модель, которая связывает уровень человека, сложность задания (b), его различающую силу (a) и шанс угадать (c)
EAP	Способ оценки уровня: среднее по всем уровням, взвешенное их правдоподобием после ответов
Информация Фишера	Насколько задание помогает отличить уровень чуть выше порога от уровня чуть ниже
GLR	Последовательная проверка гипотез: тест останавливается, как только ответ «выше порога или ниже» статистически ясен
Экспозиция	Доля кандидатов, которым показали одно и то же задание; метод Симпсона–Хеттера её ограничивает
Каппа Коэна	Согласие двух решений (например, первой и повторной сдачи) с поправкой на случайное совпадение: 1 — полное, 0 — как наугад
AUC	Как часто тест ставит более сильного кандидата выше более слабого: 1 — всегда, 0,5 — наугад
ДИ	Доверительный интервал: диапазон, в котором с заданной уверенностью лежит настоящее значение

Как устроен тест

- **Задания — параметрические семейства** (`backend/app/items/families/`, реестр `backend/app/items/registry.py`): **22** семейства — 20 с вычисляемым ответом (Python, SQL, JavaScript, алгоритмы, графы, сети, безопасность, ML-метрики, встраиваемые системы, границы и расхождения тестов, метрики и доли аналитика, непокрытые требования и шаги цепочки, пакеты и опоздания в конвейере данных, Docker, Kubernetes, Linux shell) и 2 концептуальных пула. Число на главной стенда (`/api/v1/public/stats` → `families`) — это `len(FAMILIES)` = 22. У каждого семейства 5 уровней сложности. Правильный ответ получается запуском фрагмента кода, SQL-запроса или формулы; юнит-тест `test_item_families.py` перезапускает каждый сгенерированный фрагмент отдельно — в Python, Node.js и SQLite — и сверяет ответ.
- **Ключи проверены исполнением, отдельной командой.** `cd backend && python -m app.validation.key_check --variants 20` генерирует по 20 вариантов на каждый уровень каждого семейства с кодом и заново исполняет их отдельным процессом (Python, Node.js, SQLite, bash, Go; Java — если установлена). Прогон 11.10.2026 на машине без Java: **исполнено 1875 вариантов, совпали с ключом все 1875**. Всего вариантов 1900: оставшиеся 25 — задания algo_ops с фрагментом на Java, на этой машине нет исполнителя Java, поэтому они не исполнены и не засчитаны как совпадение (с установленной Java команда проверит все 1900). Графы сверяются отдельным юнит-тестом через Беллмана–Форда. При расхождении команда завершается с ошибкой.
- **Числовой ответ вводится.** Если у задания с кодом правильный ответ — целое число, кандидат печатает его сам: вариантов на экране нет. Так устроены Python, SQL, подсчёт операций, графы, границы тестов, метрики аналитика и часть заданий по JavaScript, сетям и встраиваемым системам: в симуляции ниже это 79 % выданных заданий. Ответы вроде шестнадцатеричной строки, адреса или десятичной дроби остаются выбором из четырёх: их формат вводить неоднозначно.
- **Никакого смешения специализаций**: тест backend собирается только из семейств backend по плану (Python 30 %, SQL 25 %, алгоритмы 20 %, …). Концептуальных вопросов не больше 30 %.
- **IRT 3PL**: $P(\text{верно} \mid \theta) = c + (1 - c) \cdot \sigma(a \cdot (\theta - b))$. Угадывание c = 0,02 для вводимого числа и c = 1/4 для четырёх вариантов. Оценка способности θ считается как среднее апостериорного распределения на сетке (EAP), априорное распределение центрировано на пороге заявленного грейда. Селектор берёт ожидаемое с семейства на этом уровне (`expected_guess`): долю вводимых ответов считает по 24 сгенерированным заданиям, до того как выбран экземпляр.
- **Почему это важно на пороге.** Информация задания 3PL при θ = b и a = 1 равна $(P - c)^2 \cdot (1 - P) / ((1 - c)^2 \cdot P)$, где $P = (1 + c) / 2$, и сворачивается к $0{,}25 \cdot (1 - c) / (1 + c)$. При c = 0,25 это 0,150, при c = 0 это 0,250: четыре варианта оставляют 60 % информации задания ровно на границе грейда. Вводимое число в продукте идёт с c = 0,02 и даёт 0,240, то есть в 1,60 раза больше четырёх вариантов (`cut_information` в `app/services/cat.py`).
- **Адаптивный выбор под решение о грейде**: задание выбирается по информации Фишера в точке порога, а не в текущей оценке θ; среди пяти лучших — случайно; доли тем ограничены планом; слишком часто выдаваемые ключи получают штраф.
- **Остановка**: не меньше 8 и не больше 20 заданий. Тест заканчивается, когда апостериорная вероятность уровня выходит за 10 % или 90 % и вопрос о следующем уровне уже ясен. Раньше этого его может остановить обобщённое отношение правдоподобия (GLR) у порога: зона безразличия ±0,25, порог ln 9 при ошибках 10 % с каждой стороны. Остановка случается только если отношение и апостериорное распределение смотрят в одну сторону. Критерий Вальда в файле есть и в решение не входит. На симуляции у порога GLR не сработал ни разу (0 из 400 сессий, `cd backend && python`

`tests/stop_reasons.py --n 400`): сессии заканчиваются по апостериорной вероятности (53 %) или по лимиту 20 заданий (47 %). Пробный проход из 15 заданий и 5 минут останавливается по счётчику, без GLR. Среди пяти лучших заданий срабатывает Симпсон–Хеттер, когда по ключу уже есть доля показов.

- **Решение**: грейд подтверждён при P(θ ≥ порог) ≥ 0,75. Тест на уровень подтверждает только этот уровень: при P(θ ≥ следующий порог) ≥ 90 % сессия рекомендует отдельный тест на ступень выше, но грейд не повышает. Не сдали, а те же ответы дают P(θ ≥ порог ступени ниже) ≥ 75 % при априоре в этом пороге и нож держится — кандидат может принять ступень ниже без отдельного теста. Подтверждённый грейд не понижается; хранится отдельно для каждой специализации. Тест по вакансии и пробный проход из 15 заданий грейд не меняют. Грейд «стажёр» стоит на ступень ниже junior: порог θ = −2,5, и под него выбираются задания первого уровня.
- **Навыки, подтверждённые тестом**: ≥ 75 % верных ответов по навыку на ≥ 3 заданиях; в профиле они отмечены `test_confirmed` и отличаются от заявленных.
- **Мониторинг утечки**: по каждому ключу «семейство:уровень» (и по каждому статичному вопросу) копится z = (верных − ожидаемых)/√дисперсии, где ожидание считается по θ, оценённой **без заданий того же семейства**: так утёкшее семейство не завышает θ и не прячет сигнал. Рядом копится односторонняя сумма S = max(0, S + (ответ − P − 0,12)). При n ≥ 30 ключ уходит в карантин, если z ≥ 3,5 или S ≥ 7: больше не выдаётся, а его ответы не учитываются. Статистика открыта: `GET /api/v1/public/assessment/items`.
- **Калибровка**: `python -m app.validation.calibrate` пересчитывает сдвиги трудности по реальным ответам (тоже без своего семейства) и обнуляет счётчики дрейфа.
- **Защита сессии**: правильный ответ не покидает сервер до конца теста; у вводимого задания варианты тоже остаются на сервере. Ответ принимается только на текущее задание по HMAC-токену; время засекает сервер (3 минуты на задание с кодом, 75 секунд на вопрос); прерванная или просроченная сессия считается попыткой; пересдача того же уровня — через 7 дней; после проваленного теста тест на уровень ниже доступен сразу.

Симуляция

Кандидаты: θ ~ N(0,3; 1), специализация случайна среди всех одиннадцати направлений, включая инженерию данных. 60 % заявляют свой истинный грейд, 30 % — на ступень выше, 10 % — ниже. «Правда» специально не совпадает с моделью сервиса: у каждого ключа свой неизвестный сервису сдвиг трудности N(0; 0,3²) и множитель дискриминации U(0,8; 1,2). Задание вводимое с той долей, которую реально даёт его генератор. Перед мониторингом прогнана когорта для калибровки.

Сценарии «без утечки» и «все задания с вариантами» проходят одни и те же кандидаты, поэтому форматы ответа сравниваются напрямую. Сценарии с утечкой — отдельные когорты того же размера.

Симуляция: 2000 кандидатов на сценарий, зерно генератора 7. Команда: `python -m app.validation.assessment --n 2000 --seed 7`. Ответ вводится в 83 % выданных заданий.

Сценарий	Точность «сдал/не сдал»	Точность вне ±0,5 от порога	Сдали из тех, кто на уровне	Ложно сдали	Ложная рекомендация теста выше	Заданий: среднее / 90 %	Решено до лимита	Экспозиция по всем	В специализации (макс. по 11)	Доля концептуальных
Без утечки	77.8%	91.7%	71.9%	7.4%	0.9%	14.3 / 20	63%	41%	72%	7%
Без утечки, все задания с вариантами	72.0%	84.4%	64.4%	8.6%	0.7%	16.3 / 20	49%	46%	68%	13%
Утечка у 30 %, без мониторинга	79.8%	91.2%	79.4%	19.2%	2.9%	14.1 / 20	64%	41%	72%	7%
Утечка у 30 %, ключи на карантине	77.0%	88.9%	73.6%	13.4%	1.7%	14.7 / 20	59%	48%	83%	9%

41 % — доля от всех специализаций сразу. Внутри одной специализации самый частый ключ видят до 72 % её сессий (mobile, `js_eval:3`). Перекрытие ключей внутри специализации 33,5 %. Знаменатель этой таблицы — все сессии. Живой банк считает сессии своей специализации: на той же когорте максимум 49,7–65,0 % (`--spec-exposure`, раздел ниже). Команда таблицы та же.

Специализация	Людей	Макс. доля ключа	Ключ
analyst	193	51%	py_eval:4
backend	180	51%	py_eval:4
data	175	55%	py_eval:4
devops	193	50%	py_eval:4
embedded	166	67%	emb_bits:3
frontend	199	70%	js_eval:3
fullstack	183	68%	js_eval:3

Специализация	Людей	Макс. доля ключа	Ключ
ml	175	50%	py_eval:4
mobile	174	72%	js_eval:3
qa	187	53%	qa_checks:4
security	175	50%	emb_bits:3

Экспозиция экземпляров против порогов Duolingo English Test. Ключ — это семейство и уровень, а не одно задание: числа, таблицы и код у каждого кандидата свои. Поэтому 72 % в строке mobile — это доля сессий с ключом `js_eval:3`, а не одно и то же задание. Отдельный прогон той же чистой когорты с генерацией заданий даёт каждому выданному заданию отпечаток sha256 (семейство, уровень, условие или статический ключ, код; ответ из них следует) и считает на одних и тех же сессиях обе меры. Ориентир — рамка безопасности DET: экспозиция задания ≤ 10 %, перекрытие тестов ≤ 15 % (go.duolingo.com/security_framework, со ссылкой на Way, 1998). Ключевые доли в этом прогоне немного отличаются от таблицы выше, потому что генерация расходует тот же поток случайных чисел.

Команда: `cd backend && python -m app.validation.instance_exposure --n 2000 --seed 7`

```
Сессий 2000, зерно 7, заданий на сессию в среднем 13.9.
Порог DET: экспозиция ≤ 10%, перекрытие ≤ 15%.
| Уровень | Макс. экспозиция по всем | Макс. в специализации | Перекрытие в специализации |
|---|---|---|---|
| Ключ (семейство:уровень) | 41.2% | 71.8% | 33.8% |
| Экземпляр (sha256) | 4.8% | 14.1% | 1.1% |

| Специализация | Людей | Ключ: макс. | Ключ | Экземпляр: макс. | Перекрытие ключей | Перекрытие экземпляров |
|---|---|---|---|---|---|---|
| analyst | 193 | 50.8% | py_eval:4 | 5.7% | 34.0% | 0.4% |
| backend | 180 | 50.0% | py_eval:4 | 11.7% | 30.7% | 1.5% |
| data | 175 | 52.0% | py_eval:4 | 7.4% | 31.6% | 0.2% |
| devops | 193 | 47.2% | py_eval:4 | 9.3% | 27.5% | 0.5% |
| embedded | 166 | 65.1% | emb_bits:3 | 12.0% | 33.3% | 1.4% |
| frontend | 199 | 71.4% | js_eval:3 | 14.1% | 35.0% | 3.1% |
| fullstack | 183 | 71.6% | js_eval:3 | 9.3% | 38.5% | 1.0% |
| ml | 175 | 50.9% | py_eval:4 | 8.0% | 33.3% | 0.5% |
| mobile | 174 | 71.8% | js_eval:3 | 12.1% | 38.1% | 1.9% |
| qa | 187 | 55.1% | qa_checks:4 | 9.1% | 36.1% | 0.5% |
| security | 175 | 53.1% | emb_bits:3 | 10.9% | 33.9% | 0.7% |

Флаг low_variety (экземпляр > 10% в специализации): algo_ops 13.1%, concept_bank 11.4%, concept_http 10.6%, net_ipv4
10.4%, py_eval 10.3%
```

Итог: по экземплярам экспозиция 4,8 % по всем и до 14,1 % внутри специализации, перекрытие внутри специализации 1,1 % — ниже порога DET 15 %. По экспозиции экземпляра порог 10 % превышают пять семейств из флага `low_variety` в блоке выше: algo_ops 13,1 %, concept_bank 11,4 %, concept_http 10,6 %, net_ipv4 10,4 %, py_eval 10,3 %. Это измеримая часть F12: им нужны новые изоморфы или концепт-задания.

Повторный тест: 1000 кандидатов проходят тест дважды, задания каждый раз новые.

Формат	Тот же итог «сдал/не сдал»	Каппа Коэна	То же вне ±0,5 от порога	Тот же грейд	Корреляция оценок θ
Числовой ответ вводится	75%	0.49	87%	63%	0.86
Все задания с вариантами	71%	0.43	76%	57%	0.84

Различающая сила заданий в сценарии «без утечки». r — точечно-бисериальная корреляция ответа с оценкой способности по остальным семействам теста.

Формат ответа	Ключей с ≥ 30 ответами	Медиана r	Ключей с r ≥ 0,20	Верно у тех, кто на 0,5 слабее задания	Верно у тех, кто на 0,5 сильнее
Число вводится	61	0.24	62%	29%	72%
Выбор из четырёх	17	0.13	12%	56%	80%

- Мониторинг утечки: из 37 утёкших ключей с ≥ 30 ответами пойманы 18, ложных срабатываний среди остальных — 1; на чистых данных — 5 из 68 ключей.
- Случайное угадывание проходит тест в 0.0% случаев (все задания с вариантами — 0.0%).
- Точный грейд в сценарии «без утечки»: 57.0%, в пределах одной ступени: 84.0%, завышение: 2.2%.

- Оценка способности против истинной θ, без утечки: r(θ̂, θ) = 0,919, RMSE = 0,452; экспресс-режим: r = 0,922, RMSE = 0,448. Экспресс не хуже полного, потому что полный тест останавливается, как только ясно решение о грейде (в среднем 14,3 задания), а экспресс всегда даёт 15: точность θ̂ вдали от порога полному не нужна.
- Взвешенная каппа повторного теста (квадратичные веса по грейду): 0.58.
- Ключ от другого экземпляра того же семейства: сдают 0.0%. Ключ от того же экземпляра (фиксированный банк): сдают 100.0%.
- Второй сигнал утечки (CUSUM): поймано 13 утёкших ключей, ложных тревог на чистых данных — 2. Карантин ставится, если сработал z или CUSUM.
- Калибровка различимости, средняя ошибка a: 0.146 и 0.283. В банк пишется априор семейства.
- Порог по теме: точность 55.4%, ложные сдачи 3.0%. В решении о грейде его нет.
- Точность классификации по Руднеру у порога заявленного грейда: 83.5%, согласованность 79.0%. Вероятность принятого решения по апостериорному распределению: 83.5%.

79,0 % — согласованность без порога решения: вероятность просто быть по ту же сторону разреза. Продукт сдаёт при вероятности уровня от 0,75, то есть при θ̂ ≥ порог + 0,6745·SE. По этому правилу модель предсказывает повторное совпадение решения 76,1 %, измеренное на повторе — 73,6 %. Многоклассовая точность по Руднеру, шесть классов от «ниже стажёра» до lead, — 58,8 %. ECE вероятности уровня — 0,084: в корзине 0–0,1 доля тех, кто на самом деле не ниже порога, 15,9 % при средней p_pass 6,7 %; в корзине 0,2–0,3 — 54,8 % при средней 25,0 %. Та же команда.

- Перекрытие заданий: в среднем другой кандидат этой же симуляции видел 15.2% заданий данного кандидата. Максимальная экспозиция ключа — в таблице выше.
- Два теста одного кандидата: во втором в среднем 70.4% заданий совпадают с первым по семейству и уровню. Сами задания собираются заново. В этом прогоне списанный ключ засчитывается неверным по построению, поэтому сдача выходит 0 %. Честный прогон (`construction_check`) даёт совпадение в коде 9,3 % и сдачу 69,2 % против 58,5 % своими силами.
- Карантин на тех же людях и тех же случайных потоках: ложные сдачи 18.2% без мониторинга и 13.2% с карантином.
- Экспозиция по регионам: четыре метки назначены отдельным генератором и в выбор задания не входят. Максимальная доля одного ключа в группах: 33%, 35%, 33%, 37%. Отношение меньшей к большей: 0.88.

Посткалибровка вероятности уровня (измерение, в продукт не внесена). Изотоническая регрессия (Zadrozny & Elkan 2002, doi:10.1145/775047.775151) обучена на зерне 7 и проверена на отложенном зерне 8. Команда: `cd backend && python -m app.validation.calibration_check --n 2000 --fit-seed 7 --test-seed 8`.

```
ECE, обучение на зерне 7: до 0.084.
ECE на отложенном зерне 8: до 0.135, после изотонической 0.052.
```

На отложенном зерне ECE падает с 0,135 до 0,052. В продукте остаётся исходная p_pass: от неё зависит правило «подтверждён при p ≥ 0,75», и замена шкалы — отдельное решение владельца с полным перепрогоном валидации.

Матрица «истинный грейд по θ × грейд, который назначила сессия» в сценарии «без утечки». Строка «ниже» — способность ниже стажёра или сессия не назначила грейд.

Истинный \ назначенный	ниже	intern	junior	middle	senior	lead
ниже	3	0	0	0	0	0
intern	16	45	0	0	0	0
junior	70	88	186	13	0	0
middle	119	1	181	429	12	0
senior	91	0	0	142	342	17
lead	10	0	0	0	93	142

Лестница: все кандидаты проходят тест на middle. Доля сдавших по истинному грейду.

Истинный грейд	Кандидатов	Сдали тест middle
ниже	3	0.0%
intern	61	0.0%
junior	357	7.3%
middle	742	70.4%
senior	592	98.3%
lead	245	100.0%

Точность решения по расстоянию до порога. Те же 2000 сессий «без утечки», seed 7; корзины по |θ − порог заявленного грейда|. Одна кривая связывает 77,8 % в целом и 91,7 % вне ±0,5: ошибки собраны у порога, где решение трудно для любого теста (Rudner 2005, doi:10.7275/56a5-6b14).

|θ − порог|	Людей	Верное решение (95 % ДИ)	Ложно сдали среди тех, кто ниже	Сдали среди тех, кто на уровне
0–0,1	186	44,1 % (37,1–51,1 %)	19,7 %	24,2 %
0,1–0,2	178	49,4 % (42,1–56,7 %)	15,7 %	35,4 %
0,2–0,3	194	64,9 % (58,2–71,6 %)	13,4 %	53,5 %
0,3–0,5	354	74,0 % (69,2–78,5 %)	5,1 %	63,6 %
0,5–1	887	90,0 % (87,9–92,0 %)	2,3 %	86,7 %
> 1	201	99,5 % (98,5–100,0 %)	0,0 %	99,5 %

Точность не убывает с расстоянием. В полосе 0,1 от порога решение близко к монете: 44,1 %. Ошибки там в основном в сторону «не сдал»: правило сдачи P ≥ 0,75 строже середины, поэтому сдают только 24,2 % тех, кто на уровне. Пересняты 11.10.2026 на main после расширения банка. Воспроизвести: `cd backend && python -m app.validation.cut_distance --n 2000 --seed 7` (sha256 вывода `0157da9e…`, два прогона совпали). Код: `app/validation/cut_distance.py`, тест `tests/unit/test_cat_baselines.py`.

Три атаки, коротко

Цифры ниже — те же, что в таблицах этого файла и в README. Команда: `python -m app.validation.assessment --n 2000 --seed 7`.

Атака	Что выходит
Случайное угадывание	Сдают 0 %
30 % кандидатов знают ответы главного семейства	Точность вдали от порога 91,2 %. Карантин на тех же людях снижает ложные сдачи с 19,2 % до 13,4 % и находит 18 ключей из 34
Ключ от другого экземпляра того же семейства	В основном прогоне 0 % по построению; честный прогон: 69,2 % против 58,5 % у тех же людей, см. «Что держится по построению». Ключ от того же экземпляра, как в заранее опубликованном банке, сдаёт 100 %

Первая защита — генерация: у каждого кандидата свои числа и фрагменты кода. Карантин — вторая: ключ, по которому верных ответов больше, чем ждёт модель, перестаёт выдаваться.

Что показывают таблицы:

- **Вводимое число против выбора из четырёх.** На одних и тех же 2000 людях точность вне ±0,5 от порога растёт с 84,4 % до 91,7 %, а доля сдавших среди тех, кто действительно на уровне, — с 64,4 % до 71,9 %. Тест укорачивается с 16,3 до 14,3 задания, а до лимита заканчиваются 63 % сессий вместо 49 %.
- **Повторная сдача.** Каппа Коэна учитывает совпадение «сдал / не сдал» сверх случайного: 0,49 против 0,43 у формата с вариантами. Вне ±0,5 от порога итог совпадает в 87 % пар против 76 %, корреляция двух оценок θ — 0,86 против 0,84.
- **Различающая сила.** Медиана r = 0,24, и порог 0,20 проходят 62 % таких ключей. Кандидат на 0,5 слабее задания решает его в 29 % случаев, а на 0,5 сильнее — в 72 %. У выбора из четырёх медиана r = 0,13: слабый кандидат добирает до 56 % за счёт угадывания.
- **Где тест ошибается.** Ширина полосы одного грейда — 1,0, и вне ±0,5 от порога тест прав в 91,7 % случаев. Кандидата в пределах ±0,3 от порога за 20 заданий нельзя уверенно отнести к одной стороне: это свойство самой задачи, а не конкретного теста.
- **Утечка ответов.** Даже если 30 % кандидатов знают ответы главного семейства своей специализации, точность вне ±0,5 от порога остаётся 91,2 % без мониторинга и 88,9 % с карантином. Карантин проверяется на тех же людях и тех же случайных потоках, что и прогон без него, и снижает долю ложно сдавших с 19,2 % до 13,4 % и находит 18 утёкших ключей из 34, с одним ложным срабатыванием на остальных. На чистых данных карантин отмечает 6 ключей из 78. Первая линия защиты — генерация: каждый кандидат получает новые экземпляры заданий.
- **Угадывание и завышение.** доля случайных угадывателей, сдавших тест, — 0 %. Ложная рекомендация теста на ступень выше без утечки — 0,9 %: грейд при этом не меняется. Точный грейд совпадает в 57,4 % случаев, в пределах одной ступени — в 84,7 %, завышение — 2,1 %. Взвешенная каппа повторного теста — 0,56. Лестница «все проходят тест на middle» показывает, где тест разделяет: сдают 7,3 % настоящих junior, 70,4 % настоящих middle и 98,3 % настоящих senior. Точность классификации по Руднеру у порога заявленного грейда — 83,4 %, согласованность — 79,1 %. Другой кандидат той же симуляции видел в среднем 14,9 % заданий. Матрица выше показывает, что завышение больше чем на ступень почти не встречается.
- **Чужой ключ.** Ключ от другого экземпляра того же семейства: в основном прогоне 0 % по построению, в честном прогоне +10,8 п. п. к честной сдаче (раздел «Что держится по построению»). Ключ от того же экземпляра, как в заранее опубликованном банке, даёт 100 %. Второй сигнал утечки ловит 11 утёкших ключей и на чистых данных даёт 3 ложные тревоги. Вместе с z карантин отмечает 18 из 34.
- **Порог по теме и различимость.** Если слабая тема блокирует сдачу, ложные сдачи 3,9 % и точность решения 54,9 %. Оценка различимости даёт среднюю ошибку a 0,268 при 0,147 у априора. В решении о грейде порога по теме нет. В банк пишется априор семейства.
- **Потолок заданий одной семьи.** Команда `python -m app.validation.assessment --family-grid --n 2000 --seed 7`. Клетка принимается, только если сразу: ложные сдачи при утечке не выше 12 %, без утечки не выше 8 %, сдали из тех, кто на уровне не ниже 71 %, точность вдали от порога не ниже 91 %, средняя длина не выше 15. Потолок 5 без ножа — живой тест:

18,2 %, 7,7 %, 73,0 %, 91,5 %, 14,3. Потолок 5 с ножом: 12,0 %, 6,7 %, 69,0 %, 88,9 %, 14,3. В сессии не больше 5 заданий одной семьи.

Эти свойства закреплены в CI: при каждом изменении запускается `python -m app.validation.assessment --n 300 --seed 7 --check`, и сборка падает, если нарушен хотя бы один порог: точность вне ±0,5 не ниже 85 %, ложно сдавшие не больше 12 %, ложная рекомендация теста выше не больше 3 %, каппа пересдачи не ниже 0,35, случайное угадывание не больше 2 %, медиана r не ниже 0,20 для вводимого числа и 0,15 для выбора из четырёх, сдача с ключом от другого экземпляра того же семейства не больше 2 %, а ложных сдач с карантином не больше, чем без него на тех же людях. Всего 9 порогов.

Вводимый ответ сравнивается на сервере функцией `same_number`: она принимает лишние пробелы, знак плюс и типографский минус, так что формат записи не влияет на результат. Когда накопятся реальные ответы, команда `python -m app.validation.calibrate` уточнит трудность каждого ключа по ним.

Воспроизвести: `cd backend && python -m app.validation.assessment --n 2000 --seed 7`.

Как читать каппу 0,49 и долю показов 72 %

Каппа 0,49 — это каппа Коэна, не взвешенная. Её считают по парам «первый тест — повторный тест на новых заданиях» у одного человека и спрашивают одно: совпал ли итог «сдал / не сдал» сверх случайного совпадения. 0 — совпадение как у случайного решения, 1 — полное. По шкале Ландиса и Коха (Landis & Koch, 1977, doi:10.2307/2529310) 0,41–0,60 — умеренное согласие. С чем сравнивать:

- **С другим форматом на тех же людях.** Выбор из четырёх вариантов даёт 0,43, вводимый ответ — 0,49; совпадение итога вне ±0,5 от порога 87 % против 76 %, корреляция двух θ 0,86 против 0,84.
- **С зоной у порога.** Каппа считается по всем парам, включая кандидатов в пределах ±0,3 от порога, которых за 20 заданий нельзя уверенно отнести к одной стороне. У решений вне одной стандартной ошибки от порога повтор даёт тот же итог в 82,7 % (80,1–85,4 %), у пограничных — 54,3 % (раздел «Зона неопределённости решения о грейде»).
- **С длиной теста.** Тест в среднем занимает 14,3 задания из 20 и останавливается, как только решение уверенное. Перебор порогов остановки на этой длине не поднимает каппу выше 0,49 (раздел «Перебор порогов остановки»).
- **Со взвешенной каппой.** Квадратичные веса по грейду (`_quadratic_kappa`, `app/validation/assessment.py:446`): промах на две ступени стоит дороже промаха на одну. Для одного теста — 0,56, для итогового грейда после пути продукта — 0,85, если «не подтверждён» стоит ниже стажёра, и 0,76 при исходном порядке (раздел «Итоговый грейд после пути»); тот же грейд при повторе — около 65 %.

Поэтому одна сессия не ставит окончательный грейд у порога: человек идёт по пути продукта, и решение о найме принимает работодатель после собеседования.

Доля показов 72 % — у ключа `js_eval:3` в сессиях mobile, у frontend 70 %, у fullstack 68 % (таблица в разделе «Симуляция»). Ключ — это семейство и уровень, а не текст: каждый показ — новый экземпляр с другими числами и кодом, ответ чужого экземпляра совпадает в коде в 9,3 % случаев. Причина — глубина банка: JavaScript занимает 50 % плана frontend, 35 % mobile и 25 % fullstack (`BLUEPRINTS`, `app/items/registry.py:32`), а семейство JavaScript в банке одно (`js_eval`, `app/items/families/javascript.py:80`). Симпсон–Хеттер в опубликованном прогоне считает долю по всем сессиям, поэтому узкий ключ проходит под целью 0,40. Живой банк считает долю внутри специализации. Калибровка K на отложенных зёрнах снижала максимум до 0,51–0,58, но стоила точности, поэтому в продукт не вошла (раздел «Знаменатель Симпсон–Хеттер»). Следующий шаг — расширение банка: новые семейства для frontend, mobile и devops.

Разделяющая сила: AUC соседних грейдов

Для каждой пары соседних истинных грейдов считается AUC: вероятность того, что у случайного кандидата старшего грейда оценка $\hat{\theta}$ выше, чем у случайного кандидата младшего. Ничья считается за половину. Это U Манна–Уитни, делённое на $n_1 \cdot n_2$ (Hanley & McNeil 1982, doi:10.1148/radiology.143.1.7063747). 95 % интервал строится перцентильным бутстрапом: 1000 повторов, каждый грейд перевыбирается отдельно, поэтому размеры групп не меняются. Зерно бутстрапа фиксировано, отчёт воспроизводим. Сценарий «без утечки», 2000 кандидатов, seed 7, каждый проходит тест своего заявленного грейда.

Пара грейдов	Людей (младший / старший)	AUC	95 % ДИ
intern / junior	61 / 357	0,923	0,894–0,947
junior / middle	357 / 742	0,921	0,904–0,936
middle / senior	742 / 592	0,931	0,916–0,944
senior / lead	592 / 245	0,901	0,876–0,925
среднее по парам	—	0,919	—

Воспроизвести: `cd backend && python -m app.validation.assessment --n 2000 --seed 7` — сценарий «без утечки», n = 2000, зерно генератора 7; бутстрап B = 1000 с фиксированным зерном.

Экспресс-режим демо-стенда (до 15 вопросов, 5 минут): тот же банк, тот же выбор заданий, те же люди. Короткий тест разделяет грейды немного хуже, в среднем 0,921 против 0,930. У пары middle / senior экспресс чуть выше полного (0,926 против 0,923), разница внутри интервала.

Пара грейдов	Людей	AUC экспресс	95 % ДИ	AUC полный
intern / junior	61 / 357	0,925	0,894–0,950	0,923
junior / middle	357 / 742	0,925	0,908–0,941	0,921
middle / senior	742 / 592	0,922	0,906–0,935	0,931
senior / lead	592 / 245	0,914	0,893–0,935	0,901
среднее по парам	–	0,921	–	0,919

Та же команда (`--n 2000 --seed 7`) печатает и эту таблицу: экспресс-сессии идут на тех же 2000 людях с лимитом 15 заданий.

Истинный грейд берётся из θ симуляции, поэтому это проверка на синтетике, а не на реальных людях. Код: `auc` и `adjacent_auc` в `app/validation/assessment.py`, тесты в `tests/unit/test_adjacent_auc.py`.

Покрытие интервала θ̂ ± z·SE

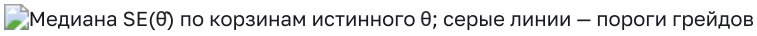
θ̂ — среднее апостериорного распределения, SE — его стандартное отклонение (априор в пороге заявленного грейда). Доля сессий, где истинная θ лежит внутри θ̂ ± z·SE. Если заявленная неопределённость честна, доля близка к номиналу, а SD величины (θ̂ − θ)/SE близко к 1 (Cook, Gelman & Rubin 2006, doi:10.1198/106186006X136976; Bock & Mislevy 1982, doi:10.1177/014662168200600405). Те же 2000 людей, seed 7.

Режим, группа	Людей	±1·SE (номинал 68,3 %)	±1,645·SE (90 %)	±1,96·SE (95 %)	SD (θ̂ − θ)/SE
полный тест, все	2000	68,7 %	89,3 %	94,0 %	0,98
полный, intern	61	73,8 %	86,9 %	93,4 %	0,94
полный, junior	357	62,7 %	85,4 %	89,9 %	1,05
полный, middle	742	68,7 %	88,4 %	93,3 %	0,97
полный, senior	592	72,6 %	92,2 %	96,3 %	0,93
полный, lead	245	66,9 %	91,4 %	96,7 %	0,95
экспресс, все	2000	67,9 %	89,2 %	94,5 %	0,99

В целом покрытие у номинала: на 95 % — 94,0 %, недобор 1,0 п. п. По грейдам разброс шире: у junior 89,9 % при номинале 95 %, у lead и senior интервал с запасом (96,7 % и 96,3 %). Это след априора в пороге заявленного грейда: он тянет θ̂ к порогу, и у краёв шкалы интервал смещён. Группа «ниже» (3 человека) в таблицу не вынесена. Калибровка вероятности сдачи — отдельная задача, см. посткалибровку p_pass. Пересняты 11.10.2026 на main после расширения банка. Воспроизвести: `cd backend && python -m app.validation.interval_coverage --n 2000 --seed 7` (sha256 `c0353ad1…`, два прогона совпали). Код: `app/validation/interval_coverage.py`.

Рисунки

Рисунки строит `python docs/tools/make_figures.py` из корня репозитория. Когорта та же: «без утечки», 2000 кандидатов, seed 7. Чисел, которых нет в таблицах выше, на рисунках нет.


Медиана SE(θ̂) по корзинам истинного θ; серые линии — пороги грейдов


θ̂ против истинного θ


Истинный и присвоенный грейд; «ниже» — тест своего грейда не сдан


AUC соседних грейдов с 95 % ДИ

Слитая база: доля знающих ответы K = 0, 5, 25, 100 %

Худший случай утечки: K % кандидатов знают ответы на главное семейство заданий своей специализации, например по опубликованному решателю. Кандидаты и их случайные потоки одни и те же во всех строках, меняется только K. Монитор утечки сначала учится на отдельной когорте с той же долей K, затем попавшие под подозрение ключи уходят в карантин, и тест проходит основная когорта. Так же устроен основной прогон, только там K = 30 %.

Команда (около 3 минут):

```
cd backend && PYTHONPATH=. python ../docs/tools/leak_grid.py --n 2000 --seed 7
```

K, %	Монитор	Верное «сдал/не сдал»	Ложно сдали (ниже грейда)	Грейд выше истины	Ключей в карантине
0	нет	78,5 %	8,1 %	2,1 %	–
0	да	78,6 %	8,1 %	2,2 %	4
5	нет	78,6 %	10,3 %	2,7 %	–

K, %	Монитор	Верное «сдал/не сдал»	Ложно сдали (ниже грейда)	Грейд выше истины	Ключей в карантине
5	да	77,2 %	8,9 %	2,4 %	11
25	нет	79,5 %	17,8 %	4,7 %	—
25	да	77,0 %	13,2 %	3,5 %	19
100	нет	83,5 %	41,1 %	10,8 %	—
100	да	79,5 %	24,0 %	6,2 %	27

Как читать:

- Главная цифра — «ложно сдали»: доля тех, кто ниже заявленного грейда, но сдал. Без утечки это 8 %. Если слито всё главное семейство и его знают все (K = 100), доля вырастает до 41 %, а с карантином монитора — до 24 %. Монитор срезает примерно половину вреда, но не весь.
- «Верное сдал/не сдал» при большой утечке растёт. Это не улучшение: знающим ответы сильным кандидатам проще сдать, и счёт в эту сторону выигрывает за счёт ложных сдач. Смотреть на эту колонку в отрыве от «ложно сдали» нельзя.
- При K = 0 монитор отправил в карантин 4 ключа. Это ложные тревоги на чистых данных, точность от них не страдает.
- Один прогон, одно зерно, без доверительных интервалов. Числа — из симуляции на тех же моделях, что и основной отчёт, а не из данных людей.

Отдельный прогон на 5000

Команда: `python -m app.validation.assessment --n 5000 --seed 7`. Ответ вводится в 82% выданных заданий. Порог по теме на этих 5000: точность 55.9%, ложные сдачи 4.3%. Оценка различимости: ошибка a 0.145 и 0.267. В решении о грейде порога по теме нет, в банк пишется априор. Эта таблица стоит рядом с прогоном на 2000 и в пороги CI не входит.

Сценарий	Точность «сдал/не сдал»	Точность вне ±0,5 от порога	Сдали из тех, кто на уровне	Ложно сдали	Ложная рекомендация теста выше	Заданий: среднее / 90 %	Решено до лимита	Макс. экспозиция ключа	Доля концептуальных
Без утечки	78.4%	92.4%	73.4%	8.4%	1.0%	14.2 / 20	63%	40%	7%
Без утечки, все задания с вариантами	70.9%	85.1%	63.3%	9.0%	0.8%	16.4 / 20	49%	47%	13%
Утечка у 30 %, без мониторинга	78.9%	91.3%	77.7%	17.8%	2.8%	14.3 / 20	63%	40%	7%
Утечка у 30 %, ключи на карантине	76.9%	89.1%	73.4%	13.3%	2.0%	14.7 / 20	59%	50%	10%

Повторный тест: 2500 кандидатов проходят тест дважды, задания каждый раз новые.

Формат	Тот же итог «сдал/не сдал»	Каппа Коэна	То же вне ±0,5 от порога	Тот же грейд	Корреляция оценок θ
Числовой ответ вводится	75%	0.49	84%	60%	0.85
Все задания с вариантами	70%	0.39	77%	55%	0.83

Формат ответа	Ключей с ≥ 30 ответами	Медиана r	Ключей с r ≥ 0,20	Верно у тех, кто на 0,5 слабее задания	Верно у тех, кто на 0,5 сильнее
Число вводится	55	0.21	56%	31%	72%
Выбор из четырёх	18	0.14	22%	55%	80%

- Мониторинг утечки: из 38 утёкших ключей с ≥ 30 ответами пойманы 25, ложных срабатываний среди остальных — 1; на чистых данных — 12 из 73 ключей.
- Случайное угадывание проходит тест в 0.0% случаев (все задания с вариантами — 0.0%).
- Точный грейд в сценарии «без утечки»: 56.8%, в пределах одной ступени: 84.6%, завышение: 2.3%.
- Взвешенная каппа повторного теста (квадратичные веса по грейду): 0.51.
- Ключ от другого экземпляра того же семейства: сдают 0.0%. Ключ от того же экземпляра (фиксированный банк): сдают 100.0%.
- Второй сигнал утечки (CUSUM): поймано 15 утёкших ключей, ложных тревог на чистых данных — 2. Карантин ставится, если сработал z или CUSUM.
- Точность классификации по Руднеру у порога заявленного грейда: 83.6%, согласованность 79.3%. Вероятность принятого решения по апостериорному распределению: 83.6%.
- Перекрытие заданий: в среднем другой кандидат этой же симуляции видел 16.9% заданий данного кандидата.

- Экспозиция по регионам на этом прогоне: 42%, 40%, 40%, 37%, отношение меньшей к большей 0.89. В опубликованную проверку справедливости входит прогон на 2000.

Истинный \ назначенный	ниже	intern	junior	middle	senior	lead
ниже	11	1	0	0	0	0
intern	58	104	6	0	0	0
junior	123	209	506	24	0	0
middle	319	2	439	988	58	0
senior	245	0	0	405	856	28
lead	24	0	0	0	218	376

Итоговый грейд после пути

Одна сессия отвечает на вопрос «сдал ли заявленный уровень». Продукт ведёт человека дальше: не сдал — до двух шагов вниз, попытка не сгорает; если те же ответы подтверждают ступень ниже, человек принимает её и путь заканчивается; сдал и сессия рекомендует ступень выше — один отдельный тест на неё. Итог — последний подтверждённый грейд или «не подтверждён». Подтверждённый грейд вниз не сдвигается. Команда та же: `python -m app.validation.assessment --n 2000 --seed 7`.

Путь	Точный грейд	±1	Завышение	Занижение	Не подтверждён	Сессий	Заданий
Первый	64,5 %	98,9 %	2,4 %	32,2 %	1,0 %	1,24	18,4
Повторный	65,4 %	98,7 %	2,3 %	31,4 %	1,1 %	1,28	18,6

Завышение после пути 2,4 %, порог проверки 5 %. Согласованность двух независимых путей: каппа Коэна итогового грейда 0,53, взвешенная каппа 0,85 при «не подтверждён» ниже стажёра и 0,76 при исходном порядке.

Повтор всей процедуры одним человеком. Команда `cd backend && python -m app.validation.assessment --n 2000 --seed S`, строка «Тот же итоговый грейд в обоих путях». «Не подтверждён» считается ступенью ниже стажёра.

Зерно	Тот же грейд	Тот же или соседний	kw двух путей	kw, «не подтверждён» ниже стажёра
7	64,8 %	99,5 %	0,76	0,85
201	62,6 %	99,5 %	0,70	0,85
202	66,8 %	99,8 %	0,73	0,85
203	65,4 %	99,7 %	0,74	0,86
204	66,3 %	99,7 %	0,80	0,86
205	64,7 %	99,6 %	0,79	0,85
Отложенные 201–205, среднее	65,2 %	99,7 %	0,75	0,85

Что измерено: точное совпадение грейда при повторе около 65 %. Расхождение почти всегда на одну ступень: сессия подтверждает заявленный грейд, а у границы второй путь может остановиться ступенью ниже. Разброс kw между зёрнами 0,70–0,80, поэтому каждое значение публикуем вместе с зерном. Как улучшим: продление у границы (раздел ниже).

Про две каппы. Взвешенная каппа штрафует промах пропорционально квадрату расстояния между категориями, значит важен их порядок. В исходном расчёте «не подтверждён» стоит последней категорией, после lead: расхождение «не подтверждён — стажёр» считается как промах через всю шкалу. На шкале способности «не подтверждён» ниже стажёра; при таком порядке kw двух путей 0,85–0,86 на всех шести зёрнах и почти не скачет. Обе цифры печатает одна команда, строка «Согласованность двух путей». Матрицы и доли совпадения от порядка не зависят. Порядок категорий мы заметили 11.10.2026 при сверке методики; исходную цифру 0,75 оставляем рядом, чтобы было видно, что изменилось только упорядочение, а не тест. Рядом с любой из двух каппа ставим «тот же грейд 65 %».

Проверка чувствительности (зерно 7, `cd backend && python -m app.validation.assessment --n 2000 --seed 7`, строка «Только пары, где оба пути подтвердили грейд»): на 98,5 % пар, где оба пути подтвердили грейд, kw = 0,84. Эта каппа не зависит от того, куда поставить «не подтверждён». Значит, 0,85 держится не на перестановке категорий. 0,76 занижена тем, что «не подтверждён» стоял после lead, и редкая пара «не подтверждён — intern» считалась промахом через всю шкалу.

До кнопки «принять ступень ниже» тот же расчёт давал 65,1 % точного грейда, 30,9 % заниженных, 1,69 сессии и 25,2 задания на человека. Принятие из той же сессии убирает в среднем 0,44 отдельного теста, точность не теряется.

Те же 2000 человек и тот же первый путь, команда не менялась. Занижение — подтверждённый грейд ниже истинного; человек без грейда в эту долю не входит. В нижней половине своего грейда (θ − порог < 0,5) занижено 48,8 % из 964, в верхней — 14,7 % из 1033. Кто заявил грейд ниже истинного, занижен в 73,7 % случаев (171 человек); кто заявил верно — в 30,1 % (1260); кто заявил

выше — в 20,7 % (566). Доля сдавших именно тест заявленного уровня по θ − порог этого грейда, шаг 0,25: 0,8 %, 3,8 %, 7,1 %, 13,7 %, 38,5 %, 63,2 %, 77,7 %, 91,7 %, 94,7 %. Доля 0,5 приходится на θ − порог = +0,116. Порог 0,75 сдвигает сдачу вверх от разреза и сильнее задевает нижнюю половину грейда.

Истинный \ итоговый	ниже	intern	junior	middle	senior	lead
ниже	3	0	0	0	0	0
intern	19	42	0	0	0	0
junior	0	112	240	5	0	0
middle	0	4	233	483	22	0
senior	0	0	1	188	388	15
lead	0	0	0	1	83	161

Ошибка модели, фиксированная форма, чужие сессии

Ответ в этих трёх строках рисуется четырёхпараметрической кривой, которую оценщик не знает: различимость умножается на exp(N(0, σ)), трудность сдвигается на N(0, σ), угадывание растёт на равномерный шаг до 0,1, верхняя асимптота 0,95. Оценщик считает номинальной трёхпараметрической моделью. Та же команда на 2000 человек, зерно 7.

σ	Точность вне ±0,5 от порога	Ложно сдали	Каппа повтора
0,15	92,8 %	10,0 %	0,47
0,30	93,1 %	9,1 %	0,44
0,50	92,6 %	9,3 %	0,39

При σ = 0,30 точность вдали от порога 93,1 % и ложные сдачи 9,1 % остаются внутри порогов 85 % и 12 %. При σ = 0,50 тоже. Калибровка на реальных ответах — первый шаг пилота: здесь ошибка модели задана, а не измерена.

Фиксированная форма: 40 заданий всех уровней специализации backend, 3000 человек, способность из нормального распределения с нулём и единичной дисперсией. Медиана скорректированной корреляции задания с суммой остальных 0,32, доля заданий с корреляцией не ниже 0,2 — 92 %. В адаптивном тесте медиана для вводимого ответа 0,24: задание видит узкий диапазон способностей, поэтому корреляция ниже.

Атака «20 сессий»: человек с истинным грейдом на ступень ниже middle сдаёт тест на middle, заранее посмотрев 20 чужих сессий на middle; верным засчитывается только повторившийся экземпляр (семейство, уровень и параметры). Фиксированный банк сдают 100 %, ключ от другого экземпляра того же семейства — 0 %. Механизм найден по счётчику совпадений: повторялись экземпляры узких семейств — подсчёт операций (algo_ops, уровни 2 и 4: 14 и 15 вариантов) и срезы Python (py_eval, уровень 2), плюс конечный банк вопросов на понимание. Исправление в продукте (11.10.2026): у этих уровней расширены диапазоны параметров (algo_ops 2: n 6–300; 4: n 5–60, k 2–7; 5: n 5–14; py_eval 2: n 16–40, i 0–5, j 6–15, k 2–4); проверка ключей запуском `python -m app.validation.key_check --variants 20` — 1875 из 1875, психометрия сессий (доля верных решений, ECE, экспозиция) не изменилась, потому что параметры заданий (a, b) те же. Воспроизводится одной командой: `cd backend && python -m app.validation.assessment --stolen` (400 человек на зерно, зёрна 7, 201–205, калибровка на 3000 человек — те же правда, калибровка и зерно, что в основном прогоне; Python 3.12 и 3.13 дают одинаковый результат). Всего 2400 человек: честная сдача 5,3 % (95 % ДИ Уилсона 4,5–6,3 %), после атаки 5,7 % (4,8–6,7 %), то есть в пределах интервала честной сдачи. До исправления (тот же расчёт на коде до c24ffa1) — 6,2 % (5,3–7,2 %): исправление убрало часть совпадений, но и до него атака давала немного. По зёрнам после исправления: 6,8 / 5,2 / 8,8 / 4,5 / 6,0 / 3,0 % против честных 4,0 / 4,0 / 8,5 / 4,0 / 7,2 / 4,0 %. Основной прогон `python -m app.validation.assessment` (80 человек, зерно 7, калибровка на 3000): честно 1,2 %, после атаки 3,8 % — это 1 против 3 человек, интервалы на 80 людях широкие (0,2–6,7 % и 1,3–10,5 %). С `--n 2000` калибровка другая и на тех же 80 людях выходит 5,0 % против 7,5 %; поэтому вывод делаем по 2400 людям. Прежняя запись «6,5 % / 7,2 % / 8,3 %» была получена разовым скриптом, которого нет в репо, и этой командой не воспроизводится; заменена. Остаток прироста даёт конечный банк вопросов на понимание: их ответ не вычисляется, его можно запомнить.

Перебор порогов остановки

Каппа повторной сдачи в опубликованном прогоне 0,49 (таблица перебора ниже — из прежнего прогона и на a63e4ca не пересчитывалась). Команда `python -m app.validation.assessment --n 2000 --seed 7 --grid` перебирает длину 20 и 24, пороги отношения правдоподобия 0,10 и 0,05 и ширину зоны 0,25 и 0,35. Клетку можно принять только если каппа не ниже 0,52, средняя длина не выше 16,5 и держатся все 9 прежних проверок. Константы теста те же: каппа в переборе не выше 0,49.

Длина	α = β	Зона	Каппа	Точность вне ±0,5	Ложно сдали	Средняя длина	9 проверок	Принять
20	0,10	0,25	0,47	91,5 %	7,7 %	14,3	нет	нет
20	0,10	0,35	0,49	92,3 %	9,0 %	14,0	да	нет
20	0,05	0,25	0,49	91,5 %	7,7 %	14,3	нет	нет
20	0,05	0,35	0,49	91,5 %	7,7 %	14,3	нет	нет

Длина	α = β	Зона	Каппа	Точность вне ±0,5	Ложно сдали	Средняя длина	9 проверок	Принять
24	0,10	0,25	–	–	–	–	нет	нет
24	0,10	0,35	–	–	–	–	нет	нет
24	0,05	0,25	–	–	–	–	нет	нет
24	0,05	0,35	–	–	–	–	нет	нет

Длина 24 в переборе не используется: для части специализаций подбор задания останавливается. Медиана r выбора из четырёх на этом зерне 0,1498, в таблице выше она печатается как 0,15. Каппа в переборе не выше 0,49 при пороге клетки 0,52.

Адаптивный тест против линейной формы и случайного набора

Зачем адаптивность. Те же 2000 человек, та же скрытая правда (сдвиги трудности и различимости), та же калибровка банка, что в основном прогоне, seed 7. Сравниваются четыре способа: (а) продукт — задания у порога, ранняя остановка, 8–20 заданий; (б) тот же выбор заданий, но всегда L штук; (в) одна линейная форма на специализацию: L заданий по темам плана и равномерно по уровням 1–5; (г) случайный набор: L ключей банка специализации, у каждого свои. Решение о грейде у всех одно и то же (`cat.decide`, априор в пороге заявленного грейда). L = 14 (средняя длина продукта, округлённо) и L = 20 (потолок продукта). Литература: Weiss & Kingsbury 1984, doi:10.1111/j.1745-3984.1984.tb01040.x; Spray & Reckase 1996, doi:10.3102/10769986021004405.

Способ	Заданий	Точность «сдал/не сдал»	Вне ±0,5 от порога	Ложно сдали	Смещение θ̂	RMSE θ̂	Грейд по правилу: точно / ±1	Грейд по θ̂: точно	kw грейда по θ̂
Продукт: выбор у порога, ранняя остановка	14,3	77,8 %	91,7 %	7,4 %	-0,127	0,466	57,4 % / 84,7 %	71,7 %	0,790
Адаптивный, всегда 14 заданий	14,0	71,9 %	83,6 %	4,6 %	-0,152	0,452	54,2 % / 88,6 %	68,5 %	0,786
Линейная форма, 14 заданий	14,0	62,4 %	74,4 %	9,1 %	-0,155	0,555	45,6 % / 86,5 %	61,6 %	0,733
Случайный набор, 14 заданий	14,0	64,7 %	76,1 %	6,9 %	-0,165	0,554	47,7 % / 86,5 %	60,7 %	0,721
Адаптивный, всегда 20 заданий	20,0	76,2 %	91,2 %	6,0 %	-0,123	0,412	57,8 % / 88,5 %	71,0 %	0,788
Линейная форма, 20 заданий	20,0	69,2 %	80,1 %	4,7 %	-0,126	0,477	54,2 % / 90,2 %	67,7 %	0,792
Случайный набор, 20 заданий	20,0	66,9 %	79,3 %	9,3 %	-0,145	0,497	50,5 % / 88,5 %	64,5 %	0,739

Продукт минус линейная форма на тех же людях, доля верных решений «сдал/не сдал», парный бутстреп 1000 повторов:

Против	Все	Вне ±0,5 от порога
линейной формы из 14	+16,1 п. п. (95 % ДИ +13,6…+18,4)	+17,0 п. п. (+14,2…+19,9)
линейной формы из 20	+8,9 п. п. (+6,6…+11,2)	+11,3 п. п. (+8,7…+13,9)

Сколько заданий нужно до SE ≤ 0,5 и SE ≤ 0,4 (прогоны по 20 заданий; среднее среди дошедших, в скобках доля дошедших): адаптивный выбор — 10,4 (95,0 %) и 16,5 (75,0 %); линейная форма — 16,5 (74,1 %) и не доходит ни один (0,0 %); случайный набор — 16,1 (71,3 %) и 19,6 (1,5 %).

Что видно, без прикрас: - Решение у порога — главное, ради чего тест адаптивный. Продукт за 14,3 задания в среднем точнее линейной формы из 20 заданий (91,7 % против 80,1 % вне ±0,5). - Ложные сдачи у продукта 7,4 %. Это больше, чем у адаптивного теста из 14 (4,6 %) и линейной формы из 20 (4,7 %): ранняя остановка по апостериорной вероятности иногда сдаёт тех, кто чуть ниже порога (GLR на симуляции у порога не срабатывает, `python tests/stop_reasons.py`). Цена укороченного теста видна здесь. - Грейд по правилу «±1» у продукта ниже, чем у любых форм фиксированной длины (84,7 % против 86–90 %): продукт решает про заявленный грейд и рано останавливается, а «не подтверждён» считается промахом. Для итогового грейда есть отдельная процедура пути (раздел «Итоговый грейд после пути»). - Грейд по θ̂ на всей шкале (kw): на 14 заданиях адаптивный выбор лучше линейной формы (0,786 против 0,733). На 20 заданиях разница пропадает (0,788 против 0,792): выбор заданий нацелен на порог, а не на точность θ по всей шкале. - Смещение θ̂ отрицательное у всех способов (−0,12…−0,17): априор стоит в нижней границе заявленного грейда и тянет оценку вниз.

Синтетика: правда берётся из модели симуляции, реальных людей здесь нет. Числа пересняты 11.10.2026 на main после расширения банка семействами frontend / devops / mobile. Воспроизвести: `cd backend && python -m app.validation.cat_baselines --n 2000 --seed 7` (около 5 минут; sha256 вывода `00430fde…`, два прогона совпали). Код: `app/validation/cat_baselines.py`, общая когорта — `app/validation/sim_base.py`, тесты — `tests/unit/test_cat_baselines.py`.

Потолок семьи и складной нож

Измерение: `python -m app.validation.assessment --family-grid --n 2000 --seed 7`. Клетка принимается, только если сразу ложные сдачи при утечке не выше 12 %, без утечки не выше 8 %, сдали из тех, кто на уровне не ниже 71 %, точность вдали от порога не ниже 91 %, средняя длина не выше 15. В продукте потолок 5 без ножа: 18,2 %, 7,7 %, 73,0 %, 91,5 %, 14,3. Потолок 5 с ножом: 12,0 %, 6,7 %, 69,0 %, 88,9 %, 14,3.

Знаменатель Симпсон-Хеттер

09.10.2026. Симпсон-Хеттер по специализации с калиброванным K

Знаменатель доли показов — сессии своей специализации, прогрев 20 сессий этой специализации. Вероятность взять ключ из окна — константа K, десять итераций на зёрнах 101–105, цель 0,40. Проверка один раз на 201–205. Команда: `python -m app.validation.assessment --sh-holdout --n 500`. Публичные зёрна не использовались.

Зерно	max в специализации было	стало	Δ точности вне ±0,5, п.п.	Δ каппы
201	0,773	0,523	−1,50	+0,020
202	0,775	0,525	−0,71	+0,002
203	0,791	0,514	+0,78	−0,017
204	0,816	0,579	−1,12	−0,034
205	0,755	0,509	+1,17	+0,007

Максимум внутри специализации в этом опыте 0,51–0,58, каппа повтора сдвигается до 0,034. Точность вдали от порога на двух зёрнах из пяти ниже больше чем на 1 процентный пункт; команда сообщает «порог не пройден» и завершается с кодом 1 (переснято 11.10.2026). Выученные константы K в продукт не входят. Живой банк считает долю показов среди завершённых сессий той же специализации и сравнивает её с той же целью 0,40. Опубликованная таблица выше по-прежнему считает знаменатель по всем сессиям: отсюда 72 % у `js_eval:3` в сессиях mobile.

11.10.2026. Доля показов внутри специализации: основной отчёт и продукт

Основная таблица считает долю показов по всем сессиям сразу, а живой банк — по сессиям своей специализации (`services/assessment.py`, `bank_state`). Поэтому 72 % в таблице описывают не тот селектор, что работает в продукте. Ниже та же чистая когорта на тех же людях и случайных потоках, с обоими знаменателями. Команда: `cd backend && python -m app.validation.assessment --n 2000 --seed S --spec-exposure`.

Зерно	Макс. доля ключа в специализации: все сессии → своя специализация	Точность	Точность вне ±0,5	к решения с истиной	Точный грейд	RMSE θ̂
7	71,4 % → 54,2 %	78,8 % → 77,7 %	93,0 % → 92,4 %	0,558 → 0,533	58,8 % → 57,6 %	0,460 → 0,466
201	70,5 % → 51,6 %	77,3 % → 78,1 %	91,0 % → 90,8 %	0,517 → 0,536	57,2 % → 58,0 %	0,471 → 0,474
202	72,8 % → 58,8 %	80,7 % → 80,4 %	94,6 % → 94,9 %	0,573 → 0,571	62,5 % → 61,2 %	0,428 → 0,444
203	70,6 % → 49,7 %	75,6 % → 74,5 %	90,0 % → 89,7 %	0,508 → 0,489	54,7 % → 53,4 %	0,480 → 0,491
204	70,5 % → 52,8 %	78,0 % → 75,9 %	93,2 % → 92,4 %	0,533 → 0,498	58,9 % → 56,6 %	0,454 → 0,466
205	74,2 % → 65,0 %	77,5 % → 78,3 %	93,0 % → 92,9 %	0,521 → 0,536	57,6 % → 58,7 %	0,456 → 0,440
201–205, среднее	71,7 % → 55,6 %	77,8 % → 77,4 %	92,8 % → 92,5 %	0,530 → 0,526	58,2 % → 57,6 %	0,458 → 0,463

В продукте самый частый ключ внутри специализации на этой когорте видят 49,7–65,0 % сессий. Цель 0,40 уже стоит в селекторе; банк узкий, поэтому доля выше цели. Калибровка K из раздела выше снижает долю до 0,51–0,58 и на двух отложенных зёрнах из пяти теряет больше 1 п. п. точности, поэтому выученные константы в продукт не входят. Дальше банк пополняется ключами JavaScript нужного уровня. Ключ — это семейство и уровень, а не сам текст: каждый показ — новый экземпляр, ответ чужого экземпляра совпадает в коде в 9,3 % случаев.

Продление у границы

Измерение: `python -m app.validation.assessment --n 2000 --seed 7 --extend`. На 2000 кандидатах, зерно 7: 76,9 % сдавших среди тех, кто на уровне при пороге 77 %, ложные сдачи 9,8 % при пороге 8,5 %, занижение после пути 27,4 % при пороге 26 %. Живой тест заканчивается на 20-м задании.

Зона неопределённости решения о грейде

Решение по первому тесту называется **пограничным**, если оценка лежит в пределах одной стандартной ошибки от порога целевого грейда: |θ̂ − порог| < SE. Это видно в момент решения, истинная θ для этого не нужна. Повторный тест с новыми заданиями считается отдельно для уверенных и пограничных решений. 95 % доверительный интервал — перцентильный бутстреп (2000 повторов, фиксированное зерно). Протокол зафиксирован до прогона, порог 1·SE не подбирался: зерно 7 — основное, зёрна 201–205 — отложенные.

Зерно	Доля пограничных	Тот же итог у уверенных (95 % ДИ)	Тот же итог у пограничных
7	28,2 %	82,7 % (80,1–85,4 %)	54,3 %
201	30,8 %	81,4 % (78,5–84,2 %)	65,6 %
202	28,6 %	80,7 % (77,7–83,8 %)	55,6 %
203	30,1 %	77,4 % (74,2–80,5 %)	59,8 %
204	28,6 %	83,3 % (80,5–85,9 %)	61,2 %
205	28,5 %	78,7 % (75,8–81,8 %)	60,7 %

Вывод. Разделение работает: уверенные решения повторяются на 15–28 п. п. чаще пограничных. Целевой порог для уверенных решений — 85 %; в среднем на отложенных зёрнах 80,3 %, и ни один интервал не лежит целиком выше 85 %. Около 30 % решений пограничные. Это свойство задачи: ширина полосы грейда 1,0, а SE теста из 8–20 заданий около 0,6 (см. «Где тест ошибается»). С 11.10.2026 карточка кандидата помечает подтверждённый тестом грейд «на границе уровня», если |θ̂ − порог| < SE (`cat.is_borderline`). Грейд при этом не меняется и не скрывается. Продление теста у границы реализовано (`cat.ALLOW_EXTEND`), но выключено: в разделе «Продление у границы» оно не прошло пороги, и без нового замера оно не включается. Команда: `python -m app.validation.assessment --n 2000 --seed 7` (и `--seed 201` … `205`). Источники метода: Eggen (1999), Thompson (2009) — классификационный CAT с зоной безразличия; Livingston & Lewis (1995) — согласованность решений.

Почему ±1 ступень — 84 %, а не 95–100 %

Тест подтверждает заявленный грейд, а не ищет настоящий. Если сессия не сдала заявленный уровень, грейд остаётся «не подтверждён», и основной отчёт считает это промахом, раз настоящий грейд у человека есть. Кроме того, классов шесть (от «ниже стажёра» до lead): чем больше ступеней, тем строже мера «±1 ступень».

Команда: `cd backend && python -m app.validation.grade_misses --n 2000 --seed 7`

```
Без утечки, 2000 кандидатов, зерно 7.
±1 по всем сессиям: 84.0%. Вне ±1: заявил выше и не подтверждён 12.9%; не подтверждён при заявленном не выше настоящего 3.0%; подтверждён, но дальше ступени 0.10%.
Подтверждено 84.0% сессий, среди них ±1: 99.9%.
```

Итог: из 16 % промахов 12,9 п. п. — люди, заявившие грейд выше настоящего и не подтвердившие его; так и задумано. Ложный отказ при честной заявке — 3,0 %. Среди подтверждённых грейдов ±1 — 99,9 %. Продукт и пороги не менялись.

Сопоставимые строки

Эти строки считают метрики в распространённых определениях (верное решение, kw против истинного грейда). Правила продукта они не меняют. Команда: `cd backend && python -m app.validation.assessment --n 2000 --seed S --parity`. Она печатает ту же чистую когорту («без утечки»), что и основной отчёт, на тех же случайных числах, поэтому 77,8 %, 92,6 % и 71,9 % на зерне 7 совпадают с основной таблицей. Грейд по θ̂ отрезается от оценки способности по тем же порогам `CUTS`, без ограничения заявленным грейдом (`ability_grade`, только в валидации). kw — квадратичные веса по шкале «ниже, intern … lead». 95 % ДИ — бутстреп, 1000 повторов.

Зерно	Анкета: точный грейд	База «верить анкете»: верное решение	Тест: верное решение	Завышение не подтверждено	Честный грейд подтверждён	kw анкеты	kw грейда продукта (строгая)	kw грейда по θ̂ (95 % ДИ)
7	63,0 %	71,5 %	77,8 %	92,6 %	71,9 %	0,828	0,513	0,864 (0,852–0,876)
201	63,1 %	72,9 %	76,9 %	89,7 %	72,1 %	0,829	0,586	0,873 (0,863–0,884)
202	64,0 %	73,5 %	82,1 %	91,9 %	78,6 %	0,832	0,537	0,881 (0,871–0,891)
203	63,7 %	72,5 %	75,5 %	94,5 %	68,4 %	0,832	0,465	0,861 (0,848–0,872)

Зерно	Анкета: точный грейд	База «верить анкете»: верное решение	Тест: верное решение	Завышение не подтверждено	Честный грейд подтверждён	kw анкеты	kw грейда продукта (строгая)	kw грейда по θ̂ (95 % ДИ)
204	65,3 %	73,8 %	77,2 %	92,4 %	71,8 %	0,845	0,529	0,870 (0,857–0,882)
205	66,0 %	75,5 %	77,4 %	93,0 %	72,3 %	0,841	0,530	0,868 (0,856–0,879)
201–205, среднее	64,4 %	73,6 %	77,8 %	92,3 %	72,6 %	0,836	0,529	0,871

Как это читать, без прикрас: - Раньше мы оценивали базу «верить анкете» примерно в 60 %, и эта оценка не подтвердилась. На деле анкета называет точный грейд в 63–66 % случаев, а решение «сдал / не сдал» по анкете верно в 71,5–75,5 %. Тест выигрывает у анкеты 4–6 п. п. по точности решения. Зато он отсекает 92 % завышений, а анкета не отсекает ни одного. - kw грейда из анкеты (0,83) выше, чем у грейда, который назначает продукт (0,51–0,53). Продукт не ставит грейд выше заявленного, а несданный грейд у него уходит в «ниже». Поэтому kw продукта против истины мала, хотя сама оценка способности точная: kw грейда по θ̂ 0,87, r(θ̂, θ) = 0,919. Строку «по θ̂» мы показываем только рядом со строгой 0,51. - Строгие цифры повтора не меняются: kw двух путей 0,85 на зерне 7 и 0,85 на отложенных зёрнах («не подтверждён» ниже стажёра; при исходном порядке 0,76 и 0,75 ± 0,04), тот же грейд при повторе 64,8 % и 65,2 %, kw повтора одного теста 0,56 и 0,55, тот же итог у уверенных решений 82,7 % и 80,3 % (`python -m app.validation.assessment --n 2000 --seed S`). - Всё это синтетика, реальных людей тут нет.

Кривые сигналов честности

Команда: `cd backend && python -m app.validation.detector_curves` (2000 честных и 2000 сессий с утечкой половины ключей на зерно, зёрна 7 и 201–205, около 35 с). Статистику считают функции продукта (`integrity_level`, `note_pace`, `person_fit`, `note_family_cusum`), скрипт только перебирает пороги. В отличие от `app.validation.integrity`, уходы со вкладки и копирование одинаковы в обеих группах, поэтому баллы риска не отделяют группы за счёт самой модели. Сигналы только просят посмотреть человека и грейд не меняют. Это проверка на модели: настоящее распределение времени ответов ещё не известно.

Баллы риска (уровень «умеренный»+)

Порог	зерно 7	зерно 201	зерно 202	зерно 203	зерно 204	зерно 205	среднее 201–205
1	91.0% / 8.1%	91.0% / 8.9%	90.5% / 9.2%	91.3% / 8.2%	90.3% / 8.8%	91.1% / 8.6%	90.9% / 8.8%
2 ★	90.5% / 0.5%	90.4% / 0.5%	90.2% / 0.8%	89.9% / 0.9%	89.5% / 0.9%	90.2% / 0.7%	90.0% / 0.7%
3	7.3% / 0.1%	7.7% / 0.0%	9.2% / 0.0%	8.3% / 0.0%	7.8% / 0.1%	7.8% / 0.1%	8.2% / 0.0%
4	0.1% / 0.0%	0.1% / 0.0%	0.3% / 0.0%	0.1% / 0.0%	0.1% / 0.0%	0.1% / 0.0%	0.1% / 0.0%

Быстрые маловероятные верные ответы

Порог	зерно 7	зерно 201	зерно 202	зерно 203	зерно 204	зерно 205	среднее 201–205
1	98.0% / 5.5%	97.3% / 6.0%	97.5% / 5.1%	97.9% / 5.6%	97.5% / 5.7%	97.7% / 5.8%	97.6% / 5.6%
2 ★	90.5% / 0.4%	90.3% / 0.4%	90.2% / 0.4%	89.9% / 0.5%	89.4% / 0.8%	90.2% / 0.6%	90.0% / 0.5%
3	78.2% / 0.1%	76.8% / 0.0%	76.6% / 0.1%	76.1% / 0.1%	76.8% / 0.1%	77.2% / 0.1%	76.7% / 0.1%
4	60.7% / 0.0%	59.5% / 0.0%	59.5% / 0.0%	58.7% / 0.1%	60.2% / 0.0%	60.5% / 0.0%	59.6% / 0.0%
5	43.1% / 0.0%	41.6% / 0.0%	43.1% / 0.0%	41.9% / 0.0%	43.1% / 0.0%	43.0% / 0.0%	42.5% / 0.0%

Согласованность ответов lz*

Порог	зерно 7	зерно 201	зерно 202	зерно 203	зерно 204	зерно 205	среднее 201–205
-1	56.7% / 12.8%	56.7% / 13.0%	54.4% / 13.1%	56.9% / 12.2%	54.5% / 13.0%	54.7% / 12.7%	55.4% / 12.8%
-1.282	48.0% / 9.1%	48.6% / 8.8%	46.7% / 9.4%	47.9% / 8.4%	46.2% / 9.0%	46.9% / 8.6%	47.2% / 8.8%
-1.645 ★	38.7% / 5.5%	38.1% / 4.9%	37.6% / 5.4%	37.8% / 4.6%	36.0% / 5.1%	37.5% / 5.3%	37.4% / 5.1%
-2	28.9% / 2.6%	29.8% / 2.9%	29.6% / 2.9%	29.8% / 2.7%	28.6% / 3.4%	29.1% / 2.7%	29.4% / 2.9%
-2.326	22.4% / 1.4%	22.3% / 1.2%	23.5% / 1.8%	22.8% / 1.6%	21.2% / 2.1%	22.2% / 1.5%	22.4% / 1.6%

CUSUM по семействам

Порог	зерно 7	зерно 201	зерно 202	зерно 203	зерно 204	зерно 205	среднее 201–205
1	98.3% / 71.1%	98.6% / 72.8%	98.3% / 72.5%	98.0% / 72.0%	98.5% / 71.4%	97.7% / 71.0%	98.2% / 71.9%
1.3	93.3% / 54.2%	93.6% / 54.9%	93.9% / 54.7%	93.0% / 53.9%	94.1% / 54.5%	92.7% / 52.6%	93.5% / 54.1%

Порог	зерно 7	зерно 201	зерно 202	зерно 203	зерно 204	зерно 205	среднее 201–205
1.6 ★	75.6% / 8.7%	74.9% / 8.4%	76.6% / 8.3%	75.9% / 8.1%	78.0% / 8.6%	76.9% / 8.3%	76.5% / 8.3%
2	48.5% / 2.6%	46.7% / 2.2%	48.9% / 2.2%	47.9% / 2.2%	50.6% / 1.9%	50.0% / 2.1%	48.9% / 2.1%
2.5	18.2% / 0.1%	17.5% / 0.2%	17.4% / 0.4%	17.8% / 0.1%	18.9% / 0.3%	18.7% / 0.2%	18.1% / 0.3%

В ячейке: поймано / ложно.

Что показывают кривые. Баллы риска и быстрые маловероятные ответы на порогах продукта ловят около 90 % сессий с утечкой при 0,5–0,7 % ложных у честных. Согласованность ответов lz* при пороге −1,645 ловит 37 % при 5 % ложных: это вспомогательный сигнал, поэтому он помечает сессию. CUSUM по семействам при пороге 1,6 даёт 8 % ложных у честных, поэтому в продукте он тоже помечает сессию, а не решает. Сигнал утечки по ключу (z и CUSUM в `item_monitor`) работает по ключам, а не по сессиям, и считается в основном отчёте.

Оценка способности: EAP против WLE

Команда: `cd backend && python -m app.validation.wle_eap --n 2000` (зёрна 7 и 201–205, около 4,5 мин). Одни и те же ответы 20 заданий, выбранных по информации из случайного пула, оцениваются дважды: EAP (`cat.posterior` с нейтральным априором N(0, 1)) и WLE Уорма. Это проверка оценщика, а не пути продукта: в продукте априор центрирован на пороге заявленного грейда и выбор идёт по информации в пороге, поэтому RMSE здесь (0,27) ниже, чем в основном отчёте (0,47).

Зерно	Оценка	Смещение	RMSE	Точный грейд	±1 грейд	kw	Смещение θ<−1,5	Смещение θ≥1,5
7	EAP	-0.002	0.270	80.8%	100.0%	0.928	+0.150	-0.164
7	WLE	-0.002	0.274	79.8%	100.0%	0.928	-0.002	-0.016
201	EAP	+0.005	0.277	78.5%	100.0%	0.921	+0.190	-0.178
201	WLE	+0.006	0.277	78.1%	100.0%	0.923	+0.047	-0.030
202	EAP	-0.006	0.266	81.5%	100.0%	0.930	+0.171	-0.135
202	WLE	-0.004	0.268	81.3%	100.0%	0.932	+0.029	+0.009
203	EAP	+0.008	0.267	79.5%	100.0%	0.927	+0.162	-0.175
203	WLE	+0.010	0.267	79.9%	100.0%	0.932	+0.016	-0.033
204	EAP	+0.003	0.267	79.5%	100.0%	0.926	+0.184	-0.157
204	WLE	+0.005	0.269	80.0%	100.0%	0.931	+0.048	-0.003
205	EAP	+0.002	0.268	79.8%	100.0%	0.923	+0.173	-0.142
205	WLE	+0.001	0.273	80.5%	100.0%	0.929	+0.024	+0.004

Среднее по отложенным зёрнам 201–205: RMSE EAP 0,269 против WLE 0,271; точный грейд 79,8 % против 80,0 %; kw 0,925 против 0,929. Разница в общем качестве в пределах разброса между зёрнами. Где они расходятся: EAP тянет крайних к центру — смещение +0,18 у θ < −1,5 и −0,16 у θ ≥ 1,5, у WLE +0,03 и −0,01. Для продукта это значит: у самых сильных EAP немного занижает оценку, то есть ошибается в осторожную сторону (не завышает lead). Переход на WLE в продукте сейчас не делаем: выигрыша по грейду нет, а EAP даёт апостериорные вероятности, на которых стоит правило решения P(θ ≥ порог) ≥ 0,75.

Что держится по построению

Часть цифр основного прогона верна отчасти потому, что так устроена симуляция. Ниже эти места названы прямо и пересчитаны без такого устройства. Команда: `python -m app.validation.construction_check --n 2000 --seed 7` (код `backend/app/validation/construction_check.py`).

Ответы из той же модели, которой тест оценивает. В основном прогоне ответы рисует та же трёхпараметрическая модель, поэтому точность, «завышенный грейд не подтверждён» и kw частично заданы устройством. Здесь оценщик прежний, а ответы рисуют две чужие модели; предварительная калибровка трудности заново снята на ответах той же чужой модели, как сделает пилот. Строка «3PL» — база этого прогона, она отличается от основного отчёта, потому что здесь нет контроля экспозиции и другой порядок случайных потоков.

Откуда ответы	«Сдал / не сдал» верно	Завышенный грейд не подтверждён	Честный грейд подтверждён	kw грейда по θ̂
3PL, как у оценщика	78,1 % (верить анкете 71,5 %)	90,0 %	73,4 %	0,86
4PL, ошибка параметров σ 0,30	77,2 % (верить анкете 71,5 %)	90,7 %	71,8 %	0,86
многомерная: сдвиг по семейству σ 0,6, срывы 5 %, угадывание +0,08	73,0 % (верить анкете 71,5 %)	84,9 %	68,3 %	0,84

Вывод: отсев завышенного грейда и kw устойчивы к смене модели (84,9–90,7 % и 0,84–0,86). Выигрыш решения «сдал / не сдал» над «верить анкете» при многомерных ответах сжимается до 1,5 п. п.: если умения по темам сильно расходятся, одна шкала способности теряет точность. Это проверит калибровка на ответах пилота.

Ключ от другого экземпляра. Строка «сдают 0 %» в основном отчёте получается по построению: совпадение ответа там мерили только у заданий на код (4 пары на уровень), вопросы на понимание получали 0, а атакующий без совпадения всегда ошибался. Честный прогон: совпадения измерены у всех семейств (40 пар на уровень), а там, где ключ не подходит, человек отвечает сам.

- Совпадение ответа у двух свежих экземпляров: код 9,3 %, вопросы на понимание 36,9 %; больше всех `concept_http`, 45,0 % (пул вопросов конечный). Специализация для замера — первая по алфавиту среди тех, где семейство доступно: обход множества зависел от `PYTHONHASHSEED` и давал то 9,1/30,9, то 9,2/34,7.
- Сдают с чужим ключом: 69,2 %; честно те же 400 человек: 58,5 %; прирост 10,8 п. п. (`python -m app.validation.construction_check`, зерно 7, 400 человек).

Защита от этого — не генерация, а контроль экспозиции и карантин ключей по статистике (раздел «Карантин») и расширение пула вопросов на понимание. Атака «20 чужих сессий» измерена без этого допущения: 5,7 % против 5,3 % у честных (2400 человек, `--stolen`).

Разметка подбора. Метка релевантности считается из скрытой правды по тем же факторам, которые взвешивает подбор: специализация, навыки, грейд (`relevance`, `app/validation/matching.py:162`). Подбор видит только зашумлённый профиль, поэтому цифра не тавтология, но и не независимое суждение HR: она проверяет, насколько подбор восстанавливает правду из шума. Независимую проверку даст разметка людьми.

Проверка подбора кандидатов

Механика подбора даёт 35 % оценки решения. Качество подбора проверено на синтетическом наборе со **скрытой правдой**: подбор видит только зашумлённые наблюдения, как в жизни, а релевантность считается по правде. Код проверки: `backend/app/validation/matching.py`. Ранжирует та же функция `app.services.matching.score`, что работает в API.

Как устроена оценка соответствия

- Попадание в список решает соответствие потребности: навыки 0,45, грейд 0,30, специализация 0,25, текст запроса 0,05, предметная область 0,05, город и формат 0,05. Город и формат выпадают из балла, если потребность их не называет. Несовпадение снижает этот фактор и человека из выдачи не убирает.
- Навык засчитывается с весом доказательства: подтверждён тестом 1,0, цитатой из резюме 0,75, заявлен 0,55. Близкая технология внутри одной группы (MySQL вместо PostgreSQL) даёт частичный зачёт 0,3. Более узкий инструмент закрывает общий навык сильнее: PostgreSQL закрывает SQL на 0,85 этого веса, SQL PostgreSQL сильнее 0,3 не закрывает. Таблица nDCG ниже снята на симметричных группах, до этой таблицы рёбер.
- Грейд: точное совпадение 1,0, на ступень выше 0,6, на ступень ниже 0,35; грейд без подтверждения тестом умножается на 0,85.
- **Порядок внутри категории** задают доказательства: тест (сила результата — нижняя граница вероятности уровня) и протокол ФСП (нижняя граница доверительного интервала, LCB), поровну. Они добавляются к соответствию с весом 0,3. Вес умножается на близость `g·r·(0,5 + 0,5·покрытие навыков)`: точный грейд 1, на ступень выше 0,6, на ступень ниже 0,35, дальше 0; та же специализация 1, соседняя 0,5, другая 0. Половина веса есть у каждого в категории, вторая половина растёт с покрытием навыков.
- Итог: внутри одной категории подтверждённый тестом грейд стоит выше заявленного, а один лишний заявленный навык не перекрывает сильный тест (закреплено в `backend/tests/unit/test_matching.py`).
- Неприменимый фактор не учитывается, веса остальных нормируются. Если в потребности нет предметной области или у кандидата нет истории ФСП, фактор выпадает, и кандидат не теряет баллы (ТЗ 2.1).
- Каждая строка объяснения несёт машинный код и источник (тест, ФСП, заявлено, вилка). У строк теста и ФСП в карточке виден их вклад в балл, например «+9 к баллу».
- Для потребности дополнительно подбираются **рекомендуемые категории**: ячейки «специализация × грейд» с самыми сильными совпадениями.

Синтетический набор

Кандидат: специализация, способность θ ~ N(0, 1), реальные навыки. Наблюдения искажены так, как это бывает в анкетах: заявленные навыки теряют 25 % реальных и добавляют около 15 % ложных; 30 % заявленных навыков подтверждены цитатой; заявленный грейд у 30 % кандидатов на ступень выше реального и у 10 % ниже; тест проходят 45 %, он подтверждает реальные навыки и уровень; у 55 % есть история ФСП с LCB, связанной с θ.

Релевантность (0–3) считается только по правде: 3 — та же специализация, все обязательные навыки реальны, грейд совпадает; 2 — специализация та же или смежная, навыки реальны, грейд отличается не больше чем на ступень; 1 — реальна хотя бы половина навыков; 0 — иначе. Формулу оценки разметка не видит. Город и формат дописываются после меток, отдельным зерном, и в релевантность не входят.

Что измеряем

- **nDCG@10** (нормированный дисконтированный совокупный выигрыш): насколько первые десять мест близки к идеальному порядку по релевантности, от 0 до 1.
- **P@10**: доля кандидатов с оценкой 2–3 в первой десятке. Если размеченных пар меньше десяти (свой файл разметки), считается по всем: P@min(10, n).
- **MAP** (средняя точность): насколько высоко стоят все релевантные кандидаты, а не только первая десятка.
- **Спирмен внутри категории**: ранговая корреляция порядка среди людей той категории, которую видит работодатель (та же специализация и показанный грейд), со скрытой способностью θ. Выше ноля — сильные стоят выше. Потребности, где в категории меньше пяти человек, пропускаются.
- **Завышенный грейд в топ-10**: доля людей в первой десятке, у которых показанный грейд выше реального.
- **95 % ДИ** (доверительный интервал): бутстреп по потребностям, 1000 повторов.
- **Поиск по ключевым словам (BM25)**: так ищет обычная полнотекстовая система. Текст потребности — специализация, грейд, обязательные и желательные навыки; текст профиля — специализация, показанный грейд и все заявленные навыки, без различия, подтверждён навык или нет. Ранжирует Okapi BM25 с учебными константами k1 = 1,2 и b = 0,75; редкость слова (IDF) и средняя длина считаются по тому же банку кандидатов. Город и формат — условия, а не ключевые слова, и в тексты не входят. Константы не подбирались на наборе. Код: `bm25_rank`, `need_text`, `profile_text` в `matching.py`.
- **Базовая линия «по резюме»**: число заявленных обязательных навыков, затем специализация и заявленный грейд. Это сильная база: она знает структуру потребности (какие навыки обязательны, какой грейд нужен) и сравнивает поля, а не слова.

Результаты

Набор с зерном 2026

Команда: `cd backend && python -m app.validation.matching`. 800 кандидатов, 78 потребностей с релевантными кандидатами. На Linux x86-64 (CI, Python 3.12 и 3.13) SHA-256 набора `1ebff9bbea1c3a3efa08684a86e3094bc6ec88a1867014ad655e9bd12ffc7e00`. Хэш считается по сырому JSON. С 11.10.2026 набор хранит проваленную попытку теста (`failed_grade`, раздел «Проваленный тест»), поэтому хэш сменился; прежний на Linux — `c83ce73df96d2f7cc7e1f4bd4b1d1fbf8f661d5fa482cf25b340b65c24100f49`. Другая сборка математики меняет хвост `gauss`, поэтому на Windows хэш другой. Таблицы ниже сняты на Linux, Python 3.12.

Порядок в таблице — тот же, что видит работодатель. Последнее место каждой десятки может занять человек без теста, если его балл не ниже чем на 0,08. Строка «без квоты для новичков» — прежняя сортировка только по баллу.

Ранжирование	nDCG@10	95 % ДИ nDCG	P@10, оценка ≥ 2	MAP	Спирмен внутри категории	95 % ДИ Спирмена	Завышенный грейд в топ-10	Настоящий грейд ниже потребности
Случайный порядок	0.034	0.025… 0.044	0.035	0.043	-0.006	-0.075… 0.064	30.0 %	44.0 %
Поиск по ключевым словам (BM25): текст потребности против текста профиля	0.562	0.522… 0.602	0.541	0.515	0.029	-0.035… 0.089	26.8 %	33.7 %
Базовая линия «по резюме»: заявленные навыки и грейд	0.615	0.575… 0.656	0.565	0.548	0.018	-0.048… 0.083	27.4 %	32.7 %
ФСП Талант: оценка соответствия	0.653	0.612… 0.695	0.662	0.613	0.359	0.302… 0.415	22.7 %	18.1 %
без квоты для новичков	0.652	0.612… 0.695	0.662	0.613	0.358	0.300… 0.413	22.8 %	18.1 %
без весов доверия к навыкам	0.661	0.621… 0.701	0.677	0.617	0.301	0.244… 0.358	24.5 %	21.9 %
без теста	0.639	0.595… 0.681	0.647	0.607	0.184	0.122… 0.245	22.9 %	19.5 %
без оси ФСП	0.653	0.614… 0.696	0.673	0.615	0.322	0.259… 0.379	23.5 %	20.8 %
без теста и без ФСП	0.665	0.626… 0.710	0.673	0.626	0.034	-0.026… 0.088	28.2 %	26.5 %
без близких технологий	0.660	0.621… 0.704	0.668	0.612	0.336	0.282… 0.391	22.8 %	19.1 %

«Завышенный грейд в топ-10» — показанный грейд выше настоящего. «Настоящий грейд ниже потребности» — настоящее ниже грейда, который просит потребность; это второе определение завышения, его используют чужие работы.

Три зерна

Та же команда печатает таблицу по зёрнам 2026, 2027 и 2028, чтобы вывод не держался на одном удачном наборе.

Ранжирование	nDCG@10	P@10, оценка ≥ 2	Спирмен внутри категории	Завышенный грейд в топ-10, %	Настоящий грейд ниже потребности, %
Поиск по ключевым словам (BM25): текст потребности против текста профиля	0.568 ± 0.009	0.526 ± 0.020	0.032 ± 0.024	27.5 ± 1.0	32.1 ± 1.6
Базовая линия «по резюме»: заявленные навыки и грейд	0.620 ± 0.005	0.559 ± 0.021	0.011 ± 0.017	28.1 ± 1.8	30.9 ± 2.1
ФСП Талант: оценка соответствия	0.652 ± 0.019	0.636 ± 0.025	0.374 ± 0.017	21.5 ± 2.9	14.7 ± 3.0
без теста	0.647 ± 0.008	0.633 ± 0.015	0.172 ± 0.012	20.9 ± 2.6	18.3 ± 1.2
без оси ФСП	0.651 ± 0.020	0.640 ± 0.031	0.340 ± 0.018	21.9 ± 3.2	16.6 ± 3.6
без теста и без ФСП	0.669 ± 0.004	0.648 ± 0.023	0.016 ± 0.016	27.6 ± 2.2	24.7 ± 1.7

> Строка продукта в этой таблице посчитана до настройки весов 11.10.2026 (#460). После неё та же команда на зёрнах 2026–2028 даёт nDCG@10 0,658 и P@10 0,632.

Та же команда следом печатает, из чего сложился завышенный грейд. На этих зёрнах он целиком заявленный: 21,5 % у полной модели и 28,1 % по резюме. Доля, где завышенную ступень подтвердил тест, — 0,0 % в обеих строках. Тест подтверждает заявленную ступень, только когда способность уже у её порога, и тогда скрытый грейд не ниже показанного. Порядок выдачи этот разрез не меняет.

На зерне 7 (`python -m app.validation.matching --seed 7 --seeds 0`): nDCG@10 0,699 у полной модели и 0,626 по резюме, Спирмен внутри категории 0,325 и −0,040, завышенный грейд в топ-10 22,3 % и 30,7 %.

Непройденный тест на заявленном грейде

Изменение от 11.10.2026 (DoR-N7). Кандидат выбирает грейд G и проходит тест на G. Тест не пройден, значит апостериорная вероятность θ ≥ порог G ниже порога сдачи (`app/services/cat.py`). Раньше такой кандидат ранжировался по заявленному G, как будто теста не было, а навыки, подтверждённые в той же попытке, поднимали его в категории G. Теперь подбор читает грейд на ступень ниже проваленного: `grade_for_matching` в `app/services/matching.py`. Карточка показывает выбранный грейд со статусом «не подтверждён» и строкой объяснения `GRADE_TEST_FAILED` («тест на «мидл» не пройден, в подборе учтён «джуниор»»). Человек не скрыт, он опущен (уточнение требований FTT.4). Подтверждённый грейд правило не трогает. Проваленная попытка на ступень выше показанного грейда тоже ничего не меняет. Поиск берёт последнюю попытку по специализации, кроме экспресс-пробы и теста по вакансии (`latest_failed_targets` в `app/services/session_marks.py`). В синтетическом наборе платформа видит ту же попытку как поле `failed_grade`. Метки релевантности и скрытая правда не изменились. Без теста (строка «без теста») правило выключено вместе с тестом. Параметров нет, на наборе ничего не подбиралось.

Принцип — ранжирование по вероятности релевантности (Robertson, 1977, doi:10.1108/eb026647): кандидата ставят по тому, что о нём известно, а непрошедшая классификационная попытка CAT — свидетельство против заявленной ступени (Eggen и Straetmans, 2000, doi:10.1177/00131640021970862).

До и после, та же команда `python -m app.validation.matching --check`, зёрна 2026–2028, среднее ± стандартное отклонение:

Полная модель	nDCG@10	P@10	Спирмен внутри категории	Завышенный грейд в топ-10, %	Настоящий грейд ниже потребности, %
до: проваленный тест не учитывается	0,649 ± 0,006	0,632 ± 0,019	0,236 ± 0,033	20,1 ± 3,1	19,5 ± 2,7
после: грейд на ступень ниже проваленного	0,652 ± 0,019	0,636 ± 0,025	0,374 ± 0,017	21,5 ± 2,9	14,7 ± 3,0

> Строка продукта в этой таблице посчитана до настройки весов 11.10.2026 (#460). После неё та же команда на зёрнах 2026–2028 даёт nDCG@10 0,658 и P@10 0,632.

Порядок внутри категории стал заметно точнее: Спирмен 0,236 → 0,374. Людей с настоящим грейдом ниже потребности в первой десятке меньше: 19,5 % → 14,7 %. nDCG@10 и P@10 выросли в пределах разброса. Доля «показанный грейд выше настоящего» выросла на 1,4 п. п.: кандидат с проваленным G теперь попадает в выдачу потребностей на G−1, а карточка показывает выбранный им G со статусом «не подтверждён». По второму определению завышения, настоящий грейд ниже потребности, этот же человек не завышен. С проваленным тестом полная модель на зерне 2028 обгоняет строку «без теста» по nDCG@10 со значимой разницей: +0,0245, 95 % ДИ +0,0034…+0,0449, p = 0,0222. До изменения было +0,0054 и p = 0,4732.

Разделы «Ярус соответствия», «Сначала категория», «Доля завышенного грейда», «Полоса у сдавшего грейд», «Подъём оси ФСП», «Атака накруткой навыков», «Неподтверждённый грейд» и предрегистрированный эксперимент сняты до этого изменения. Таблицы «Ярус соответствия» и «Сначала категория» пересняты на коммите `7aacb2f` (до N7, `FSP_LIFT = "boost"`, Linux): их строка текущего порядка совпадает со строкой «до» в таблице выше.

Ярус соответствия

Ярус — целая часть от деления балла соответствия (навыки, грейд, роль, текст, отрасль) на ширину Δ. Внутри яруса выше стоит грейд, подтверждённый тестом, затем текущий балл. Человек из более низкого яруса не обгоняет верхний за счёт теста.

Перебор ширин 0,02, 0,03, 0,05, 0,08 и 0,10 на зёрнах 2026, 2027 и 2028, 800 кандидатов и 80 потребностей. Команда: `python -m app.validation.matching --tier-delta 0.02,0.03,0.05,0.08,0.10`. Правило выбора: наибольший Спирмен среди ширин, у которых nDCG полной модели не ниже nDCG без теста и без ФСП больше чем на 0,005. В этом прогоне ярус включён у обеих строк, поэтому nDCG строки «без теста и без ФСП» в таблице относится к порядку с ярусом и не заменяет 0,669 у текущего порядка.

Δ	nDCG полная	nDCG без теста и без ФСП	разница	Спирмен	завышенный грейд, %	проходит порог nDCG
0.02	0.660	0.668	-0.009	0.094	24.6	нет
0.03	0.659	0.668	-0.009	0.107	24.3	нет
0.05	0.657	0.668	-0.011	0.126	22.7	нет
0.08	0.652	0.668	-0.016	0.156	21.0	нет

Δ	nDCG полная	nDCG без теста и без ФСП	разница	Спирмен	завышенный грейд, %	проходит порог nDCG
0.10	0.647	0.668	-0.022	0.169	19.0	нет

Ни одна ширина не удерживает nDCG в пределах 0,005 от строки без теста и без ФСП. Ширина 0,02 даёт Спирмен 0,094 и завышенный грейд 24,6 %. У порядка по баллу эти цифры 0,236 и 20,1 %. Выдача сортируется по баллу.

Сначала категория

Класс 0 — показанные специализация и грейд совпадают с потребностью. Класс 1 — та же специализация и грейд на одну ступень рядом, либо соседняя специализация с точным грейдом. Класс 2 — остальные. Внутри класса порядок прежний: балл, затем нижняя граница оси ФСП. Параметров нет. Команда: `python -m app.validation.matching --order category_first`. Поиск сортирует по баллу.

Приёмка на среднем по зёрнам 2026, 2027 и 2028: nDCG@10 не ниже 0,665, P@10 не ниже 0,651, Спирмен внутри категории не ниже 0,22, завышенный грейд не выше 19,8 %.

Порядок	nDCG@10	P@10	Спирмен	завышенный грейд, %
Текущий, только балл	0,649 ± 0,006	0,632 ± 0,019	0,236 ± 0,033	20,1 ± 3,1
Сначала категория	0,648 ± 0,014	0,608 ± 0,028	0,236 ± 0,033	21,5 ± 2,5

Спирмен 0,236 в обоих порядках. У «сначала категория» nDCG@10 0,648, P@10 0,608, завышенный грейд 21,5 %. У порядка по баллу — 0,649, 0,632 и 20,1 %. Человек точной категории без нужных навыков иначе оказывается выше человека соседней категории с полным набором навыков. Поиск сортирует по баллу.

Парные сравнения

Метка релевантности — роль, навыки и грейд. Скрытая сила θ внутри грейда в неё не входит, поэтому её не подгоняли под результат. nDCG отвечает за категорию. Спирмен внутри категории — вторая метрика, объявленная заранее: она как раз про порядок внутри грейда.

Полная модель минус сравнение, по потребностям. 95 % интервал — бутстреп среднего, 10 000 повторов. p — доля перестановок знака, у которых модуль среднего не меньше наблюдаемого. Команда: `python -m app.validation.matching`.

Зерно	Сравнение	Δ nDCG@10	95 % ДИ	p	Δ Спирмен
2026	поиск по ключевым словам	+0,0901	+0,0562…+0,1252	0,0000	+0,3306
2026	по резюме	+0,0373	+0,0001…+0,0743	0,0531	+0,3413
2026	без теста	+0,0138	−0,0083…+0,0354	0,2321	+0,1748
2026	без оси ФСП	−0,0006	−0,0090…+0,0083	0,8899	+0,0375
2026	без доказательств	−0,0122	−0,0316…+0,0076	0,2319	+0,3257
2027	поиск по ключевым словам	+0,0553	+0,0216…+0,0882	0,0014	+0,3824
2027	по резюме	+0,0131	−0,0232…+0,0485	0,4920	+0,4001
2027	без теста	−0,0213	−0,0461…+0,0035	0,0888	+0,2316
2027	без оси ФСП	+0,0035	−0,0033…+0,0107	0,3535	+0,0350
2027	без доказательств	−0,0349	−0,0603…−0,0103	0,0074	+0,3909
2028	поиск по ключевым словам	+0,1065	+0,0673…+0,1447	0,0000	+0,3152
2028	по резюме	+0,0465	+0,0058…+0,0858	0,0259	+0,3492
2028	без теста	+0,0245	+0,0034…+0,0449	0,0222	+0,2023
2028	без оси ФСП	+0,0016	−0,0065…+0,0092	0,6975	+0,0310
2028	без доказательств	−0,0023	−0,0307…+0,0266	0,8726	+0,3574

На зерне 2027 Δ nDCG@10 против строки «без доказательств» равна −0,0349, интервал не накрывает ноль (p = 0,0074). На том же зерне Спирмен внутри категории выше на 0,391. На зёрнах 2026 и 2028 интервал накрывает ноль. Потребности без единой метки ≥ 2: 2, 3 и 3 из 80. В среднее nDCG они не входят. У полной модели P@min(10, R) — 0,662, 0,612 и 0,660, R-precision — 0,568, 0,535 и 0,569.

Против двух баз: заголовочные цифры

Одна и та же разметка, те же зёрна 2026–2028 и те же метрики. Для каждой потребности с релевантными кандидатами считается разность «полная модель минус база», потребности трёх зёрен объединяются (232), 95 % ДИ — парный перцентильный бутстреп среднего, 10 000 повторов, зерно повторной выборки 11. Модель, пороги `--check` и предрегистрация не менялись; BM25 добавлен как ещё одна строка сравнения. Таблицу печатает та же команда `cd backend && python -m app.validation.matching --check`:

База	nDCG@10 базы	Δ nDCG@10	95 % ДИ	P@10 базы	Δ P@10	95 % ДИ	nDCG модели выше / ниже
поиск по ключевым словам (BM25)	0,568	+0,089	+0,068… +0,111	0,526	+0,106	+0,084… +0,129	73 % / 27 %
сортировка по резюме	0,620	+0,038	+0,014… +0,061	0,559	+0,074	+0,048… +0,100	60 % / 40 %

Полная модель на тех же потребностях: nDCG@10 0,658, P@10 0,632. Обе разности положительны, интервалы не накрывают ноль. По отдельным зёрнам интервал Δ против BM25 выше нуля на всех трёх (таблица «Парные сравнения»), против резюме — на 2026 и 2028.

Как читать размер разрыва: в синтетическом наборе текст профиля — ровно специализация, грейд и названия навыков из той же таксономии, без синонимов, опечаток и длинных описаний опыта. Словам здесь не мешают синонимы и опечатки. Цифры чужих работ «против BM25» сняты на других данных и разметке, напрямую с этой не сравниваются. Сильнее всего модель отличается от обеих баз порядком внутри категории: Спирмен со скрытой силой 0,410 против 0,032 у BM25 и 0,011 у резюме.

Выводы

- Против поиска по ключевым словам (BM25) nDCG@10 выше на +0,089 (95 % ДИ +0,068…+0,111), P@10 — на +0,106 (+0,084… +0,129). Против сортировки по резюме — +0,038 (+0,014…+0,061) и +0,074 (+0,048…+0,100). Парный бутстреп по 232 потребностям трёх зёрен.
- Подбор обходит базовую линию «по резюме» на всех трёх зёрнах: nDCG@10 0,658 ± 0,022 против 0,620 ± 0,005, P@10 0,632 ± 0,028 против 0,559 ± 0,021.
- Внутри категории порядок у базовой линии случаен (Спирмен 0,011), у полной модели 0,410 ± 0,021. Без теста он падает до 0,162, без оси ФСП — до 0,384, без обоих — до 0,015. Тест и ФСП работают именно здесь: они поднимают сильных внутри показанного грейда, а проваленная попытка опускает тех, кто тест на свой грейд не сдал.
- Завышенный грейд в первой десятке — показанный грейд выше настоящего: 28,1 % по резюме, 22,1 % у полной модели. Весь этот остаток — заявленный грейд, не подтверждённый тестом (0,0 % в обеих строках). Второе определение, настоящий грейд ниже грейда потребности, на тех же трёх зёрнах 30,9 % и 13,6 %. Работодатель реже видит наверху человека, который приписал себе ступень.
- Доказательства почти не двигают nDCG: без теста и ФСП он 0,669, с ними 0,658. Разметка ставит 3 за точный грейд и полный набор навыков и не различает силу внутри категории, поэтому порядок по силе её не улучшает. Польза теста и ФСП — точный порядок внутри категории и меньше завышенных грейдов наверху. Город и формат в метку не входят, поэтому их вес 0,05 чуть сдвигает nDCG и Спирмен относительно набора без этих полей.
- Слепая разметка смотрит только на заявленный профиль и в одном случае из пяти сдвигает оценку на ступень. Против меток, посчитанных по скрытой правде, каппа Коэна 0,46 при зерне разметчика 2027. Веса по каппе не подгоняются. В своём файле разметки скрытой правды нет, поэтому каппа и Спирмен там не считаются.

Проверка в CI

`python -m app.validation.matching --check` падает, если полная модель не выше базовой линии и случайного порядка по nDCG@10, если nDCG@10 на зерне 2026 ниже 0,65, если тест или ось ФСП не поднимают Спирмен внутри категории и если завышенных грейдов в топ-10 не меньше, чем по резюме. Шаг стоит в задании `test-backend`.

Критерий для теста и оси ФСП сменён с фиксированного порога на доверительный интервал (11.10.2026). Раньше `--check` требовал, чтобы изъятие каждого доказательства снижало Спирмен на зерне 2026 не меньше чем на 0,04. Это число было выбрано вручную, смотрело на одно зерно и не учитывало разброс по потребностям. Теперь правило задаёт знак эффекта, а не его размер: для каждого изъятия разность «полная модель минус строка» по Спирмену внутри категории считается парно по потребностям, потребности трёх зёрен 2026–2028 объединяются, и нижняя граница 95 % бутстреп-интервала среднего (10 000 повторов, тот же расчёт, что у парного Δ nDCG@10) должна быть выше нуля. Правило принято до прогона, порог не подбирался. Таблицу печатает та же команда:

Сравнение	Δ Спирмен, 2026	Δ Спирмен, 2027	Δ Спирмен, 2028	Сводно	95 % ДИ	Потребностей	Проходит
без теста	+0,1748	+0,2316	+0,2023	+0,2018	+0,1701…+0,2335	206	да
без оси ФСП	+0,0375	+0,0350	+0,0310	+0,0346	+0,0260…+0,0443	206	да

Тест и ось ФСП поднимают порядок внутри категории на каждом из трёх зёрен, и интервал обоих эффектов не накрывает ноль. Вклад ФСП меньше, чем до учёта проваленного теста: часть категории, которую раньше упорядочивала только ось ФСП, теперь упорядочивает проваленная попытка.

Своя разметка

Своих пар найма нет. Файл для проверки — JSONL, одна пара на строку: потребность, наблюдаемый профиль кандидата и оценка 0…3. Оценки независимы от формулы. Команда из каталога репозитория:

```
cd backend && python -m app.validation.matching --pairs ../docs/examples/jury_pairs.jsonl
```

Она печатает nDCG@10, P@min(10, n) и MAP. Без консоли то же считает страница `/evaluate`: файл или текст уходит в `POST /api/v1/public/evaluate` (до 500 пар и 256 КБ, 10 запросов в минуту с адреса), ответ — три строки: модель, «по резюме» и случайный порядок, плюс sha256 набора. Ещё в ответе есть `per_need` — по каждой потребности P@min(10, n) и nDCG@10 модели и сортировки «по резюме» и топ-5 модели с оценками, — и `paired_delta`: средняя по потребностям разница nDCG@10 «модель − по резюме» с 95 % бутстреп-ДИ (1000 повторов, зерно 11, как у агрегата). На странице это раскрывающаяся таблица «По потребностям». В ней подсвечены строки, где у модели доля релевантных в топе ниже, чем у сортировки по резюме. Пары не сохраняются, банк кандидатов не читается. Три строки примера:

```
{"need": {"id": "n1", "role": "backend", "grade": "middle", "skills": ["Python", "PostgreSQL"]}, "candidate_id": "c1",
"candidate": {"observed": {"role": "backend", "grade": "middle", "grade_source": "assessed", "skills": {"Python":
"test_confirmed", "PostgreSQL": "test_confirmed"}, "test_p": 0.9, "fsp_lcb": 0.6, "industry": "fintech"}}, "label": 3}
{"need": {"id": "n1", "role": "backend", "grade": "middle", "skills": ["Python", "PostgreSQL"]}, "candidate_id": "c2",
"candidate": {"observed": {"role": "backend", "grade": "junior", "grade_source": "declared", "skills": {"Python":
"self_reported"}, "test_p": null, "fsp_lcb": null, "industry": "fintech"}}, "label": 1}
{"need": {"id": "n1", "role": "backend", "grade": "middle", "skills": ["Python", "PostgreSQL"]}, "candidate_id": "c3",
"candidate": {"observed": {"role": "ml", "grade": "senior", "grade_source": "declared", "skills": {"Python":
"self_reported"}, "test_p": null, "fsp_lcb": 0.4, "industry": "fintech"}}, "label": 0}
```

Тот же расчёт принимает JSON в формате `--export`: `python -m app.validation.matching --import pairs.json`.

Как проверить подбор самому за 2 минуты. Откройте `/evaluate`, режим «Разметить самому», выберите потребность и нажмите «Начать». Страница по одной покажет восемь карточек из слепого листа `blind_pairs_2029.csv`: трое из верха модели, трое из верха сортировки по резюме и двое случайных, перемешаны. Балла модели и того, из какого списка карточка, не видно. На каждую ставите 0–3 кнопкой или клавишей: 0 — не подходит, 1 — слабо, 2 — с оговорками, 3 — подходит полностью. После восьмой карточки или по кнопке «Готово» оценки уходят в тот же `POST /api/v1/public/evaluate` в формате JSONL выше, и страница показывает nDCG@10 и P@5 модели, сортировки по резюме и случайного порядка на ваших метках. nDCG@10 — насколько высоко стоят те, кому вы дали больше баллов; P@5 — доля оценок 2 и 3 среди первых пяти. Восемь оценок одного человека — маленькая выборка, итог может перевернуть пара других оценок, поэтому это проверка на глаз, а не замена разметки двумя людьми. Оценки хранятся только в браузере (localStorage). Лист для страницы — `frontend/src/data/blindSheet.json`, его собирает `python scripts/blind_sheet_web.py docs/examples/blind_pairs_2029.csv frontend/src/data/blindSheet.json` из листа и ключа; тексты карточек те же, что в CSV, кроме строки о тесте. В CSV у 33 карточек из списка в разделе про двух ИИ-оценщиков ошибочно стоит «Теста нет», и ещё у восьми — тоже, хотя человек проходил тест. Скрипт повторяет генератор с параметрами выгрузки, сверяет его с ключом и пишет правду: «Тест на грейд … не сдан», а если навыки отмечены «тест», то добавляет, что они подтверждены этой попыткой. Если генератор с ключом не совпадёт, строку о тесте он убирает. Профили, которые уходят на расчёт, остаются как в ключе, поэтому метрики не меняются.

Слепая разметка: лист и расчёт

Лист на двоих оценщиков готов. Команда из каталога `backend`:

```
python -m app.validation.matching --blind-export ../docs/examples/blind_pairs_2029.csv --needs 25 --per-need 8 --seed 2029
```

200 пар: 25 потребностей, в каждой 8 карточек (три из верха модели, три из верха сортировки по резюме, две случайные, затем перемешаны). В листе только наблюдаемый профиль: специализация, грейд заявлен или подтверждён тестом, навыки, сводка теста и оси ФСП. Скрытой правды и балла подбора нет. Кандидаты в листе синтетические, из генератора с зерном 2029. Колонка `label` пустая: двое ставят 0…3 независимо, не видя друг друга.

Взвешенная каппа двух заполненных файлов: `python -m app.validation.matching --kappa a.csv b.csv`. Вес расхождения `(i - j)² / 9`. Порог годности 0,6. Ниже него правила уточняют на десяти парах и размечают лист заново.

Слияние, среднее с округлением вниз: `python -m app.validation.matching --merge-raters a.csv b.csv --merged merged.jsonl`. Рядом с первым файлом должен лежать ключ `a.key.json` из той же выгрузки: в нём те же наблюдаемые факты, без скрытой правды. Дальше `python -m app.validation.matching --pairs merged.jsonl` печатает nDCG@10, P@min(10, n) и MAP модели и сортировки по резюме на этих парах, с 95 % доверительным интервалом по потребностям.

Слепая разметка: два ИИ-оценщика (11.10.2026)

Кто размечал лист. Оба оценщика — автоматические, на основе языковых моделей (LLM-as-judge): оценщик A и оценщик B. Работали они порознь, по одной и той же рубрике из шкалы 0–3. Каждый видел только `blind_pairs_2029.csv`: ни ключа `*.key.json`, ни баллов модели, ни чужого листа. Разметка лежит в `docs/examples/ai_rater_a.csv` и `ai_rater_b.csv`, слитые метки — в `ai_raters_merged.jsonl`, полный вывод команд — в `ai_raters_result.md`.

Рубрика в сжатом виде: 3 — специализация совпадает, грейд не ниже нужного, есть все или почти все навыки, ключевые подтверждены тестом или резюме; 2 — специализация та же или смежная, грейд на ступень ниже или часть навыков только заявлена; 1 — смежная специализация, грейд сильно ниже или не подтверждён, навыки в основном заявлены или отсутствуют; 0 — другая специализация и нет ключевых навыков. Ось ФСП и предметная область — слабый плюс при прочих равных, завышенный грейд без теста — минус.

Согласие. Квадратично взвешенная к 0,974, к Коэна 0,937, точное совпадение 191 пара из 200 (95,5 %). Все девять расхождений — ровно на одну ступень, через ступень оценщики не расходились ни разу. Порог годности 0,6 пройден с большим запасом.

Оговорка к этой цифре обязательна: оба оценщика — языковые модели и, скорее всего, работают на одной базовой модели. Высокая к между двумя ИИ говорит о том, что рубрика однозначна для такого читателя: два автоматических оценщика по ней приходят к одному ответу. Согласия экспертов-людей она не заменяет. Общая базовая модель означает общие искажения: где она ошибается, оба оценщика ошибаются одинаково, и к этого не покажет. Поэтому 0,974 — проверка рубрики, а не подтверждение того, что HR оценил бы подбор так же.

Метрики модели на согласованных метках. Пары слиты штатным `--merge-raters` (среднее с округлением вниз; распределение 0/1/2/3 — 37/72/67/24), метрики посчитал `--pairs`. В расчёт попали 25 потребностей и 171 кандидат.

Ранжирование	nDCG@10	95 % ДИ nDCG	MAP
Случайный порядок	0,788	0,746…0,830	0,628
Базовая линия «по резюме»	0,866	0,835…0,900	0,746
ФСП Талант: оценка соответствия	0,928	0,895…0,957	0,904
без весов доверия к навыкам	0,884	0,852…0,916	0,843
без теста	0,941	0,915…0,966	0,885
без оси ФСП	0,923	0,891…0,953	0,878
без теста и без ФСП	0,959	0,934…0,980	0,893
без близких технологий	0,942	0,912…0,967	0,931

Как это читать:

- Модель ставит подходящих выше, чем сортировка по резюме: nDCG@10 0,928 против 0,866, MAP 0,904 против 0,746. Интервалы почти расходятся (0,895…0,957 и 0,835…0,900), но парного теста эта команда не делает, поэтому строгой значимости мы не заявляем.
- Абсолютные числа высокие из-за устройства листа, а не только из-за модели. В каждой потребности всего 8 карточек, и шесть из них уже взяты из верха модели или резюме, поэтому даже случайный порядок получает 0,788. С синтетической таблицей выше (0,65 на 800 кандидатах) эти цифры не сравниваются.
- P@10 одинаковый у всех строк, 0,455: карточек меньше десяти, P@min(10, n) считается по всем восьми и порядок не различает. В таблицу он не вынесен.
- Без теста и ФСП nDCG выше, 0,959 против 0,928. Это то же, что на синтетике: рубрика оценивает категорию и навыки и почти не различает силу внутри категории, а тест и ФСП работают именно там. Скрывать эту строку мы не стали.
- Спирмен внутри категории и завышенный грейд здесь не считаются: в парах нет скрытой правды. AUC модуль не считает вовсе.
- Вывод не держится на одном оценщике. На метках только A модель даёт 0,939 против 0,866 по резюме, на метках только B — 0,918 против 0,865 (тот же `--merge-raters` с одним файлом дважды, затем `--pairs`).

Где рубрика оказалась неясной. Оба оценщика выписали места, где рубрике не хватает определений. Их нужно закрыть до разметки людьми, иначе люди разойдутся там же, где ИИ договорились молча.

Оценщик A и как он решал:

1. Не определено, какие специализации «смежные». A считал смежными ML, аналитику и инженерию данных между собой, а также Fullstack, фронтенд и бэкенд.
2. Для завышенного грейда без теста не задана величина штрафа. A снимал одну ступень.
3. Не сказано, как сочетать грейд, подтверждённый тестом, с навыками, которые только заявлены. A ставил 2.
4. Не определено «почти все» при трёх требуемых навыках.
5. Противоречие в данных: в карточке «Теста нет», а навыки помечены «(тест)» (p003, p011, p013, p015, p029).

Оценщик B:

1. То же противоречие: «Теста нет» при навыках с пометкой «(тест)».
2. Как считать «почти все» при трёх навыках: два из трёх — это уже «почти все» или нет?
3. Смежная специализация без нужных навыков: по букве рубрики это 1, хотя по сути такая пара ближе к 0.

Противоречие «Теста нет» — ошибка генератора листа, а не рубрики. Мы проверили пары по ключу. У p003, p011, p013, p015 и p029 в `observed` есть навыки `test_confirmed`, но `test_p`, `test_n` и `test_correct` пустые, грейд `declared`. Таких карточек в листе 33 из 200: p003, p011, p013, p015, p021, p029, p030, p039, p047, p058, p062, p063, p067, p082, p088, p093, p100, p103, p104, p117, p125,

p139, p148, p156, p170, p171, p183, p185, p187, p188, p189, p191, p193. Причина в `make_dataset`: человек прошёл тест, но не набрал порог грейда (`test_p < 0,75`). Тогда `test_p` обнуляется, а навыки, подтверждённые тем же тестом, остаются помеченными «тест». `_candidate_card` смотрит только на `test_n` и пишет «Теста нет».

Список исправлений для генератора листа (в этом PR не сделаны: правка `make_dataset` меняет синтетический набор и все хеши выше):

1. В `make_dataset` не обнулять результат несданного теста: хранить `test_p`, `test_n` и `test_correct` всегда, когда тест был, а порог 0,75 решает только `grade_source`.
2. Если менять набор нельзя, то хотя бы в `_candidate_card`: при навыках `test_confirmed` и пустом `test_n` писать «Тест пройден ниже порога грейда» вместо «Теста нет».
3. Добавить в `write_blind_sheet` проверку: строка «Теста нет» и пометка «(тест)» в одной карточке — ошибка выгрузки.
4. Unit-тест на это сочетание и перевыгрузка `blind_pairs_2029.csv` с ключом перед разметкой людьми.
5. В рубрику дописать: список смежных специализаций, штраф за завышенный грейд без теста (например, одна ступень, как у А), правило для «грейд по тесту, навыки заявлены», порог «почти все» (например, все, кроме одного, при трёх и более навыках) и что ставить смежной специализации без нужных навыков.

На ИИ-разметку эта ошибка влияет так: 33 пары оценщики видели с противоречивой карточкой и разрешали его каждый по-своему. Числа выше посчитаны на листе как есть, без исправлений.

Что дальше. Дальше лист разметят люди, и он для этого готов: `blind_pairs_2029.csv` и та же рубрика. Перед раздачей лист стоит перевыгрузить с исправленным генератором, а в рубрику внести уточнения из списков выше. Числа людей впишем сюда же рядом с этими, какими бы они ни вышли. До тех пор результат выше — проверка рубрики и пайплайна на ИИ-оценщиках, а не подтверждение качества подбора у HR.

Воспроизвести из каталога `backend`:

```
python -m app.validation.matching --kappa ../docs/examples/ai_rater_a.csv ../docs/examples/ai_rater_b.csv
python -m app.validation.matching --pairs ../docs/examples/ai_raters_merged.jsonl
```

Чтобы пересобрать `ai_raters_merged.jsonl`, скопируйте оба листа во временный каталог, положите рядом с первым ключ `blind_pairs_2029.key.json` под именем `ai_rater_a.key.json` и запустите `--merge-raters`.

Доля завышенного грейда

Команда из каталога `backend`:

```
python -m app.validation.matching --overclaim 0,0.25,0.5,0.75 --seed 2026
```

Доля — это люди, заявившие грейд на ступень выше настоящего. Остальные в этом прогоне заявляют истинный грейд. Обычный набор, где 30 % завышают и 10 % занижают, эта кривая не заменяет.

Доля	nDCG модели	nDCG по резюме	Спирмен модели	Спирмен по резюме	Завышенный грейд в топ-10, модель	Завышенный грейд в топ-10, по резюме
0 %	0,767	0,706	0,147	0,050	0,0 %	0,0 %
25 %	0,706	0,643	0,227	0,055	18,5 %	25,4 %
50 %	0,677	0,603	0,270	0,056	34,0 %	45,5 %
75 %	0,639	0,555	0,210	0,024	56,7 %	66,7 %

Модель выше сортировки по резюме по nDCG на каждой напечатанной доле, включая нулевую. Разрыв растёт вместе с долей завышения: 0,061, 0,063, 0,074 и 0,084. Когда никто не завышает грейд, тест всё равно отличает людей по навыкам и роли, поэтому сортировка по резюме не обгоняет модель.

Полоса у сдавшего грейд

Непроверенный грейд даёт полосе «Тест» 0,20. Сдавший на грейд потребности не опускается ниже 0,25, даже если нижняя граница доли верных ответов меньше. Команда на отложенных зёрнах, публичные 7 и 2026–2028 в решении не участвовали:

```
python -m app.validation.matching --pass-floor --seeds 4001,4002,4003,4004,4005
```

На всех пяти зёрнах Δ nDCG@10 и Δ Спирмен в категории равны нулю: в генераторе нижняя граница у сдавших уже выше полосы, порядок не сдвинулся. Вместе 0,0000, 95 % ДИ [0,0000; 0,0000]. Порог «не хуже» выполнен, полоса включена. На зерне 2026 nDCG и P@10 с полосой и без неё совпали.

Подъём оси ФСП

11.10.2026. Подъём включён в продукт

С 11.10.2026 в продукте `FSP_LIFT = "boost"`. Решение нормативное: по ТЗ 2.1 и уточнению требований E08 результаты ФСП — дополнительное доказательство, и их привязка не должна опускать кандидата. Синтетическая разметка ставит 3 за точный грейд и полный набор навыков и не различает силу внутри категории, поэтому она не может решить этот вопрос. Цена на трёх публичных зёрнах: nDCG@10 0,650 → 0,649, P@10 0,631 → 0,632, Спирмен внутри категории 0,237 → 0,236. `--check` проходит. Свойство «история ФСП не понижает оценку» закреплено property-тестом `tests/unit/test_fsp_lift_property.py` (hypothesis, 3 000 примеров).

11.10.2026. Подъём на отложенных зёрнах: результат приёмки и следующий шаг

Команда `python -m app.validation.matching --holdout` (зёрна 4001–4005, нижняя граница из событий движка) на main @ 0507edc завершается с кодом 1:

Зерно	Δ nDCG@10	Δ Спирмен в категории
4001	−0,0156	+0,0025
4002	−0,0185	+0,0044
4003	−0,0156	−0,0014
4004	−0,0072	+0,0083
4005	−0,0180	+0,0198

nDCG@10 −0,0150 (95 % ДИ −0,0195…−0,0104), Спирмен +0,0065. Нижняя граница nDCG ниже порога −0,005, порог по Спирмену выполнен. Подъём оставлен в продукте по нормативной причине (абзац выше); его цена на отложенных зёрнах — около 0,015 nDCG@10. Повтор на a63e4ca, после настройки весов, тоже завершается с кодом 1: nDCG@10 −0,0100 (95 % ДИ −0,0136…−0,0065), Спирмен +0,0107 (зёрна 4001–4005: −0,0124, −0,0099, −0,0136, −0,0066, −0,0075). Следующий шаг — перекалибровать величину подъёма на разметке пилота (демо и промышленный контур).

Ошибка измерения теста в синтетике

В генераторе вероятность уровня y прошедших тест — точная функция скрытой θ, без ошибки измерения. Настоящая сессия оценивает θ с ошибкой (RMSE ≈ 0,47, `python -m app.validation.assessment --n 2000 --seed 7`). Команда `python -m app.validation.matching --test-noise 0.47` добавляет такую ошибку; эталон по умолчанию и его SHA-256 не меняются.

Три зерна 2026–2028	Без ошибки (эталон)	Ошибка 0,47
nDCG@10 полной модели	0,652	0,647
P@10	0,636	0,631
Спирмен внутри категории	0,374	0,315
Δ nDCG@10 против сортировки по резюме (95 % ДИ)	+0,032 (+0,011…+0,054)	+0,027 (+0,007…+0,048)
Δ nDCG@10 против BM25 (95 % ДИ)	+0,084 (+0,064…+0,105)	+0,079 (+0,060…+0,098)

> Значения в этой таблице посчитаны до настройки весов 11.10.2026 (#460). После неё эталон на зёрнах 2026–2028 даёт nDCG@10 0,658 и P@10 0,632.

Около 16 % порядка внутри категории даёт «идеальный» тест генератора. С реалистичной ошибкой преимущество над обеими базами остаётся значимым.

09.10.2026. Ось только поднимает доказательства

Нижняя граница — максимум по подмножествам не длиннее пяти событий, пустое подмножество остаётся в переборе. В подборке фактор ФСП может только поднять доказательства над тестом: `T + 0,5·max(0, F - T)`. Команда:

```
python -m app.validation.matching --fsp-events --seeds 4001,4002,4003,4004,4005
```

Столбец «до» в этом опыте тоже считается на нижней границе по подмножествам событий, а не на пяти сильнейших событиях выдачи: сравнили только правило подъёма.

Зерно	Δ nDCG@10	Δ Спирмен в категории
4001	−0,0156	+0,0025
4002	−0,0185	+0,0044

Зерно	Δ nDCG@10	Δ Спирмен в категории
4003	−0,0156	−0,0014
4004	−0,0072	+0,0083
4005	−0,0180	+0,0198

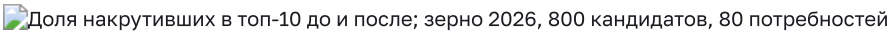
Вместе (переснято 11.10.2026 на текущем main): nDCG@10 −0,0150, 95 % ДИ [−0,0195; −0,0104]. Спирмен в категории +0,0065. Собственный порог команды (нижняя граница Δ nDCG@10 ≥ −0,005) **не пройден**, команда завершается с кодом 1; порог Спирмена (≥ −0,01) пройден. Подъём ФСП оставлен в продукте как нормативное решение (ТЗ 2.1, ответ E08), а не потому что он улучшает подборку. На этом варианте движка ферма из 40 онлайн-участий даёт LCB 0,16, две победы — 0,54, Спирмен LCB на зёрнах 2026–2028 — 0,752 ± 0,002. В выдаче пять сильнейших событий и среднее теста с осью ФСП. Участие входит в нижнюю границу: история с нулевым классом говорит о силе. Ответ E08: достижения не заменяют тест. Поднять доказательства над пустой историей можно результатом выше её уровня.

Атака накруткой навыков

20 % кандидатов дописывают в профиль все навыки своей специализации. Истина и метки релевантности не меняются. Сравнивается доля этих же людей в топ-10 до накрутки и после. Команда: `python -m app.validation.matching --stuffing 0.2` (зерно 2026, 800 кандидатов, 80 потребностей). 95 % ДИ — бутстреп по потребностям. Контрольная строка «всё подтверждено» показывает, что было бы, если бы платформа считала каждый заявленный навык подтверждённым.

Метод	В топ-10 до накрутки	В топ-10 после (95 % ДИ)	Доля в своей специализации	nDCG@10 после
Продукт	17,8 %	30,5 % (26,5 %…34,5 %)	18,4 %	0,6346
Ключевые слова	20,2 %	44,9 % (40,5 %…49,6 %)	18,4 %	0,5668
Продукт, если всё заявленное считать подтверждённым	17,8 %	66,8 % (62,5 %…71,0 %)	18,4 %	0,5558

Ограничение. Накрутка в продукте даёт +12,7 п. п. в топ-10. Это вдвое меньше, чем при поиске по словам (+24,7 п. п.), и в четыре раза меньше, чем если бы заявления не отличались от подтверждений (+49,0 п. п.). Полностью эффект не гасится: незаявленный навык ничего не даёт, а заявленный без подтверждения тестом или резюме немного весит. План: снизить вес только заявленных навыков, когда у кандидата есть тест той же специализации, и показывать работодателю долю подтверждённых навыков на карточке. Методика близка к исследованию чувствительности ранжирования к возмущениям (arXiv:2501.04316). Код: `stuff_skills` и `stuffing_study` в `app/validation/matching.py`, тесты в `tests/unit/test_skill_stuffing.py`.


Доля накрутивших в топ-10 до и после; зерно 2026, 800 кандидатов, 80 потребностей

Неподтверждённый грейд: показывать, опускать или скрывать

Вопрос из уточнения требований FTT.4: что делать с кандидатом, грейд которого тест ещё не подтвердил. Сравниваются три политики на одной и той же выдаче продукта. Метки релевантности не меняются, поэтому скрытый релевантный кандидат засчитывается как потеря. Команда: `python -m app.validation.matching --unconfirmed` (зерно 2026, 800 кандидатов, 80 потребностей, из них 78 с релевантными). Δ считается к порядку продукта, парно по потребностям; 95 % ДИ — бутстреп по потребностям, 1000 повторов.

Политика	nDCG@10	Δ nDCG@10 (95 % ДИ)	P@10	Δ P@10 (95 % ДИ)	Релевантные без подтверждения в топ-20
Как в продукте: общий порядок по баллу	0,6528	0	0,6526	0	47,0 %
Все неподтверждённые после подтверждённых	0,4735	−0,1792 (−0,2151… −0,1390)	0,4654	−0,1872 (−0,2282… −0,1449)	0,0 %
Неподтверждённые скрыты	0,4735	−0,1792 (−0,2151… −0,1390)	0,4654	−0,1872 (−0,2282… −0,1449)	0,0 %
Балл, неподтверждённым × 0	0,4742	−0,1785 (−0,2142… −0,1385)	0,4654	−0,1872 (−0,2282… −0,1449)	0,0 %
Балл, неподтверждённым × 0,25	0,4742	−0,1785 (−0,2142… −0,1385)	0,4654	−0,1872 (−0,2282… −0,1449)	0,1 %
Балл, неподтверждённым × 0,5	0,4821	−0,1707 (−0,2031… −0,1325)	0,4769	−0,1756 (−0,2141… −0,1359)	12,7 %
Балл, неподтверждённым × 0,75	0,5727	−0,0801 (−0,1001… −0,0585)	0,6026	−0,0500 (−0,0731… −0,0269)	32,9 %
Балл, неподтверждённым × 1	0,6525	−0,0003 (−0,0009… +0,0001)	0,6513	−0,0013 (−0,0038… +0,0000)	47,1 %

В продукте неподтверждённый кандидат остаётся в выдаче с пометкой «Грейд не подтверждён», а место ему задаёт балл. Опускание и скрытие совпадают по построению: в пуле 166 кандидатов с подтверждённым грейдом из 800, поэтому при опускании топ-20 целиком состоит из подтверждённых, то есть из тех же людей, что и при скрытии. Сетка множителей монотонна: чем сильнее штраф неподтверждённым, тем ниже nDCG@10 и тем меньше релевантных возвращается в топ-20. Множитель 1 практически совпадает с порядком продукта. Обе политики теряют около 0,18 nDCG@10, и интервал далеко от нуля. Синтетика: доля протестированных 45 %, грейд подтверждается при p ≥ 0,75 (`make_dataset`). Код: `unconfirmed_study` в `app/validation/matching.py`, тесты в `tests/unit/test_unconfirmed_policy.py`.

Ранжирование по ожидаемому выигрышу: предрегистрированный эксперимент (11.10.2026)

Протокол, гипотезы и правило решения зафиксированы в `docs/PREREG_MATCHING_2026-10.md` коммитом `37fcb8f` **до** первого запуска на отложенных зёрнах 5001–5010. Отложенные зёрна запускались один раз. Команды: `python -m app.validation.expected_gain --holdout` и то же с `--overclaim 0.0` и `--overclaim 0.45`. Это измерение: порядок в продукте не менялся.

Названия строк: `eg_full` — ожидаемый выигрыш с тестом и ФСП; `eg_no_evidence` — та же модель, откалиброванная без теста и ФСП; `product_full` и `product_no_evidence` — продукт и его абляция из основной таблицы.

Основной запуск (генератор по умолчанию)

```
seeds 5001…5010 (10), λ=0.3, overclaim=None
| Ранжирование | nDCG@10 | Спирмен в категории | Завышенный грейд в топ-10 |
|---|---|---|---|
| eg_full | 0.6931 | 0.140 | 25.2 % |
| eg_no_evidence | 0.6878 | -0.004 | 28.1 % |
| product_full | 0.6709 | 0.231 | 21.2 % |
| product_no_evidence | 0.6867 | 0.007 | 27.9 % |

| Сравнение | Метрика | Δ | 95 % ДИ | p |
|---|---|---|---|---|
| eg_full - eg_no_evidence | ndcg | +0.0052 | +0.0007…+0.0097 | 0.0221 |
| eg_full - eg_no_evidence | rho | +0.1448 | +0.1341…+0.1557 | 0.0000 |
| eg_full - product_full | ndcg | +0.0221 | +0.0118…+0.0323 | 0.0000 |
| eg_full - product_full | rho | -0.0906 | -0.1064…-0.0748 | 0.0000 |
| eg_full - product_no_evidence | ndcg | +0.0064 | -0.0023…+0.0151 | 0.1443 |
| eg_full - product_no_evidence | rho | +0.1332 | +0.1192…+0.1472 | 0.0000 |
```

Устойчивость: убеждения заморожены, у генератора другая доля завышения

```
seeds 5001…5010 (10), λ=0.3, overclaim=0.0
| Ранжирование | nDCG@10 | Спирмен в категории | Завышенный грейд в топ-10 |
|---|---|---|---|
| eg_full | 0.7721 | 0.104 | 0.0 % |
| eg_no_evidence | 0.7864 | -0.030 | 0.0 % |
| product_full | 0.7701 | 0.189 | 0.0 % |
| product_no_evidence | 0.7926 | -0.014 | 0.0 % |

| Сравнение | Метрика | Δ | 95 % ДИ | p |
|---|---|---|---|---|
| eg_full - eg_no_evidence | ndcg | -0.0143 | -0.0181…-0.0104 | 0.0000 |
| eg_full - eg_no_evidence | rho | +0.1341 | +0.1222…+0.1463 | 0.0000 |
| eg_full - product_full | ndcg | +0.0020 | -0.0088…+0.0126 | 0.7023 |
| eg_full - product_full | rho | -0.0844 | -0.1000…-0.0684 | 0.0000 |
| eg_full - product_no_evidence | ndcg | -0.0205 | -0.0297…-0.0114 | 0.0000 |
| eg_full - product_no_evidence | rho | +0.1180 | +0.1036…+0.1326 | 0.0000 |
```

```
seeds 5001…5010 (10), λ=0.3, overclaim=0.45
| Ранжирование | nDCG@10 | Спирмен в категории | Завышенный грейд в топ-10 |
|---|---|---|---|
| eg_full | 0.7061 | 0.126 | 37.6 % |
| eg_no_evidence | 0.6914 | -0.011 | 40.2 % |
| product_full | 0.6963 | 0.205 | 31.8 % |
| product_no_evidence | 0.6904 | -0.006 | 39.9 % |

| Сравнение | Метрика | Δ | 95 % ДИ | p |
|---|---|---|---|---|
| eg_full - eg_no_evidence | ndcg | +0.0147 | +0.0104…+0.0190 | 0.0000 |
| eg_full - eg_no_evidence | rho | +0.1369 | +0.1264…+0.1476 | 0.0000 |
| eg_full - product_full | ndcg | +0.0098 | -0.0006…+0.0198 | 0.0619 |
| eg_full - product_full | rho | -0.0794 | -0.0944…-0.0646 | 0.0000 |
| eg_full - product_no_evidence | ndcg | +0.0157 | +0.0070…+0.0243 | 0.0003 |
| eg_full - product_no_evidence | rho | +0.1316 | +0.1182…+0.1454 | 0.0000 |
```

Итог по гипотезам

- **H1 подтверждена.** Тест и ФСП поднимают nDCG@10 модели над её же абляцией: +0,0052, 95 % ДИ +0,0007…+0,0097, p 0,022. Эффект малый.
- **H2 подтверждена.** Новая модель выше продукта по nDCG@10: +0,022 (ДИ +0,012…+0,032). Над абляцией продукта (0,687) она **не** значима: +0,0064, p 0,14.
- **H3 — цена, как и объявлено.** Спирмен внутри категории 0,140 против 0,231 у продукта (−0,091, ДИ −0,106…−0,075).
- **H4 — ниже продукта.** Завышенный грейд в топ-10: 25,2 % против 21,2 %.
- **Устойчивость не подтверждена полностью.** Если в генераторе никто не завышает грейд, а убеждения откалиброваны на 30 % завышения, полная модель проигрывает своей абляции: −0,014, значимо. При 45 % завышения выигрывает: +0,015. Выигрыш держится на том, что калибровка совпадает с реальной долей завышения.

Решение по правилу предрегистрации. H1 и H2 подтверждены, значит переключать продукт допустимо только отдельным PR за флагом, и решает владелец. Наша рекомендация — **не переключать**: выигрыш по nDCG малый и зависит от калибровки, а цена (порядок внутри категории и завышенный грейд наверху) — как раз то, ради чего в продукте есть тест и ФСП. На реальных данных частоты P(навык | доверие) и P(грейд | источник) нужно пересчитать по тестам (`grade_posterior.fail_table`) и своей разметке, после этого повторить проверку.

Неблагоприятное воздействие: правило 4/5 и Skew@k

Раньше `app.validation.fairness` проверял правило только на синтетическом примере. Теперь он считает его на этом же наборе подбора, в продуктовом порядке `rank(..., "full")`.

- **Пул** — кандидаты специализации потребности (отклик на вакансию). **Отбор** — место в топ-10 пула. **Балл** — место в верхней половине пула, это вариант для скоринговых инструментов из правил DCWP к NYC Local Law 144 (редакция 06.04.2023: доля выше медианы).
- **Impact ratio (IR)** — доля группы, делённая на самую высокую долю. Ниже 0,8 — признак неблагоприятного воздействия по 29 CFR 1607.4(D). Доли суммируются по всем потребностям и зёрнам, 95 % ДИ — бутстреп по потребностям (B = 1000). Группы с n < 5 выводятся, но не сравниваются.
- **Skew@k** (Geyik, Ambler, Kenthapadi, KDD 2019, doi:10.1145/3292500.3330691) = ln(доля метки в топ-k / доля в пуле), объединённо по потребностям. 0 — топ повторяет пул, −0,22 — примерно на пятую часть меньше.
- **Метки.** Регион — четыре округа, отдельный генератор, в балл не входит. Язык кода — один язык из реальных навыков кандидата; он связан со специализацией, поэтому пул и режется по специализации. Участие в ФСП — **ось балла по ТЗ**, разница там ожидаема и не является дискриминацией, строка дана отдельно для прозрачности. Пол и национальность продукт не собирает, проверить их нельзя и не нужно.

Команда (≈ 1 мин):

```
cd backend && python -m app.validation.fairness
```

Вывод:

```
Зёрна 2026..2035 (10), кандидатов 800, потребностей 80 на зерно, всего потребностей 800. Пул = кандидаты роли потребности.
| Метка | Группа | n | Доля в топ-10 | IR отбора [95 % ДИ] | Доля выше медианы | IR балла [95 % ДИ] | Skew@10 | Skew@20 |
|---|---|---|---|---|---|---|---|---|
| Регион | ЦФО | 15258 | 0.136 | 0.962 [0.908; 1.000] | 0.501 | 1.000 [0.982; 1.000] | -0.008 | +0.007 |
| Регион | СЗФО | 14800 | 0.142 | 1.000 [0.975; 1.000] | 0.497 | 0.993 [0.970; 1.000] | +0.031 | +0.003 |
| Регион | СФО | 14291 | 0.136 | 0.958 [0.903; 1.000] | 0.497 | 0.993 [0.970; 1.000] | -0.012 | -0.002 |
| Регион | ПФО | 13893 | 0.136 | 0.957 [0.905; 1.000] | 0.491 | 0.981 [0.957; 0.999] | -0.013 | -0.008 |
| Язык кода | Python | 16952 | 0.144 | 0.875 [0.738; 0.965] | 0.507 | 0.898 [0.833; 0.945] | +0.050 | +0.033 |
| Язык кода | без языка | 16398 | 0.129 | 0.780 [0.656; 0.865] | 0.478 | 0.847 [0.786; 0.892] | -0.066 | -0.044 |
| Язык кода | JavaScript | 6004 | 0.142 | 0.863 [0.725; 0.960] | 0.504 | 0.893 [0.827; 0.941] | +0.036 | +0.018 |
| Язык кода | TypeScript | 5727 | 0.133 | 0.804 [0.665; 0.898] | 0.505 | 0.896 [0.831; 0.947] | -0.034 | -0.008 |
| Язык кода | Kotlin | 2945 | 0.143 | 0.869 [0.711; 0.983] | 0.498 | 0.883 [0.815; 0.936] | +0.042 | +0.011 |
| Язык кода | Swift | 2554 | 0.130 | 0.786 [0.657; 0.888] | 0.486 | 0.862 [0.799; 0.910] | -0.058 | -0.025 |
| Язык кода | C++ | 2093 | 0.139 | 0.840 [0.683; 0.987] | 0.494 | 0.875 [0.806; 0.937] | +0.009 | -0.002 |
| Язык кода | Rust | 1952 | 0.135 | 0.817 [0.670; 0.954] | 0.490 | 0.868 [0.801; 0.932] | -0.019 | -0.002 |
| Язык кода | C | 1581 | 0.154 | 0.936 [0.754; 1.000] | 0.544 | 0.964 [0.883; 1.000] | +0.117 | +0.044 |
| Язык кода | Go | 558 | 0.127 | 0.771 [0.571; 0.966] | 0.462 | 0.820 [0.723; 0.913] | -0.077 | -0.077 |
| Язык кода | C# | 506 | 0.121 | 0.731 [0.521; 0.949] | 0.470 | 0.834 [0.709; 0.956] | -0.130 | -0.082 |
| Язык кода | Java | 491 | 0.165 | 1.000 [0.845; 1.000] | 0.564 | 1.000 [0.947; 1.000] | +0.183 | +0.177 |
| Язык кода | PHP | 481 | 0.135 | 0.819 [0.596; 1.000] | 0.495 | 0.877 [0.778; 0.961] | -0.016 | +0.058 |
| Участие в ФСП (ось балла по Т3) | есть ФСП | 31845 | 0.148 | 1.000 [1.000; 1.000] | 0.509 | 1.000 [1.000; 1.000] |
+0.071 | +0.045 |
| Участие в ФСП (ось балла по Т3) | нет ФСП | 26397 | 0.125 | 0.848 [0.812; 0.884] | 0.482 | 0.947 [0.931; 0.965] | -0.094
| -0.058 |
Регион: мин. IR отбора 0.957, мин. IR балла 0.981, порог 0.8 — проходит
Язык кода: мин. IR отбора 0.731, мин. IR балла 0.820, порог 0.8 — ниже 4/5
Участие в ФСП (ось балла по Т3): мин. IR отбора 0.848, мин. IR балла 0.947, порог 0.8 — проходит
```

Вывод. Регион проходит с запасом: минимальный IR отбора 0,957, балла 0,981, |Skew@10| ≤ 0,031. Участие в ФСП проходит порог (0,848 и 0,947), хотя входит в балл. **Находка:** по языку кода точечный IR отбора ниже 4/5 у двух малых групп: C# 0,731 (506 мест, ДИ 0,521…0,949) и Go 0,771 (558 мест, ДИ 0,571…0,966); обе около 1 % пула, оба ДИ включают 0,8, IR по баллу 0,834 и 0,820. По правилам DCWP такие малые группы отчитываются с оговоркой; порог мы не правим. Это синтетика, на живых данных отчёт нужно повторить.

Разбор причины (11.10.2026). Та же команда теперь печатает таблицу «Разбор по языку кода». Порог 4/5 не трогаем, балл подбора тоже не менялся.

Язык	Мест в пулах	Доля в пуле	IR против самой большой группы (Python)	Релевантных мест (метка ≥ 2)	Доля в топ-10 среди релевантных
Python	16952	29,1 %	1,000	7592	0,236
Go	558	1,0 %	0,881	6	0,333
C#	506	0,9 %	0,835	14	0,571
Java (опорная группа в IR)	491	0,8 %	1,142	11	0,455
PHP	481	0,8 %	0,936	8	0,625

Почему так выходит: - **Опорная группа сама крошечная.** IR делится на самую высокую долю, а её дала Java: 491 место, 0,8 % пула, всего 11 релевантных мест. Если делить на самую большую группу, Python, то IR у C# 0,835, у Go 0,881. Оба выше 0,8. - **Подбор не отсекает релевантных из этих групп.** В топ-10 попадают 57 % релевантных C#-мест и 33 % релевантных Go-мест. У Python таких 24 %. - **Почему у этих групп вообще мало мест в топ-10.** Генератор потребностей почти не просит C#, Go, Java и PHP: релевантных мест у них 1–3 %, у Python 45 %. Нерелевантные кандидаты попадают в топ-10 за счёт факторов, не связанных с навыками, а при n ≈ 500 точечная оценка IR сильно шумит. У обоих 95 % ДИ включает 0,8. - **Пробела в словаре навыков нет.** Отбор среди релевантных у этих групп выше, чем у Python, поэтому синонимы и маппинг навыков менять не нужно. - **Правило DCWP.** Группы меньше 2 % пула (Go, C#, Java, PHP) можно не включать в расчёт IR, и в таблице они помечены «< 2 %». Вывод «ниже 4/5» в основной строке мы оставляем как есть. - Это синтетика. На реальных откликах отчёт нужно повторить.

IR при равной истинной релевантности. Та же команда печатает ещё одну таблицу. Сырой IR смешивает две причины: у группы меньше мест, которые действительно подходят потребности, или порядок опускает её при той же релевантности. Нечестна только вторая. Поэтому доля в топ-10 сравнивается с опорой (Python) внутри слоёв одной скрытой метки 0–3, слои сводятся отношением Мантеля–Хензеля (точечная оценка, 10 зёрен, ДИ не считали).

Язык	Мест в пулах	Сырой IR против Python	IR при равной релевантности
Go	558	0,881	1,728
C#	506	0,835	1,618

Язык	Мест в пулах	Сырой IR против Python	IR при равной релевантности
Java	491	1,142	2,153
PHP	481	0,936	1,829
TypeScript	5727	0,919	0,932
Kotlin	2945	0,992	0,932
Swift	2554	0,897	**0,758**

- Для C# и Go низкий сырой IR объясняется релевантностью: при той же метке они попадают в топ-10 чаще Python.
- **Находка: Swift.** Сырой IR 0,897 проходит порог, а при равной релевантности выходит 0,758, ниже 0,8. Значит, при той же истинной пригодности Swift-кандидаты стоят ниже. Балл подбора до сдачи не меняем, в планах пилота — разобрать этот слой.

Лестница баз

Та же разметка и те же потребности (с хотя бы одной меткой ≥ 2), что у `evaluate`. P@5 и P@10 — доля кандидатов с меткой ≥ 2 среди первых 5 и 10. «Лучше / хуже» — доля потребностей, где nDCG@10 полной модели строго выше или ниже базы. «Самоописание» сортирует по заявленным специализации и грейду, без навыков, теста и ФСП (метод `self`, только в валидации). Команда: `cd backend && python -m app.validation.matching --ladder`; для отложенных зёрен добавить `--seeds 4001,4002,4003,4004,4005`.

Зёрна 2026–2028, 232 потребности:

Ранжирование	nDCG@10	P@5	P@10	Модель лучше / хуже
случайный порядок	0,040	0,028	0,032	100 % / 0 %
самоописание	0,530	0,543	0,525	78 % / 22 %
BM25	0,568	0,605	0,526	73 % / 27 %
сортировка по резюме	0,620	0,652	0,559	60 % / 40 %
ФСП Талант	0,658	0,650	0,632	–

Отложенные зёрна 4001–4005, 390 потребностей:

Ранжирование	nDCG@10	P@5	P@10	Модель лучше / хуже
случайный порядок	0,041	0,032	0,032	100 % / 0 %
самоописание	0,577	0,570	0,528	66 % / 34 %
BM25	0,582	0,587	0,511	69 % / 31 %
сортировка по резюме	0,637	0,650	0,551	56 % / 44 %
ФСП Талант	0,659	0,655	0,626	–

У нас разрыв с базой «по резюме» небольшой, потому что эта база сильная: она уже сортирует по заявленным навыкам, специализации и грейду. Значимость разницы против резюме и BM25 приведена в разделе «Против двух баз» (парный бутстреп).

Настройка весов (11.10.2026)

Веса соответствия и вес доказательств подобраны перебором на **отдельных зёрнах 301–305** (800 кандидатов и 80 потребностей на зерно). Зёрна 2026–2028 и 4001–4005, на которых опубликованы все цифры, в выборе не участвовали. Разметку, протоколы и генератор синтетики не меняли. Цель перебора — среднее nDCG@10 по трём протоколам (строгий, A, B).

- Перебирали: веса навыков, грейда и специализации; `EVIDENCE_WEIGHT` 0,2–1,0; `DECLARED_GRADE` 0,5–1,0; `UNTESTED` 0,1–0,3; `SKILL_GATE` 0–1; `RELATED_CREDIT` 0–0,5; соотношение теста и ФСП. Около 70 вариантов.
- Выбрано: навыки 0,50 → 0,45, грейд 0,20 → 0,30, специализация 0,20 → 0,25, `EVIDENCE_WEIGHT` 0,40 → 0,30. Остальные константы без изменений.
- `DECLARED_GRADE = 1,0` на зёрнах 301–305 давал чуть больше, но оставлен 0,85 по продуктовому правилу: заявленный грейд не приравнивается к подтверждённому тестом. При 1,0 завышенный грейд в первой десятке растёт с 21,5 % до 24,2 %.

Отложенная проверка, парная разность по потребностям (после минус до), 95 % ДИ — бутстреп по потребностям, 2000 повторов:

Протокол	Зёрна	nDCG@10 до → после, Δ [95 % ДИ]	P@10 до → после, Δ	MRR до → после, Δ
Строгий	2026–2028	0,652 → 0,658, +0,005 [−0,001; +0,011]	0,636 → 0,632, −0,003	0,817 → 0,805, −0,011 [−0,024; −0,001]
Строгий	4001–4005	0,652 → 0,659, +0,007 [+0,002; +0,011]	0,628 → 0,626, −0,002	0,813 → 0,803, −0,010 [−0,016; −0,004]
A	2026–2028	0,607 → 0,621, +0,013 [+0,007; +0,020]	0,540 → 0,547, +0,007	0,852 → 0,859, +0,007

Протокол	Зёрна	nDCG@10 до → после, Δ [95 % ДИ]	P@10 до → после, Δ	MRR до → после, Δ
A	4001–4005	0,598 → 0,612, +0,014 [+0,010; +0,018]	0,514 → 0,526, +0,013 [+0,007; +0,019]	0,845 → 0,851, +0,007
B	2026–2028	0,666 → 0,684, +0,018 [+0,012; +0,025]	0,638 → 0,649, +0,011 [+0,003; +0,020]	0,918 → 0,922, +0,003
B	4001–4005	0,660 → 0,680, +0,020 [+0,016; +0,025]	0,627 → 0,643, +0,016 [+0,010; +0,021]	0,887 → 0,890, +0,003

Цена: по строгому протоколу MRR ниже на 0,010–0,011, интервал не накрывает ноль. Что ещё сдвинулось на зёрнах 2026–2028: Спирмен внутри категории 0,374 → 0,410, завышенный грейд в первой десятке 21,5 % → 22,1 %, «настоящий грейд ниже потребности» 14,7 % → 13,6 %; `--check` проходит. Правило 4/5 на зёрнах 2026–2028 и 4001–4005 не хуже: минимальный IR отбора по языку кода 0,759 → 0,800 (теперь на пороге 0,8), по участию в ФСП 0,826 → 0,857, по региону 0,957 → 0,971; минимальный IR балла по языку кода 0,887 → 0,883 (в пределах шума бутстрепа) (`python -m app.validation.fairness --seeds 2026,2027,2028,4001,4002,4003,4004,4005`).

Таблицы абляций и парных сравнений по зёрнам в разделах выше, датированные до 11.10.2026, посчитаны на прежних весах; заголовочные цифры (лестница, протоколы, против двух баз, выводы) пересчитаны.

Почему nDCG@10 ниже, чем в других опубликованных работах

На внешнем открытом наборе, где у каждого кандидата есть проверенный грейд, наш балл даёт nDCG@10 0,835, а если подать ему заявленный грейд и навыки без теста — 0,740 (раздел «Проверка на внешнем открытом наборе»). У нас тест прошли 45 % кандидатов, 30 % завышают грейд, 15 % заявленных навыков ложны. Разрыв в цифрах объясняется в основном данными, а не формулой.

Разбор ошибок на отложенных зёрнах 4001–4005 (строгий протокол, 4000 мест в первых десятках, 39 % мест с меткой < 2): 51 % таких мест — у кандидата нет нужного навыка и он его не заявлял; это добор, когда подходящих меньше десяти (у 103 из 390 потребностей меньше десяти кандидатов с меткой ≥ 2, потолок P@10 0,875). 25 % — ложный заявленный навык, ещё 8,5 % — ложный навык вместе с недостающим. Завышенный или далёкий грейд — около 7 %. Город и формат в строгую метку не входят, поэтому их вес 0,05 по ней только шумит.

Проверка на внешнем открытом наборе

Источник набора — открытый репозиторий miliantr/fsp-talent, коммит `d45eb75`: 600 синтетических кандидатов, 60 вакансий, файл `evaluation/data/matching_benchmark.json`. Лицензия в репозитории не указана, поэтому ни данные, ни код мы не копируем: команда скачивает набор и справочник навыков по закреплённому адресу и кэширует их в `~/.cache/fsp-external`. Справочник читается через `ast`, не исполняется.

- **Разметка 0–3** — правило, опубликованное вместе с набором: специализация × грейд × покрытие обязательных навыков (с учётом близких технологий) × положение внутри грейда × зарплата. Наша реализация совпадает с авторскими метками на всех 36 000 парах.
- **Метрики** как у авторов: nDCG@10 с выигрышем 2^(rel) − 1, P@10 и MRR по меткам ≥ 2. 95 % ДИ — бутстреп по вакансиям.
- **Сопоставление полей наше:**
- специализация → роль;
- проверенный грейд → грейд, подтверждённый тестом;
- оценка уровня θ → вероятность уровня 1 / (1 + e^(−3(θ − граница грейда)));
- заявленные навыки → навыки, подтверждённые тестом (в наборе проверены все кандидаты);
- формат работы → формат.

История ФСП, зарплата, город и отрасль в наш балл не передаются. Другое сопоставление даст другие цифры.

Вход нашего балла	nDCG@10 [95 % ДИ]	P@10	MRR
проверенный грейд и тест	0,835 [0,802; 0,864]	0,785	0,947
заявленный грейд и навыки, без теста	0,740 [0,697; 0,781]	0,700	0,842

Авторы набора публикуют для своего ранжирования nDCG@10 0,808. Наш балл на тех же метках — 0,835, интервалы перекрываются. Разница между строками таблицы показывает, сколько даёт проверенный грейд.

Команда: `cd backend && python -m app.validation.external_dataset` (и `--declared`). Тест сопоставления без сети: `tests/unit/test_external_dataset.py`.

Сценарий «чистые данные» на нашем генераторе

Флаг `--clean` меняет только то, что видит подбор: все кандидаты прошли тест, показан настоящий грейд, ложных навыков нет. Скрытая правда, метки и потребности те же, что в основном мире (SHA набора тот же, закреплено `tests/unit/test_clean_scenario.py`). Это допущения внешнего набора выше. Веса не перенастраивались. Основная цифра остаётся строгой: 0,659 на 4001–4005.

Протокол, `--clean`	Зёрна	nDCG@10 [95 % ДИ]	P@10	MRR
Строгий	4001–4005	0,863 [0,846; 0,878]	0,696	0,914
A	4001–4005	0,780 [0,767; 0,792]	0,666	0,951
B	4001–4005	0,936 [0,927; 0,944]	0,800	0,995
Строгий	2026–2028	0,869 [0,850; 0,887]	0,716	0,929

Команда: `cd backend && python -m app.validation.protocols --protocol all --clean`.

По протоколам опубликованных методов

Чтобы цифры подбора можно было сравнить с цифрами других опубликованных решений, тот же продуктовый порядок посчитан ещё по двум градуированным протоколам. На этих зёрнах ничего не подбирали: веса настроены на отдельных зёрнах 301–305 (раздел «Настройка весов»). Меняются только правило разметки по скрытой истине, отбор потребностей и определения метрик.

- **Протокол A**, шкала 0–3. 0: другая специализация, истинный грейд дальше чем на ступень или нет ни одного обязательного навыка. 1: соседний грейд или меньше 1/3 обязательных навыков. 2: целевой грейд и приемлемые навыки, либо грейд на ступень выше запроса и не меньше 1/3 навыков. 3: целевой грейд, не меньше 2/3 обязательных навыков и θ в верхней половине грейда. Минус 1 за несовпадение формата работы (зарплату наш синтетический мир не моделирует). Потребности, где меньше трёх кандидатов с оценкой ≥ 2, пропускаются.
- **Протокол B**, полезность u = специализация (1, смежная 0,4) × грейд (1 тот же, 0,75 на ступень выше, 0,45 на ступень ниже) × (0,25 + 0,75 × доля обязательных навыков) × положение внутри грейда. Оценка 3 при u ≥ 0,8, 2 при u ≥ 0,6, 1 при u ≥ 0,4. Пропускаются потребности без кандидатов с оценкой ≥ 2.
- **В обоих протоколах:** nDCG@10 с выигрышем 2^(rel) − 1, скидкой 1 / $\log_2(i + 1)$ и идеалом по всему пулу, P@10 как доля оценок ≥ 2 на десяти местах, MRR по первому кандидату с оценкой ≥ 2. Пул 800 кандидатов, 80 потребностей на зерно, 95 % ДИ — бутстреп по потребностям.

Протокол	Зёрна	nDCG@10 [95 % ДИ]	P@10	MRR	Потребностей
Строгий (наш, основной)	2026–2028	0,658 [0,633; 0,683]	0,632	0,805	232
Строгий (наш, основной)	4001–4005	0,659 [0,641; 0,675]	0,626	0,803	390
A	2026–2028	0,621 [0,602; 0,641]	0,547	0,859	214
A	4001–4005	0,612 [0,596; 0,627]	0,526	0,851	354
B	2026–2028	0,684 [0,662; 0,706]	0,649	0,922	237
B	4001–4005	0,680 [0,665; 0,695]	0,643	0,890	393
A, поиск по ключевым словам (BM25)	2026–2028	0,399 [0,379; 0,420]	0,356	0,627	214

Одинаковый протокол не делает цифры сопоставимыми с чужими: синтетические миры разные. У нас тест прошли только 45 % кандидатов, 30 % завышают грейд, а заявленные навыки теряют четверть настоящих и на 15 % ложны. Поэтому сравнивать стоит с базами на той же разметке (строка BM25), а не с числами из других проектов.

Проверка: `python -m app.validation.matching --protocol all` (то же, что `python -m app.validation.protocols`); база: `python -m app.validation.protocols --protocol a --method bm25`.

Нагрузочная проверка

ТЗ 3.5: «стабильная работа при одновременном использовании десятками пользователей; основные операции — формирование подборки, открытие профиля, поиск — выполняются за приемлемое время».

Как проверяли

Скрипт `backend/tests/load_api.py` регистрирует нужное число пользователей через API (коды подтверждения берёт из писем, как сквозная проверка), затем запускает их одновременно. Каждый пользователь повторяет обычные действия с паузой 0,5–1,5 секунды между запросами, как человек, который читает страницу:

- работодатели ищут по банку кандидатов (шесть разных запросов), открывают подборку по потребности, свои вакансии и профиль компании;
- кандидаты открывают профиль, вакансии под профиль, задачу недели и уведомления.

Команда на запущенном стенде:

```
make load
# или
docker compose exec -T -e SMOKE_BASE=http://localhost:8000/api/v1 -e MAILPIT_URL=http://mailpit:8025 \
  backend python tests/load_api.py --users 40 --seconds 60
```

Результат

Время ответа дано перцентилями: p50 — медиана, p95 — время, быстрее которого пришли 95 % ответов, p99 — 99 %.

Стенд с демо-данными (153 кандидата, 9 вакансий), API в двух процессах uvicorn, PostgreSQL 16 и Redis 7 на той же машине: 2 ядра Intel Xeon 2,8 ГГц. Прогон 06.10.2026.

40 пользователей одновременно (20 работодателей, 20 кандидатов), 60 секунд, 2416 запросов, 39,3 запроса в секунду.

Операция	Запросов	Ошибок 5xx и сетевых	Ответов 429	p50, мс	p95, мс	p99, мс
Поиск по банку кандидатов	417	0	0	36	69	364
Подборка по потребности	11	0	0	52	274	344
Профиль кандидата	293	0	0	10	21	264
Вакансии под профиль	314	0	0	11	25	147
Задача недели	283	0	0	12	26	283
Уведомления	326	0	0	7	22	313
Вакансии компании	388	0	0	6	17	157
Профиль компании	384	0	0	6	15	158

Ни одной ошибки. 95 % ответов на поиск быстрее 70 мс, на открытие профиля — быстрее 25 мс. Тяжелее всего подборка по потребности: она ранжирует весь банк по оценке соответствия и считает перцентиль внутри категории, её 95-й перцентиль — 274 мс.

Одновременные ответы

Ответ на приглашение берёт строку с блокировкой. Второй ответ на то же приглашение получает «уже обработано». Черновик ответа в тесте хранит номер версии: из нескольких сохранений одной версии остаётся одно, остальные получают отказ.

Команда не трогает демо-приглашения. Одноразовые идентификаторы задают переменными:

```
docker compose exec -T -e SMOKE_BASE=http://localhost:8000/api/v1 \
  -e LOAD_SESSION_ID=... -e LOAD_ITEM_TOKEN=... -e LOAD_DRAFT_VERSION=0 \
  -e LOAD_OFFER_ID=... \
  backend python tests/load_api.py --scenario races
```

Таблица из 40 пользователей выше от этого сценария не меняется.

Масштабирование

- **Поиск** ограничен 30 запросами в минуту на работодателя: это защита от выгрузки базы, а не предел производительности. В прогоне лимит не сработал ни разу.

- **Подборка** считается на сервере приложения по кандидатам, прошедшим SQL-фильтры. В Redis на 300 секунд лежит один порядок и баллы: ключ `shortlist:{потребность}:{версия}:{эпоха банка}`. Эпоха увеличивается, когда меняется профиль, заканчивается тест, приходит синхронизация ФСП, меняются видимость, согласие или статус поиска, человек удаляет аккаунт или компания записывает итог собеседования. Одинаковые потребности без итога собеседования считают этот порядок один раз. Пометки компании («в шорт-листе», «уже приглашён», «контакты закрыты») в Redis не пишутся и подставляются после чтения, только для той компании, которая спросила. Правка потребности увеличивает версию, и старая запись больше не читается. Таблица ниже снята, когда готовые страницы хранились 60 секунд.
- **API без состояния**: процессы uvicorn масштабируются горизонтально (`--workers` или реплики контейнера `backend`), состояние сессий входа через ФСП ID и лимиты лежат в Redis.

Большой банк

Команда `make load-big` добавляет синтетические профили, гоняет нагрузку и снова прячет их из поиска. Это не люди: почта `extra00001@load.talent`, имя «Профиль 00001», без телефона и даты рождения. Демо-аккаунты команда не удаляет. На 2000 профилях в поиске 1674, на 10 000 — 8497 (`python -m seed.generate_seed --extra-candidates N`, строка `in_search`).

Сценарий подборки: 20 работодателей, у каждого три потребности, страницы со смещением 0, 20 и 40, 60 секунд. Запросов больше 300, лимит 30 поисков в минуту не срабатывает.

Подборка кеширует общий порядок на 300 секунд. Карточки собираются только для запрошенной страницы. Команда на 10 000 профилях и двух процессах: `python tests/load_api.py --scenario shortlist --seconds 60 --reuse`. Напечатано: 367 запросов, 5,8 в секунду, ошибок 5xx нет, ответов 429 нет, медиана 18 мс, 95-й перцентиль 360 мс, 99-й 596 мс. Прогон сделан на коммите `8ebfa0f` (08.10.2026), который ввёл этот кеш; эту цифру цитирует README. После него подбор менялся (#282, #288, #348, #366), а нагрузку заново не мерили.

Строка «10 000 / 2» таблицы ниже — этот прогон. Остальные три строки и таблицы смешанного сценария сняты раньше, на коммите `19a922a` (08.10.2026, `make load-big`), когда кешировалась каждая страница подборки на 60 секунд; после перехода на общий порядок их не переснимали. Тогда строка «10 000 / 2» давала 346 запросов и p95 1120 мс — это был прежний кеш, к текущему коду она не относится.

Банк	Процессы	Запросов	В секунду	Ошибок 5xx	429	p50, мс	p95, мс	p99, мс
2000	2	358	5,7	0	0	15	761	1628
2000	4	361	5,7	0	0	14	435	979
10 000	4	358	5,6	0	0	47	745	1183
10 000	2	367	5,8	0	0	18	360	596

Четыре процесса API задаёт `docker-compose.load.yml`. Обычный стенд остаётся на двух: в конце `make load-big` контейнер API поднимается снова из основного файла.

Смешанный сценарий тот же, что `make load`: работодатели ищут и открывают подборку, кандидаты открывают профиль. Ошибок 5xx и ответов 429 не было. Ниже — банк 10 000, он тяжелее.

40 пользователей, 2 процесса, 2331 запрос, 38,2 в секунду.

Операция	Запросов	Ошибок 5xx и сетевых	Ответов 429	p50, мс	p95, мс	p99, мс
Поиск по банку кандидатов	396	0	0	168	316	1103
Подборка по потребности	11	0	0	14	1278	1689
Профиль кандидата	288	0	0	8	122	737
Вакансии под профиль	312	0	0	9	125	250
Задача недели	280	0	0	10	136	536
Уведомления	318	0	0	6	112	677
Вакансии компании	362	0	0	5	99	506
Профиль компании	364	0	0	5	75	495

100 пользователей, 2 процесса, 4766 запросов, 77,5 в секунду.

Операция	Запросов	Ошибок 5xx и сетевых	Ответов 429	p50, мс	p95, мс	p99, мс
Поиск по банку кандидатов	782	0	0	536	1270	1549
Подборка по потребности	11	0	0	394	1330	1585
Профиль кандидата	634	0	0	196	748	1444
Вакансии под профиль	586	0	0	215	743	1420
Задача недели	617	0	0	177	612	1230

Операция	Запросов	Ошибок 5xx и сетевых	Ответов 429	p50, мс	p95, мс	p99, мс
Уведомления	606	0	0	156	814	1347
Вакансии компании	738	0	0	134	529	967
Профиль компании	792	0	0	134	526	801

40 пользователей, 4 процесса, 2348 запросов, 38,3 в секунду.

Операция	Запросов	Ошибок 5xx и сетевых	Ответов 429	p50, мс	p95, мс	p99, мс
Поиск по банку кандидатов	399	0	0	185	320	438
Подборка по потребности	11	0	0	13	784	1007
Профиль кандидата	289	0	0	8	128	171
Вакансии под профиль	313	0	0	8	128	206
Задача недели	283	0	0	9	120	176
Уведомления	322	0	0	6	110	197
Вакансии компании	365	0	0	5	91	160
Профиль компании	366	0	0	5	90	145

100 пользователей, 4 процесса, 5604 запроса, 90,8 в секунду.

Операция	Запросов	Ошибок 5xx и сетевых	Ответов 429	p50, мс	p95, мс	p99, мс
Поиск по банку кандидатов	920	0	0	240	586	790
Подборка по потребности	11	0	0	27	1052	1383
Профиль кандидата	730	0	0	15	247	796
Вакансии под профиль	697	0	0	14	238	689
Задача недели	726	0	0	15	227	383
Уведомления	725	0	0	10	251	713
Вакансии компании	887	0	0	10	192	297
Профиль компании	908	0	0	9	187	307

На 2000 профилях смешанный сценарий легче. 40 пользователей и 2 процесса: 2389 запросов, 39,0 в секунду, подборка p95 137 мс, поиск p95 238 мс. 100 пользователей и 2 процесса: 5557 запросов, 90,5 в секунду, подборка p95 309 мс, поиск p95 585 мс. Четыре процесса, 40 пользователей: 2397 запросов, 39,0 в секунду, подборка p95 829 мс. Четыре процесса, 100 пользователей: 5813 запросов, 94,7 в секунду, подборка p95 677 мс. Ошибок 5xx нет. У подборки в смешанном сценарии 11 запросов, поэтому 95-й перцентиль там шумный; сценарий из 346–361 запроса выше — основная цифра.

Интеграция с реестром ФСП и ФСП ID

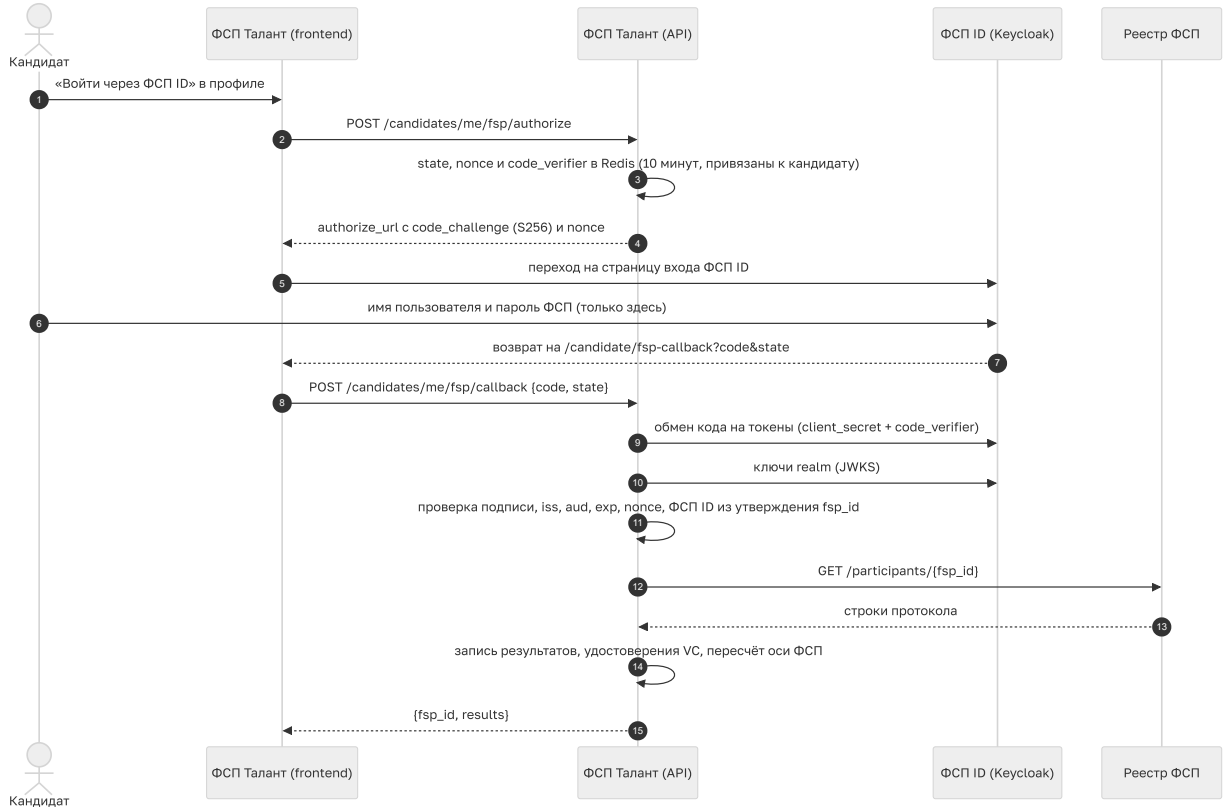
ТЗ не задаёт API и структуру данных ФСП: команда сама решает, что получать и как показывать (разделы 2 и 3.4). Ниже мы описываем, что предлагаем Федерации и что уже работает.

Короткий итог:

- **Вход через ФСП ID работает по OpenID Connect** — открытому стандарту входа через внешний сервис. Платформа — обычный клиент Keycloak (сервера учётных записей, на котором будет работать ФСП ID): вход по коду авторизации с защитой от перехвата кода (PKCE). В демо роль Keycloak играет помеченная страница этого сайта. Для подключения настоящего ФСП ID код менять не нужно, достаточно настроек в `.env`.
- **Из реестра берём только строки протокола:** соревнование, дисциплина, статус по ЕКП, место, число участников, командность, дата. Пол, контакты и документы не запрашиваем.
- **Результаты ФСП — отдельная ось профиля.** На грейд они не влияют. Кандидат без истории ФСП ничего не теряет: ось просто не учитывается (ТЗ 2.1).

1. Вход через ФСП ID (OpenID Connect, Keycloak)

ТЗ: «В дальнейшем ФСП ID будет работать на базе Keycloak, поэтому совместимость с этим решением упростит последующую интеграцию».



Свойства, которые проверяют тесты `backend/tests/unit/test_fsp_identity.py`:

- пароль ФСП вводится только на странице провайдера, платформа его не видит;
- `state` одноразовый и принадлежит одному кандидату: чужой `state` отклоняется;
- код одноразовый и привязан к `code_challenge`: перехваченный код без `code_verifier` этой сессии бесполезен;
- код выдаётся только для зарегистрированного `client_id` и адреса возврата;
- ID-токен проверяется по ключам пространства Keycloak (realm): подпись, издатель, аудитория, срок и одноразовый `nonce` этого запроса. Токен для другого клиента, просроченный, без `nonce` или подписанный чужим ключом отклоняется;
- вычисление `code_challenge` сверено с примером из RFC 7636.

Режимы

`FSP_ID_MODE`	Где вводят пароль	Что нужно
`mock` (демо)	Страница `/fsp-id/authorize` этого сайта с пометкой «Демо-провайдер». Реестр FSP2000–FSP2099, пароль `test`	Ничего

FSP_ID_MODE	Где вводят пароль	Что нужно
`oidc`	Страница входа Keycloak Федерации	`FSP_ID_ISSUER`, `FSP_ID_CLIENT_ID`, `FSP_ID_CLIENT_SECRET`, `FSP_ID_CLAIM`

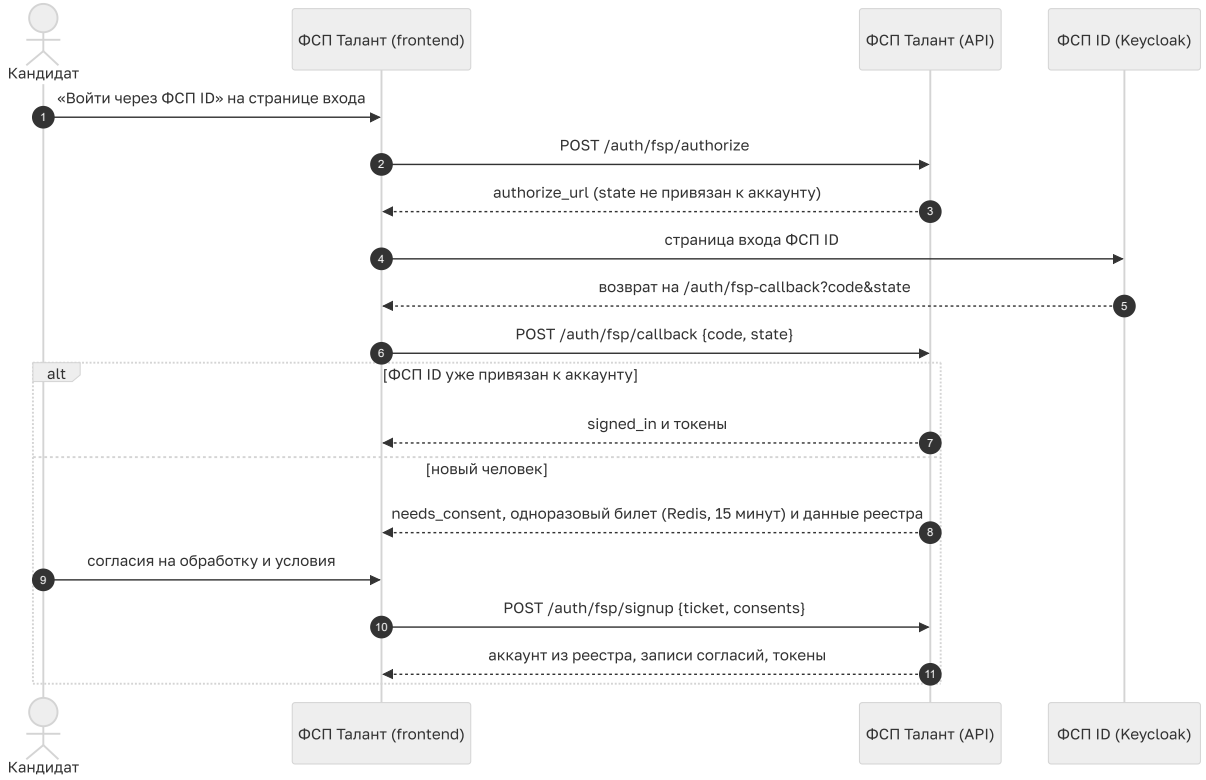
Что настроить в Keycloak Федерации

1. Клиент `fsp-talent`: Client authentication — On (конфиденциальный клиент), Standard flow — On, остальные потоки выключены.
2. Valid redirect URIs: `https://<адрес стенда>/candidate/fsp-callback` (привязка) и `https://<адрес стенда>/auth/fsp-callback` (вход).
3. Advanced → Proof Key for Code Exchange Code Challenge Method: `S256`.
4. Client scopes → mapper типа «User Attribute»: атрибут пользователя с номером ФСП ID → утверждение `fsp_id` в ID-токене.
5. В `.env` платформы: `FSP_ID_MODE=oidc`, `FSP_ID_ISSUER=https://<keycloak>/realms/<realm>`, `FSP_ID_CLIENT_ID=fsp-talent`, `FSP_ID_CLIENT_SECRET=<секрет клиента>`.

Код: `backend/app/services/fsp/identity.py` (`KeycloakFspIdProvider`, `MockFspIdProvider`), методы API — `backend/app/api/v1/candidates.py` (`/me/fsp/authorize`, `/me/fsp/callback`), демо-провайдер — `backend/app/api/v1/fsp_id_demo.py`.

Вход в платформу через ФСП ID

ФСП ID — единая точка входа, а на демонстрационном стенде вход можно имитировать. Поэтому ФСП ID работает двумя способами: привязка к готовому аккаунту (схема выше) и вход без регистрации по email. Регистрация по email (ТЗ 2.2, п. 4) остаётся.



- Тот же поток кода с PKCE, что и привязка, но свой адрес возврата: `/auth/fsp-callback` (привязка — `/candidate/fsp-callback`). Код, выданный для одного адреса, другой поток не примет. `state` и код одноразовые.
- Без обязательных согласий (`terms`, `pd_candidate`) аккаунт не создаётся: 422 `consent_required`, билет не тратится. Повтор билета — 400 `fsp_ticket_invalid`.
- Такой аккаунт работает без email и пароля: личность подтверждает ФСП ID. Для связи кандидат добавляет Telegram или телефон в профиле, уведомления приходят в колокольчик.
- В Keycloak добавляется второй адрес возврата: `https://<адрес стенда>/auth/fsp-callback`.
- Как показать: «Вход» → «Войти через ФСП ID» → `FSP001` / `test` (аккаунт `fsp001@demo.talent` уже привязан, вход сразу) или любой из FSP2000–FSP2099 / `test` (новый человек, экран согласий).

Код: `backend/app/api/v1/auth.py` (`/fsp/authorize`, `/fsp/callback`, `/fsp/signup`), тесты — `backend/tests/unit/test_fsp_signin.py` (вход привязанного аккаунта, согласия до создания, одноразовые `state` и билет, разделение адресов возврата).

2. Контракт реестра ФСП

Предлагаемый контракт — один метод только на чтение. В режиме `FSP_MODE=real` его вызывает `HttpFSPAdapter` (`backend/app/services/fsp/registry.py`). В демо (`FSP_MODE=mock`) те же структуры отдаёт `MockFSPAdapter` с синтетическими протоколами.

```
GET {FSP_API_URL}/participants/{fsp_id}
Authorization: Bearer {FSP_API_KEY}
Accept: application/json
```

```
{
  "fsp_id": "FSP001",
  "display_name": "Иван Иванов",
  "region": "Томская область",
  "birth_date": "2003-05-12",
  "results": [
    {
      "event_id": "CR2025-ALG",
      "event_name": "Чемпионат России по спортивному программированию 2025",
      "discipline": "algorithmic",
      "competition_status": "russia_championship",
      "place": 3,
      "total_participants": 180,
      "regions_count": 52,
      "is_team": false,
      "team_size": 1,
      "role": "participant",
      "event_date": "2025-11-20",
      "protocol_url": "https://…"
    }
  ]
}
```

Ответ	Что делает платформа
200	Записывает строки протокола, выдаёт по каждой подписанное удостоверение, пересчитывает ось ФСП
404	«Протоколы ФСП не найдены» (`fsp_not_found`), профиль остаётся без оси ФСП
другой код или неверная структура	503 «Реестр ФСП сейчас не отвечает», ранее загруженные данные не трогаются

Строка с неизвестной дисциплиной или местом вне `1…total_participants` пропускается, а не угадывается.

Поля и зачем они нужны

Поле	Зачем	Где используется
`discipline`	Одна из пяти дисциплин ВРВС «спортивное программирование» (`algorithmic`, `product`, `security`, `bas`, `robotics`) или `ai` — расширение вне ВРВС для протоколов соревнований по системам ИИ, загруженных вручную. Реестр Федерации `ai` не присылает: в ВРВС такой дисциплины нет (fsp-russia.ru, Положение 2026). Все соревнования весят одинаково, поэтому `ai` баллов не добавляет	Ось ФСП по дисциплинам, фильтр поиска
`competition_status`	Статус соревнования по ЕКП	Подпись строки протокола. В оценку не входит
`place`, `total_participants`	Класс результата: победитель, призёр, топ-10 %, верхняя половина, участие	Качество результата
`regions_count`	Показывается в карточке события, в оценку не входит	Карточка события
`ranks`	Разряд, дата присвоения и номер приказа по дисциплине	Профиль, фильтр «Разряд не ниже», PDF. В оценку не входит
`is_team`, `team_size`	Командный результат	Вес доказательства делится на √(размер команды): уверенность ниже, уровень не снижается
`event_date`	Давность	Затухание с полупериодом 18 месяцев

Поле	Зачем	Где используется
`birth_date`	Совершеннолетие	Профили младше 18 лет скрыты от работодателей без согласия законного представителя
`region`	Подпись в профиле	В балл подбора не входит
`display_name`	Подпись, если кандидат не указал имя сам	Видна работодателю только после согласия кандидата

Не запрашиваем: пол, контакты, паспортные данные, место учёбы. Принцип минимизации из ст. 5 152-ФЗ: берём только то, что нужно для оценки.

Значения `competition_status`: `world`, `russia_championship`, `russia_cup`, `russia_youth`, `all_russian`, `federal_district`, `regional`, `other`.

Аутентификация сервиса

В демо-контракте это ключ `Bearer`. В рабочей эксплуатации удобнее токен Keycloak по `client_credentials` того же клиента `fsp-talent` с отдельной ролью «чтение протоколов». Тогда доступ к реестру отзывается в одном месте вместе с входом.

3. Обновление данных

Протоколы загружаются при привязке ФСП ID. Повторно их забирает почасовая задача `sync_fsp_changes`: она спрашивает реестр, кто изменился после сохранённой отметки, и дописывает только новые строки уже привязанным участникам.

```
GET {FSP_API_URL}/participants/changes?since=2026-10-01T00:00:00Z
→ {"fsp_ids": ["FSP001", "FSP017"], "next_since": "2026-10-06T12:00:00Z"}
```

Отметку хранит Redis (`fsp:changes:since`). Первый запуск смотрит на два часа назад. Повтор той же строки безопасен: пара (участник, соревнование) в таблице `results` уникальна, уже лежащая строка не пишется второй раз. В демо ту же выборку отдаёт `MockFSPAdapter`. Если Федерации удобнее самой присылать изменения, подойдёт вебхук (уведомление на наш адрес) «протокол утверждён» с той же структурой и подписью HMAC.

Исправления и аннулирования. Для изменившегося участника строки с `trust_level = verified_fsp` сверяются с ответом реестра по `event_id`. Строка, которой больше нет в реестре, удаляется (аннулирован протокол). Строка, у которой изменились место, размер поля, команда, дата, роль или дисциплина, заменяется исправленной. Подписанное достижение (VC) удалённой или заменённой строки отзывается (`revoked_at`), профиль пересчитывается. Протоколы, загруженные кандидатом из файла, не трогаются. Пустой ответ реестра ничего не удаляет, как и ответ, после которого пропало бы больше половины строк профиля (и больше одной): такой ответ считается неполным, в журнал пишется предупреждение, исправления мест при этом применяются. Сбой или обрезанная страница не должны стирать историю.

Затухание пересчитывается каждую ночь. Давность считается от даты расчёта, поэтому задача `rescore_fsp_nightly` (03:20 МСК) пересобирает профили, у которых оценка ФСП старше 20 часов или посчитана прежней версией методики. Иначе двух кандидатов сравнивали бы по разным опорным датам.

4. Как результат становится доказательством

1. Строка протокола → класс результата по месту и размеру поля (`result_class`, методика `v4.2.0`): победитель 1,00 · призёр (2–3 место) 0,85 · топ-10 % 0,65 · верхняя половина 0,45 · участие 0. Победитель и призёр — при шести участниках и больше. В нижнюю границу входит лучшее подмножество до пяти результатов. Статус соревнования не учитывается: по совету Федерации все соревнования одинаково важны. Разряд из этой строки не считается.
2. Вес результата = затухание с полупериодом 18 месяцев / √(размер команды) · (1 − e^(−N/30)), N — размер поля. В одном подмножестве не больше двух результатов дисциплины за календарный месяц; отбор идёт по нижней границе и видит все результаты месяца. Вклад внутри команды не разделяется: команда снижает уверенность, а не уровень.
3. По дисциплине — байесовское сжатие к среднему: у человека с короткой историей оценка ближе к средней. Затем берётся осторожная нижняя граница уровня (LCB, lower confidence bound), чтобы один удачный старт не выглядел как стабильный уровень. В подборке по потребности эта граница берётся по профильной дисциплине целиком, а по остальным — с весом 0,5 (`app/services/fsp/relevance.py`). Фильтр «дисциплина» оставляет одну дисциплину. Статус соревнования в выборе не участвует. Один случайный успех не обгоняет стабильную серию. Участие весит 0 и в балл не входит: оно остаётся в профиле, но оценку не поднимает и не опускает. Профиль нельзя набрать одними онлайн-участиями. Проверка — `python -m app.validation.fsp_axis --seeds 3` из каталога `backend`: 2000 человек, зёрна 2026, 2027 и 2028, место в протоколе зависит от скрытой силы, а число онлайн-участий в нижней половине — нет. Считается тот же движок оси, что работает в продукте. В корреляцию не входят люди без единой строки протокола. Корреляция Спирмена показывает, насколько порядок людей по показателю совпадает с порядком по скрытой силе (1 — полностью).

Зерно	В корреляции	Спирмен LCB	Спирмен среднего места	Спирмен числа строк
2026	1969	0,768	0,695	0,148
2027	1966	0,766	0,668	0,144
2028	1966	0,769	0,680	0,089
среднее ± стандартное отклонение		0,768 ± 0,002	0,681 ± 0,014	0,127 ± 0,033

На каждом зерне LCB выше среднего места, а среднее место выше числа строк. Среднее место — средняя доля поля, которую человек оставил позади (выше значит финиш лучше). Накрутка из 40 онлайн-участий в нижней половине даёт LCB 0,16 (как пустая история), две победы — LCB 0,54. Грейд эта проверка не трогает. 4. Ось ФСП влияет на место кандидата внутри категории и никогда — на грейд: грейд даёт только тест специализации. Разряд — факт реестра (разряд, дата, приказ), а не расчёт по месту. Он показывается в профиле и фильтре «Разряд не ниже» и в оценку не входит. Формулы балла — в README и `backend/app/services/scoring/engine.py`, версия `v4.2.0`: новое событие никогда не опускает нижнюю границу.

ЕВСК 2026–2029

Действующий документ — приказ Минспорта России от 09.04.2026 № 299 «Об утверждении Единой всероссийской спортивной классификации», зарегистрирован Минюстом 05.06.2026 № 86925, вступил в силу 19.06.2026. Нормы по спортивному программированию — приложение № 77 (пункт 1.77). Прежний приказ от 11.01.2022 № 6 утратил силу.

Разряды присваиваются по нормам приложения № 77 к приказу № 299; платформа получает разряд из реестра ФСП готовым фактом и сама не вычисляет. Демо-реестр стенда выдаёт демо-разряды по своей таблице (`_DEMO_RANK_TABLE` в `backend/app/services/fsp/mock.py`), номер приказа начинается с `Приказ-демо-`: это демонстрационная запись, не приказ органа власти. Срок действия разряда в балл не входит.

В вид спорта «спортивное программирование» во Всероссийском реестре видов спорта входят пять дисциплин: алгоритмическое, продуктовое, беспилотных авиационных систем, робототехники и систем информационной безопасности (приказ Минспорта России от 31.01.2023 № 58; те же пять дисциплин — в федеральном стандарте спортивной подготовки, приказ от 24.06.2025 № 445). Код `ai` («программирование систем ИИ») — направление календаря ЕКП 2026, а не дисциплина ВРВС; в подборе он относится к специализации ML (`backend/app/services/categorization.py`). Подпись «вне ЕВСК» не ставится.

- Обзор приказа: https://ppt.ru/obzory/vstupaet-v-silu/prikaz-minsporta-rossii-09-04-2026-299
- Текст приказа без приложений: https://pravo.ppt.ru/prikaz/minsport/n-299-340165
- Страница Минспорта: https://minsport.gov.ru/evsk-2026-2029-sportivnoe-programmirovanie/
- Дисциплины в ВРВС, приказ № 58: https://pravo.ppt.ru/prikaz/minsport/n-58-277547
- Федеральный стандарт, приказ № 445: https://pravo.ppt.ru/prikaz/minsport/n-445-316320

5. По каждой строке выдаётся удостоверение W3C Verifiable Credentials 2.0 (`eddsa-jcs-2022`). Любой может проверить подпись: `GET /api/v1/public/issuer`, `POST /api/v1/public/credentials/verify`.

Сейчас издатель удостоверений — платформа: она подтверждает, что результат получен из системы ФСП. В рабочей эксплуатации издателем может стать сама Федерация со своим ключом, формат при этом не меняется.

5. Что видит работодатель

Карточка в выдаче анонимна: статус соревнования, год и диапазон процентиля («топ-5 %»). Названия события и точного места в ней нет, чтобы по ним нельзя было найти человека в открытом протоколе. После того как кандидат принял приглашение или откликнулся сам, работодатель видит имя и контакты. Каждый просмотр записывается в журнал доступа.

Защита от фиктивных вакансий и интеграция с ATS

ТЗ (раздел 2) просит концепцию защиты от фиктивных вакансий и связи с ATS работодателя. Ниже мы разделили, что уже работает, а что пока остаётся концепцией.

1. Защита от фиктивных вакансий и недобросовестных работодателей

От чего защищаемся

Угроза	Чем вредит
Фиктивная вакансия ради сбора контактов	Утечка персональных данных, спам кандидатам
Вилка-приманка: в приглашении 300 тыс., на интервью 120 тыс.	Подрывает главное отличие платформы: условия видны до общения
Массовая рассылка одного шаблона всем подряд	Обратная механика превращается в спам
Компания-однодневка или выдача себя за другую компанию	Кандидат доверяет бренду, которого нет
Сбор открытых протоколов ФСП, чтобы найти человека	Обход анонимности карточки

Что уже работает

Мера	Где в коде
Контакты кандидата закрыты до его согласия: принятого приглашения или собственного отклика. Каждый просмотр пишется в журнал доступа	`app/api/v1/offers.py`, `app/api/v1/vacancies.py`, таблица `access_logs`
Кандидат закрывает контакты для одной компании, не отзывая согласие целиком: «Приглашения» → «Закрыть контакты для этой компании». Закрываются все принятые приглашения и отклики этой компании, пока она не получит новое принятое приглашение; компания видит 403 `contacts_revoked`, попытки после закрытия пишутся в журнал	`POST /candidates/me/offers/{id}/revoke-contacts`, `consent_logs`, `access_logs`
Переписка открывается только после принятия приглашения, после закрытия контактов — только чтение. Не больше 60 сообщений в час, уведомление без текста сообщения	`app/api/v1/messages.py`
Зарплатная вилка «от–до» обязательна в приглашении и вакансии, не ниже 27 093 ₽ (МРОТ с 01.01.2026, 429-ФЗ) в месяц до вычета НДФЛ (минимум с 1 января 2025 года, `MIN_MONTHLY_GROSS`), «до» не меньше «от», есть верхняя граница. Ссылка `http` или `www` в способе связи не принимается	`OfferCreate`, `VacancyIn`, `InviteDialog.tsx`
Не больше 50 приглашений в сутки от одной компании	`_OFFERS_PER_DAY`, `app/api/v1/offers.py`
Одно активное приглашение на пару «компания — кандидат» (уникальный индекс)	миграции, `app/services/offers.py`
Приглашение истекает через 14 дней, кандидат видит срок	`expires_at`, задача `expire_offers`
ИНН проверяется по контрольной сумме ФНС без внешних запросов. Кандидат видит в приглашении, указан ли ИНН	`app/services/inn.py`, `company_profile.inn_given`
Профиль компании (сфера, сайт, описание) кандидат видит до принятия решения	`/candidate/offers`, «О компании»
Отзывчивость на отклики против «вакансий-призраков»: в карточке вакансии, в приглашении и на странице компании видно, какую долю откликов за 90 дней компания открыла в течение 7 дней (нижняя граница Уилсона) и через сколько дней в среднем (медиана). Свежий неоткрытый отклик (меньше 7 дней) не считается. Если откликов меньше пяти, показывается «мало данных». Там же доля принятых приглашений компании среди отвеченных. Это факты, а не балл	`app/services/reputation.py` (`responsiveness`, `acceptance`), `GET /vacancies/for-me` → `company_responsiveness`, `GET /public/companies/{id}` → `reputation.applications`
Карточка в выдаче анонимна: статус соревнования, год и диапазон процентиля без названия события и места	`app/services/search/pipeline.py`
Уведомления не несут данных другой стороны	`app/services/notify.py`
Лимиты на вход, регистрацию и письма. Если хранилище лимитов недоступно, эти методы закрываются, а не пропускают запросы	`app/core/ratelimit.py`

Репутация из фактов. За 180 дней кандидат видит три доли и срок на платформе: как часто компания отвечает после согласия, как часто дело доходит до собеседования, сколько жалоб пришлось на приглашения. Доля — нижняя граница Уилсона (коэффициент 1,64) и появляется от пяти случаев, иначе написано «мало данных». Одной цифры «хорошая компания» нет: демо-балл рядом с ней по-прежнему помечен как демо-таблица. Число считает `compute_trust_score`: старт 3, затем отзывы, отметка реестра, тендеры и разрыв вилки. Реестр Минцифры и госзакупки не вызываются. На публичной странице компании балл написан той же фразой, что в приглашении.

Уровни доверия, которые уже считаются. Уровень не хранится: он выводится из фактов. 0 — почта подтверждена, 10 приглашений в сутки и одна открытая вакансия. 1 — ИНН прошёл контрольную сумму (ЕГРЮЛ не подключён, так и написано в метке), 50 приглашений и 3 вакансии. 2 — домен почты совпадает с доменом сайта, 50 приглашений и 10 вакансий. 3 — партнёр

ФСП, флаг ставит только `python -m app.services.trust partner <почта>`, 50 приглашений и 30 вакансий. Кандидат видит метку рядом с названием. Через API уровень не записывается.

Что предлагаем дальше

Проверка компании по ЕГРЮЛ и подтверждение домена записью DNS встраиваются как усиление уровней 1 и 2 на контуре Федерации, не заменяя их.

Жалоба, которая уже работает. В приглашении кандидат нажимает «Пожаловаться» и выбирает причину: не вакансия, сбор данных, зарплата не та, оскорбление или другое. Одна жалоба на одно приглашение или одну вакансию. За 30 дней компания уходит на проверку, если пожаловались не меньше трёх разных людей и нижняя граница Уилсона доли жалоб среди её приглашений не ниже 0,10 (коэффициент 1,64). Новые приглашения тогда отвечают 403: «Приглашения приостановлены до проверки». Снять паузу можно только вручную: `python -m app.services.trust release <ИНН>`. Жалобы при этом не удаляются. Одна жалоба или три жалобы от одного человека компанию не останавливают.

Автоматические правила. Порог жалоб замораживает рассылку до ручной проверки. Вилка, у которой верх ниже половины медианы ожиданий в категории, помечается и видна кандидату как «вилка ниже рынка». Вилка, у которой низ выше этой медианы втрое, попадает на стол модерации. Отправке обе не мешают. Медиана та же, что на странице рынка: ожидания кандидатов категории, и только если их не меньше пяти. Одинаковый текст в сотне приглашений за час снижает лимит.

Решение остаётся за человеком. Автоматика только ставит в очередь модерации и ограничивает лимит. Блокировка — решение модератора, компания видит причину и может её оспорить.

2. Интеграция с ATS работодателя

Цель — чтобы рекрутер работал в своей ATS (Хантфлоу, Talantix, E-Staff, Greenhouse), а платформа была источником кандидатов с подтверждённой категорией.

Принципы

- **Только согласившиеся.** ATS получает только кандидатов, которые сами согласились на контакт: приняли приглашение или откликнулись. До этого из платформы уходит только анонимная категория.
- **Те же правила, что в кабинете.** Когда кандидат закрывает контакты для компании («Приглашения» → «Закрыть контакты для этой компании»), ATS этой компании получает событие `contacts.revoked` с `candidate.ref` и `delete: true` и должна удалить контакты (ст. 21 152-ФЗ). Отзыв согласия на обработку целиком (`DELETE /api/v1/me/consents/pd_candidate`) и удаление аккаунта событий в ATS сейчас не отправляют: карточка уходит из поиска, кабинет и API работодателя перестают отдавать контакты (`contacts_allowed` в `app/services/privacy.py`), через 30 дней данные стираются на платформе (`purge_withdrawn`). Отправка `contacts.revoked` всем компаниям с доступом при отзыве согласия — следующий шаг.
- **Два направления.** Из платформы в ATS идут вебхуки и экспорт, из ATS в платформу — статусы найма: они нужны для воронки и для будущей калибровки подбора.

Вебхуки (платформа → ATS)

Работодатель указывает адрес на странице «Интеграции» и один раз видит секрет. Событие подписывается по Standard Webhooks (`webhook-id`, `webhook-timestamp`, `webhook-signature: v1,…` от строки `id.timestamp.body`) и старым заголовком `X-FSP-Talent-Signature`. Приёмник старше пяти минут по часам отклоняется. Повтор: 1 мин, 5 мин, 30 мин, 2 ч, 12 ч; ответ 410 повторов не получает. Чужой адрес — только https и только публичный хост, редиректы не идём. Приёмник стенда — кнопка на той же странице. Живое событие несёт категорию и, после согласия, контакты. Навыки, результаты ФСП и PDF из примера ниже в эту доставку не входят. Квитанция на странице «Компания» по-прежнему подписывает пример и никуда его не отправляет.

Событие	Когда	Что внутри
`offer.accepted`	Кандидат принял приглашение	Приглашение, категория, подтверждения, контакты, PDF-профиль по ссылке
`offer.declined`	Кандидат отклонил	Приглашение, причина, если указана
`application.created`	Кандидат откликнулся на вакансию	Вакансия, категория и `ref` кандидата. Контактов нет: работодатель открывает их в «Вакансиях», обращение пишется в журнал доступа
`contacts.revoked`	Кандидат закрыл контакты для этой компании	`candidate.ref` и `delete: true` — удалить у этой компании

```
{
  "event_id": "8c1d…",
  "type": "offer.accepted",
  "created_at": "2026-10-06T12:00:00Z",
  "employer_id": "…",
  "candidate": {
    "ref": "c_5f2…",
    "name": "…", "email": "…", "telegram": "…",
    "category": {"specialization": "backend", "grade": "middle", "grade_source": "assessed"},
    "skills": [{"name": "Python", "source": "test", "confirmed_at": "2026-09-30"}],
    "fsp": [{"discipline": "algorithmic", "lcb": 0.42, "best_rank": "KMC"}],
    "profile_pdf": "https://…/api/v1/…/profile.pdf"
  },
  "offer": {"role": "Backend-разработчик", "salary_from": 230000, "salary_to": 300000, "work_format": "remote"}
}
```


Формат кандидата совместим по смыслу с JSON Resume. Для ATS, которые принимают только резюме-файл, есть стандартизированный PDF-профиль: он уже формируется (`/api/v1/candidates/me/profile.pdf`).

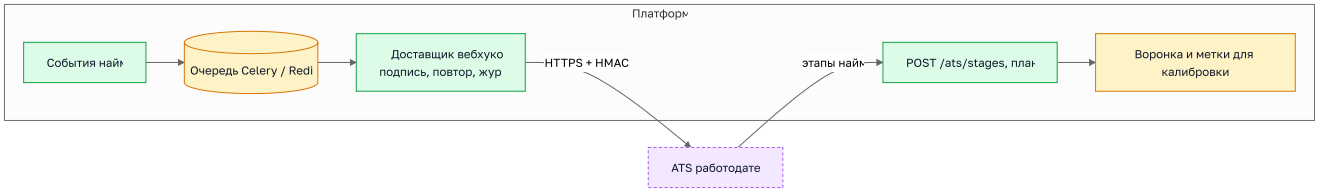
Статусы найма (ATS → платформа)

Итог собеседования уже приходит в платформу: `POST /api/v1/offers/{offer_id}/interview/outcome` со значением `passed`, `failed` или `no_show` (`backend/app/api/v1/interviews.py:166`). Следующий шаг интеграции — приёмник этапов ATS `POST /api/v1/ats/stages` с `candidate_ref` и этапом `screening`, `interview`, `offer`, `hired`, `rejected`: платформа покажет этапы в воронке работодателя и накопит размеченные пары «потребность — кандидат — итог». Это и есть данные для калибровки подбора: сейчас валидация идёт на синтетике, потому что меток найма ещё нет (`docs/VALIDATION_MATCHING.md`). Скрипт `python -m app.validation.matching --import pairs.json` уже принимает такие пары.

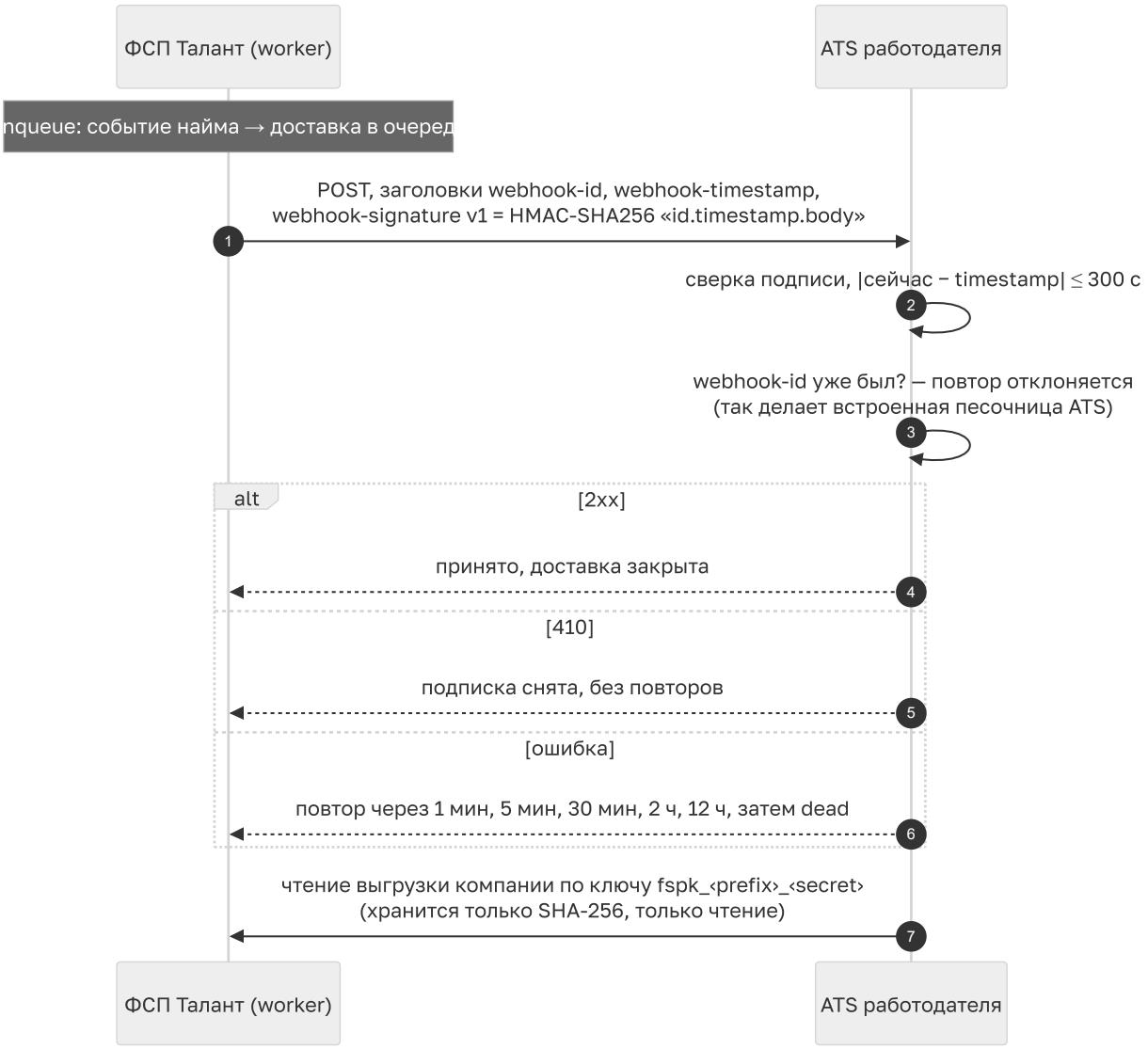
Пример: Хантфлоу

1. Вебхук `offer.accepted` → создание кандидата методом API Хантфлоу «добавить кандидата» с источником «ФСП Талант» и прикреплённым PDF-профилем.
2. Привязка к вакансии Хантфлоу по `need_id` или `vacancy_id`, который работодатель сопоставил в кабинете.
3. Вебхук Хантфлоу о смене этапа → приёмник этапов `POST /api/v1/ats/stages` (следующий шаг, контракт выше).

Архитектура



Подпись и защита от повтора (`app/services/webhooks.py`, `api_keys.py`):



Доставщик — отдельная задача Celery: очередь и повторы уже есть в стенде. Журнал доставок виден работодателю в кабинете.

3. Карта соответствия: EU AI Act и NIST AI RMF

Регламент (ЕС) 2024/1689 (AI Act) в России не действует. Мы берём его как ориентир лучшей практики: ИИ для подбора, фильтрации откликов и оценки кандидатов отнесён к высокому риску (Приложение III, п. 4(а)). Сроки применения обязанностей для систем Приложения III здесь не называем, они зависят от решений ЕС. Вторая рамка — NIST AI RMF 1.0 (NIST AI 100-1, doi:10.6028/NIST.AI.100-1), функции GOVERN, MAP, MEASURE, MANAGE. Таблица не сертификат: каждая строка указывает на код или документ, который это делает, и на то, чего нет.

Требование	Что у нас	Где	Чего нет
AI Act ст. 9, NIST MAP — управление рисками	Риски и ограничения записаны рядом с каждой валидацией; атаки (утечка ключей, накрутка навыков, угадывание) смоделированы	`docs/VALIDATION_ASSESSMENT.md`, `docs/VALIDATION_MATCHING.md`	Отдельного реестра рисков с владельцами
AI Act ст. 10 — данные	Проверочный набор для подбора синтетический, генератор и SHA-256 набора печатаются; пол и национальность продукт не собирает	`backend/app/validation/matching.py`	Проверки на живых размеченных парах
AI Act ст. 12 — журналирование	Каждый ответ теста — строка `test_session_items`, сессия — `test_sessions`; согласия — `consent_logs` (только добавление); просмотр контактов — `access_logs`; споры о грейде — `grade_disputes`	`backend/app/models/assessment.py`, `backend/app/models/audit.py`	Журнала каждой выдачи подбора с версией ранжирования

Требование	Что у нас	Где	Чего нет
AI Act ст. 13 — прозрачность	Причины места в выдаче («Почему в подборке»); формулы опубликованы; статистика заданий открыта без входа: `GET /api/v1/public/assessment/items`	`backend/app/services/matching.py` (`Reason`), `backend/app/api/v1/public.py`	Карточки модели на каждую версию ранжирования
AI Act ст. 14 — человеческий надзор	Решение о найме принимает работодатель: платформа ранжирует, но не отсеивает; пересмотр решения теста вручную за 5 рабочих дней; тест не понижает подтверждённый грейд	`backend/app/services/session_marks.py` (`DISPUTE_TEXT`), `backend/app/services/assessment.py`, `docs/SOLUTION.md` §10	Интерфейса оператора для разбора споров со сроком
AI Act ст. 15, NIST MEASURE — точность и устойчивость	AUC, nDCG, κ, экспозиция заданий, правило 4/5 по метке, которая не входит в балл	`docs/VALIDATION_*.md`, `python -m app.validation.fairness`	Мониторинга этих чисел на живом трафике
NIST MANAGE — реакция	Утёкшие ключи заданий уходят на карантин (z и CUSUM); публичные рейтинги показываются только при k ≥ 5 в ячейке	`backend/app/services/assessment.py`, `backend/app/api/v1/public.py` (`BANK_MIN`)	Регламента вывода модели из эксплуатации
NIST GOVERN — ответственность	Роли и права модерации выдаются только консольной командой; 152-ФЗ: согласия, отзыв, удаление	`docs/SOLUTION.md` §10	Назначенного ответственного за ИИ-риски у оператора

Коды ошибок API

Любая ошибка возвращается как RFC 9457 (`Content-Type: application/problem+json`) и сохраняет прежние поля:

```
{"type": "/problems/E01", "title": "Вход", "status": 403, "instance": "/api/v1/auth/participant/token",
 "problem": "E01",
 "error": {"code": "email_not_verified", "message": "Email не подтверждён: введите код из письма", "details": {"email":
"a@b.ru", "role": "participant"}},
 "detail": "Email не подтверждён: введите код из письма"}
```

`problem` — класс E01–E20. Конкретный код по-прежнему в `error.code`. Каталог классов: `GET /api/v1/problems`. Поле `detail` повторяет текст для старых клиентов. Для ошибки проверки полей (422) `detail` и `error.details` содержат список полей. У ответа 429 есть заголовок `Retry-After`.

Повторная регистрация уже подтверждённого адреса отвечает 202 `verification_sent`, как новая, и не показывает код. Письмо на этот адрес говорит, что аккаунт уже есть. Код `email_taken` остаётся на гонку двух запросов, которые создают одну строку.

Класс	Название
`E01`	Вход
`E02`	Доступ
`E03`	Не найдено
`E04`	Конфликт
`E05`	Поля запроса
`E06`	Слишком часто
`E07`	Подтверждение почты
`E08`	Согласие
`E09`	Тест
`E10`	Приглашение
`E11`	Задание
`E12`	Метод
`E13`	Уже нет
`E14`	Слишком большой запрос
`E15`	Сбой сервера
`E16`	Сервис недоступен
`E17`	Грейд
`E18`	Сессия теста
`E19`	Вилка
`E20`	Демо-аккаунт

Код	Значение
`bad_request`	HTTP 400
`unauthorized`	HTTP 401
`forbidden`	HTTP 403
`not_found`	HTTP 404
`method_not_allowed`	HTTP 405
`conflict`	HTTP 409
`gone`	HTTP 410
`validation_error`	HTTP 422
`payload_too_large`	HTTP 413
`rate_limited`	HTTP 429
`internal_error`	HTTP 500
`unavailable`	HTTP 503

Код	Значение
`invalid_credentials`	Неверный email или пароль
`email_not_verified`	Email не подтверждён: введите код из письма
`email_taken`	Аккаунт с таким email уже есть
`verification_invalid`	Код или ссылка не подходят
`verification_expired`	Срок кода истёк, запросите новый
`verification_attempts`	Слишком много неверных попыток, запросите новый код
`resend_cooldown`	Новый код можно запросить через минуту
`consent_required`	Нужно согласие на обработку данных
`consent_unknown`	Нет такого документа
`consent_withdrawn`	Кандидат отозвал согласие на обработку данных: контакты недоступны
`delete_confirm`	Чтобы удалить аккаунт, введите слово «удалить»
`fsp_invalid`	Неверные учётные данные ФСП
`fsp_taken`	Этот ФСП ID уже привязан к другому аккаунту
`fsp_already_linked`	К аккаунту уже привязан ФСП ID
`fsp_not_found`	Протоколы ФСП не найдены
`fsp_required`	Строка протокола крепится к ФСП ID. Сначала привяжите его в профиле.
`survey_required`	Сначала пройдите опрос: выберите специализацию
`grade_locked`	Подтверждённый грейд пока нельзя менять
`too_young`	Регистрация открыта с 15 лет
`birth_date_locked`	Дата рождения уже сохранена. Исправить её можно только через поддержку.
`reserved_email`	Этот адрес зарезервирован для демо-стенда
`decline_pause`	Кандидат отклонил приглашение этой компании: повторно — через 30 дней
`retake_cooldown`	Пересдача доступна позже
`question_bank_empty`	Не хватает заданий для теста
`session_closed`	Сессия теста уже завершена
`lower_offer_closed`	Ступень ниже по этой сессии принять уже нельзя
`session_expired`	Время сессии истекло
`item_token_invalid`	Задание не принадлежит этой сессии
`offer_closed`	Приглашение уже обработано
`offer_expired`	Срок приглашения истёк
`contacts_hidden`	Контакты откроются после согласия кандидата
`salary_required`	Укажите зарплатную вилку от и до
`need_not_found`	Потребность не найдена
`contacts_revoked`	Кандидат закрыл контакты для вашей компании
`task_closed`	Задание закрыто или срок сдачи прошёл
`task_answered`	Ответ на это задание уже отправлен
`sandbox_unavailable`	Проверка кода временно недоступна. Ответ не сохранён
`fsp_ticket_invalid`	Вход через ФСП ID устарел: начните вход заново
`invites_paused`	Приглашения приостановлены до проверки
`complaint_duplicate`	Жалоба на это приглашение или вакансию уже отправлена
`demo_protected`	Демо-аккаунт: пароль и удаление закрыты, чтобы опубликованный вход работал у всех
`demo_reset_forbidden`	Сброс доступен только демо-аккаунтам `jury1` … `jury10` на демо-стенде
`totp_required`	Нужен код из приложения-аутентификатора
`totp_invalid`	Код не подошёл или уже использован
`refresh_superseded`	Сессия уже обновлена в другом окне
`account_suspended`	Вход закрыт модерацией

Код	Значение
`csrf_invalid`	Запрос с cookie сессии без совпадающего токена
`nul_byte`	Нулевой байт в адресе, параметрах или теле запроса (HTTP 422, класс E05)
`age_mismatch`	Дата рождения не совпадает с возрастом 15–17 из регистрации: исправить можно через поддержку

Дизайн-система «Протокол»

Интерфейс ФСП Талант построен на брендбуке Федерации спортивного программирования. Главный объект продукта — подборка кандидатов, поэтому язык интерфейса взят из протокола соревнований: стартовый номер категории, место, подтверждения, табличные цифры, тонкие линейки. Версия 5 (октябрь 2026) сохраняет этот язык. Она добавляет общую шкалу компонентов, отдельный цвет ошибок, знак из логотипа и мобильную навигацию.

Что взято из брендбука ФСП

Источник: fsp-russia.ru/press/brandbook.

Элемент брендбука	Как используется
`#402FFF` — основной	Действия, ссылки, фокус, подтверждённый грейд, витрина категорий, кольцо оценки, линии сети на карте
`#EC1D35` — красный	Только приглашение: полоса над шапкой, окно отправки приглашения, полоса у ожидающего приглашения, счётчик приглашений, узел «Москва» на карте. Кнопка «Пригласить» на шаг темнее (`#D9172F`): так белый текст проходит AA
`#EDEDED` — светло-серый	Фон страниц светлой темы, основной текст тёмной
`#C8C9CA` — серый	Контуры карточек, точки карты
`#1B1C21` — тёмный	Основной текст светлой темы, панели тёмной, финальная полоса на главной
Шрифт JetBrains Mono (OFL 1.1)	Заголовки, цифры, места, грейды, подписи карты. Длинный русский текст набран Golos Text, чтобы читаться без усталости. Оба шрифта подключены локально через `@fontsource-variable`
Логотип `docs/assets/logo.png`	Из него вырезан знак — бегущий человек между скобками кода (`frontend/src/assets/mark.webp`, иконка вкладки `favicon.png`). Он читается на светлом и тёмном фоне, а на фирменной синей панели стоит на белой плашке
Карта России и соединительные линии	Главный графический элемент главной и входа: точечная карта и сеть региональных центров, которая один раз прорисовывается при загрузке

Токены

Цвета лежат в `frontend/src/index.css` в виде RGB-троек, поэтому Tailwind умеет накладывать прозрачность (`bg-accent/45`). Имена в `tailwind.config.ts`:

Токен	Светлая	Тёмная	Для чего
`paper`	`#EDEDED`	`#101115`	Фон страницы
`panel`	`#FFFFFF`	`#1B1C21`	Карточки, окна
`sunken`	`#F5F5F7`	`#16171C`	Подложки внутри карточек, шапки таблиц
`ink`, `ink-2`, `ink-3`	`#1B1C21`, `#484A53`, `#60626B`	`#EDEDED`, `#B8B9C0`, `#989AA4`	Текст трёх уровней
`line`, `line-strong`	`#DEDEE2`, `#C8C9CA`	`#2E2F38`, `#42444E`	Линейки и контуры
`field`	`#868892`	`#70727E`	Рамка поля, 3,5:1 к фону
`accent` (+ `hover`, `soft`, `ink`)	`#402FFF`	`#968CFF`	Действие, фокус, подтверждение
`invite` (+ `soft`, `ink`)	`#D9172F`	`#FF5C6E`	Только приглашение
`danger` (+ `soft`)	`#B03A00`	`#FF925C`	Ошибки, удаление, отзыв, таймер
`ok`, `warn`, `medal`	зелёный, янтарный, золото	светлее	Успех, предупреждение, разряды МС и КМС

Шкала. Кегли 11, 13, 14, 16, 18, 21, 26, 28, 34, 44, 56 px. Заголовок страницы — 28/34 px моноширинным с плотным трекингом, заголовок раздела — 16 px. Отступы кратны 4 px. Радиусы растут с размером объекта: метка 5 px, поле и кнопка 8 px, карточка 14 px, окно 18 px. Тень в светлой теме — один уровень `shadow-xs` у карточек и `shadow-lift` при наведении. В тёмной теме теней нет, только контур.

Движение. Анимация отвечает на действие: заполнение полос, раскрытие разбора, появление окна и шторки. Один раз при загрузке главной прорисовывается сеть регионов. При `prefers-reduced-motion` всё статично, включая скелетоны.

Правила

- **Одно главное действие на экран.** Это синяя кнопка. Остальные — контурные или текстовые.
- **Красный — только приглашение.** Это действие работодателя. Ошибки, «Отозвать», «Прервать», «Удалить аккаунт», последние секунды таймера и недостающий навык окрашены отдельным цветом `danger`, жжёным оранжевым. Его не спутать с приглашением.

- **Категория — это стартовый номер «специализация × грейд».** Если грейд подтверждён тестом, его ячейка залита синим и отмечена галочкой. Если заявлен — серая, за пунктирной линией. Грейды всегда названы по-русски: стажёр, джуниор, мидл, сеньор, лид.
- **Оценка всегда с разбором.** Кольцо с числом из 100, под ним полосы только тех факторов, которые участвуют в оценке. У полосы есть значение, а на широком экране её ширина пропорциональна весу фактора. Остальные факторы перечислены строкой «Не учитываются». Если критериев нет, подпись меняется на «сила профиля из 100».
- **Цели не меньше 24 px**, кнопки 40 px (малые 32 px), пункты мобильного меню 44 px. Ссылка внутри текста подчёркнута.
- **Без внешних загрузок.** Шрифты, знак, Swagger UI и карта лежат в проекте. CSP стенда: `script-src 'self'`, `font-src 'self'`, тема выставляется до отрисовки файлом `theme-init.js`, без встроенных скриптов.

Компоненты

Все классы — в `@layer components` файла `index.css`, React-обёртки — в `components/ui.tsx`.

- **Кнопки:** `btn-primary`, `btn-secondary`, `btn-ghost`, `btn-invite` (только окно приглашения), `btn-invite-quiet` (красный контур в длинных списках), `btn-danger`, `btn-icon`, модификатор `btn-sm`.
- **Поля:** `input`, `select`, `field-label`, `field-hint`, `field-error`. Фокус — синяя рамка и мягкое кольцо, у ошибочного поля (`aria-invalid`) рамка цвета ошибки.
- **Метки:** `badge-ok`, `badge-warn`, `badge-danger`, `badge-invite`, `badge-vc`, разряды `badge-mc` и `badge-kmc`; навыки — `chip-default`, `chip-active`, `chip-missing`. Счётчики — `count` (синий) и `count-invite` (красный, только у приглашений).
- **Плашка категории** `CategoryPlate` — стартовый номер с ролью и грейдом.
- **Карточка кандидата** — место, плашка, подтверждения, навыки, кольцо оценки `ScoreRing`, полосы факторов шириной по весу, разбор «Почему в выдаче».
- **Витрина** `BankGrid` — тепловая таблица «специализация × грейд» на обзоре работодателя и в поиске. Четыре ступени основного цвета, легенда «меньше — больше». Ячейка ведёт в поиск с фильтрами, ячейки меньше пяти человек скрыты. Текст в ячейках проходит 4,5:1 в обеих темах.
- **Шкала уверенности теста** — полоса с зонами двух порогов остановки (10 % и 90 %) и маркером текущей уверенности.
- **Вкладки** `tabs` / `tab` с `role="tab"` и `aria-selected`.
- **Таблица** `protocol` — тонкие линейки, шапка на подложке, табличные цифры. Таблица с прокруткой обёрнута в область с `tabIndex=0` и подписью.
- **Уведомления** `notice-info`, `notice-ok`, `notice-error` — цветная линия слева, текст основного цвета.
- **Пустое состояние** `Empty` и **скелетон** `Skeleton`.
- **Окна** — нативный `<dialog>` (`Modal`), на телефоне нижняя шторка. Фокус остаётся внутри, Escape закрывает.
- **Шапка кабинета** — знак, плашка роли, уведомления, тема, «Выйти». На широком экране под ней лента разделов. Уже 768 px вместо ленты — кнопка «Открыть меню» и шторка со всеми разделами и сменой темы.
- `hero-ink` — фирменная поверхность: `#402FFF` и точечная фактура карты, затухающая к левому краю. Одна на экран, только для главного итога: первый экран главной, категория кандидата, воронка работодателя, первое место рейтинга, итог теста. Внутри — `btn-on-hero`, `btn-on-hero-ghost`, `chip-on-hero`, `on-hero-muted`.
- `plural(n, [one, few, many])` — склонение числительных во всех счётчиках.

Контраст (WCAG 2.2 AA)

Посчитан по формуле относительной яркости:

Пара	Контраст
`#402FFF` на белом	6,84
`#402FFF` на `#EDEDED`	5,85
Белый на `#D9172F` (кнопка «Пригласить»)	5,10
`#1B1C21` на `#EDEDED`	14,53
Третичный текст `#60626B` на `#EDEDED`	5,19
Ошибка `#B03A00` на белом / на своей подложке	6,08 / 5,43
Рамка поля `#868892` на белом	3,53
Синий тёмной темы `#968CFF` на `#101115`	6,73
Третичный текст тёмной темы на `accent-soft`	5,11 (в версии 4 было 4,43)
Красный тёмной темы `#FF5C6E` и тёмный текст на нём	6,30
Золото разрядов `#8A5A00` на `#FCF3DC`	5,36

Контраст пар в таблице посчитан по формуле WCAG. Отдельно axe-core проходит 15 экранов: правила не отключаются, серьёзных и критичных нарушений быть не должно.

Карта

`frontend/src/assets/russia-network.json` (25 КБ) содержит 1 772 точки и 30 региональных центров, соединённых минимальным остовным деревом от Москвы. Как она собрана:

- геометрия — Natural Earth 1:50m (public domain) из пакета `world-atlas`;
- проекция — коническая равновеликая, параллели 52° и 68°;
- генератор — `frontend/scripts/gen-russia-network.mjs`;
- зависимости генератора ставятся отдельно и в `package.json` не попадают.

На странице карта — один SVG из трёх путей: точки нарисованы как `M x y h0` с круглым концом.

Практики, на которые опирались

- **Схемы, сетки и моноширинный шрифт вместо декоративного стекла.** Тренд описан в Tubik, UI Design Trends 2026. Пример — Vercel Geist: моно для данных, высокий контраст, сетка.
- **Анимация только в ответ на действие.** Учитывается `prefers-reduced-motion`; гибкий кегль заголовка задан через `clamp()`.
- **WCAG 2.2: размер цели.** Критерий 2.5.8 заложен в компоненты: кнопки 40 и 32 px, мелкие ссылки в таблицах 24 px, пункты мобильного меню 44 px.
- **Анонимная карточка до согласия и зарплата в приглашении.** Так устроены платформы обратного найма. У нас к этому добавлены разбор оценки и грейд, подтверждённый тестом.

Проверка интерфейса

Интерфейс проверяют семь сквозных сценариев Playwright (`frontend/e2e/journeys.spec.ts`) на свежей базе с демо-данными (`seed.generate_seed`), 210 модульных тестов Vitest и 60 проверок доступности (`frontend/e2e/a11y.spec.ts`). Один сценарий открывает кабинеты на ширине 390 px в тёмной теме. Снимки кабинетов лежат в `docs/assets/ui`. Попиксельного сравнения снимков в проекте нет.

Итог

- **Доступность:** актуальные цифры прогона — в ACCESSIBILITY.md. `@axe-core/playwright` 4.13.0, только зависимость для разработки фронтенда. 15 экранов (главная, вход, регистрация, методика, «Режим жюри», документы, кабинет и профиль кандидата, тест, экран задания, приглашения, кабинет работодателя, потребности, поиск и диалог приглашения) × светлая и тёмная темы × 1440 и 390 px. Набор правил wcag2a, wcag2aa, wcag21aa, wcag22aa. Серьёзных и критичных нарушений ноль, страница не шире окна. Правила не отключаются. Контраст ключевых пар — в таблице выше.
- **Телефон:** те же 15 экранов открываются на 390 px, плюс один сквозной сценарий кабинетов в тёмной теме. Разделы открываются из меню-шторки, а не из обрезанной ленты.
- **Тёмная тема:** фирменная поверхность `#402FFF` с белым текстом даёт 6,8:1. Витрина категорий в тёмной теме строится от светлого синего `#968CFF`, текст в ячейках не ниже 5,2:1.
- **Сквозные сценарии:** `frontend/e2e/journeys.spec.ts` — 7. Узкий экран в тёмной теме, регистрация с пробным тестом, контакты только после согласия, отклик на вакансию, вход через ФСП ID, два проверяющих подряд под демо-аккаунтами, тихий вход по просроченному access-токену. Вместе с проверкой доступности `npx playwright test --list` показывает 67.

Что изменилось в версии 5

- **Красный больше не значит «ошибка».** Появился цвет `danger`. Красный остался за приглашением: окно отправки с красной полосой, ожидающее приглашение у кандидата, счётчик приглашений.
- **Главная.** На первом экране — демо-вход одной кнопкой («Войти как жюри»). Второй экран объясняет обратный найм: путь кандидата и путь работодателя сходятся на витрине «специализация × грейд».
- **Режим жюри.** Таблица «Маршрут на три минуты»: кто, что нажать, что увидеть.
- **Обзор кандидата.** Приглашения стоят сразу под категорией, грейд назван по-русски.
- **Тест.** Шкала уверенности с порогами остановки. Таймер на плашке, варианты ответа с явным переключателем.
- **Работодатель.** Карточка кандидата с кольцом оценки и полосами шириной по весу. Банк кандидатов стал тепловой витриной и появился на обзоре. Сравнение показывает полосы по факторам. Шорт-лист — плашку категории.
- **Шапка.** Знак из логотипа, плашка роли, мобильное меню-шторка. «Выйти» видна на любой ширине.

Частые вопросы и ответы на экране

Вопрос	Где ответ
«Где здесь ФСП, кроме логотипа?»	Цвета и шрифт брендбука. Карта России с сетью отделений на главной и во входе. Отдельная ось «ФСП» в каждой оценке соответствия. Подписанные протоколы в разделе «Результаты ФСП»
«Как устроен обратный найм?»	Второй экран главной: два пути и витрина категорий. У работодателя — «Банк кандидатов» на обзоре и в поиске
«Чем подтверждённый грейд отличается от неподтверждённого?»	Синяя ячейка с галочкой на плашке категории и отметка «Грейд подтверждён тестом». Без теста карточка говорит «Грейд не подтверждён»

Вопрос	Где ответ
«Почему этот кандидат первый?»	Кольцо оценки, полосы только у действующих факторов (ширина — вес), строка «Не учитываются», разбор «Почему в выдаче», сравнение до четырёх кандидатов
«Как тест зависит от вакансии?»	Вкладка «Как проверим» в потребности и «Какой тест получится» в вакансии: доли тем, цитаты, примеры без ответа
«Что видит кандидат до согласия и после?»	В приглашении: компания, роль, вилка до вычета НДФЛ, «О компании». Контакты работодатель получает после нажатия «Принять и открыть контакты»
«Работает ли на телефоне?»	15 экранов на 390 px в обеих темах и сквозной сценарий кабинетов; разделы — в меню-шторке
«Это доступно?»	axe-core на 15 экранах, обе темы, 1440 и 390 px: серьёзных и критичных нарушений нет. Контраст пар посчитан по WCAG. Видимый фокус, цели не меньше 24 px, поддержка `prefers-reduced-motion`, нативный `<dialog>`, который держит фокус внутри

Сторонние компоненты и лицензии

Здесь перечислено всё стороннее, что входит в решение или используется при сборке. Собственный код решения открыт под лицензией MIT. Все компоненты ниже — открытое ПО или открытые материалы; закрытых библиотек и обфусцированного кода в решении нет.

Версии — те, на которых проходят тесты: backend ставит пакеты из `backend/requirements.lock`, frontend — из `frontend/package-lock.json`, инструменты документации — из `docs/tools/package-lock.json`. Лицензии взяты из метаданных пакетов (PyPI, поле `license` в `package-lock.json`).

Backend (Python 3.12)

Пакет	Версия	Лицензия	Зачем
FastAPI	0.142.2	MIT	HTTP API, OpenAPI
Starlette	1.7.0	BSD-3-Clause	ASGI-основа FastAPI
Uvicorn	0.54.0	BSD-3-Clause	ASGI-сервер
Pydantic	2.13.5	MIT	Схемы и валидация
pydantic-settings	2.15.0	MIT	Настройки из окружения
SQLAlchemy	2.1.3	MIT	ORM, асинхронный доступ
asyncpg	0.31.0	Apache-2.0	Драйвер PostgreSQL
Alembic	1.20.0	MIT	Миграции
pgvector (Python)	0.5.0	MIT	Тип vector
redis-py	6.4.0	MIT	Лимиты, состояние входа ФСП ID
Celery	5.6.3	BSD-3-Clause	Фоновые задачи и расписание
PyJWT	2.15.1	MIT	JWT, проверка ID-токена ФСП ID
argon2-cffi	25.1.0	MIT	Хеши паролей Argon2id
bcrypt	5.0.0	Apache-2.0	Проверка старых хешей при входе
cryptography	50.0.2	Apache-2.0 OR BSD-3-Clause	Ed25519, AES-256-GCM
httpx	0.28.1	BSD-3-Clause	Клиент реестра ФСП и Keycloak
pypdf	6.19.0	BSD-3-Clause	Текст из PDF-резюме
fpdf2	2.8.9	LGPL-3.0-only	PDF-профиль кандидата; используется как библиотека без изменений

fpdf2 распространяется по LGPL-3.0. Решение не меняет его исходный код и не встраивает его в свой: пакет ставится из PyPI отдельной зависимостью (`fpdf2==2.8.9` в `backend/requirements.lock`) и подключается обычным импортом Python во время работы (`from fpdf import FPDF` в `app/services/pdf_profile.py`). Это динамическое связывание в смысле LGPL: любой получатель может заменить библиотеку своей сборкой той же или совместимой версии, поменяв строку в lock-файле и пересобрав образ backend, а лицензия на собственный код решения от этого не зависит.

Только для тестов и проверки кода:

Пакет	Версия	Лицензия
pytest	9.1.1	MIT
pytest-asyncio	1.4.0	Apache-2.0
pytest-cov	7.1.0	MIT
Hypothesis	6.140.4	MPL-2.0

Версии выше — из `backend/requirements.lock`. Линтер Ruff (MIT) в lock-файл не входит: версия 0.16.10 закреплена в `.github/workflows/ci.yml` и `Makefile` (`pip install ruff==0.16.10`), в `pyproject.toml` указан нижний предел `ruff ≥ 0.4`.

Необязательные, в образ стенда не входят: `gigachat` ≥ 0.1.39 (MIT, клиент GigaChat), `sentence-transformers` ≥ 3.0 (Apache-2.0) и `torch` ≥ 2.3 (BSD-3-Clause) для смыслового поиска — ставятся аргументом сборки `INSTALL_RERANKER=1`.

Frontend (Node.js 20)

Пакет	Версия	Лицензия	Зачем
React, React DOM	18.3.1	MIT	Интерфейс
React Router DOM	6.30.6	MIT	Маршруты
TanStack Query	5.104.1	MIT	Запросы и кеш

Пакет	Версия	Лицензия	Зачем
Zustand	4.5.7	MIT	Состояние входа
React Hook Form	7.89.0	MIT	Формы
Zod	3.25.76	MIT	Проверка данных форм
@hookform/resolvers	3.10.0	MIT	Связка форм и Zod
Axios	1.20.0	MIT	HTTP-клиент
lucide-react	0.447.0	ISC	Иконки
qrcode-generator	2.0.4	MIT	QR-код паспорта навыков
date-fns	3.6.0	MIT	Даты
clsx, tailwind-merge	2.1.1, 2.6.1	MIT	Классы CSS
@fontsource-variable/golos-text, jetbrains-mono	5.3.0	OFL-1.1	Шрифты, раздаются с того же адреса, без внешних запросов

Сборка и тесты интерфейса:

Пакет	Версия	Лицензия
Vite	6.4.3	MIT
@vitejs/plugin-react	4.7.0	MIT
TypeScript	5.9.3	Apache-2.0
Tailwind CSS	3.4.19	MIT
PostCSS, Autoprefixer	8.5.28, 10.6.1	MIT
Vitest	5.0.3	MIT
Testing Library (react, jest-dom, user-event)	16.3.3, 7.0.1, 14.6.7	MIT
jsdom	29.1.1	MIT
Playwright Test	1.63.0	Apache-2.0
@axe-core/playwright	4.13.0	MPL-2.0

Образы контейнеров и инфраструктура

Компонент	Образ или версия	Лицензия	Где
PostgreSQL с pgvector и pg_trgm	`pgvector/pgvector:pg16`	PostgreSQL License	База, контейнер резервных копий
Redis	`redis:7.2-alpine`	BSD-3-Clause	Лимиты, брокер Celery
nginx	`nginx:1.28-alpine`	BSD-2-Clause	Вход локального стенда, раздача frontend
Caddy	`caddy:2.10-alpine`	Apache-2.0	HTTPS на сервере
Mailpit	`axllent/mailpit:v1.31.4`	MIT	Почтовый ящик локального стенда
Keycloak	`quay.io/keycloak/keycloak:26.0.7`	Apache-2.0	Проверка настоящего OIDC, профиль `fsp-id`
Python	`python:3.12-slim`, `python:3.12-slim-bookworm`	PSF-2.0	Образы backend и sandbox
Node.js	`node:20-alpine`, `node:20-bookworm-slim`	MIT	Сборка frontend; бинарник node в sandbox
Swagger UI	`swagger-ui-dist` 5.33.1, лежит в `backend/app/static/swagger-ui/` вместе с LICENSE и NOTICE	Apache-2.0	Страница `/docs`

Инструменты документации

Компонент	Версия	Лицензия	Зачем
Mermaid	11.17.2	MIT	Схемы в PDF документации
Matplotlib	3.11.2	Matplotlib License (на основе PSF)	Рисунки проверки (`docs/tools/make_figures.py`)
PptxGenJS	4.0.1	MIT	Слайды презентации
python-pptx	1.0.2	MIT	Сборка презентации

Модели и внешние сервисы

Что	Лицензия или условия	Как используется
GigaChat (Сбер), YandexGPT (Яндекс)	Условия облачных провайдеров	Необязательно, по умолчанию выключено (`AI_ENABLED`). Получает обезличенный текст; балл, порядок выдачи и грейд не меняет
`BAAI/bge-reranker-v2-m3`	Apache-2.0	Необязательная локальная модель пары; веса в репозиторий не входят и на стенде не используются
sslip.io	Публичный DNS-сервис	Имя сервера вида `203-0-113-10.sslip.io` без покупки домена
Let's Encrypt	Условия ISRG	Сертификат TLS, его получает Caddy

Материалы

- Фирменная палитра и стиль интерфейса — по брендбуку Федерации спортивного программирования; использование брендбука разрешено организаторами спецтрека.
- Демо-компании, кандидаты, вакансии и протоколы соревнований вымышлены и сгенерированы `backend/seed/`.
- Задания теста — собственные параметрические семейства решения (`backend/app/items/`).